

Modern C++ Programming

Resource Safety, Generic Programming, the Standard Library, and Modern Language Design

Dr. Charles Dorner

2026

Modern C++ Programming

1. [About this expanded volume](#)
 1. [Audience and progression](#)
 2. [Technical scope and primary references](#)
2. [How to use this book](#)
 1. [Read for a decision, then trace the mechanism](#)
 2. [Build the supplied examples](#)
 3. [Source, output, and exercise conventions](#)
 4. [Use failure deliberately and safely](#)
3. [Chapter 1 — Understanding the Philosophy of Modern C++](#)
 1. [1.1 Understanding Zero-Cost Abstractions](#)
 1. [1.1.1 Comparing Abstraction with Runtime Overhead](#)
 2. [1.1.2 Understanding Compile-Time Optimization](#)
 2. [1.2 Designing Around Value Semantics](#)
 1. [1.2.1 Treating Objects as Values](#)
 2. [1.2.2 Distinguishing Value and Reference Semantics](#)
 3. [1.3 Managing Ownership Explicitly](#)
 1. [1.3.1 Identifying Resource Owners](#)
 2. [1.3.2 Avoiding Ambiguous Ownership](#)
 4. [Worked example — Compare values, borrows, and an algorithmic abstraction](#)
 1. [Requirement and design](#)
 2. [Requirement and simplest approach](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A snapshot that was not a snapshot](#)
 5. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Write a value-and-lifetime contract](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
4. [Chapter 2 — Designing Robust C++ Classes](#)
 1. [2.1 Establishing Strong Class Invariants](#)
 1. [2.1.1 Preventing Invalid Object States](#)
 2. [2.1.2 Restricting Mutation Through Interfaces](#)
 2. [2.2 Applying const Correctness](#)
 1. [2.2.1 Designing Const Member Functions](#)
 2. [2.2.2 Passing Data Without Unnecessary Mutation](#)
 3. [2.3 Applying the Rule of Zero](#)
 1. [2.3.1 Delegating Resource Management to Library Types](#)
 2. [2.3.2 Eliminating Manual Special-Member Functions](#)
 4. [Worked example — Enforce a percentage and copy an owned reading collection](#)
 1. [Requirement and design](#)

2. [Establish the accepted-state boundary](#)
3. [Complete implementation](#)
4. [Build, run, and inspect](#)
5. [Evidence and interpretation](#)
6. [Failure clinic — Validation after mutation](#)
5. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Replace a validated interval atomically](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
5. [Chapter 3 — Managing Resources with RAII](#)
 1. [3.1 Understanding Resources Beyond Memory](#)
 1. [3.1.1 Managing Files, Locks, and Handles](#)
 2. [3.1.2 Associating Resources with Object Lifetime](#)
 2. [3.2 Building RAII Resource Wrappers](#)
 1. [3.2.1 Acquiring Resources During Construction](#)
 2. [3.2.2 Releasing Resources During Destruction](#)
 3. [Worked example — Own a temporary file and verify cleanup during an exception](#)
 1. [Requirement and design](#)
 2. [Trace acquisition, use, completion, and fallback](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — The destructor that never ran](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — A fake resource with observable release](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
6. [Chapter 4 — Understanding Copy Semantics](#)
 1. [4.1 Understanding Object Copying](#)
 1. [4.1.1 Using Copy Constructors](#)
 2. [4.1.2 Using Copy Assignment](#)
 2. [4.2 Distinguishing Shallow and Deep Copies](#)
 1. [4.2.1 Recognizing Shared Raw Resources](#)
 2. [4.2.2 Implementing Independent Ownership](#)
 3. [Worked example — Implement independent buffer copies with a commit point](#)
 1. [Requirement and design](#)
 2. [Walk through the state transition](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — Delete, then allocate](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Test assignment at the failure boundary](#)

- 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)
- 7. [Chapter 5 — Understanding Move Semantics](#)
 - 1. [5.1 Understanding Lvalues and Rvalues](#)
 - 1. [5.1.1 Identifying Persistent Objects](#)
 - 2. [5.1.2 Identifying Temporary Values](#)
 - 2. [5.2 Transferring Resources Efficiently](#)
 - 1. [5.2.1 Implementing Move Constructors](#)
 - 2. [5.2.2 Implementing Move Assignment](#)
 - 3. [5.3 Using `std::move` Correctly](#)
 - 1. [5.3.1 Requesting Move Semantics](#)
 - 2. [5.3.2 Understanding Moved-From Objects](#)
 - 4. [Worked example — Transfer a buffer and preserve a defined empty source](#)
 - 1. [Requirement and design](#)
 - 2. [Trace both objects, not only the destination](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — A nonzero size with no storage](#)
 - 5. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Observe relocation without making it a contract](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)
- 8. [Chapter 6 — Managing Ownership with Smart Pointers](#)
 - 1. [6.1 Expressing Exclusive Ownership with `std::unique\_ptr`](#)
 - 1. [6.1.1 Creating Unique Resources](#)
 - 2. [6.1.2 Transferring Unique Ownership](#)
 - 2. [6.2 Expressing Shared Ownership with `std::shared\_ptr`](#)
 - 1. [6.2.1 Understanding Reference Counting](#)
 - 2. [6.2.2 Recognizing Shared-Ownership Costs](#)
 - 3. [6.3 Breaking Ownership Cycles with `std::weak\_ptr`](#)
 - 1. [6.3.1 Recognizing Cyclic References](#)
 - 2. [6.3.2 Observing Objects Without Owning Them](#)
 - 4. [Worked example — Destroy a hierarchy without retaining an ownership cycle](#)
 - 1. [Requirement and design](#)
 - 2. [Trace the hierarchy's ownership edges](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — The observer that owned the answer](#)
 - 5. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Publish an immutable snapshot](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)

9. [Chapter 7 — Designing Natural Types with Operator Overloading](#)
 1. [7.1 Overloading Arithmetic and Comparison Operators](#)
 1. [7.1.1 Implementing Value-Type Arithmetic](#)
 2. [7.1.2 Defining Meaningful Comparisons](#)
 2. [7.2 Overloading Stream Operators](#)
 1. [7.2.1 Formatting Objects with operator<<](#)
 2. [7.2.2 Reading Objects with operator>>](#)
 3. [Worked example — Define arithmetic, ordering, and safe extraction for a quantity](#)
 1. [Requirement and design](#)
 2. [Define the operator contract before running it](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A correct constructor after an invalid addition](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Add checked compound assignment](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
10. [Chapter 8 — Reusing Behavior Through Inheritance](#)
 1. [8.1 Defining Base and Derived Classes](#)
 1. [8.1.1 Establishing Is-A Relationships](#)
 2. [8.1.2 Controlling Member Accessibility](#)
 2. [8.2 Understanding Construction Through Inheritance Hierarchies](#)
 1. [8.2.1 Base-Class Construction Order](#)
 2. [8.2.2 Derived-Class Destruction Order](#)
 3. [Worked example — Trace base and derived lifetimes without hidden allocation in cleanup](#)
 1. [Requirement and design](#)
 2. [Trace the complete lifetime, including the observer](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — The initializer list that looked ordered](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Add a member with a visible lifetime](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
11. [Chapter 9 — Designing Runtime Polymorphism](#)
 1. [9.1 Using Virtual Functions](#)
 1. [9.1.1 Overriding Base-Class Behavior](#)
 2. [9.1.2 Understanding Dynamic Dispatch](#)
 2. [9.2 Designing Abstract Interfaces](#)
 1. [9.2.1 Declaring Pure Virtual Functions](#)
 2. [9.2.2 Using Virtual Destructors Safely](#)

3. [Worked example — Apply interchangeable filters through exclusive base ownership](#)
 1. [Requirement and design](#)
 2. [Separate dispatch from ownership](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — The almost-matching override](#)
4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Add a range filter and test substitution](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
12. [Chapter 10 — Favoring Composition and Interface-Based Design](#)
 1. [10.1 Building Objects from Smaller Components](#)
 1. [10.1.1 Modeling Has-A Relationships](#)
 2. [10.1.2 Delegating Responsibilities](#)
 2. [10.2 Reducing Coupling Through Interfaces](#)
 1. [10.2.1 Programming Against Abstractions](#)
 2. [10.2.2 Replacing Inheritance with Composition](#)
 3. [Worked example — Test a composed reporter without external I/O](#)
 1. [Requirement and design](#)
 2. [Identify the owned value and the borrowed collaborator](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A borrowed sink escaped its owner](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Exercise a failing collaborator](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
13. [Chapter 11 — Creating Generic Functions with Templates](#)
 1. [11.1 Defining Function Templates](#)
 1. [11.1.1 Declaring Type Parameters](#)
 2. [11.1.2 Understanding Template Argument Deduction](#)
 2. [11.2 Controlling Generic Function Behavior](#)
 1. [11.2.1 Overloading Templates](#)
 2. [11.2.2 Constraining Acceptable Types](#)
 3. [Worked example — Generalize comparison without confusing text and addresses](#)
 1. [Requirement and design](#)
 2. [Trace one instantiation and one overload](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — Generic syntax, wrong semantics](#)

4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Specify a generic selection contract](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
14. [Chapter 12 — Creating Reusable Generic Classes](#)
 1. [12.1 Defining Class Templates](#)
 1. [12.1.1 Parameterizing Data Types](#)
 2. [12.1.2 Using Multiple Template Parameters](#)
 2. [12.2 Customizing Templates](#)
 1. [12.2.1 Applying Full Specialization](#)
 2. [12.2.2 Applying Partial Specialization](#)
 3. [Worked example — Build a fixed-shape matrix and specialize a type category](#)
 1. [Requirement and design](#)
 2. [Trace a coordinate into contiguous storage](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — One unchecked multiplication](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Extend a fixed matrix without breaking its contract](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
15. [Chapter 13 — Writing Constrained Generic Code](#)
 1. [13.1 Understanding Compile-Time Type Requirements](#)
 1. [13.1.1 Inspecting Types with Type Traits](#)
 2. [13.1.2 Performing Compile-Time Decisions](#)
 2. [13.2 Expressing Requirements with Concepts](#)
 1. [13.2.1 Defining Concepts](#)
 2. [13.2.2 Applying requires Clauses](#)
 3. [Worked example — Constrain integer addition and reject overflow before evaluation](#)
 1. [Requirement and design](#)
 2. [Prove the checks do not overflow first](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A concept that promised too much](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Build a checked range total](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
16. [Chapter 14 — Writing Functional-Style Code with Lambdas](#)
 1. [14.1 Creating Lambda Expressions](#)
 1. [14.1.1 Defining Parameters and Return Values](#)

- 2. [14.1.2 Capturing Local Variables](#)
- 2. [14.2 Building Generic and Stateful Lambdas](#)
 - 1. [14.2.1 Creating Generic Lambdas](#)
 - 2. [14.2.2 Using Mutable Closures](#)
- 3. [Worked example — Distinguish snapshot captures, live borrows, and owned counter state](#)
 - 1. [Requirement and design](#)
 - 2. [Trace the two timelines](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — The callback outlived its capture](#)
- 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Compare three configuration policies](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)
- 17. [Chapter 15 — Traversing Containers with Iterators](#)
 - 1. [15.1 Understanding the Iterator Abstraction](#)
 - 1. [15.1.1 Working with Begin and End Iterators](#)
 - 2. [15.1.2 Understanding Iterator Categories](#)
 - 2. [15.2 Managing Iterator Validity](#)
 - 1. [15.2.1 Recognizing Container Reallocation](#)
 - 2. [15.2.2 Avoiding Invalidated Iterators](#)
 - 3. [Worked example — Erase safely and preserve position across a deliberate reallocation](#)
 - 1. [Requirement and design](#)
 - 2. [Trace consecutive removals](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — Reserve did not save the cached end](#)
 - 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Build an invalidation decision sheet](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)
- 18. [Chapter 16 — Selecting Standard Library Containers](#)
 - 1. [16.1 Using Sequential Containers](#)
 - 1. [16.1.1 Comparing vector, deque, and list](#)
 - 2. [16.1.2 Evaluating Insertion and Access Costs](#)
 - 2. [16.2 Using Associative Containers](#)
 - 1. [16.2.1 Comparing Ordered Maps and Sets](#)
 - 2. [16.2.2 Comparing Hash-Based Containers](#)
 - 3. [Worked example — Exercise sequence and key-based container contracts](#)
 - 1. [Requirement and design](#)
 - 2. [Match each container to one observable property](#)
 - 3. [Complete implementation](#)

- 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — A report inherited hash order](#)
- 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Separate ownership from indexes](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)
- 19. [Chapter 17 — Processing Data with Standard Algorithms](#)
 - 1. [17.1 Searching and Sorting Collections](#)
 - 1. [17.1.1 Finding Elements with Standard Algorithms](#)
 - 2. [17.1.2 Sorting According to Custom Criteria](#)
 - 2. [17.2 Transforming and Reducing Collections](#)
 - 1. [17.2.1 Applying Element Transformations](#)
 - 2. [17.2.2 Accumulating Aggregate Results](#)
 - 3. [Worked example — Stably order jobs and transform a numeric report](#)
 - 1. [Requirement and design](#)
 - 2. [Trace the report contract before selecting algorithms](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — Four plausible but incorrect replacements](#)
 - 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Make ties, boundaries, and failure visible](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)
- 20. [Chapter 18 — Building Data Pipelines with Ranges and Views](#)
 - 1. [18.1 Understanding Ranges](#)
 - 1. [18.1.1 Treating Sequences as Range Objects](#)
 - 2. [18.1.2 Applying Range-Based Algorithms](#)
 - 2. [18.2 Building Lazy Views](#)
 - 1. [18.2.1 Filtering Without Copying](#)
 - 2. [18.2.2 Transforming Through Composable Pipelines](#)
 - 3. [Worked example — Build a lazy selection pipeline and an independent result](#)
 - 1. [Requirement and design](#)
 - 2. [Separate the recipe from the result](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — A returned pipeline that borrows a dead vector](#)
 - 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Choose an ownership boundary for report rows](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)

21. [Chapter 19 — Modeling Optional and Variant Data](#)
 1. [19.1 Representing Missing Values with `std::optional`](#)
 1. [19.1.1 Returning Optional Results](#)
 2. [19.1.2 Checking Presence Safely](#)
 2. [19.2 Representing Multiple Possible Types](#)
 1. [19.2.1 Using `std::variant`](#)
 2. [19.2.2 Visiting Variant Alternatives](#)
 3. [19.3 Grouping Heterogeneous Values](#)
 1. [19.3.1 Using Tuples](#)
 2. [19.3.2 Decomposing Values with Structured Bindings](#)
 4. [Worked example — Model absence, typed errors, and grouped report values](#)
 1. [Requirement and design](#)
 2. [Preserve information from parsing to presentation](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A default that erased an error](#)
 5. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Design a result type that preserves the reason](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
22. [Chapter 20 — Working with Files, Paths, Time, and Randomness](#)
 1. [20.1 Performing File Input and Output](#)
 1. [20.1.1 Reading and Writing Text Files](#)
 2. [20.1.2 Reading and Writing Binary Files](#)
 2. [20.2 Manipulating Filesystem Paths](#)
 1. [20.2.1 Inspecting Files and Directories](#)
 2. [20.2.2 Performing Portable Filesystem Operations](#)
 3. [20.3 Measuring Time and Generating Random Values](#)
 1. [20.3.1 Using `std::chrono`](#)
 2. [20.3.2 Using Standard Random Engines and Distributions](#)
 4. [Worked example — Round-trip text and explicit binary data in an isolated workspace](#)
 1. [Requirement and design](#)
 2. [Establish the experiment’s boundaries](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A successful computation with failed publication](#)
 5. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Design a bounded file-processing contract](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)

- 23. [Chapter 21 — Building a Modern C++ Application](#)
 - 1. [21.1 Designing the Ownership Model](#)
 - 1. [21.1.1 Assigning Clear Resource Ownership](#)
 - 2. [21.1.2 Avoiding Raw Owing Pointers](#)
 - 2. [21.2 Integrating Modern Language Features](#)
 - 1. [21.2.1 Combining Templates, Algorithms, and Ranges](#)
 - 2. [21.2.2 Using RAII Throughout the Application](#)
 - 3. [Worked example — Load a complete dataset transactionally and produce an ordered report](#)
 - 1. [Requirement and design](#)
 - 2. [Acceptance contract for the compact application](#)
 - 3. [Follow one candidate to its commit point](#)
 - 4. [Complete implementation](#)
 - 5. [Build, run, and inspect](#)
 - 6. [Evidence and interpretation](#)
 - 7. [From the compact reference to the hardened extension](#)
 - 8. [Failure clinic — The test caught its own failed assertion](#)
 - 9. [Separate three recovery questions](#)
 - 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Perform a release review rather than a feature demonstration](#)
 - 3. [Review and solution reasoning](#)
 - 4. [Primary reference notes](#)
- 24. [Appendix A — A Modern C++ Diagnostic Desk Reference](#)
 - 1. [Start with the failed contract](#)
 - 2. [Gate 1 — Translation and constraints](#)
 - 3. [Gate 2 — Linking and target composition](#)
 - 4. [Gate 3 — Ownership and lifetime](#)
 - 5. [Gate 4 — Invariants and failure guarantees](#)
 - 6. [Gate 5 — Algorithms and representation](#)
 - 7. [Gate 6 — External completion and evidence](#)
 - 8. [A reusable review record](#)
- 25. [Appendix B — Executable Ownership and Failure Labs](#)
 - 1. [B.1 Audit a resource from acquisition to release](#)
 - 1. [Requirement and model](#)
 - 2. [Complete source — resource_audit.cpp](#)
 - 3. [Evidence and interpretation](#)
 - 4. [Practice boundary](#)
 - 2. [B.2 Preserve a destination when copying fails](#)
 - 1. [Requirement and model](#)
 - 2. [Complete source — copy_transaction.cpp](#)
 - 3. [Evidence and interpretation](#)
 - 4. [Practice boundary](#)
 - 3. [B.3 Observe relocation without overclaiming move behavior](#)
 - 1. [Requirement and model](#)
 - 2. [Complete source — move_relocation.cpp](#)

- 3. [Evidence and interpretation](#)
 - 4. [Practice boundary](#)
- 4. [B.4 Retain immutable revisions with shared ownership](#)
 - 1. [Requirement and model](#)
 - 2. [Complete source — ownership_snapshot.cpp](#)
 - 3. [Evidence and interpretation](#)
 - 4. [Practice boundary](#)
- 26. [Appendix C — Generic Arithmetic and Range-Lifetime Labs](#)
 - 1. [C.1 Constrain the domain and check the arithmetic](#)
 - 1. [Requirement and simplest approach](#)
 - 2. [Complete source — checked_numeric.hpp](#)
 - 3. [Complete source — generic_pipeline.cpp](#)
 - 4. [Evidence and interpretation](#)
 - 5. [Guided extension](#)
 - 2. [C.2 Distinguish a range wrapper from its storage](#)
 - 1. [Requirement and model](#)
 - 2. [Complete source — range_lifetime.cpp](#)
 - 3. [Evidence and interpretation](#)
 - 4. [Guided extension](#)
- 27. [Appendix D — A Bounded, Transactional Reporting Application](#)
 - 1. [D.1 Define the extension’s contract](#)
 - 2. [D.2 Inspect the public boundary](#)
 - 1. [Complete source — report.hpp](#)
 - 3. [D.3 Follow parsing, validation, and commit](#)
 - 1. [Complete source — report.cpp](#)
 - 4. [D.4 Keep command-line policy outside the domain](#)
 - 1. [Complete source — main.cpp](#)
 - 2. [Build and run a known case](#)
 - 5. [D.5 Test the boundary, not just the happy path](#)
 - 1. [Complete source — tests.cpp](#)
 - 6. [D.6 Acceptance review and remaining work](#)
- 28. [Appendix E — Test Harness, Fixtures, and Release Evidence](#)
 - 1. [E.1 Keep assertions active in every build](#)
 - 1. [Complete source — course_test.hpp](#)
 - 2. [E.2 Run a contract in an isolated working directory](#)
 - 1. [Complete source — run_case.py](#)
 - 3. [E.3 Interpret the verification matrix](#)
 - 4. [E.4 A practical acceptance worksheet](#)
- 29. [Appendix F — Working Vocabulary and Design Questions](#)
 - 1. [F.1 Values, owners, and borrows](#)
 - 2. [F.2 Interfaces and guarantees](#)
 - 3. [F.3 Traversal, algorithms, and result models](#)
 - 4. [F.4 External operations and evidence](#)
- 30. [Primary references and further reading](#)
 - 1. [C++20 working draft N4861 — Abstract classes \[class.abstract\]](#)
 - 2. [C++20 working draft N4861 — Algorithms library \[algorithms\]](#)

3. [Clang AddressSanitizer documentation](#)
4. [C++20 working draft N4861 — Associative-container requirements \[associative.reqmts\]](#)
5. [C++20 working draft N4861 — Binary-search operations \[alg.binary.search\]](#)
6. [C++20 working draft N4861 — Structured bindings \[dcl.struct.bind\]](#)
7. [C++20 working draft N4861 — Value categories \[basic.lval\]](#)
8. [C++20 working draft N4861 — Time library \[time\]](#)
9. [CMake build-system manual](#)
10. [C++20 working draft N4861 — Concepts library \[concepts\]](#)
11. [C++20 working draft N4861 — constexpr declarations \[dcl.constexpr\]](#)
12. [C++20 working draft N4861 — Template constraints \[temp.constr\]](#)
13. [C++20 working draft N4861 — Construction and destruction behavior \[class.ctor\]](#)
14. [C++20 working draft N4861 — Standard conversions \[conv\]](#)
15. [C++20 working draft N4861 — Copy and move assignment \[class.copy.assign\]](#)
16. [C++20 working draft N4861 — Copy and move constructors \[class.copy.ctor\]](#)
17. [C++20 working draft N4861 — C library file operations \[c.files\]](#)
18. [C++20 working draft N4861 — Template argument deduction \[temp.deduct\]](#)
19. [C++20 working draft N4861 — Destructors \[class.dtor\]](#)
20. [C++20 working draft N4861 — Construction, destruction, and exception cleanup \[except.ctor\]](#)
21. [C++20 working draft N4861 — Filesystem library \[fs\]](#)
22. [C++20 working draft N4861 — Filter view \[range.filter\]](#)
23. [C++20 working draft N4861 — Numeric parsing with from_chars \[charconv.from.chars\]](#)
24. [C++20 working draft N4861 — File streams \[file.streams\]](#)
25. [C++20 working draft N4861 — Fundamental types \[basic.fundamental\]](#)
26. [GCC: C dialect and language-standard options](#)
27. [GCC: optimization options](#)
28. [GCC: options controlling the kind of output](#)
29. [C++20 working draft N4861 — Derived classes \[class.derived\]](#)
30. [C++20 working draft N4861 — General principles \[intro\]](#)
31. [C++20 working draft N4861 — Input streams \[istream\]](#)
32. [C++20 working draft N4861 — Iterator operations \[iterator.operations\]](#)
33. [C++20 working draft N4861 — Iterator requirements \[iterator.requirements\]](#)
34. [C++20 working draft N4861 — Lambda expressions \[expr.prim.lambda\]](#)
35. [C++20 working draft N4861 — Object lifetime \[basic.life\]](#)
36. [C++20 working draft N4861 — Numeric limits \[numeric.limits\]](#)
37. [C++20 working draft N4861 — List \[list\]](#)
38. [C++20 working draft N4861 — Initializing bases and members \[class.base.init\]](#)
39. [C++20 working draft N4861 — Member functions \[class.mfct\]](#)
40. [Microsoft C++: selecting the language-standard version](#)
41. [C++20 working draft N4861 — Nonmodifying sequence operations \[alg.nonmodifying\]](#)
42. [C++20 working draft N4861 — Generalized numeric operations \[numeric.ops\]](#)
43. [C++20 working draft N4861 — Overloaded operators \[over.oper\]](#)
44. [C++20 working draft N4861 — Optional objects \[optional\]](#)
45. [C++20 working draft N4861 — Output streams \[ostream\]](#)
46. [C++20 working draft N4861 — Overloading \[over\]](#)
47. [C++20 working draft N4861 — Partial specialization \[temp.class.spec\]](#)

48. [C++20 working draft N4861 — Random-number generation \[rand\]](#)
49. [C++20 working draft N4861 — Range adaptors \[range.adaptors\]](#)
50. [C++20 working draft N4861 — Ranges library \[ranges\]](#)
51. [C++20 working draft N4861 — References \[dcl.ref\]](#)
52. [C++20 working draft N4861 — Sequence-container requirements \[sequence.reqmts\]](#)
53. [C++20 working draft N4861 — Shared-ownership pointers \[util.smartptr.shared\]](#)
54. [C++20 working draft N4861 — Sorting and related operations \[alg.sorting\]](#)
55. [C++20 working draft N4861 — Three-way comparison \[expr.spaceship\]](#)
56. [C++20 working draft N4861 — Explicit specialization \[temp.expl.spec\]](#)
57. [C++20 working draft N4861 — Array container \[array\]](#)
58. [C++20 working draft N4861 — Templates \[temp\]](#)
59. [C++20 working draft N4861 — Type traits \[meta\]](#)
60. [C++20 working draft N4861 — Transform algorithm \[alg.transform\]](#)
61. [C++20 working draft N4861 — Tuples \[tuple\]](#)
62. [Clang UndefinedBehaviorSanitizer documentation](#)
63. [C++20 working draft N4861 — Unique-ownership pointers \[unique.ptr\]](#)
64. [C++20 working draft N4861 — Unordered associative requirements \[unord.req\]](#)
65. [C++20 working draft N4861 — Utility components \[utility\]](#)
66. [C++20 working draft N4861 — Variant \[variant\]](#)
67. [C++20 working draft N4861 — Vector \[vector\]](#)
68. [C++20 working draft N4861 — Vector capacity \[vector.capacity\]](#)
69. [C++20 working draft N4861 — Vector modifiers \[vector.modifiers\]](#)
70. [C++20 working draft N4861 — Virtual functions \[class.virtual\]](#)
71. [C++20 working draft N4861 — Weak pointers \[util.smartptr.weak\]](#)
72. [Additional primary references used in this expansion](#)
 1. [C++ Core Guidelines — resource, interface, and class design](#)
 2. [Current C++ working draft — filter view behavior](#)
 3. [Current C++ working draft — library object states](#)
 4. [CMake — requesting a target language standard](#)
 5. [GCC — diagnostic options](#)

About this expanded volume

Modern C++ Programming

Resource Safety, Generic Programming, the Standard Library, and Modern Language Design

C++ From Beginner to Expert · Book II

Dr. Charles Dorner · Expanded manuscript · 2026

A program can compile, print the expected answer, and still leave its next caller with a dangling reference. A resource wrapper can release memory correctly while a failed update publishes half of a new dataset. A view can make a transformation elegant while hiding that its input has already disappeared. Modern C++ is not a collection of newer syntax that makes these problems vanish. It is a way to make ownership, values, constraints, and state transitions explicit enough to reason about them.

This volume continues the language foundation established in Book I. Its goal is to help you design ordinary C++ code whose behavior remains understandable when values are copied, resources move, containers change, parsing fails, and external operations stop halfway through. It preserves the complete twenty-one-chapter curriculum, including every numbered section and subsection. The expansion adds sustained explanation, failure analysis, guided practice, executable supplemental labs, and a stronger application acceptance exercise.

The recurring organization, **HarborWorks**, and its task-reporting scenarios are fictional teaching material. They are not descriptions of an employer, customer, classified project, or production infrastructure. Names, records, limits, and failure cases are deliberately invented to make the mechanisms inspectable. The examples do not contain confidential operational data.

Audience and progression

Begin after you can write functions, use basic containers, understand references and pointers, define simple classes, and compile a multi-file application. Book I supplies that foundation. This volume revisits lifetime and class invariants when modern features make those ideas more demanding; it does not assume that a short introduction to pointers already resolves every ownership question.

Chapters 1–6 establish values, invariants, RAII, copying, moving, and smart ownership. Chapters 7–10 develop meaningful operators, inheritance, runtime polymorphism, and composition. Chapters 11–18 cover templates, concepts, lambdas, iterators, containers, algorithms, and ranges. Chapters 19–21 integrate richer result types, external resources, and a complete reporting application. The appendices provide complete supplemental programs and a reusable review method.

The book intentionally stops short of teaching concurrency, lock-free algorithms, operating-system networking, GPU programming, or hard-real-time guarantees. Those subjects belong to later volumes. A shared pointer is discussed as a lifetime mechanism, not as proof that shared data is thread-safe. A steady clock is discussed as a timing vocabulary, not as proof that a program meets a deadline. Keeping those boundaries visible is part of professional competence.

Technical scope and primary references

The executable baseline is C++20 with a conforming standard library supporting the features used. CMake requests that mode explicitly and disables compiler-specific language extensions for the examples. Most source is portable standard C++, but actual execution is verified only in the environments recorded in the accompanying `TEST_REPORT.md`. Compiler configuration for another platform is a portability provision, not a claim that the platform was tested.

The reference directory retains links to the C++20 working draft N4861 and relevant original project documentation. It also identifies living primary documents used for editorial cross-checks. A living draft can contain features or wording newer than C++20; it is not silently treated as the deployed language baseline. The examples avoid relying on later conveniences such as C++23 range-to-container helpers or optional monadic operations.

Standards describe required behavior; compiler and library implementations determine which facilities are available in your installed environment. Use the exact compiler, standard-library version, flags, and source when reproducing an issue. A different front end using the same standard library is useful additional evidence, but it is not an independent test of every library implementation. The verification report states this distinction.

The new prose, scenarios, experiments, and application extension are authored teaching material. They expand the supplied curriculum rather than quoting a standards document into a textbook. Technical reference notes support the language and library mechanisms. The measured length comparison in `REVISION_NOTES.md` uses the same EPUB-spine counting method as the Linux template and revised Book I; it is not an assertion that all word-count tools produce the same number.

How to use this book

Read for a decision, then trace the mechanism

Before studying a feature, identify the question it should answer. Does a caller need a value or a borrow? Does a failed operation preserve accepted state or only prevent a leak? Is a view a retained recipe or a completed snapshot? Is a search result a boundary or a match? These distinctions are more useful than memorizing a list of type names.

Each chapter begins with a concrete problem, develops the numbered concepts, and walks through complete source from the companion. The guided lab then changes one requirement or introduces one failure boundary. Read the explanations between listings. They identify owners, starting state, permitted inputs, and the difference between a local observation and a general guarantee.

Keep an operations notebook: requirement, mechanism, evidence, and remaining uncertainty. Draw ownership, mark commit points, state algorithm preconditions, and identify external effects that destruction cannot undo.

Build the supplied examples

Extract the companion ZIP and run the following commands from its top-level directory. A C++20 compiler, CMake, and Python 3 are required. Building the examples does not require a GPU, administrative privileges, an AI service, or a network download of third-party source.

```
cmake -S source -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug --parallel 2
ctest --test-dir build/debug --output-on-failure
```

Each chapter also includes a smaller build command for its own directory. Supplemental labs are included in the root build. For a multi-configuration generator, add `--config Debug` when building and `-C Debug` when running CTest. Build and publishing instructions are separate: the C++ companion does not need Pandoc or the PDF-rendering dependencies.

A failed negative-input application run can be the expected outcome of a passing fixture. Read the fixture's specified exit status and diagnostic before treating every nonzero process exit as a broken test. Conversely, a successful process exit is not enough when its output or retained state violates the contract. The runner checks the stated behavior, not just whether a process started.

Source, output, and exercise conventions

Every complete code listing is inserted from a real source file at assembly time. The editable chapter files contain include markers; the finished manuscript and books contain the expanded listings. This prevents a separately edited listing from quietly drifting away from the delivered program. Smaller inline fragments illustrate a local expression and are not necessarily standalone programs.

Commands assume the extracted package is the current directory. Do not run every block as one script. Filesystem exercises use private temporary directories; they do not alter system configuration. Copy a chapter before changing it so the reference remains available.

The common **CHECK** harness stays active in optimized builds. Its internal check count is diagnostic information, not a count of independent CTest cases and not a quality score. Some programs perform many checks inside one test process. Observed move/copy counts are labeled as observations where the standard permits implementation differences.

Use failure deliberately and safely

Supplemental experiments inject failures through controlled test types and stream buffers. They do not rely on exhausting the host's memory, corrupting the filesystem, or executing known undefined behavior. A distinct simulated-failure type prevents an expected-exception catch from swallowing an assertion failure. The tests verify the precise state boundary promised by the operation.

Passing tests and sanitizer runs are evidence for the exercised cases, not a proof of universal correctness. Windows and macOS execution, independent standard-library implementations, hostile-input security assessment, and official EPUBCheck conformance are not implied by the local checks. Consult the actual report rather than inferring broader validation from a successful build.

The central rule remains: **establish the contract, make ownership visible, preserve invariants, verify the boundary, and report only the guarantee the evidence supports.**

Chapter 1 — Understanding the Philosophy of Modern C++

The first task ledger worked without a single raw owning pointer. That is not a temporary beginner shortcut. It demonstrates a central modern C++ idea: use types that already express ownership and useful behavior, then inspect costs where requirements make them relevant.

This volume develops that approach into resource-safe classes, generic algorithms, and composable standard-library code. Modern C++ is not a competition to use the newest syntax. It is a discipline of making lifetime, effects, and contracts explicit while preserving efficient implementation choices.

HarborWorks is extending its small inventory tools into a reporting program. A report must retain the dataset it was built from, reject an invalid replacement without damaging the previous data, and make its ownership relationships understandable to the next maintainer. None of those requirements says “use a particular pointer.” The requirements describe observable behavior; the language facilities are implementation choices that must support it.

Keep three questions separate throughout this volume. What does an object mean? Who keeps its resources alive? Which operations are expensive enough to matter? A design can answer the first two correctly before measuring the third. Reversing that order often produces clever code whose result is unclear. This chapter establishes the vocabulary used in the remaining chapters, especially the distinction between a copied value, a borrowed object, and a transferred resource.

1.1 Understanding Zero-Cost Abstractions

An abstraction provides a useful model while hiding irrelevant implementation details. A vector exposes an owned sequence instead of making every caller manage allocation and element destruction. A generic algorithm exposes an operation over a range instead of repeating traversal mechanics at every call site.

The zero-overhead principle is a design goal, not a claim that every abstraction has zero cost under every compiler and workload. Understand which work is required by the chosen semantics and measure implementation overhead separately.

1.1.1 Comparing Abstraction with Runtime Overhead

A hand-written sum loop and `std::accumulate` can express the same operation over the same contiguous data. An optimizing compiler may produce equivalent machine code, but that result depends on the types, operations, optimization settings, and surrounding context.

Conversely, shared ownership has bookkeeping and virtual dispatch may introduce an indirect call. Those costs can be justified by the semantics they provide. Calling every abstraction free is as misleading as banning abstractions because some have costs. Begin by stating the behavior the program needs, then evaluate whether the chosen mechanism pays for something unnecessary.

Suppose a report sums the priorities of the open tasks. A counted loop exposes an index, a boundary, an accumulator, and the update operation. An algorithm call exposes the same traversal through a smaller interface. Neither spelling determines how many memory allocations the surrounding program performs. If one version first copies a million records and the other borrows them, a timing comparison primarily measures that semantic difference, not the cost of the loop abstraction.

Define a fair comparison before reaching for a stopwatch. Both implementations should receive the same elements, use the same accumulation type, apply the same overflow policy, and return the same result. They should either both include dataset preparation or both exclude it. If the required answer fits in `long` but the handwritten version accumulates in `int`, equal results on a tiny fixture do not establish equivalence over the intended domain.

There are several kinds of cost. A checked accessor may perform a conditional test. A container can allocate capacity. A type-erased callable can require indirect dispatch. A debug iterator can carry bookkeeping absent from a release configuration. These are different mechanisms and cannot be evaluated with the sentence “C++ is fast.” Identify the mechanism and the environment, then inspect or measure it. The worked example deliberately checks results rather than reporting an unearned speedup.

1.1.2 Understanding Compile-Time Optimization

`constexpr` allows suitable functions and objects to participate in constant evaluation. A `static_assert` can require a particular expression to be evaluated as a constant expression and reject an invalid property during translation. A `constexpr` function can also execute at runtime when given runtime inputs.

Do not confuse this language facility with the optimizer’s freedom to fold ordinary expressions. A compiler can optimize computations that are not declared `constexpr`, while a `constexpr` declaration does not force every call to disappear from runtime. The example uses `static_assert(square(7) == 49)` to state an actual compile-time requirement over a small representable input.

A compile-time requirement is useful when violating it should prevent the executable from existing. Consider a fixed packet layout whose field counts must add up to an expected total. An assertion over constants can reject a mistaken layout during the build. That is different from a runtime validation of a length read from a file. The former concerns the program’s construction; the latter concerns information not known until execution.

The declaration `constexpr int square(int x)` does not promise that every integer input can be squared safely. If signed multiplication overflows, the expression is not made valid by the keyword. A constant-expression context can reject some invalid computations, while a runtime call still needs the program’s usual domain rules. Our example chooses seven specifically because its square is representable. Later chapters add checked arithmetic rather than silently extending this small demonstration into a universal numeric utility.

Also separate translation time from execution time in your engineering budget. Larger template instantiations and constant computations can reduce repeated runtime work while increasing compiler memory use, build time, or binary size. For HarborWorks, a table that rarely changes may be a good candidate for precomputation. A table supplied by a customer is not suddenly a compile-time object

because precomputation sounds efficient. Choose the stage at which the required information actually becomes available.

Strong Integer Wrapper Optimized Like Its Underlying Value

Purpose. A zero-cost abstraction aims to improve type safety and expressiveness without imposing unavoidable runtime overhead compared with the equivalent manual representation.

```
#include <iostream>
struct UserId{ int value; };
bool operator==(UserId a, UserId b){ return a.value==b.value; }
int main(){ UserId a{7}, b{7}; std::cout << std::boolalpha << (a==b) << '\n'; }
```

How it works.

- `UserId` adds semantic meaning without virtual dispatch or heap allocation.
- The equality operator is a tiny value operation that the compiler can inline.

Expected behavior/output. The program prints `true`.

Experiment. Compile with optimization and inspect generated assembly for the equality operation.

1.2 Designing Around Value Semantics

Value semantics make an object behave as an independently meaningful value. Copying a point should ordinarily give another point whose later coordinates can change independently. This makes local reasoning and rollback easier because one value is not silently tied to another's mutable identity.

Not every type should be copyable. A unique file handle represents an ownership relationship rather than an independently duplicable value unless a real duplication operation is defined.

1.2.1 Treating Objects as Values

A `Point` containing two integers is naturally a value. A vector of such points owns those values, and copying the vector copies the points. Modifying the copy's first coordinate does not alter the original vector's corresponding point.

This behavior follows the member types' semantics. A struct containing a raw pointer copies the pointer value, not the pointed-to object. Therefore a class does not acquire deep value semantics merely because its syntax looks like a record. Inspect each member's ownership role before deciding what copying the enclosing object means.

Treat a report snapshot as a small contract. After making snapshot B from dataset A, a change to B's title must not alter A's title. Destroying A must not make B unusable. Saving B should not accidentally read a later revision of A. Those requirements make an owning value representation attractive: a vector of records whose titles are strings provides independent nested values without custom allocation code.

Independence does not require different values at the moment of copying. Immediately after the copy, A and B can compare equal. The distinction is what happens after mutation or destruction. A good copy test therefore has at least two phases: establish equality, then change one side and verify that the other preserves its previous value. Merely comparing their sizes immediately after construction is too weak to detect shared storage.

Value semantics also have limits. A copied record may contain an identifier for an external database row. Copying the identifier does not copy the external database. A copied filesystem path does not duplicate the file it names. Be explicit about the boundary of the value: the C++ object can be independent even while both values refer to the same external identity. This is why resource ownership and application identity need separate diagrams rather than one ambiguous arrow labelled “has.”

1.2.2 Distinguishing Value and Reference Semantics

Reference semantics emphasize access to an existing object’s identity. A reference to the first point observes that point rather than creating another point. Its validity depends on the owner’s lifetime and relevant invalidation rules.

Neither model is universally superior. Use values when independence is intended, and references when borrowing an existing object is intended. Mixing the two without naming the relationship causes defects: a supposed snapshot unexpectedly reflects later changes, or an observer dangles after its source reallocates. Interfaces should make that choice visible.

A borrow is often the correct choice for a function that only formats an existing report during the call. Passing a large report by const reference can avoid creating another owned dataset. The formatter does not decide when the report is destroyed and must not save an internal address for later use unless the interface establishes a longer lifetime contract.

The type alone does not express every part of that contract. A const reference prevents certain modifications through that access path, but another alias may still change the underlying object. It also does not stop the owner from being destroyed after the function returns. A retained callback that refers to a local report can therefore be wrong even though every reference is const and the program compiled cleanly.

Use an ownership timeline when a diagram becomes ambiguous. Mark the report’s creation, the start and finish of a borrowing call, any container mutation, and destruction. Every use of a borrowed element must occur while the element is alive and the particular reference remains valid. In a vector, the owner’s lifetime is necessary but not sufficient: reallocation can invalidate references while the vector itself continues to exist. Chapters 14–18 return to this distinction for closures, iterators, and views.

Independent Copies of a Value Object

Purpose. Value semantics make ordinary assignment and copying behave like independent mathematical values unless a type deliberately models shared identity.

```
#include <iostream>
struct Point{ int x; int y; };
int main(){ Point a{1,2}; Point b=a; b.x=9; std::cout << a.x << ' ' << b.x <<
'\n'; }
```

How it works.

- Copying `a` creates a distinct `Point` value in `b`.
- Mutating `b.x` does not change `a.x`.

Expected behavior/output. The output is `1 9`.

Experiment. Place two `Point` objects in a vector, copy the vector, and verify the copies remain independent.

1.3 Managing Ownership Explicitly

Ownership identifies who releases a resource and when that responsibility may change. Borrowing identifies who can access a resource without becoming responsible for its destruction. Separating those concepts prevents cleanup from becoming a convention scattered across unrelated functions.

The modern default is one obvious owner, with shared ownership introduced only when the lifetime genuinely has several independent controlling participants.

1.3.1 Identifying Resource Owners

A local vector owns its elements and their storage. A const reference to one element is a borrower. The borrower must not delete the element, and the owner must not invalidate it while the borrower is used. This relationship can be drawn as an owning edge and a non-owning observation edge.

For dynamically allocated polymorphic objects, `std::unique_ptr` often expresses a single owner. For genuinely shared lifetimes, `std::shared_ptr` expresses a different policy. These types are not interchangeable spellings for an address. Their copy, move, destruction, and cost models reflect different ownership requirements.

List resources rather than just pointers. The input stream has an open-file resource, the candidate dataset owns strings and vector storage, the output stream has an external destination, and an error object may own diagnostic text. Some resources are managed by local values and some are borrowed from the caller. A raw pointer used to choose between `std::cin` and a locally owned file can be a legitimate non-owning selector; it is not automatically a defect merely because a raw pointer appears.

For each resource, identify the release operation and the owner that invokes it. A vector destroys its elements and releases its storage. A stream object closes its associated file as part of its cleanup. An exclusive native-handle wrapper invokes the matching native release function. If the design cannot name the owner, it has not finished designing the failure paths.

Then ask whether ownership changes. Passing an object for inspection need not transfer anything. Returning a newly constructed value hands a result to the caller. Moving a unique pointer transfers one explicit responsibility. Copying a shared pointer adds a participant in shared lifetime management, not an independently allocated copy of the pointee. These distinctions make function signatures reviewable before reading the entire implementation.

1.3.2 Avoiding Ambiguous Ownership

Ambiguous ownership appears when a function accepts a raw pointer and callers cannot tell whether it will delete, retain, or merely inspect the object. Improve the interface or document a narrow borrowing contract rather than relying on naming alone.

A parameter accepting an ordinary reference usually communicates borrowing for the call. A value parameter can own its local value. A smart-pointer parameter should be used when the ownership relationship itself is relevant to the operation, not simply because the caller happens to store the object in a smart pointer. Borrow the object directly when lifetime participation is unnecessary.

Ambiguity often enters through “convenient” helper functions. Imagine `install_report(Report* p)`. Does the function inspect the report, store the pointer, delete the report on failure, or assume ownership only on success? Each policy can be implemented, but callers cannot safely guess which one was selected. A parameter representing an owned value or a unique owner can make a transfer visible. A reference plus an explicit no-retention contract can make synchronous borrowing visible.

Avoid solving that ambiguity by putting shared pointers into every signature. A formatting function that only needs a report’s values should not force stack-allocated callers to invent shared ownership. Conversely, a background task that must keep a report alive after the caller returns needs a real lifetime arrangement; accepting a reference and hoping the caller waits is not a substitute.

Review ownership on failure as carefully as on success. If a consuming operation takes a value parameter, the caller may already have moved its source before the function begins validation. Rejecting the operation does not automatically restore the caller’s old object. A design that promises preservation should validate before transferring, or stage a separate candidate. The transaction examples later in the book make this boundary explicit.

Exclusive Ownership Encoded with `std::unique_ptr`

Purpose. Ownership is explicit when the type communicates who is responsible for destroying a resource.

```
#include <iostream>
#include <memory>
struct Job{ int id; };
int main(){ auto owner=std::make_unique<Job>(Job{4}); std::cout << owner->id <<
'\n'; }
```

How it works.

- `make_unique` creates the object and immediately places it under an owner.
- The pointer type is move-only, preventing accidental ownership copies.
- Destruction of the owner releases the object automatically.

Expected behavior/output. The program prints `4`.

Experiment. Try copying `owner`; inspect the compile-time error, then move it into another `unique_ptr`.

Worked example — Compare values, borrows, and an algorithmic abstraction

Requirement and design

Copy a vector of points, modify the copy, and prove the original remains unchanged. Compare a hand-written sum with `std::accumulate`, and verify a small calculation at compile time. Finally borrow an existing point without copying it.

The example demonstrates semantic equivalence and independence, not measured performance. It contains no timing claim and does not prove that all compilers emit identical code.

Requirement and simplest approach

The simplest implementation copies a vector, edits its first element, and prints both values. That is useful for exploration, but printing alone does not enforce the requirement. A reviewer must notice an incorrect number, and an automated build cannot reliably distinguish a deliberate output change from an accidental one. The supplied example makes the expected independence an executable check.

Trace the test as two owners and one borrow. The original vector owns its points. The copied vector owns different point objects containing equal initial values. The final reference aliases an element of the original vector and does not acquire cleanup responsibility. No operation in the test changes the original vector's capacity while the reference is used. That last detail is part of why the demonstration is valid, not incidental decoration.

Do not add timing to this small correctness executable and then label the numbers a benchmark. The checks, process startup, and tiny input would dominate many measurements. Keep the semantic experiment small enough to understand by hand; build a separate controlled measurement only when there is an actual performance question.

Complete implementation

The complete, buildable chapter project is `source/book2/ch01/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <array>
#include <numeric>
#include <vector>

struct Point { int x{}; int y{}; };
constexpr int square(int value) { return value * value; }

int main() {
    static_assert(square(7) == 49);
    std::vector<Point> original{{1, 2}, {3, 4}};
    auto copy = original;
    copy.front().x = 99;
    CHECK(original.front().x == 1);
    CHECK(copy.front().x == 99);
    const std::array<int, 4> values{1, 2, 3, 4};
    int loop_sum{};
    for (int value : values) loop_sum += value;
    const int algorithm_sum = std::accumulate(values.begin(), values.end(), 0);
    CHECK(loop_sum == 10 && algorithm_sum == loop_sum);
    const Point& borrowed = original.front();
    CHECK(borrowed.y == 2);
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch01 -B build/b2-ch01 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch01 --parallel 2
ctest --test-dir build/b2-ch01 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch01`.

Evidence and interpretation

The original first point retains `x` equal to one after the copy's `x` becomes 99. Both summation forms produce ten. The static assertion establishes that the chosen constant expression equals 49, while the borrowed point still exposes its original `y` coordinate of two.

Notice that the test checks externally relevant behavior rather than counting every constructor invocation. Optimization and copy elision can change which intermediate operations are visible in carefully instrumented experiments. The intended ownership and value relationships should remain the primary contract.

The shared `CHECK` facility throws on failure and is not removed by defining `NDEBUG`. A successful program reports a check count. This matters when later chapters test optimized builds: correctness checks must not silently disappear merely because the standard `assert` macro would be disabled.

Failure clinic — A snapshot that was not a snapshot

An engineer changes `Point` so its `x` coordinate is represented by a pointer to an external integer. The compiler still generates copy operations, and the vector still owns its point objects. Yet the independence test now fails because both points designate the same external integer. The vector did not malfunction. The meaning of copying one element changed.

Repair the model at the level of the requirement. If a coordinate is a value, store the integer as a value. If the point intentionally observes a shared measurement, rename and document the observation relationship rather than pretending it is a snapshot. Replacing the pointer with `shared_ptr<int>` keeps the integer alive but still does not create independent integer values when the point is copied.

A useful regression test changes the external integer while all owners are still alive and observes both records. This demonstrates aliasing without ever dereferencing a dangling pointer. Deliberately running undefined behavior is not required to understand why an ownership design is wrong.

Practice and implementation lab

Replace each point's integer `x` member with a pointer to an external integer in a separate experimental type. Copy the type and explain why the external integer is now shared rather than independently copied. Keep the external owner alive during the experiment.

Inspect optimized assembly for the loop and algorithm versions using your installed compiler, recording version and options. Report what that build does without promoting one observation into a language guarantee. Then list the owners and borrowers in the original example.

Review answers and solution notes

Copying a pointer copies an address value, so both records can designate the same external integer. Copying a vector of ordinary points instead copies the integer members. The owner of the vector's storage is the vector; the point reference is a borrower.

A `constexpr` function is eligible for constant evaluation but can run at runtime. An algorithmic abstraction can optimize well without a universal zero-cost guarantee. Choose semantics first, then use measurements or generated-code inspection to answer a specific performance question.

Engineering takeaway. Use values for independence, explicit owners for lifetime, and measured evidence for abstraction costs.

Guided lab — Write a value-and-lifetime contract

Design a `ReportSnapshot` containing a report name and a sequence of integer readings. Define copying as independence, inspection as a temporary borrow, and replacement as acceptance of a new owned value. Start with `std::string` and `std::vector<int>` members and do not write a destructor.

Test a populated snapshot, an empty snapshot, a copy that outlives the source’s scope, and mutations made independently to each copy. Add a function that accepts a `const` reference and returns a count by value. Confirm that it can inspect both a local object and a temporary without retaining an address. Explain why returning a reference to a local count would violate a different lifetime rule.

For the cost investigation, write two equivalent total functions using a loop and `std::accumulate`. Fix the input domain so the chosen accumulator cannot overflow. Verify equality over empty, one-element, mixed-sign, and repeated-value cases before inspecting an optimized build. Record the compiler and exact command. A finding such as “these two functions contain the same arithmetic loop in this build” is appropriately narrow; “algorithms never cost anything” is not.

Review and solution reasoning

The strongest evidence for independence is a change to one value that does not affect the other, followed by use of the survivor after the source is gone. `const` qualification does not supply lifetime. Shared ownership does not supply deep copying. An optimization observation does not redefine a language contract. Keep those three distinctions in the written acceptance notes; they recur throughout the book.

Primary reference notes

Modern mechanisms: [constant expressions](#), [copy construction](#), [vectors](#), and [numeric algorithms](#).

Chapter 2 — Designing Robust C++ Classes

A class validates its constructor but exposes a setter that accepts anything. Another class writes five special member functions even though its only resource is a vector. Both designs add work while weakening the ability to reason locally about valid state.

Robust classes maintain one coherent invariant through every public operation and delegate ordinary resource management to members that already know how to perform it. This chapter applies that approach to a percentage and an owned reading collection.

A class is more than a group of fields. It is a promise about which combinations of those fields callers can encounter. HarborWorks represents completion as a percentage. The meaning is not “whatever integer happens to be stored”; it is an integer in the inclusive range zero through one hundred. A useful class prevents callers from accidentally turning that promise into a comment that no code enforces.

The harder case is an invariant involving several members. A report window might require `first <= last`, or a dataset might require a unique identifier for every row. Validating each field independently does not establish either relationship. Think in terms of accepted states and permitted transitions. The interface should expose the transitions the application needs while withholding arbitrary representation edits.

2.1 Establishing Strong Class Invariants

An invariant describes all states that a successfully constructed object may expose during ordinary use. For `Percentage`, the value is between zero and one hundred inclusive. Validation belongs at every state-changing entry point, not only the constructor.

A class can prevent invalid states more effectively when the representation is private and the public interface expresses meaningful transitions.

2.1.1 Preventing Invalid Object States

The constructor routes its input through a validation function before storing it. An invalid value throws and no usable percentage is returned. This avoids a two-phase “construct invalid, then initialize” protocol that every caller could accidentally misuse.

Some domains need an empty or unavailable state. Model that state explicitly, perhaps through an optional value, rather than selecting an out-of-range sentinel and hoping every operation remembers it. A representation with fewer hidden states often produces simpler tests and fewer exceptional branches.

Write the invariant before writing the constructor. For a percentage, the accepted set is small and explicit. Zero means a valid starting point; one hundred means complete; neither should be repurposed as an error sentinel. A negative input is not an incomplete percentage but an invalid candidate. That distinction

determines whether construction throws, returns a typed error through a factory, or applies a deliberately documented conversion policy.

The constructor and setter should use the same validation rule. Duplicated validation tends to drift: one path accepts one hundred while another accidentally excludes it. A private validation helper returning the accepted integer lets both operations share the boundary test. It also makes the setter's order obvious: validate first, then assign. A failed validation leaves the stored value unchanged because mutation has not begun.

Validation must cover the actual type. For floating-point percentages, comparisons alone need thought about non-finite values such as NaN. For an integer percentage, that issue does not exist, but representability can still matter at a parsing boundary. Do not parse a huge external number into a narrow type and assume the class can recover the intended value afterward. Parsing and domain validation are separate gates: first establish a representable integer, then establish a valid percentage.

The constructor's guarantee is about successful construction. If it rejects a candidate by throwing, no complete percentage object is returned. The calling code should not expect to inspect a partially valid instance. Later RAII examples explain what happens to already-constructed members when a larger object's construction fails.

2.1.2 Restricting Mutation Through Interfaces

The setter validates the candidate before assigning it. Rejection therefore leaves the old percentage unchanged. This is a local strong-guarantee pattern: perform the fallible check first, then commit a simple nonthrowing scalar assignment.

Do not expose a mutable reference to the private value, because that would bypass validation. Likewise, a bulk-update operation involving several fields must validate their relationship as a whole rather than calling independent setters that temporarily create inconsistent combinations. The invariant belongs to the object, not to a checklist of field names.

A getter returning `int&` would allow a caller to bypass percentage validation by assigning minus one directly. That one accessor would undo the purpose of the private field. Return the small integer by value, or offer a narrowly defined operation such as `set`, `increase_to`, or `mark_complete`. Encapsulation is measured by what callers can do, not by how many fields carry the `private` label.

Coupled state deserves a combined update operation. Suppose a date interval exposes `set_first` and `set_last`, each requiring a valid interval immediately afterward. Moving the interval from one disjoint range to another can become awkward or impossible through the intermediate states. A single `replace_interval(first, last)` operation can validate a candidate pair and commit it together. This is a small transaction, even though no database is involved.

Avoid returning a mutable container when the class promises unique IDs or sorted data. An unrestricted `std::vector<Record>&` lets callers erase, duplicate, or reorder elements without passing the invariant checks. A read-only query, a copied snapshot, an iterator range with a documented borrowing period, or a domain-specific mutation may be a better fit. Each has different costs and lifetime rules; choose deliberately rather than exposing the representation simply because it is convenient during early development.

Constructor-Enforced Positive Dimension Invariant

Purpose. Strong invariants simplify the rest of a class because every public operation can assume the object began and remains valid.

```
#include <stdexcept>
#include <iostream>
class Rectangle{ int w_,h_; public: Rectangle(int w,int h):w_(w),h_(h)
{ if(w<=0||h<=0) throw std::invalid_argument("dimensions"); } int area()
const{return w_*h_;} };
int main(){ Rectangle r{3,4}; std::cout << r.area() << '\n'; }
```

How it works.

- Construction validates all state needed by later member functions.
- Because invalid dimensions cannot survive construction, `area()` needs no repeated validity check.

Expected behavior/output. The program prints `12`.

Experiment. Try constructing width 0 and catch the exception.

2.2 Applying const Correctness

Const correctness lets interfaces distinguish observation from mutation. A const member function can be called on a const object and promises not to modify ordinary non-mutable state through that object access.

Use const as a semantic tool. It does not automatically provide thread safety, freeze every aliased resource, or replace a clear ownership model.

2.2.1 Designing Const Member Functions

`int value() const noexcept` is a small read-only query that returns an independent scalar. A const object can use it without giving the caller a path to mutate the representation. The `noexcept` specification is appropriate because this simple query performs no throwing operation.

A logically const operation may maintain an internal cache through `mutable`, but that design needs careful synchronization and observable-behavior reasoning. It is not necessary for the examples here. Start with genuinely simple const queries before introducing caching that makes physical mutation less visible.

A const member function treats the object through a const-qualified access path. It is useful for queries such as `size() const` or `value() const` because callers can use them on const objects and understand that ordinary direct mutation is not part of the interface. It is not a universal promise that the outside world stays unchanged. The function could write a log, inspect a clock, or modify an object reached through a pointer member.

This distinction is especially important with containers of owning pointers. A const container prevents replacing its pointer elements through that container interface, but it does not necessarily make each pointee const. For a deeply read-only public model, the pointee type or the exposed interface must express that requirement too. Constness follows types and access paths; it is not an invisible recursive freeze operation.

Use paired accessors carefully. A mutable `at` can be appropriate for a plain value sequence with no cross-element rules, while a const overload supports inspection. A percentage class does not need a mutable integer reference because any integer mutation must be validated. The same language facility is therefore appropriate for one abstraction and inappropriate for another.

Finally, const is not a synchronization primitive. Two threads concurrently calling a const method are not automatically safe if the method updates shared mutable state. This volume uses single-threaded examples unless explicitly stated otherwise; the later systems volume develops synchronization as its own contract.

2.2.2 Passing Data Without Unnecessary Mutation

The reading collection's `at` query returns an integer value from checked access, while `replace` is a separate mutating operation. Calling a mutator on a const collection is rejected by the type system. This makes the code's intent clearer than a single accessor that returns a mutable reference in every context.

Passing read-only data by const reference can avoid copying, but returning a small scalar by value is often simpler than exposing a long-lived borrow. Decide where the lifetime boundary should be. A fast-looking reference return is a poor interface when callers can retain it after the owner changes or disappears.

Parameter design should reflect use. Small arithmetic values are usually straightforward by-value parameters. A large object inspected only during the call can often be borrowed by const reference. A value that the callee will store can be accepted by value and moved into a member, allowing callers to choose copying or transfer at the call site. These are defaults to evaluate, not laws that eliminate measurement or domain reasoning.

Consider `Readings(std::vector<int> values)`. When the caller supplies an lvalue, a copy constructs the parameter and the constructor can then move its storage into the member. When the caller supplies a suitable rvalue, ownership can be transferred into the parameter and then into the member. The extra move is usually a small operation for the default vector representation, but the exact costs depend on the participating type and allocator. Chapter 5 examines the distinction in detail.

A const reference parameter can bind to a temporary for the duration of a call, but storing that reference as a member does not grant the temporary a new lifetime. A constructor that retains a borrowed vector must require an owner that outlives the constructed object, and its name or documentation should make that requirement visible. For an ordinary readings value, owning the vector is easier to reason about than making every caller manage an invisible lifetime dependency.

Const and Non-Const Member Access

Purpose. Const correctness makes mutability part of the interface and allows read-only access to be enforced by the compiler.

```
#include <iostream>
class Counter{ int n_{}; public: void increment(){++n_;} int value()
const{return n_;} };
int main(){ Counter c; c.increment(); const Counter& view=c; std::cout <<
view.value() << '\n'; }
```

How it works.

- `increment` is non-const because it changes state.
- `value()` is const and therefore callable through a const reference.
- The const reference prevents mutation through `view`.

Expected behavior/output. The program prints `1`.

Experiment. Attempt `view.increment()` and inspect the diagnostic.

2.3 Applying the Rule of Zero

The Rule of Zero is a design guideline: compose members whose ownership semantics already match the class, so the compiler-generated special members are appropriate and no custom destructor, copy, or move logic is needed.

It is not a language rule and not a prohibition against resource wrappers. A low-level wrapper may need custom special members; application classes should usually build on those wrappers instead of repeating them.

2.3.1 Delegating Resource Management to Library Types

Readings owns a vector of integers. The vector already knows how to release storage, copy elements, and transfer ownership through moves. The enclosing class can therefore focus on its public behavior instead of manually allocating and freeing an array.

The constructor accepts a vector by value and moves that local value into the member. This gives a straightforward ownership boundary: callers can provide a temporary or an existing collection, and the class owns its resulting member. The exact copy cost depends on the argument and is a deliberate consequence of the interface, not hidden raw-pointer borrowing.

The Rule of Zero works because member types already provide appropriate resource operations. A string owns its character sequence; a vector owns its elements and storage. Their enclosing class can rely on member destruction and memberwise copying when those actions match the class's intended semantics. Removing an unnecessary destructor is not merely a style preference: it avoids accidentally suppressing an implicitly generated move operation.

Memberwise behavior must still be reviewed. A class storing both a vector and a pointer into that vector has a relationship that memberwise copying does not automatically repair. The copied pointer may still designate the original vector. Likewise, a class storing an owned sequence and a cached count can have a broken moved-from relationship if the sequence becomes empty but the cached scalar does not change. Prefer representations that do not duplicate derivable facts when that makes the invariant simpler.

Rule-of-Zero design is therefore a representation choice, not a command to ignore special members forever. If the class genuinely has a native exclusive handle, put that handle inside an appropriate RAI member. If copying the wrapper would not make sense, let that member make copying unavailable. The public class then follows its real ownership policy without maintaining a second, parallel cleanup implementation.

2.3.2 Eliminating Manual Special-Member Functions

A destructor added merely to call a member's destructor is wrong; members are already destroyed automatically. Hand-written copies that repeat vector copying add maintenance burden without improving semantics. They can also accidentally suppress implicit move generation or omit a newly added member.

Review the class whenever a new member changes its ownership role. A raw pointer observer may be fine if its lifetime contract is explicit, but a raw owning handle changes the special-member problem. The correct response may be to wrap that handle in a dedicated RAI type, restoring Rule-of-Zero composition at the application level.

Before deleting custom special members, record the behavior the class currently promises. Does copying create independent values? Is copying prohibited? Is a moved-from object allowed to be empty? Is assignment required to preserve the old state if allocation fails? Different answers can require different representations even when both versions use standard-library members.

A hand-written copy loop may conceal a defect that a vector member removes. Conversely, a generated copy operation may copy a reference member so that two objects borrow the same external service. That may be intentional for a reporter but wrong for an independently owned dataset. "Compiler-generated" means the language supplies memberwise operations under its rules; it does not mean the domain design has been approved.

Use compile-time checks to document the selected capabilities where useful. Type traits can show whether a type is copy-constructible or nothrow-move-constructible. Runtime checks should then verify the observable consequences: independent edits, preserved invariants after rejection, and correct empty-state behavior. Neither kind of test replaces the other. A trait can be true for a type whose copy operation implements the wrong application semantics.

Resource-Containing Class Requiring No Custom Special Members

Purpose. The Rule of Zero says a class built from well-behaved value members often should define no destructor, copy constructor, move constructor, or assignment operators.

```
#include <iostream>
#include <string>
#include <vector>
struct Report{ std::string title; std::vector<int> values; };
int main(){ Report a{"run",{1,2,3}}; Report b=a; b.values[0]=9; std::cout <<
a.values[0] << ' ' << b.values[0] << '\n'; }
```

How it works.

- `std::string` and `std::vector` already manage their own resources.
- The compiler-generated copy/move/destructor operations compose the members' correct behavior.

Expected behavior/output. The output is `1 9`.

Experiment. Add another standard-library value member and verify no custom special member is needed.

Worked example — Enforce a percentage and copy an owned reading collection

Requirement and design

Create a percentage at forty, reject 101 without changing it, accept both endpoints, and reject construction from minus one. Copy a reading collection, modify only the copy, and verify that the original remains independent.

The example intentionally gives the resource-owning vector the cleanup responsibility, leaving the application class without manual special members.

Establish the accepted-state boundary

Start the percentage at forty and attempt to replace it with 101. The crucial assertion is the one after rejection: the object must still report forty. Testing only that an exception occurred would miss a setter that assigned 101 before throwing. The endpoint checks then clarify that the range is inclusive. Test names should state those distinctions rather than collecting them under an opaque “works” label.

For readings, perform a copy and then change one element in the copy. The vector's value semantics should keep the original element unchanged. This is a separate requirement from percentage validity, and the example intentionally checks both. Together they demonstrate that class correctness has several dimensions: local domain invariants, ownership, and copy behavior.

Complete implementation

The complete, buildable chapter project is `source/book2/ch02/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <stdexcept>
#include <vector>
#include <utility>

class Percentage {
    int value_;
    static int validate(int value) {
        if (value < 0 || value > 100) throw std::out_of_range("percentage");
        return value;
    }
public:
    explicit Percentage(int value) : value_(validate(value)) {}
    int value() const noexcept { return value_; }
    void set(int value) { value_ = validate(value); }
};

class Readings {
    std::vector<int> values_;
public:
    explicit Readings(std::vector<int> values) : values_(std::move(values)) {}
    int at(std::size_t index) const { return values_.at(index); }
    void replace(std::size_t index, int value) { values_.at(index) = value; }
};

int main() {
    Percentage p{40};
    CHECK(course::throws<std::out_of_range>([&] { p.set(101); }));
    CHECK(p.value() == 40);
    p.set(0); CHECK(p.value() == 0);
    p.set(100); CHECK(p.value() == 100);
    CHECK(course::throws<std::out_of_range>([] { Percentage invalid{-1}; }));
    const Readings first{1, 2, 3};
    auto second = first;
    second.replace(0, 9);
    CHECK(first.at(0) == 1 && second.at(0) == 9);
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch02 -B build/b2-ch02 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch02 --parallel 2
ctest --test-dir build/b2-ch02 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch02`.

Evidence and interpretation

The failure-path check immediately after `p.set(101)` proves that the previous value forty survives rejection. Endpoint tests establish the inclusive nature of the domain. The reading-copy test then distinguishes independent vector values from shared mutable storage.

The setter's validation function returns the accepted integer, so assignment is reached only after validation succeeds. The constructor uses the same function, preventing constructor and setter policies from drifting apart. This is small-scale reuse with a precise purpose, not a generic validation framework.

The vector constructor uses `std::move` to transfer from its local parameter, which the class no longer needs afterward. Chapter 5 explains why the named parameter is otherwise an lvalue expression and why the cast requests a move rather than moving anything by itself.

Failure clinic — Validation after mutation

The tempting setter writes its candidate into the field and then calls a validation function. Its happy path looks identical to the correct setter. Its failure path leaves an invalid object behind. A later unrelated query may expose the defect, making the original cause difficult to locate.

Reverse the order and make the transition visible: compute an accepted candidate, then commit it. For a single integer, the final assignment is simple. For a collection, a candidate object and a suitable nonthrowing swap can provide a stronger boundary. Do not generalize the integer example into “all assignment is nonthrowing”; the member operations of the actual type determine that property.

A second subtle failure is an accessor that leaks mutable representation. A carefully validated setter cannot protect a class if another method returns an unrestricted reference to the stored integer. Review the entire public interface, including conversion operators and inherited operations, rather than checking the setter in isolation.

Practice and implementation lab

Add tests for setting minus one, repeated rejection, and copying after mutation. Add a read-only size query to **Readings**, then test checked access at its boundary and beyond it.

Review a class with a hand-written destructor and vector member. Remove unnecessary special members in a controlled refactor and verify its intended copy behavior. State which change would require reconsideration if the class later acquired a native file handle.

Review answers and solution notes

A rejected percentage update should preserve the old value after every failure, not only the first. Checked vector access reports an invalid index; the class should not expose unchecked mutation accidentally through a different query.

A vector-owning class normally needs no custom destructor or copy implementation. A native owning handle requires a cleanup policy, usually isolated in an RAII member. Const qualification narrows permitted operations but does not make shared external state immutable or synchronized.

Engineering takeaway. Preserve one invariant through every entry point, and let well-designed members manage their own resources.

Guided lab — Replace a validated interval atomically

Build an interval value with two integer endpoints. Successful construction and replacement require the first endpoint to be no greater than the last. Expose read-only endpoint queries and one replacement operation taking both candidates. Avoid separate unrestricted reference accessors.

Use accepted intervals entirely below zero, crossing zero, and reduced to a single point. Attempt an inverted interval after a successful construction and compare both endpoints before and after rejection. Then replace the interval with a disjoint valid interval and verify that no caller needs to create an invalid intermediate state.

Repeat the design with a name and a vector of readings. Decide whether an empty name is valid, whether an empty reading set is valid, and whether callers may mutate individual readings. Those are domain choices; document them before selecting getters. Add a copy-independence test and a const-use compilation check.

Review and solution reasoning

A valid public state is stronger than a private field. The interval's invariant belongs to the pair of endpoints, so its update boundary should cover the pair. The percentage's setter preserves the old value because validation precedes mutation, not because an exception magically rolls changes back. A Rule-of-Zero representation reduces ownership code only when the members' copy and move behavior matches the enclosing class's meaning.

Primary reference notes

Class and ownership rules: [member functions](#), [copy construction](#), [copy assignment](#), and [vectors](#).

Chapter 3 — Managing Resources with RAII

A file is opened, data is written, and a later check throws. A manual close at the bottom of the function is never reached. Adding a close before each current return works until the next maintenance change adds another path.

RAII connects release to an object’s lifetime rather than to a programmer’s memory of every exit. The same idea applies to file handles, locks, transactions, and other resources, but each resource’s failure semantics still need to be understood.

A file operation has at least three outcomes to distinguish: acquiring the handle, completing the requested work, and releasing the handle. Successfully opening a file proves only the first. Writing can fail later, and a buffered output error can become visible during an explicit completion step. RAII makes cleanup reliable along ordinary control-flow paths; it does not make every operation on the resource succeed.

This distinction is central to production-oriented C++. HarborWorks must not leak an input handle when parsing throws, but it also must not announce “report saved” merely because an output object reached its destructor. A resource wrapper should make ownership automatic while leaving meaningful operation failures observable at an appropriate boundary.

3.1 Understanding Resources Beyond Memory

A resource is something whose acquisition creates a corresponding obligation: release a file, unlock a mutex, return a buffer, or close a native handle. Memory is only one example. Ownership must identify which object performs that release and whether responsibility can be copied or transferred.

RAII is a lifetime technique, not an assurance that every possible external failure becomes recoverable.

3.1.1 Managing Files, Locks, and Handles

A file wrapper owns an open handle and releases it once. A lock guard owns the obligation to unlock a successfully acquired mutex. A native handle wrapper must use the matching platform release operation, not whichever cleanup function has a similar name.

The owned thing may remain represented by a raw pointer or integer internally, because that is the external API’s representation. The wrapper’s public semantics should prevent accidental duplication of the release obligation. A borrower can inspect or use the handle under a bounded contract without becoming another owner.

Think of a resource as something with a finite lifetime and a release obligation. Memory is one example. A file, mutex lock, transaction reservation, device context, or operating-system handle can have the same shape: acquire, use, release. The details of the release function differ, but the design question remains who is responsible for invoking it exactly once for each successful acquisition.

A resource may be scarce even when its C++ representation is small. An integer descriptor can refer to an open file in the operating system. Copying that integer does not necessarily create an independently releasable resource. Deleting an integer object does not close the descriptor. This is why ownership cannot be inferred from the number of bytes in a handle or from whether its representation is a pointer.

Some acquisitions are logically paired with a different completion operation. Holding a lock requires unlocking; beginning a transaction may require commit or rollback; opening an output stream may require checking a flush or close result. Define the fallback behavior first. If ordinary work throws before completion, what should destruction do? A rollback-like fallback is often appropriate for an uncommitted transaction, while a file wrapper normally attempts release without claiming that data is durable.

Do not build a single universal wrapper before understanding two concrete resources. Their policies may differ on invalid values, release failure, duplication, thread affinity, or whether release can block. Encapsulate the native mechanism locally, then expose an interface that states its actual guarantees.

3.1.2 Associating Resources with Object Lifetime

An automatic wrapper is destroyed when its scope exits normally and during applicable exception unwinding. That makes cleanup compose across helper functions: each local owner is responsible for its own resource, rather than an outer catch block guessing which partial operations succeeded.

Lifetime-based cleanup does not run after every possible process termination mechanism or hardware failure. It also cannot prove that buffered data reached durable storage. Separate resource release, successful output completion, and durability requirements. A file destructor closing a stream is not a substitute for a reviewed persistence protocol.

Object lifetime supplies a structured cleanup boundary. A fully constructed automatic object is destroyed when its scope exits normally and during ordinary exception unwinding. Its members are destroyed according to the language's lifetime rules. By placing the release obligation inside that object, the cleanup follows all those exits without a separate cleanup label at each return statement.

The phrase “fully constructed” matters. If a constructor throws before finishing, the destructor of the incomplete enclosing object is not called. Members that completed construction are still cleaned up. Therefore an owning resource should be represented by a member whose own construction establishes ownership, rather than by an unprotected raw handle stored only after several other potentially failing actions. Chapter 18 of Book I introduced construction order; here that order becomes a failure-management tool.

RAII is not a guarantee against every termination mode. Abrupt process termination, a fatal signal, a power loss, or deliberate termination mechanisms can bypass ordinary stack unwinding. A server requiring recovery after a crash needs persistent formats and external recovery procedures in addition to C++ cleanup. Keep claims local: the example demonstrates cleanup during a controlled exception, not crash consistency or storage durability.

Also include borrowers in the timeline. Calling `get ( )` exposes a handle without transferring its release obligation. After explicit close, that borrowed handle is no longer usable even though the wrapper object still exists. “The owner is still in scope” is insufficient when the owner has already released the underlying resource.

File Stream Lifetime as a Resource

Purpose. RAII applies to files, sockets, locks, and many other resources whose cleanup must happen reliably.

```
#include <fstream>
#include <iostream>
void write(){ std::ofstream out("resource.txt"); out << "data\n"; }
int main(){ write(); std::ifstream in("resource.txt"); std::string s; in>>s;
std::cout<<s<<'\\n'; }
```

How it works.

- The resource is an operating-system file handle, not heap memory.
- `ofstream` acquires and releases that handle through object lifetime.

Expected behavior/output. The program prints `data`.

Experiment. Return early from `write` after opening the file and reason about why cleanup still occurs.

3.2 Building RAI Resource Wrappers

A resource wrapper has two central responsibilities: establish a valid owned state after successful construction, and release that state safely when it ends. Its copy and move rules must agree with the ownership model.

The chapter's temporary-file wrapper deliberately deletes copying and exposes an explicit checked close operation in addition to destructor cleanup.

3.2.1 Acquiring Resources During Construction

The constructor calls `std::tmpfile` and throws if no file is obtained. After successful construction, the wrapper owns one handle. If later work throws, its destructor can close that handle without the caller duplicating cleanup logic.

For wrappers that acquire several resources, acquire them through already-owning members or local guards. If the second acquisition fails, the first resource must not depend on the destructor of an object whose construction never completed. This is where composition provides real exception safety rather than merely a destructor-shaped promise.

Construction should either establish the wrapper's stated usable state or report failure without leaving a leaked acquisition behind. The temporary-file wrapper calls its acquisition function and checks the result. Copying is prohibited because a shallow copy would give two wrappers the same apparent release obligation. Move support is postponed until the next ownership chapters so the essential boundary stays visible.

Some APIs naturally use an invalid sentinel such as a null pointer or minus one. Store that sentinel whenever no resource is owned. A wrapper that can be explicitly closed needs a well-defined empty state and methods that respect it. Avoid leaving a released numeric handle in the member as if it were still live; a later release could affect a reused external resource in APIs that recycle handle numbers.

For more complicated acquisition sequences, build each stage as an independently owned local or member. If opening a second resource fails, the first owner should clean up automatically. This is simpler than maintaining a Boolean flag for each completed step and trying to update all flags correctly across every exception path.

A factory returning an optional or variant result can be suitable when acquisition failure is an expected branch rather than an exception-based interface. The lifetime principle stays the same. The returned success alternative must own the resource, and the error alternative must not conceal a live handle with no owner. Chapters 19 and 21 develop typed results after the ownership foundations are established.

3.2.2 Releasing Resources During Destruction

The destructor attempts cleanup without throwing. Since `close()` can itself fail, the explicit `close()` method returns whether the operation succeeded and first removes the stored ownership claim so a later destructor will not close the same handle twice.

A destructor cannot conveniently report a late write failure to a caller that has already committed success. Therefore code with a meaningful output-completion requirement should explicitly finish or close and check before reporting success. The destructor remains the fallback for failure paths. After explicit close, the raw borrowed handle returned by `get()` is no longer usable; the interface is intentionally narrow and its precondition must be respected.

A destructor is a poor place for a recoverable error conversation. The caller may already be handling another exception, and the object is ending its lifetime. Throwing another exception from cleanup can terminate execution during unwinding. A practical wrapper therefore separates explicit checked completion from fallback cleanup. The normal path calls `close()` and examines the result; destruction releases any remaining handle without pretending to report every possible failure.

Clear the stored ownership state before invoking a native release operation when that is the wrapper's chosen one-shot policy. The example moves the old `FILE*` into a local variable, sets the member to null, and then calls `fclose` once. A second `close()` does nothing. This does not imply that every operating-system API has identical retry semantics. A descriptor or device wrapper must follow its own API's documented release behavior rather than copying a file-stream pattern blindly.

Destructors should also avoid unrelated work that can fail. Logging into a growing vector may allocate; formatting a diagnostic string may throw. The chapter's live-count observer modifies an integer in an enclosing scope, so its instrumentation does not require allocation. Later lifecycle examples reserve fixed observation capacity before recording destructor events. Instrumentation is part of the program and can introduce exactly the failure being investigated.

The result is a narrow but useful guarantee: along the tested ordinary paths, one successful acquisition leads to one release attempt, and exceptions do not bypass the owner's cleanup. Claims about successful writes, durable storage, or external consistency require separate tests and policies.

RAII Wrapper for a C `FILE`

Purpose. The wrapper converts a manual acquire/release protocol into deterministic lifetime semantics.

```
#include <cstdio>
#include <stdexcept>
class File{ std::FILE* f_; public: explicit File(const char* p)
{ f_ =std::fopen(p,"w"); if(!f_) throw std::runtime_error("open"); } ~File()
{ if(f_) std::fclose(f_); } File(const File&)=delete; File& operator=(const
File&)=delete; std::FILE* get() const{return f_;} };
int main(){ File f{"raii.txt"}; std::fputs("ok\n",f.get()); }
```

How it works.

- The constructor acquires the C resource or fails immediately.
- The destructor conditionally closes the handle.
- Copying is disabled because two owners closing the same `FILE` would be incorrect.

Expected behavior/output. The program creates `raii.txt` containing `ok`.

Experiment. Add move construction and transfer ownership between two wrapper objects.

Worked example — Own a temporary file and verify cleanup during an exception

Requirement and design

Write known text to a temporary file, seek back, read it, compare the bytes, and explicitly check close. Separately create a scope observer, throw a controlled exception, and verify that its destructor restores the live count.

The file wrapper is noncopyable. It does not implement move semantics in this chapter, and its raw `get()` result is a borrowed handle valid only while the file remains open.

Trace acquisition, use, completion, and fallback

Draw four states for the temporary-file wrapper: not yet constructed, open, explicitly closed, and destroyed. Successful construction moves from the first state to open. `get()` borrows the open handle but does not change ownership. Explicit close transitions to closed even if its completion result reports a failure. Destruction of an open owner attempts release; destruction of a closed owner has no handle left to release.

The worked test writes a known byte sequence, seeks to the beginning, reads back bytes, and checks their content before closing explicitly. These are independent checks. A correct write count does not prove the

seek worked, and a matching prefix does not prove the expected number of bytes was read. Keep the evidence attached to the operation it validates.

The exception test uses a distinct controlled exception type so a failed **CHECK** cannot be accidentally swallowed by the catch intended for the simulated failure. This small test-harness detail is important: an exception test that catches every `runtime_error` can hide a failed assertion inside its own setup. Failure injection should be distinguishable from a genuine test failure.

Complete implementation

The complete, buildable chapter project is `source/book2/ch03/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <cstdio>
#include <stdexcept>
#include <string>

class TemporaryFile {
    std::FILE* file_;
public:
    TemporaryFile() : file_(std::tmpfile()) {
        if (!file_) throw std::runtime_error("tmpfile failed");
    }
    ~TemporaryFile() { if (file_) std::fclose(file_); }
    TemporaryFile(const TemporaryFile&) = delete;
    TemporaryFile& operator=(const TemporaryFile&) = delete;
    std::FILE* get() const noexcept { return file_; }
    bool close() noexcept {
        std::FILE* old = file_;
        file_ = nullptr;
        return !old || std::fclose(old) == 0;
    }
};

class ActiveScope {
    int& live_;
public:
    explicit ActiveScope(int& live) : live_(live) { ++live_; }
    ~ActiveScope() { --live_; }
    ActiveScope(const ActiveScope&) = delete;
    ActiveScope& operator=(const ActiveScope&) = delete;
};

struct SimulatedFailure final {};

int main() {
    TemporaryFile file;
    const std::string text{"resource owned\n"};
    CHECK(std::fwrite(text.data(), 1, text.size(), file.get()) == text.size());
    CHECK(std::fseek(file.get(), 0, SEEK_SET) == 0);
    char buffer[64]{};
    const auto bytes = std::fread(buffer, 1, sizeof buffer, file.get());
    CHECK(std::string(buffer, bytes) == text);
    CHECK(file.close());
    int live{};
    try {
        ActiveScope scope{live};
```

```

        CHECK(live == 1);
        throw SimulatedFailure{};
    } catch (const SimulatedFailure&) {}
    CHECK(live == 0);
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book2/ch03 -B build/b2-ch03 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch03 --parallel 2
ctest --test-dir build/b2-ch03 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch03`.

Evidence and interpretation

The write count must equal the text length, seeking must succeed, and the bytes read back must match the original text. These checks distinguish successful handle acquisition from successful use. The explicit close result is checked before the example reports success.

During the controlled exception, `ActiveScope` decrements the referenced live count as its scope unwinds. The outer check sees zero. The live integer belongs to the enclosing function and therefore outlives the observer. The observer’s copy operations are deleted so cleanup accounting cannot be duplicated accidentally.

The file destructor ignores close errors because throwing during cleanup could be unsafe, particularly while another exception is active. That policy is not concealed: the checked explicit close is the normal successful path, while destructor cleanup is a best-effort release path. Production wrappers should document comparable distinctions for their actual native APIs.

Failure clinic — The destructor that never ran

A novice places a raw handle in an enclosing object and expects that object’s destructor to clean it up if later constructor work throws. The expectation is wrong when the enclosing constructor never completed. The repair is not another broad catch in every caller. Put the handle into a completed RAII member or a local owner before beginning the later work, so a completed subobject has the release obligation.

A second failure arises when a wrapper is copied. Both copies contain the same handle, and both destructors attempt release. Suppressing one destructor call manually merely moves the uncertainty elsewhere. Delete copy operations when exclusive ownership is intended, and later add a move operation that empties the source deliberately. A raw address copy and an ownership transfer are different operations.

A third failure is reporting successful output after destructor cleanup without checking stream state. An application can release the file and still have failed to write all intended bytes. The resource has been cleaned up, but the business operation failed. The book's capstone distinguishes accepted in-memory state from external publication for precisely this reason.

Practice and implementation lab

Add a repeated-close test and verify that the second call is harmless under this wrapper's policy. Add a test that throws after opening the file but before explicit close, using an external observation or instrumentation appropriate to the platform.

Design, without calling a real device API, a wrapper for a fake integer handle whose release function records each released handle. Test successful acquisition, failed acquisition, scope exit, and forbidden copying. Decide whether an explicit checked completion operation is needed in addition to destruction.

Review answers and solution notes

After the first close clears the stored handle, a second close performs no additional release and reports success. A wrapper must never call a release function on a handle it failed to acquire. Copying is deleted when it would duplicate exclusive ownership.

RAII makes ordinary unwinding release dependable, but it does not guarantee data durability or report every close failure automatically. Checked completion and fallback cleanup are different operations. A borrowed handle becomes invalid after its owner closes or destroys the resource.

Engineering takeaway. Use destructors for dependable release, and explicit checked operations for failures that must be reported before success is committed.

Guided lab — A fake resource with observable release

Use the supplied resource-audit laboratory in Appendix B to study cleanup without depending on a real device. The fake resource has a small handle number, a live-handle registry, and a release counter. Its owner is noncopyable and movable. The registry outlives all owners, so observations do not dangle. There is no global handle allocator or external service to make the test nondeterministic.

Test successful acquisition and scope exit first. Then throw a distinct controlled exception after acquisition and verify that the registry is empty after the catch. Attempt failed acquisition and verify that no release is recorded for a resource that never existed. Move an owner and confirm that the original no longer releases the transferred handle. Close explicitly twice and verify that the second call changes no counters.

Do not count an exact number of constructor calls in a return-by-value expression as the resource contract. Copy elision can change intermediate object operations. Count successful acquisitions, currently live handles, and release attempts instead. Those events represent the policy the wrapper is supposed to enforce.

Review and solution reasoning

RAII addresses release responsibility. It does not guarantee that the resource was acquired, that every operation succeeded, or that external data is durable. A completed member can clean up when an enclosing constructor fails; the incomplete enclosing object's destructor is not the fallback. An explicit completion method can surface errors while ordinary destruction supplies a nonthrowing cleanup path. These distinctions let a wrapper remain both convenient and honest about failure.

Primary reference notes

Resource lifetime: [destructors](#), [exception unwinding](#), and [C file operations](#).

Chapter 4 — Understanding Copy Semantics

A buffer class owns an array but copies only its raw address. Two objects now believe they own one allocation. A mutation appears in both, and destruction eventually tries to release the same storage twice. The problem is not that copying is bad; the class never defined what a copy should mean.

This chapter implements independent buffer values with explicit deep-copy semantics and copy-and-swap assignment. It also explains when the better answer is to use a vector and avoid writing these operations at all.

A report editor needs a working copy of the currently accepted dataset. The editor may change its copy, discard it, or propose it as a replacement. Those operations are only predictable if “copy” has a defined meaning. Copying an address, copying a sequence of values, and duplicating an operating-system resource are three different operations, even when all three can be triggered by similar-looking C++ syntax.

This chapter deliberately implements a small owned integer buffer. It is not an argument for replacing `std::vector` in ordinary application code. The buffer makes allocation, element copying, failure, and commit visible so that the Rule of Zero has a mechanism behind it rather than functioning as an unexplained slogan.

4.1 Understanding Object Copying

Copy construction creates a new object from an existing object. Copy assignment replaces the value of an object that already exists. Their cleanup responsibilities differ because assignment begins with a live destination that may already own a resource.

Specify the intended relationship between source and destination before choosing an implementation. Independent values, shared ownership, and forbidden copying are different valid designs.

4.1.1 Using Copy Constructors

A copy constructor commonly accepts `const T&` because copying should observe the source without changing it. In the buffer, construction first allocates storage with the source’s size and then copies each integer. If allocation fails, no successfully constructed buffer is returned.

The source retains its allocation and values. The new buffer owns a different allocation with equal initial contents. This gives the caller an ordinary value model: modifying the copy later does not alter the source. For nontrivial element types, element-copy exceptions and partial construction need additional care; the teaching buffer deliberately stores integers.

A copy constructor initializes a new object from an existing object. At its entry, the destination’s lifetime is being established; there is no old destination value to preserve. The constructor must create a usable

new object or fail without leaking any completed resource acquisition. In the buffer, delegated construction allocates storage first, and the copy loop then copies the integer values.

A useful copy-construction test has more than one assertion. Compare size and element values immediately after copying. Mutate the new object and verify the source is unchanged. Then destroy the source while retaining the copy and inspect the surviving values. Each step answers a different question: initial equality, independence, and independent lifetime.

Memberwise copying is not inherently shallow or deep in a useful domain sense. It copies each member according to that member's semantics. A vector member copies its element values. A shared pointer member copies participation in a shared ownership group. A raw pointer member copies its stored pointer value. Therefore the enclosing record's semantics emerge from its representation. A class name such as `Snapshot` cannot turn a shared mutable pointee into an independent value.

Consider graph-shaped data as an extension. Deep copying one node recursively may duplicate shared nodes unintentionally or loop through cycles. A real graph copy needs a policy about identity and sharing, often a mapping from original nodes to their copies. The integer buffer has no such relationships, which is why its copy loop is sufficient for this teaching domain. Do not advertise that loop as a universal deep-copy recipe.

4.1.2 Using Copy Assignment

Copy assignment must consider both the incoming value and the destination's existing allocation. Deleting the destination first and then attempting allocation can lose the old value if the allocation fails. It can also break self-assignment when source and destination are the same object.

The example constructs a temporary copy first. Only after that succeeds does it swap size and storage with the destination. The temporary then destroys the old destination storage. This gives a clear commit point and naturally handles self-assignment, at the cost of allocating a replacement even when existing capacity could potentially be reused.

Copy assignment replaces the value of an already existing destination. Its old resources must eventually be released, but releasing them too early can destroy the only accepted value if allocation of the replacement fails. That distinction from construction is the reason assignment deserves its own design rather than a pasted constructor body.

The candidate-and-swap implementation stages the new value in a temporary buffer. Until that temporary is complete, the destination is untouched. Swapping transfers the old destination into the temporary, whose normal destruction releases it. The swap must exchange every member that participates in the representation invariant. Exchanging storage without the corresponding size would leave both objects internally inconsistent.

The strong exception guarantee has a boundary. Here it means assignment failure leaves the destination buffer's observable value unchanged. It does not promise that an allocator made no internal bookkeeping changes, that a diagnostic log did not record the attempt, or that an external input stream was rewound. State the relevant observable object rather than writing "nothing changes" without qualification.

Self-assignment is a useful stress case because source and destination name the same object. Candidate-and-swap handles it by making an independent candidate before modifying the destination. A direct self-

assignment check can avoid unnecessary work, but it is an optimization after correctness, not a substitute for a sound failure path. Test self-assignment through an alias so the test examines the intended semantics without relying on a compiler to approve an obviously redundant expression.

Instrumented Copy Construction

Purpose. Object copying constructs a new object from an existing value and can have observable cost when a type defines nontrivial copy behavior.

```
#include <iostream>
struct Trace{ int value; Trace(int v):value(v){} Trace(const Trace&
other):value(other.value){std::cout<<"copy\n";} };
int main(){ Trace a{5}; Trace b=a; std::cout<<b.value<<'\n'; }
```

How it works.

- The copy constructor receives a const reference to the source object.
- The new object initializes its own member from the source.
- The message exposes exactly when copying occurs.

Expected behavior/output. The program prints `copy` then `5`.

Experiment. Pass `Trace` by value to a function and inspect when additional copies or moves occur.

4.2 Distinguishing Shallow and Deep Copies

“Shallow” and “deep” describe which relationships are duplicated, not a universal ranking of good and bad. Copying a non-owning pointer is a useful shallow observation. Copying an exclusively owning raw pointer as though it created another independent owner is usually a defect.

A deep copy should follow the intended value boundary, which may be more complicated than recursively copying every reachable object.

4.2.1 Recognizing Shared Raw Resources

A raw member pointer copied by a generated copy operation still designates the same object. If both enclosing objects later delete it, ownership is duplicated incorrectly. If neither owns it, the copy may be an intentional shared borrow—but both copies now depend on an external lifetime.

Shared pointers represent yet another policy: copying them participates in shared lifetime management, not independent object duplication. When reviewing a class, classify each pointer-like member as owner, shared owner, or observer. Only then decide whether generated copy behavior matches the class’s contract.

The dangerous shallow-resource example has a size and a raw owning pointer. A generated copy creates a second size and a second pointer containing the same address. Both objects appear complete, and a

simple read test can pass. Later mutation changes what the other object observes; later destruction can release the same allocation twice. The defect begins at the ownership model, not at the eventual crash site.

A raw pointer is not always an owner. If both records intentionally borrow a separately owned immutable table, copying the pointer may be exactly right. The owner must outlive both records, and neither record may release it. Avoid teaching “all pointer copying is wrong.” Teach that pointer copying preserves an observation of identity and does not independently duplicate the object or release obligation.

The distinction also appears in standard types. Copying `std::string_view` copies a view, not the characters. Copying `std::span` copies an access range, not the underlying array. Copying a shared pointer keeps the same pointee alive through another ownership participant. All of these can be useful when their semantics match the interface. They become defects when an API promises an independent snapshot but returns one of these relationships without naming it.

Inspect nested members too. A vector of pointers owns its pointer objects, not automatically the pointed-to objects. A vector of strings owns independent string values. Both are vectors, yet copying them supports different application contracts. Tests should change the nested data that the domain cares about, not only compare the outer container’s address or capacity.

4.2.2 Implementing Independent Ownership

The buffer uses `std::unique_ptr<int[]>` internally to ensure each allocation has one release owner. The explicit copy constructor allocates a fresh owned array, so exclusivity remains true while value copying becomes possible at the enclosing type level.

The invariant couples `size_` and `data_`: a positive size must correspond to storage for that many integers; a zero-sized buffer uses an empty pointer. `swap` exchanges both parts together and is nonthrowing for this representation. Exchanging only one member would violate the invariant even though each individual member remained a valid C++ object.

Independent ownership needs both storage independence and value preservation. Allocate enough storage for the accepted size, copy or construct each element according to its semantics, and establish cleanup for partially completed work. For integer elements, assignment cannot throw and the example is simple. For arbitrary element types, element construction can fail partway through, so a generic container must track and destroy the elements that were actually constructed. This is one reason standard containers are preferable to casually generalized raw-storage exercises.

An assignment can preserve the old value by finishing a candidate before commit. The tradeoff is extra peak memory: both the old allocation and the new allocation can exist simultaneously. A memory-constrained application may select a different guarantee or representation, but it should do so explicitly. Reducing peak memory by overwriting the old buffer in place changes the failure story when element operations can throw.

The supplemental copy-transaction laboratory injects an exception at a chosen copy step. It does not rely on exhausting the machine’s memory. The destination begins with known values, a replacement is attempted, and a specific copy failure is triggered before the swap. The test compares the destination with its earlier snapshot and verifies the source is unchanged. This directly tests the selected commit boundary.

A passing injected-failure test demonstrates the exercised failure points in the laboratory type. It does not prove every allocator or element type has the same guarantees. Keep the custom buffer's contract narrow, and use standard components when they already express the application's value semantics.

Deep Copy Through Owning `std::vector` Storage

Purpose. A deep copy duplicates owned state, unlike copying a raw pointer value that would make two objects point at the same allocation.

```
#include <iostream>
#include <vector>
struct Buffer{ std::vector<int> data; };
int main(){ Buffer a{{1,2,3}}; Buffer b=a; b.data[0]=9; std::cout<<a.data[0]<<'
'<<b.data[0]<<'\n'; }
```

How it works.

- Each vector copy owns a separate element buffer.
- Changing the destination's element does not change the source.

Expected behavior/output. The output is `1 9`.

Experiment. Replace the vector with `int*` temporarily and explain why the default copy would then be shallow and dangerous.

Worked example — Implement independent buffer copies with a commit point

Requirement and design

Copy a three-element buffer, modify the copy, and verify that the original retains its value. Assign into an existing differently sized buffer, test self-assignment, and assign an empty buffer. Checked access must reject an index in the resulting empty value.

This custom buffer is an ownership lesson. A vector would normally be the preferred application implementation.

Walk through the state transition

Before assignment, the destination might contain one integer while the source contains three. The algorithm first constructs a three-element candidate. At that stage there are three independent buffer objects: source, old destination, and candidate. The source never changes. If candidate construction fails, its completed resources are cleaned up and the old destination remains accepted.

At the swap, the destination takes the candidate's representation. The temporary takes the old destination's representation. No element-by-element modification of the accepted destination is required

after commit. Finally the temporary is destroyed, releasing the previous destination. Draw sizes next to storage owners at every step; this catches an incomplete swap immediately.

The integer example proves visible values and checked-access behavior. It does not attempt to force an allocation failure in the normal chapter executable. Appendix B supplies a separate deterministic failure-injection laboratory so the successful teaching example and the exceptional experiment remain independently readable.

Complete implementation

The complete, buildable chapter project is `source/book2/ch04/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <cstdint>
#include <memory>
#include <utility>

class Buffer {
    std::size_t size_;
    std::unique_ptr<int[]> data_;
public:
    explicit Buffer(std::size_t size)
        : size_(size), data_(size ? std::make_unique<int[]>(size) : nullptr) {}
    Buffer(const Buffer& other) : Buffer(other.size_) {
        for (std::size_t i = 0; i < size_; ++i) data_[i] = other.data_[i];
    }
    Buffer& operator=(const Buffer& other) {
        Buffer copy{other};
        swap(copy);
        return *this;
    }
    void swap(Buffer& other) noexcept {
        std::swap(size_, other.size_);
        data_.swap(other.data_);
    }
    std::size_t size() const noexcept { return size_; }
    int& at(std::size_t index) {
        if (index >= size_) throw std::out_of_range("buffer index");
        return data_[index];
    }
    const int& at(std::size_t index) const {
        if (index >= size_) throw std::out_of_range("buffer index");
        return data_[index];
    }
};

int main() {
    Buffer original{3};
    original.at(0) = 7;
    Buffer copy{original};
    copy.at(0) = 99;
    CHECK(original.at(0) == 7 && copy.at(0) == 99);
    Buffer assigned{1};
    assigned = original;
    CHECK(assigned.size() == 3 && assigned.at(0) == 7);
}
```

```

    const Buffer& same = assigned;
    assigned = same;
    CHECK(assigned.at(0) == 7);
    Buffer empty{0};
    assigned = empty;
    CHECK(assigned.size() == 0);
    CHECK(course::throws<std::out_of_range>([&] { (void)assigned.at(0); }));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book2/ch04 -B build/b2-ch04 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch04 --parallel 2
ctest --test-dir build/b2-ch04 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch04`.

Evidence and interpretation

The original first element remains seven while the copy becomes 99. Assignment changes the destination’s size from one to three and copies the source value. Self-assignment leaves it valid, and assigning the empty buffer produces size zero with checked access failing.

Trace assignment as three states: construct the candidate, swap the candidate with the destination, destroy the former destination now held by the temporary. Before the swap, failure leaves the destination unchanged. After the swap, cleanup is handled by the temporary’s owning member. The integer-element model keeps the copy loop itself nonthrowing.

This chapter intentionally focuses on copies. Declaring copy operations affects implicit move generation, so do not assume this class has the move behavior shown in the next chapter. Special member functions form a related design set; changing one requires reviewing the others.

Failure clinic — Delete, then allocate

The naïve assignment first frees the destination’s old allocation, then allocates replacement storage. If the second step fails, the old value has already been destroyed. A catch block cannot recreate it without some other saved copy. Setting the pointer to null avoids a double release but does not restore the promised value. This implementation may support a weaker documented guarantee, but it cannot honestly claim preservation of the previous accepted buffer.

A different defect swaps only the pointer. Tests that inspect only the first element may still pass because both old and new allocations have an element at index zero. A size query or boundary access exposes the inconsistency. This is why the supplied tests assign differently sized values, include an empty source, and examine checked access afterward.

A third defect counts a move as a deep copy. Transferring a resource can efficiently give ownership to another object, but it does not preserve two independently usable copies. Use copy when the source and destination must both retain independent values; use move when transferring the source's resource is allowed by the surrounding operation.

Practice and implementation lab

Add tests for copying an empty buffer, assigning a larger value into a smaller destination, and modifying each copy after construction. Verify that changes remain independent in both directions.

Sketch an allocation-failure injection interface and identify the exact point where assignment should preserve the old value. Then replace the implementation with a vector member and compare the amount of custom ownership code required to provide the same visible behavior.

Review answers and solution notes

Copy-and-swap preserves the destination until a complete candidate exists. It also handles self-assignment without a special branch, although it may perform unnecessary allocation in that case. Swapping every coupled member preserves the size/storage invariant.

A vector member already provides suitable value and resource semantics for this application, so the enclosing class can often follow the Rule of Zero. A shared pointer copy would not create an independent buffer value. Deep-copy behavior must be chosen according to the intended abstraction, not assumed from the presence of a class.

Engineering takeaway. Define copying as a relationship between values and resources, then place failure-prone work before the assignment commit point.

Guided lab — Test assignment at the failure boundary

Run the copy-transaction laboratory in Appendix B. Begin with a destination containing two distinct values and a source containing three. Inject failure while constructing the candidate's second element. Verify destination size, every destination value, source size, and every source value after rejection. Then disable the injected failure and repeat the assignment successfully.

Extend the test to failure on the first and last copied elements. Include an empty source and self-assignment. Reset the injection policy explicitly between cases so one case cannot silently change the next case's starting conditions. Compare semantic values rather than allocator addresses; an allocator is allowed to reuse released addresses.

For a design exercise, replace the custom buffer with a class containing one vector. Remove the custom destructor, copy constructor, and assignment operator if the vector's semantics satisfy the intended contract. Keep the same public behavior tests. Explain which tests remain meaningful even though the implementation shrank substantially.

Review and solution reasoning

Construction creates a destination; assignment replaces one. The old destination is preserved by delaying mutation until a complete candidate exists. A deep-copy requirement is about the domain's independently owned state, not merely about using a class or a smart pointer. Copy-and-swap is a technique with costs and assumptions, especially around allocator-aware swap and generic element behavior. The compact teaching type uses a representation for which its chosen swap is nonthrowing.

Primary reference notes

Copy contracts: [copy constructors](#), [copy assignment](#), [unique ownership](#), and [utility operations](#).

Chapter 5 — Understanding Move Semantics

A large temporary buffer is ready to become the next stage’s input. Copying every element would preserve independence that no caller needs: the original temporary is about to be discarded. Transferring ownership is the more appropriate operation.

Move semantics let types express that transfer. They do not mean that every object is physically relocated, every move is free, or every use of `std::move` causes a move operation. This chapter makes the source and destination states explicit.

HarborWorks can build a report dataset correctly, but returning it or placing it in a container appears to require repeated allocation and copying. Move semantics offer a different operation: transfer an existing resource when the source is no longer required to retain its old value. The benefit is not a mysterious faster copy. The benefit comes from changing what must be preserved.

The source still exists after a move unless its lifetime has separately ended. It must remain usable according to its documented post-move contract. This chapter gives its custom buffer a precise empty state so the transfer can be traced and tested. Standard-library types and user-defined types do not all promise the same visible post-move values.

5.1 Understanding Lvalues and Rvalues

Value categories describe expressions and influence overload selection. An lvalue commonly identifies an existing object whose identity can be used; prvalues and xvalues participate in the broader rvalue category. An object’s type and an expression’s value category are related but separate questions.

Avoid explaining the categories only as “left side” and “right side” of assignment. Many valid expressions do not fit that mnemonic.

5.1.1 Identifying Persistent Objects

A named variable expression is an lvalue, including the expression naming an rvalue-reference parameter. The parameter may have been bound from an expiring object, but using its name does not automatically treat every subsequent access as expendable.

This matters inside a move constructor. The parameter `other` is named, so transferring one of its owned members normally uses `std::move(other.member)`. Without the cast, overload resolution can select a copy or reject the operation when copying is forbidden. The cast expresses that this particular source value may be consumed under the move contract.

Value categories classify expressions, not permanent personality traits attached to variables. A named variable is normally an lvalue expression even when the variable’s declared type is an rvalue reference. Inside `consume(Buffer&& value)`, the expression `value` names an existing object and is an lvalue. To pass it onward as a move candidate, the implementation must make that intent explicit.

Lvalues are useful for identifying objects that can be accessed again. Passing a rvalue to a by-value parameter normally preserves the caller's report by copying, provided the type is copyable. A noncopyable owner rejects that path. This is useful feedback: the call site has not yet said that surrendering the resource is allowed.

Do not interpret "lvalue" as "stored on the stack" or "mutable." A const object can be named by an lvalue expression. A dynamically allocated object can be accessed through an lvalue expression after dereferencing its pointer. Storage duration, const qualification, and value category are separate aspects of the language. Mixing them produces explanations that work for one tiny example and fail immediately in a real function.

When diagnosing overload selection, write the expression being passed and its cv-qualification, not just the variable's declaration. A named `Buffer&&` and the expression `std::move(buffer)` can designate the same object while selecting different overloads. This is the mechanical reason move syntax belongs at transfer boundaries rather than on every use of an object.

5.1.2 Identifying Temporary Values

A temporary value can initialize an owning destination directly, and many return-value situations benefit from guaranteed or permitted copy elision. Do not insert `std::move` into every return statement; it can interfere with useful elision in cases where returning the local by name is the right expression.

An xvalue designates an object whose resources may be reused by a move-aware operation. `std::move(existing)` produces such a cast result; it does not itself call the object's move constructor. The receiving operation still determines what happens. A const source often cannot supply the mutable source required by an ordinary ownership-transferring move.

Rvalues include prvalues and xvalues. A prvalue such as a freshly computed integer supplies a value; an xvalue identifies an object whose resources may be reused. `std::move(existing)` produces an xvalue expression referring to the existing object. No allocation is freed and no member is changed merely by forming that expression.

An overload accepting an rvalue reference can then perform a move operation. What it does is determined by the selected constructor or assignment operator. A custom type might move a pointer, copy a small array, or perform a more expensive operation. The phrase "move is constant time" is not a property of every move operation in C++.

Return-by-value deserves special care. A function returning a local value can benefit from copy elision or automatic move selection under the applicable language rules. Adding `std::move` to an eligible local return can prevent named return value optimization. Start with the direct return of the local and inspect evidence before trying to optimize the language's existing return mechanisms manually.

The book's C++20 baseline distinguishes guaranteed elision in specified prvalue cases from optional named return value optimization. Do not write a portable correctness test requiring an exact number of temporary constructors for every return expression. Test the returned value and resource ownership; use instrumentation only for a narrowly labelled implementation experiment.

Overload Selection for Persistent and Temporary Strings

Purpose. Value categories influence overload resolution and whether an operation may safely transfer resources.

```
#include <iostream>
#include <string>
void inspect(const std::string&){ std::cout<<"lvalue-compatible\n"; }
void inspect(std::string&&){ std::cout<<"rvalue\n"; }
int main(){ std::string s="named"; inspect(s); inspect(std::string{"temp"}); }
```

How it works.

- A named object expression is an lvalue.
- A temporary string is an rvalue and binds to the rvalue-reference overload.

Expected behavior/output. The output identifies the named argument first and the temporary second.

Experiment. Call `inspect(std::move(s))` and note which overload is selected.

5.2 Transferring Resources Efficiently

A move operation must leave both objects in states allowed by their contracts. The destination acquires the intended value or resource, and the source remains suitable for the operations the type promises after a move.

For a custom type, document the moved-from state. Do not rely on vague folklore about moved-from objects being “destroyed.”

5.2.1 Implementing Move Constructors

The move constructor transfers the unique pointer and uses `std::exchange` to replace the source size with zero. This preserves the source’s empty-buffer invariant while the destination receives the previous size and allocation.

Simply defaulting the move would transfer the unique pointer but copy the scalar size, leaving the source with a positive size and no storage. That would make its checked accessor’s size test misleading. A defaulted operation is correct only when the memberwise result preserves the enclosing invariant; Rule of Zero works best when the representation itself composes cleanly.

The custom buffer has coupled members: its size describes the allocation owned by its unique pointer. Moving only the pointer would leave the source with a nonzero size and no storage. Its `at` method could then accept an index and attempt to access absent storage. The move constructor therefore transfers the pointer and exchanges the source’s size with zero as part of establishing the chosen empty-state contract.

A useful move-construction test records the source’s values, moves into a new buffer, and verifies the destination. It then checks that the source is empty and rejects checked access. Finally it assigns a new

buffer to the source and uses it again. That last step distinguishes a valid reusable moved-from object from an object the test simply avoids touching until destruction.

Mark a move operation `noexcept` only when its implementation can honor that guarantee. For the default unique-pointer ownership transfer and scalar exchange in this buffer, the selected operations support it. Adding an allocating diagnostic string or a throwing callback inside the move could invalidate the claim. An exception escaping a nonthrowing function is not converted into a recoverable move failure.

Containers use exception properties when choosing relocation strategies. A nonthrowing move can allow a vector to transfer elements while preserving stronger failure guarantees. The exact path still depends on the element type and operation; teach the mechanism rather than promising one universal constructor count.

5.2.2 Implementing Move Assignment

Move assignment begins with an already-live destination, so it must release or replace the destination's old resource as well as acquire the source's resource. Assigning the unique pointer handles that release, while exchanging the source size completes the coupled-state transfer.

The example guards self-move assignment explicitly. Both move operations are declared `noexcept` because their member operations are nonthrowing for this representation. This promise can affect container strategies for relocating elements; do not add `noexcept` to an operation that may actually propagate an exception, since an escaping exception would terminate the program.

Move assignment has two resources to consider: the source's resource to acquire and the destination's old resource to release. The unique pointer member handles release of the destination's previous allocation when it receives the new ownership. The size member must then reflect the new representation, and the source must be put into the class's documented empty state.

Self-move is a special case because source and destination are the same object. The teaching buffer explicitly checks this and leaves itself unchanged. That is its chosen contract, not a claim that every standard-library type preserves the same value after self-move assignment. For an unfamiliar type, consult the actual operation requirements and avoid depending on an unstated value guarantee.

Think about failure before calling a consuming function. If the caller writes `accept(std::move(report))`, construction of the receiving parameter may already transfer the caller's state before `accept` validates a business rule. A later error result does not undo that transfer automatically. A preservation requirement may require inspection before transfer, a copy of the accepted state, or a staged candidate owned separately from the caller's current value.

Move assignment should be reviewed alongside destruction, copying, and move construction. Declaring one special member can affect which others are generated. A design that was copy-only does not acquire efficient moves simply because a call site begins using `std::move`; overload resolution may still select a copy or reject the expression.

Move-Constructing a Large Vector

Purpose. Move operations are designed to transfer ownership of expensive resources when the source value is no longer needed.

```
#include <iostream>
#include <utility>
#include <vector>
int main(){ std::vector<int> a(100000,7); auto before=a.data(); std::vector<int>
b=std::move(a); std::cout<<std::boolalpha<<(b.data()==before)<<'
'<<b.size()<<'\n'; }
```

How it works.

- `std::move(a)` permits the vector move constructor to transfer its internal allocation.
- The destination receives the elements without element-by-element copying in the usual implementation.
- The source remains valid but in an unspecified state.

Expected behavior/output. Typically the output begins `true` and reports size 100000; pointer preservation is expected for the standard allocator case but the conceptual guarantee is resource transfer, not a universal address promise.

Experiment. After the move, call `a.clear()` and reuse `a` to reinforce that moved-from objects remain valid.

5.3 Using `std::move` Correctly

`std::move` is a cast that enables move-aware overload selection. Ownership transfer occurs only when the receiving constructor, assignment operator, or function performs it. A use of the cast should therefore identify a real handoff point in the program.

After a handoff, stop assuming that the old value remains available unless the type’s contract explicitly guarantees it.

5.3.1 Requesting Move Semantics

The example constructs `second` from `std::move(first)`, then verifies that `first` is empty by this class’s documented rule. Later it moves a buffer into a vector, making the vector’s element the owner of the allocation.

Moving a scalar such as an integer ordinarily just copies its value. Moving a string or vector may transfer storage, but allocator rules and representation details can affect costs. Use the term “move” for the semantic operation and measure its cost when performance matters. Do not promise constant time for every movable type.

`std::move` is a cast expressing permission to treat an object as a move candidate. The receiving operation determines whether resources move. For a const string, the resulting expression is still const-qualified. The usual string move constructor requires a non-const source because transferring ownership changes the source's state, so an available const-reference copy constructor can be selected instead.

This explains why sprinkling `std::move` into a function is not a performance strategy. On a small integer it normally changes no meaningful resource behavior. On a const owner it may copy. On a returned local it may inhibit an optimization. On an object needed later it can violate the caller's assumptions. Use it where the program's lifetime and value contract says that the old resource can be surrendered.

A useful review reads each move as a sentence: "After this point, this scope no longer relies on the object's previous value." If that sentence is false, the move is suspicious. Reinitialization or a documented query can be valid afterward, but the old content should not be casually assumed. The source code should make the transition visible enough that a later maintainer does not insert a use of the old value between transfer and reinitialization.

5.3.2 Understanding Moved-From Objects

Standard-library moved-from objects generally have a valid but unspecified state unless a more specific guarantee applies. Operations without unmet preconditions, such as destruction or assigning a new value, remain available. Calling `front()` on a moved-from vector without checking emptiness is not justified by that general validity guarantee.

This custom buffer provides the stronger specific rule of size zero after a non-self move. Its tests may therefore inspect that state. The source is then assigned a fresh buffer and reused safely. The lesson is to rely on the actual type contract, not to infer another type's moved-from behavior from this example.

The standard library generally specifies valid but unspecified states for moved-from objects unless a particular operation provides a more specific guarantee. "Valid" means the object's invariants are preserved and operations can be used when their preconditions are satisfied. "Unspecified" means the caller should not guess the retained value from one implementation's behavior.

For example, asking a container whether it is empty can be valid after a move. Accessing its first element still requires that it actually contain an element. A programmer who sees an empty vector in one test should not elevate that observation into a promise for every container move or allocator configuration. The custom buffer is different because this chapter explicitly defines and implements zero size for its moved-from state.

Moving a container and invalidating a particular borrow are also operation-specific questions. Some operations preserve references to elements while changing which container owns them; others can relocate elements. Do not use "moved" as a shortcut for either "all references survive" or "all references dangle." Follow the documented operation, allocator assumptions, and actual owner of the referenced elements.

Treat moved-from tests as contract tests. Check only promised properties for library types, and check the explicit stronger properties chosen for your own type. The distinction keeps tests portable without making them vague.

`std::move` as a Cast Rather Than an Operation

Purpose. `std::move` changes the value category of an expression; a move only happens if the selected operation implements one.

```
#include <iostream>
#include <string>
#include <utility>
int main(){ std::string s="payload"; auto&& r=std::move(s); std::cout<<s<<'
'<<r<<'\n'; std::string destination=std::move(s);
std::cout<<destination<<'\n'; }
```

How it works.

- Binding `r` to `std::move(s)` does not itself move characters anywhere.
- The actual resource transfer can occur when the rvalue expression is used to construct `destination`.

Expected behavior/output. Before destination construction both names refer to the same string object; afterward `destination` contains `payload` and `s` is valid but unspecified.

Experiment. Replace destination construction with `std::string destination=s` and compare source state.

Worked example — Transfer a buffer and preserve a defined empty source

Requirement and design

Move a buffer containing 42 into another buffer, reuse the emptied source, move-assign again, and finally transfer the value into a vector of buffers. Copies are deleted so an accidental copy cannot silently replace the intended ownership transfer.

The custom moved-from invariant is explicit: size zero and no owned allocation. This is stronger than the generic valid-but-unspecified rule for many library types.

Trace both objects, not only the destination

Write the buffer's representation beside each operation: size and ownership. Before the first move, the source owns three zero-initialized integers, one of which is changed to 42. After move construction, the destination owns that allocation and the source has size zero with no allocation. Reassigning the source establishes a new resource independent of the transferred one.

The vector demonstration then transfers buffers into a larger owner. The vector owns buffer objects; each buffer owns its own integer allocation. These are two ownership levels. A vector reallocation can move

the buffer objects without copying their underlying integer arrays because the buffer's move operation transfers that inner ownership.

The example proves selected states and a nonthrowing move trait. It does not benchmark allocation throughput or promise that every moved type will behave like a pointer-owning buffer.

Complete implementation

The complete, buildable chapter project is `source/book2/ch05/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <cstdint>
#include <memory>
#include <type_traits>
#include <utility>
#include <vector>

class Buffer {
    std::size_t size_;
    std::unique_ptr<int[]> data_;
public:
    explicit Buffer(std::size_t size)
        : size_(size), data_(size ? std::make_unique<int[]>(size) : nullptr) {}
    Buffer(const Buffer&) = delete;
    Buffer& operator=(const Buffer&) = delete;
    Buffer(Buffer&& other) noexcept
        : size_(std::exchange(other.size_, 0)), data_(std::move(other.data_)) {}
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            data_ = std::move(other.data_);
            size_ = std::exchange(other.size_, 0);
        }
        return *this;
    }
    std::size_t size() const noexcept { return size_; }
    int& at(std::size_t index) {
        if (index >= size_) throw std::out_of_range("buffer index");
        return data_[index];
    }
};

int main() {
    static_assert(std::is_nothrow_move_constructible_v<Buffer>);
    Buffer first{3}; first.at(0) = 42;
    Buffer second{std::move(first)};
    CHECK(first.size() == 0 && second.at(0) == 42);
    first = Buffer{1};
    CHECK(first.size() == 1 && first.at(0) == 0);
    first = std::move(second);
    CHECK(second.size() == 0 && first.at(0) == 42);
    std::vector<Buffer> buffers;
    buffers.push_back(std::move(first));
    buffers.emplace_back(100);
    CHECK(buffers.front().at(0) == 42);
    CHECK(first.size() == 0);
}
```

```
    course::report();  
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch05 -B build/b2-ch05 \  
  -DCMAKE_BUILD_TYPE=Debug  
cmake --build build/b2-ch05 --parallel 2  
ctest --test-dir build/b2-ch05 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch05`.

Evidence and interpretation

The tests check both sides of each transfer: the destination retains 42 and the source reports zero size. A fresh assignment restores the source to a usable one-element buffer. Moving into the vector preserves the value even when another element is subsequently emplaced.

The `static_assert` checks that the move constructor is recognized as nonthrowing. This is a property of the declared and composed operation, not a benchmark. The vector can use the type’s move behavior without requiring a copy constructor.

Inspect the paired members during every transfer. The unique pointer and size are one logical ownership state. Leaving either half behind incorrectly would allow a live object to report storage it no longer owns. Move semantics make efficient transfer possible, but only the class’s implementation can preserve that invariant.

Failure clinic — A nonzero size with no storage

A defaulted move operation can be wrong for a representation with redundant or coupled state. If one member owns storage and another independently records its length, moving the owner and copying the scalar may leave the source’s invariant broken. The fix is either a custom move that repairs the relationship or a simpler representation whose size is derived from the owning member.

For an application class containing only a vector, querying `vector.size()` avoids maintaining a second length. That representation can support the Rule of Zero. The custom buffer retains the pair deliberately to teach why moving members individually is not the same as validating the enclosing object.

The other common defect is reading the source’s old content after transfer because the move “looked like a copy.” Add a test that reuses the source through a documented operation, not through an assumption about its previous contents. A moved-from state is part of the API, not an opportunity for the test to guess what the implementation happened to leave behind.

Practice and implementation lab

Add self-move assignment through an alias and verify the documented policy. Test moving an empty buffer and repeatedly assigning fresh values to moved-from objects. Confirm that an attempted copy is a compile-time error in a separate negative-compilation exercise.

Replace the custom representation with a vector and compare the moved-from contract you may safely assume. Review return statements and remove unnecessary moves that interfere with direct value return or elision. Record semantic reasoning before measuring any timing difference.

Review answers and solution notes

This implementation leaves a self-move unchanged and makes ordinary moved-from sources empty. A moved-from standard vector should instead be used according to its own guarantees, with preconditions such as nonemptiness checked before element access.

`std::move` does not move by itself, and a named rvalue-reference parameter is an lvalue expression. The custom move must update size together with storage. `Noexcept` is a real exception promise, not an optimization annotation to add without examining every operation.

Engineering takeaway. Transfer ownership at explicit handoff points and preserve a documented, usable state on both sides of every move.

Guided lab — Observe relocation without making it a contract

Use the supplemental relocation laboratory to compare a type with a nonthrowing move and a type whose move may throw but whose copy is available. Both types hold small values and maintain operation counters only for observation. Trigger a vector capacity change and verify that values remain correct.

Record the copy and move counts for the installed library, but do not make those exact counts a cross-platform acceptance rule. The stable requirements are value preservation, valid source states where specified, and the correct exception properties. Then remove unnecessary `std::move` from a returned local in a separate experiment and inspect the effect on that build.

Review and solution reasoning

A named rvalue-reference variable is still an lvalue expression. A cast does not itself transfer a resource. A custom buffer must preserve its size/storage invariant in both source and destination. Nothrow moves can help container operations, but a false `noexcept` promise makes failure less recoverable rather than making the code faster by definition.

Primary reference notes

Move rules: [copy/move constructors](#), [assignment operators](#), [value categories](#), [utility casts](#), and [unique pointers](#).

Chapter 6 — Managing Ownership with Smart Pointers

A tree owns its children, while each child needs to find its parent. Making every edge a shared owner seems convenient, but it creates a cycle: the parent keeps the child alive and the child keeps the parent alive. Neither object's last owner ever disappears.

Smart pointers solve specific ownership problems. They do not decide the ownership graph for you. This chapter compares exclusive ownership, shared ownership, and observation without ownership, then verifies the destruction behavior of a small hierarchy.

Smart pointers make particular ownership policies executable. They do not choose the correct policy for the application. HarborWorks might need one owner for a report generator, shared lifetime for an immutable published snapshot, and a non-owning link back to a parent object. Those are three different relationships and should not be collapsed into one pointer type selected by habit.

Begin every design with a graph of ownership. Label an owning edge differently from a borrowing edge. Then trace what happens when one owner disappears. The important question is not whether a pointer looks modern; it is whether the last intended owner can release the resource without leaving a live user behind or retaining a cycle forever.

6.1 Expressing Exclusive Ownership with `std::unique_ptr`

`std::unique_ptr<T>` represents exclusive ownership of a dynamically managed object through its deleter. It cannot be copied, because copying would create another exclusive owner. It can normally be moved when responsibility should pass to another owner.

Prefer exclusive ownership when one object or scope is clearly responsible for lifetime. Borrowing access does not require changing that ownership policy.

6.1.1 Creating Unique Resources

`std::make_unique<T>(arguments)` constructs an owned object and returns its unique owner. The array form supports owned arrays, though a vector is usually more convenient when size and sequence operations are needed together.

Use `get()` only for borrowed access under the owner's lifetime. Do not construct a second owning smart pointer from that raw pointer, and do not delete it manually. `release()` relinquishes ownership without destroying the object, transferring a cleanup obligation to the caller; it is not an ordinary way to inspect the pointer. `reset()` replaces or clears ownership and releases the previously owned resource according to the deleter.

An exclusive owner is a good default for a dynamically allocated object that has one clear lifetime manager. `std::unique_ptr<T>` cannot be copied, so accidentally duplicating its release obligation is rejected by the type system. `std::make_unique` creates the owned object and the owner together, avoiding an exposed owning raw pointer in ordinary application code.

Exclusive ownership does not imply that no other code can inspect the object. A function can borrow `T&`, `const T&`, or an appropriate pointer during a documented period. The unique owner remains responsible for lifetime. Passing the smart pointer itself is useful when the function participates in ownership; passing the pointee is usually clearer when the function only needs behavior or data.

Distinguish `get`, `reset`, and `release`. `get` observes the stored pointer without surrendering ownership. `reset` replaces or clears ownership and disposes of the previously owned object under the deleter policy. `release` gives up ownership without disposing of the object, leaving the caller responsible for arranging a new owner or matching cleanup. A casually ignored `release()` result leaks responsibility even though the code contains a smart pointer.

Custom deleters let an exclusive owner release resources through a matching function rather than ordinary `delete`. The deleter becomes part of the owner's type and behavior. It must be safe under the conditions where the owner invokes it. Do not use an array owner for a single object or a single-object deleter for an allocation that requires a different release operation.

6.1.2 Transferring Unique Ownership

Moving a unique pointer transfers its ownership and leaves the source empty under the library's specified move behavior. The receiving object now controls destruction. Passing a unique pointer by value to a function therefore makes ownership transfer visible at the call boundary.

A function that only reads the object should usually accept the object by `const` reference or a documented pointer, not demand ownership transfer. A function that optionally observes it can accept a nullable borrowed pointer. Choose the interface based on the relationship needed by the callee, not merely the representation used by the caller.

A consuming function can take a unique pointer by value. The caller must transfer the pointer explicitly when passing an existing owner. If the function returns normally without storing or returning the owner, the local parameter destroys its pointee. This is often appropriate for a sink operation whose contract includes consumption, but it would be surprising for a function named `inspect`.

A reference to a unique pointer is a different interface. It can allow the function to replace the caller's owner through `reset` or assignment. That authority should be required by the operation, not granted merely because the caller stores its object in a unique pointer. Borrowing the pointee narrows what the callee can do to the lifetime manager.

After a successful move of a unique pointer, the source owner is empty. The pointee itself may remain at the same address under the new owner. A borrower can remain usable only while the transferred object remains alive and its own validity conditions hold. Moving the owner does not grant a borrower permission to outlive the new owner, nor does it transfer the borrower automatically into a different scope.

For a factory returning an exclusive polymorphic object, use a base interface with an appropriate virtual destructor when deletion through that base is intended. Smart ownership removes manual cleanup syntax; it does not repair an incorrect base-class destruction contract.

Transferring Exclusive Ownership Between Functions

Purpose. Exclusive ownership is visible in function signatures and cannot be accidentally copied.

```
#include <iostream>
#include <memory>
struct Task{ int id; };
std::unique_ptr<Task> make_task(int id){ return
std::make_unique<Task>(Task{id}); }
void consume(std::unique_ptr<Task> t){ std::cout<<t->id<<'\n'; }
int main(){ auto t=make_task(9); consume(std::move(t));
std::cout<<std::boolalpha<<(t==nullptr)<<'\n'; }
```

How it works.

- The factory returns a move-only owner.
- `consume` takes ownership by value.
- Moving leaves the caller's `unique\_ptr` empty.

Expected behavior/output. The output is `9` then `true`.

Experiment. Change `consume` to take `Task&` when it should borrow instead of own and compare the API meaning.

6.2 Expressing Shared Ownership with `std::shared_ptr`

`std::shared_ptr<T>` allows several owners to participate in a resource's lifetime. The managed object is destroyed when the last participating strong owner releases it, subject to the stored deleter and ownership arrangement.

Shared ownership is appropriate when independent participants genuinely determine lifetime. It is not a universal cure for difficult ownership design.

6.2.1 Understanding Reference Counting

Shared pointers coordinate ownership through a control structure associated with the shared lifetime. Copying a shared pointer adds an owning participant; destroying or resetting one removes that participation. `std::make_shared` creates an object with a compatible shared ownership arrangement.

The stored pointer and the managed ownership identity are not always identical concepts, as aliasing constructors demonstrate, but ordinary application code should begin with straightforward ownership. Never create unrelated shared ownership groups independently from the same raw pointer. Each group can then try to destroy the same object, defeating the intended coordination.

Shared ownership represents a resource whose lifetime is controlled by several independent owners. Copying a `shared_ptr` adds an ownership participant. Destruction of one participant does not destroy the pointee while another owning participant remains. A weak observer does not keep the pointee alive, although bookkeeping associated with the ownership group can outlive the pointee itself.

The ownership group matters more than the numerical address. Creating two unrelated shared pointers from the same raw pointer can create two independent groups that each believe they must destroy the same object. Copy an existing shared pointer when sharing an ownership group; do not reconstruct a new owner from its `get()` result. The code may appear to work until the groups are destroyed, which makes a direct ownership audit more valuable than waiting for a crash.

The aliasing constructor illustrates the distinction between a stored pointer and an owned lifetime. A shared pointer can expose a subobject while keeping a larger owning object alive through the same control block. This is useful but requires careful understanding: the stored pointer is what dereferencing accesses, while the ownership group determines what stays alive. Do not infer ownership solely from the exposed address.

For the introductory examples, `make_shared` is the simplest construction path. It commonly uses a combined allocation for ownership bookkeeping and the object. Treat the allocation strategy as an implementation and lifetime consideration, not as proof that shared ownership is free or that it is required for all heap objects.

6.2.2 Recognizing Shared-Ownership Costs

Reference counting adds bookkeeping, and shared lifetime can make destruction happen at an unexpected point when the last owner disappears. The pointee is not automatically thread-safe merely because shared ownership bookkeeping supports certain concurrent operations on separate shared-pointer objects.

Avoid using `use_count()` as a synchronization or exclusive-access proof. Another owner can appear, and the count does not describe raw borrowers. Performance and concurrency requirements need their own design. A shared pointer is a lifetime mechanism, not a mutex, a transaction, or a guarantee that access patterns are race-free.

Shared ownership adds work beyond an ordinary borrow or exclusive transfer. Copying an owner adjusts lifetime bookkeeping; separate object access may involve indirection; destruction of the final owner triggers cleanup at the point where that final ownership disappears. Those costs can be worthwhile when independent consumers genuinely need to keep an object alive.

The harder cost is conceptual. A resource can remain alive long after the creating function returns because an unrelated callback retained an owner. If that callback is never released, the object remains retained even without a cycle. A memory investigation should therefore identify retained owners as well as allocations. “No raw new” is not evidence that lifetime is bounded as intended.

Shared-pointer lifetime management does not make the pointee’s data thread-safe. Different shared-pointer objects participating in the same ownership group have specified bookkeeping behavior, but unsynchronized writes to a shared mutable report remain a separate concurrency problem. Even a read-only-looking interface can expose mutable nested objects. The single-threaded labs test ownership, not a concurrency guarantee.

Avoid using `use_count()` as a general permission check for mutation. A numerical observation is not an application synchronization protocol, and the meaning can change as owners are created or destroyed. Prefer explicit exclusive mutation before publication, followed by immutable shared snapshots, when that matches the workload. The supplemental snapshot laboratory demonstrates this separation without introducing threads.

Two Owners Sharing One Object Lifetime

Purpose. Shared ownership is appropriate only when multiple components genuinely participate in lifetime control.

```
#include <iostream>
#include <memory>
struct State{ int value=5; };
int main(){ auto a=std::make_shared<State>(); auto b=a; b->value=8;
std::cout<<a->value<<' '<<a.use_count()<<'\n'; }
```

How it works.

- Copying the shared pointer increments shared ownership count.
- Both pointers identify the same `State`` object.
- The object is destroyed only after the final owning `shared_ptr`` is released.

Expected behavior/output. The output is ``8 2``.

Experiment. Reset ``b`` and inspect ``a.use_count()`` again.

6.3 Breaking Ownership Cycles with `std::weak_ptr`

A `std::weak_ptr<T>` observes a shared ownership group without keeping the object alive. It is useful for back references, caches, and other relationships that should not determine lifetime.

The observer must attempt to acquire a temporary strong owner before using an object that might otherwise disappear.

6.3.1 Recognizing Cyclic References

An owning cycle can keep every strong count positive even after external roots are removed. In a parent/child model, choose which direction owns. The example gives the parent strong ownership of children while each child holds only a weak reference to its parent.

Not every graph should be forced into this pattern. A central arena or registry can own all nodes while edges use non-owning identifiers, often making lifetime simpler. The important step is to draw the graph and distinguish ownership edges from navigation edges. A data relationship does not automatically need an owning edge.

A parent owning its children and each child owning its parent creates a cycle of strong ownership. Dropping all external owners leaves the internal owners keeping one another alive. Reference counting cannot conclude that the cycle is unreachable merely by seeing nonzero counts. The destructor may never run even though no useful part of the program can reach the objects.

An ownership graph should distinguish containment from navigation. The parent can own children while each child merely observes the parent. A weak pointer is suitable for that observing edge when the parent is shared-owned and can disappear independently. Another design might use unique child ownership and a carefully bounded raw parent borrow. The correct choice depends on the whole lifetime model, not on a rule that every back pointer must have one particular spelling.

Test the disappearance of the ownership graph from the outside. Keep weak observers, release the strong owners in a planned order, and examine whether the weak observers expire. This verifies lifetime without retaining an extra owner just for the test. A test that stores an additional shared pointer to every node would itself keep the nodes alive and obscure the intended result.

Do not solve a cycle by calling `reset()` at an arbitrary shutdown location unless the application can guarantee that location executes on every relevant path. A weak observation edge expresses the intended non-ownership throughout the object's lifetime, including failure paths.

6.3.2 Observing Objects Without Owning Them

Call `lock()` on a weak pointer to obtain a shared pointer when the object is still available. Test that result and keep it alive for the duration of access. Checking `expired()` and then separately trying to use an old raw address is not an equivalent lifetime guarantee.

A weak pointer can remain as an expired observer after the object is destroyed. Its associated bookkeeping may remain until weak observers are released, depending on the ownership allocation arrangement. That is different from keeping the managed object alive. Use the distinction when interpreting memory retention in shared-ownership systems.

A weak pointer must be promoted with `lock()` before accessing an object that may have disappeared. The result is either an empty shared pointer or an owning participant keeping the object alive for the duration of its use. Store the result locally and use that same result. A separate `expired()` check followed by an unprotected assumption is not the lifetime operation the interface is designed to provide.

Promotion secures the object's lifetime, not an arbitrary consistency property of its fields. The parent may remain alive while its application state changes. A multithreaded design still needs synchronization for mutable data. In the single-threaded hierarchy example, the important distinction is simply between an available parent and a parent that has been released.

Weak observers can also support caches that should not keep values alive indefinitely. A cache lookup can attempt promotion and recompute a missing value. That policy differs from a cache of strong owners, which retains every stored value until explicit eviction. Write down which retention behavior the application wants before choosing a map's value type.

A weak pointer is not a universal safer raw pointer. It only observes objects participating in shared ownership, and its bookkeeping has a cost. For a synchronous call whose owner is unquestionably alive,

a reference may express the contract more directly. Use the smallest lifetime mechanism that actually supports the required relationship.

Parent Back-Reference That Does Not Extend Lifetime

Purpose. A `weak_ptr`` breaks shared-ownership cycles while still allowing temporary checked access through `lock()`.

```
#include <iostream>
#include <memory>
struct Parent;
struct Child{ std::weak_ptr<Parent> parent; };
struct Parent{ std::shared_ptr<Child> child=std::make_shared<Child>(); ~Parent()
{std::cout<<"parent destroyed\n";} };
int main(){ auto p=std::make_shared<Parent>(); p->child->parent=p; }
```

How it works.

- The parent owns the child with `shared_ptr``.
- The child observes the parent through `weak_ptr`` without adding another owning reference.
- The cycle is therefore not ownership-strong in both directions.

Expected behavior/output. Program exit prints `parent destroyed``.

Experiment. Replace the weak pointer with `shared_ptr`` and use a destructor trace to observe the leak cycle.

Worked example — Destroy a hierarchy without retaining an ownership cycle

Requirement and design

Transfer an exclusively owned integer, then create a shared root and child with a weak parent back reference. Remove the root's external owner and verify that it is destroyed while an independently owned child remains alive. Finally remove the child's remaining strong owner and verify its expiration.

The example deliberately keeps weak observers so destruction can be checked without dereferencing destroyed objects.

Trace the hierarchy's ownership edges

The root has one child in its vector, and the child has a weak parent link. The local `child` variable is another strong owner of the child. Resetting the root therefore destroys the root and releases its ownership of the child, but the child remains alive through the local variable. Its parent observation expires. Resetting the last local child owner then ends the child's lifetime.

These intermediate states are intentional. A test expecting both nodes to disappear immediately after `root.reset()` would misunderstand the remaining local owner. Draw every strong edge, including test variables, before interpreting reference-counted lifetime. Hidden owners in lambdas, containers, and fixtures are common reasons for apparently mysterious retention.

Complete implementation

The complete, buildable chapter project is `source/book2/ch06/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <memory>
#include <string>
#include <vector>
#include <utility>

struct Node {
    std::string name;
    std::weak_ptr<Node> parent;
    std::vector<std::shared_ptr<Node>> children;
    explicit Node(std::string value) : name(std::move(value)) {}
};

int main() {
    auto owner = std::make_unique<int>(42);
    auto next = std::move(owner);
    CHECK(owner == nullptr && *next == 42);
    auto root = std::make_shared<Node>("root");
    auto child = std::make_shared<Node>("child");
    child->parent = root;
    root->children.push_back(child);
    const std::weak_ptr<Node> root_observer = root;
    const std::weak_ptr<Node> child_observer = child;
    {
        auto parent = child->parent.lock();
        CHECK(parent && parent->name == "root");
    }
    root.reset();
    CHECK(root_observer.expired());
    CHECK(child->parent.expired());
    CHECK(!child_observer.expired());
    child.reset();
    CHECK(child_observer.expired());
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch06 -B build/b2-ch06 \
  -DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build/b2-ch06 --parallel 2
ctest --test-dir build/b2-ch06 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch06`.

Evidence and interpretation

The moved unique pointer is empty and its destination still owns 42. The child's parent can initially be locked into a temporary strong owner. That temporary is scoped so it no longer retains the root when `root.reset()` executes.

Resetting the root destroys the root and releases its child ownership edge, but the local `child` variable still keeps the child alive. The child's weak parent becomes expired. Resetting the local child then removes its final strong owner, and the child observer expires too.

The checks describe the graph rather than relying on destructor print timing. A strong parent back reference would change the result by forming a cycle. That alternative is a design error for this ownership policy, not a defect in the reference counter.

Failure clinic — The observer that owned the answer

A test creates a second shared pointer called `observer`, resets the original owner, and expects destruction. The object remains alive because the observer is not an observer at all; it is another owner. Renaming the variable cannot change the policy. Replace it with a weak pointer when the test only needs to detect lifetime, or accept that a strong owner deliberately extends lifetime.

Another defect uses `shared_ptr(existing.get())`. The pointer value looks right, but the new ownership group is unrelated to the existing one. A sanitizer may later report double deletion, yet the repair belongs at construction: copy the existing shared pointer or redesign the transfer. Do not run the double-owner defect in the normal suite; reason about the invalid ownership and keep the shipped examples defined.

For immutable report publication, construct the complete snapshot before sharing it, then expose it through a const pointee interface without retaining a mutable alias that bypasses that policy. Const shared ownership is only as strong as the full alias graph permits.

Practice and implementation lab

Draw the strong and weak edges before and after each reset. Add a second external child owner and predict which expiration check must move later. Keep the `lock()` result in a longer scope and explain how it extends the parent's lifetime.

Design a registry-owned graph using integer identifiers for edges. Compare its cleanup responsibilities with the shared-pointer version. Do not switch every edge to shared ownership merely to avoid documenting the registry's lifetime.

Review answers and solution notes

The last strong owner controls destruction; weak observers do not. Keeping a successful `lock()` result alive creates a strong owner and therefore intentionally extends lifetime. Another child owner delays child destruction but does not require retaining the parent through a weak edge.

Shared-pointer bookkeeping does not protect the object's mutable fields from races. Unique ownership transfers through moves, while ordinary inspection can remain a borrow. The ownership graph must be acyclic where reference counting alone is expected to reclaim every object.

Engineering takeaway. Choose smart pointers to encode a deliberate ownership graph, and use weak observation for relationships that must not retain objects.

Guided lab — Publish an immutable snapshot

Use the ownership-snapshot laboratory in Appendix B. Publish an owned snapshot through `shared_ptr<const Snapshot>`, obtain two reader owners, and retain a weak observer. Replace the publisher's current pointer with a new snapshot. The old readers should still observe the original values until they release their owners; the publisher should expose only the new values.

Verify that releasing one reader does not expire the old weak observer while the other reader remains. Then release the final reader and verify expiration. Attempt a weak promotion after expiration and require an empty result. This models lifetime across revisions without threads or a hidden global cache.

Review and solution reasoning

Exclusive ownership can coexist with borrowing. Shared ownership duplicates lifetime participation, not the pointee's value. Weak observation breaks an owning cycle only when it replaces the appropriate strong edge. A smart pointer can enforce a selected policy, but the application must still define mutation rights, retention boundaries, and what failure means.

Primary reference notes

Smart-pointer contracts: [unique pointers](#), [shared pointers](#), and [weak pointers](#).

Chapter 7 — Designing Natural Types with Operator Overloading

An engineering quantity looks like a number, but its accepted range is bounded. Giving it `+`, comparisons, and stream operations can make client code natural—provided those operators preserve the same invariant as the named interface.

Operator overloading should make a type behave predictably, not turn familiar syntax into a surprise. This chapter designs a small value type with bounded arithmetic and transactional stream extraction.

An operator is part of a type’s public language. If `a + b` behaves like arithmetic, readers can use familiar reasoning. If it unexpectedly opens a file, mutates both operands, or performs an unrelated conversion, the short spelling hides more than it clarifies. HarborWorks uses a bounded quantity type to show how natural syntax can preserve domain rules rather than bypass them.

The chapter’s quantity is deliberately small: its permitted values lie between minus one million and one million. Its arithmetic rejects results outside that domain. This is not a currency library, a physical-units system, or a universal overflow wrapper. The narrow domain makes the meaning of equality, ordering, arithmetic, and stream extraction independently testable.

7.1 Overloading Arithmetic and Comparison Operators

An overloaded operator is a function selected through operator syntax. It cannot invent a new operator symbol or change the language’s precedence rules. Its meaning should remain close enough to the conventional operation that a reader can reason about an expression without opening every definition.

For value arithmetic, avoid mutating operands unexpectedly and define how out-of-domain results are reported.

7.1.1 Implementing Value-Type Arithmetic

The quantity accepts integers between minus one million and one million. Adding two quantities first computes a representable intermediate on the supported hosted baseline with at least 32-bit `int`, then constructs the result through the same range-checked constructor. A result outside the quantity domain is rejected even though the machine integer addition itself is representable.

This separates representation overflow from domain overflow. For arbitrary integer operands, the addition would need checks before evaluation to avoid signed overflow. A constructor cannot repair undefined behavior that already occurred while computing its argument. Later generic arithmetic examples make the precheck explicit for full integer ranges.

Begin with the result contract. Adding two quantities should not alter either operand. The result should represent their mathematical sum when that sum is within the type’s accepted range, and it should reject the operation otherwise. These requirements suggest a nonmutating binary operator returning a new

value. A compound assignment such as `+=` would have a different mutation contract and should preserve its target on failure if that is what the type promises.

Check representability before relying on arithmetic in the implementation type. A constructor that rejects an out-of-range result cannot repair signed overflow that already occurred while computing that result. The example restricts both operands and explicitly requires an integer representation large enough for their intermediate sum. A general checked arithmetic type needs bounds checks or a suitable wider representation before the addition, not only validation afterward.

An explicit constructor prevents accidental conversions from unrelated integers at arbitrary call sites. Requiring `Quantity{3}` can make a unit or domain choice visible. This does not mean all implicit conversions are wrong; it means conversions belong to the type's interface and should be justified by the domain rather than included to save a few characters in a demonstration.

Prefer familiar algebraic behavior where the domain permits it. Test identity with zero, symmetry of addition, and rejection at both boundaries. Be careful with algebraic laws that assume closure: a bounded type can reject an intermediate operation even when another grouping would remain representable. Document the bounded domain instead of claiming the type implements unrestricted integer mathematics.

7.1.2 Defining Meaningful Comparisons

The defaulted three-way comparison compares the quantity's value representation and supplies a coherent ordering for this type. Equality is consistent with that value model: two quantities with the same accepted integer are equal.

Do not default comparison blindly for a type containing caches, raw ownership addresses, or fields that are not part of its logical value. Memberwise ordering is only appropriate when member order expresses the intended semantics. Floating-point types can also have partial-order concerns such as NaN, unlike this bounded integer model. Define the ordering relation required by algorithms and containers before selecting an implementation.

Equality asks whether two values mean the same thing under the type's contract. Ordering asks how values should be arranged. Those relations often use the same fields for a simple quantity, but a larger record can have several meaningful orderings. A task's identity might be its ID, while a displayed report sorts by priority and then ID. Defining one global operator for every possible report ordering can make the model misleading.

The defaulted three-way comparison in the quantity delegates to its integer member, giving a natural ordering for the represented value. For a record with several members, a defaulted comparison considers the members in declaration order according to the language rules. That order is a design decision when exposed publicly. Reordering fields for layout should not silently redefine business ordering unless that is intentionally part of the API change.

Floating-point members deserve additional attention because NaNs do not behave like ordinary totally ordered numbers. A sorting comparator must still satisfy the algorithm's requirements over the inputs it actually accepts. Rejecting non-finite values at a domain boundary, using a defined total-order policy, or selecting an appropriate comparison relation can be necessary. Do not assume a spaceship token makes every numeric domain safely sortable.

For HarborWorks, keep report ordering as an explicit projection or comparator when it represents a presentation choice rather than the intrinsic meaning of a task. This reduces the chance that one report's sorting policy will accidentally control a set, map, or another part of the application with different identity needs.

Value-Type Arithmetic for a Money Amount

Purpose. Operator overloading is most effective when the overloaded syntax matches the natural semantics of the domain type.

```
#include <compare>
#include <iostream>
struct Cents{ long value; friend Cents operator+(Cents a,Cents b){return
{a.value+b.value};} auto operator<=>(const Cents&) const = default; };
int main(){ Cents a{125},b{75}; auto c=a+b; std::cout<<c.value<<
'<<std::boolalpha<<(c>Cents{100})<<'\n'; }
```

How it works.

- `operator+` returns a new value rather than mutating either operand.
- Defaulted three-way comparison derives relational operators from the stored value.

Expected behavior/output. The output is ``200 true``.

Experiment. Add subtraction and decide how negative results should be represented.

7.2 Overloading Stream Operators

Stream operators integrate a type with ordinary input and output composition. Insertion should format a value; extraction should parse a candidate and update the destination only when the accepted operation succeeds.

Stream state communicates parsing failure separately from the object being parsed. Preserve both contracts.

7.2.1 Formatting Objects with `operator<<`

`operator<<` accepts the output stream by reference, inserts the numeric representation, and returns the same stream so calls can be chained. It does not add a newline automatically, because line structure belongs to the caller's surrounding format.

A formatter should document whether its output is intended for humans, round-trip serialization, or both. This example's integer form can be read by its extractor, but it does not define a versioned file format. More complex domain types need delimiters, escaping, units, and compatibility rules that a convenient insertion overload does not supply automatically.

An insertion operator should return the stream it received so operations can be chained. Its output should be documented: raw numeric value, units, diagnostic decoration, or a stable interchange record are

different formats. The quantity's operator emits only its integer value. That makes it suitable for simple composition, but it is not a promise of a complete self-describing file format.

The stream already has formatting state. Base selection, precision, fill characters, locale, and error state can affect output. A type that changes persistent formatting settings should decide whether to restore them. Otherwise printing one quantity can change how a later unrelated value is formatted. A diagnostic helper with a local string stream can isolate formatting choices when predictable text matters more than reusing every caller setting.

Insertion can fail because the destination fails. Returning the stream allows the caller to inspect its state or use an exception mask. Do not clear a failed output stream inside a value's insertion operator merely to make the operator appear successful. The operator should cooperate with the stream's error model, leaving publication and retry policy to an appropriate outer boundary.

Test output with an in-memory stream and an exact expected string when the format is part of the contract. Use a separate test for output failure if the application needs to detect it. Comparing only a console screenshot does not verify whitespace, chaining, or error handling reliably.

7.2.2 Reading Objects with `operator>>`

`operator>>` reads into a local candidate. If extraction succeeds but the candidate violates the quantity range, it sets `failbit` and leaves the previous quantity unchanged. If extraction itself fails, the destination is likewise preserved.

The extractor follows formatted stream conventions, so it may accept a numeric prefix and leave trailing characters for later parsing. A complete-record parser must separately require the intended end of input, as Book I's `from_chars` example did. Stream extraction and whole-field validation are different interfaces; do not claim that this operator rejects every trailing suffix by itself.

Extraction should parse into a candidate and commit only after the candidate is acceptable. In the quantity operator, failure to read an integer leaves the target unchanged. A representable integer outside the quantity domain sets `failbit` and also preserves the old value. The existing value remains available for the caller's recovery logic.

This is token extraction, not whole-line validation. Reading an integer from `12ms` can consume a numeric prefix and leave the suffix for a later operation. If the application requires the entire field to be a valid integer, use a parser that checks the full range, as the `from_chars` examples do. The same input can therefore be acceptable as the beginning of a stream token sequence and unacceptable as a complete numeric field. Name the grammar being tested.

Calling `setstate` can itself throw when the stream's exception mask requests it. The operator's preservation rule should still hold because it has not modified the target yet. Do not label extraction unconditionally `noexcept` just because the domain check looks simple. Stream policy participates in its behavior.

Recovery from invalid interactive input is separate from defining the value type. A user interface might clear the stream and discard the rest of a line before prompting again. A batch parser may stop at the first invalid record. The quantity operator should not silently swallow an entire line or choose a retry policy that belongs to its caller.

Streaming a Domain Object Without Exposing Its Representation Logic

Purpose. Stream operators let domain types participate in the standard formatting interface naturally.

```
#include <iostream>
struct Point{ int x,y; };
std::ostream& operator<<(std::ostream& out,const Point& p){ return out << '(' <<
p.x << ',' << p.y << ')'; }
int main(){ std::cout << Point{2,3} << '\n'; }
```

How it works.

- The left operand is the output stream, so the nonmember overload matches normal stream syntax.
- Returning the stream enables chaining with further `<<` operations.
- The point is passed by const reference because formatting should not copy or mutate it.

Expected behavior/output. The program prints `(2,3)`.

Experiment. Chain two points in one output expression to verify the returned stream reference is necessary.

Worked example — Define arithmetic, ordering, and safe extraction for a quantity

Requirement and design

Verify ordinary addition and comparison, reject an out-of-domain sum, read a valid quantity, and reject an invalid one without overwriting the last accepted value. Format the resulting value through an output stream.

The quantity's range is deliberately small enough for its intermediate sum on the documented integer baseline.

Define the operator contract before running it

The valid examples establish that two plus three produces five and two orders before three. The invalid sum checks the upper domain boundary. Extraction begins with a known value, accepts twelve, then rejects one million and one while preserving twelve. Finally insertion produces the exact string 12.

These are deliberately separate observations. An arithmetic operator could be correct while extraction overwrites its target before validation. A comparison could be correct while output changes the caller's formatting state unexpectedly. Avoid one large expression that obscures which operation failed. The shared check harness records the expression and source location for each contract.

The implementation includes a compile-time requirement for an `int` wide enough to represent the constrained intermediate sum. That keeps the arithmetic explanation honest on implementations with smaller integer ranges. The program does not claim to support every imaginable C++ target through accidental assumptions about desktop type widths.

Complete implementation

The complete, buildable chapter project is `source/book2/ch07/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <compare>
#include <limits>
#include <istream>
#include <ostream>
#include <sstream>

static_assert(std::numeric_limits<int>::max() >= 2000000);

class Quantity {
    int value_{};
public:
    explicit Quantity(int value = 0) : value_(value) {
        if (value < -1000000 || value > 1000000) throw std::out_of_range("quantity");
    }
    int value() const noexcept { return value_; }
    auto operator<=>(const Quantity&) const = default;
    friend Quantity operator+(Quantity left, Quantity right) {
        // The constrained operands make the intermediate addition representable.
        return Quantity{left.value_ + right.value_};
    }
    friend std::ostream& operator<<(std::ostream& out, Quantity value) {
        return out << value.value_;
    }
    friend std::istream& operator>>(std::istream& in, Quantity& value) {
        int candidate{};
        if (in >> candidate) {
            if (candidate < -1000000 || candidate > 1000000) {
                in.setstate(std::ios::failbit);
            } else { value.value_ = candidate; }
        }
        return in;
    }
};

int main() {
    CHECK(Quantity{2} + Quantity{3} == Quantity{5});
    CHECK(Quantity{2} < Quantity{3});
    CHECK(course::throws<std::out_of_range>([] {
        (void)(Quantity{1000000} + Quantity{1});
    }));
    Quantity value{7};
    std::stringstream valid{"12"}; valid >> value;
    CHECK(valid && value == Quantity{12});
    std::stringstream invalid{"1000001"}; invalid >> value;
    CHECK(invalid.fail() && value == Quantity{12});
    std::ostringstream output; output << value;
    CHECK(output.str() == "12");
}
```

```
    course::report();  
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch07 -B build/b2-ch07 \  
  -DCMAKE_BUILD_TYPE=Debug  
cmake --build build/b2-ch07 --parallel 2  
ctest --test-dir build/b2-ch07 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch07`.

Evidence and interpretation

Two plus three becomes five, and two compares less than three. One million plus one throws because the resulting quantity is outside its domain. The valid input twelve replaces the previous value seven. Reading one million one sets the input stream’s failure state while the object retains twelve.

The output stream then produces the exact text `12`. This check verifies the formatting contract without forcing line termination into the operator. The failed extraction check verifies both the stream state and preserved object state; checking only one would miss part of the interface.

A friend operator is used where convenient access to private representation makes the implementation clear. Friendship is a narrow access decision, not evidence that every operation should become a member or that encapsulation has no value.

Failure clinic — A correct constructor after an invalid addition

A developer writes a constructor that checks a range and assumes every expression returning that type is now safe. The addition operator performs signed addition before invoking the constructor. For unrestricted operands, overflow can happen before any range validation runs. The constructor protects the object state only after it receives a valid representable candidate; it cannot make an earlier undefined operation well-defined.

A second defect assigns an extracted candidate directly into the object and validates afterward. As in Chapter 2, a thrown exception or failed stream state does not automatically roll back that assignment. The correction is candidate-first parsing and a visible commit.

A third defect treats a successful numeric prefix as a complete record. The stream operator is not wrong if its token contract permits the suffix to remain, but the record parser is wrong if it forgets to reject the suffix. Fix the layer that promised whole-field validation rather than making a generic stream operator unexpectedly consume unrelated input.

Practice and implementation lab

Add subtraction with a documented result-range policy. Test equality, both endpoints, and a result just outside each endpoint. Add extraction fixtures for nonnumeric input and a numeric prefix followed by letters, distinguishing token extraction from complete-record acceptance.

Review a type with a cached field and decide whether defaulted comparison should include it. Replace surprising operator behavior with a named function when the operation no longer resembles ordinary arithmetic.

Review answers and solution notes

A failed extraction should preserve the previous value and leave the stream in a failure state. A successful numeric prefix can leave trailing text, which the enclosing parser must validate when whole-input consumption is required.

A cache normally should not define logical equality merely because it is a member. Bounded domain validation must happen before committing a value, while machine arithmetic overflow must be prevented before the arithmetic expression is evaluated. Operators should preserve conventional expectations and the type's invariant together.

Engineering takeaway. Overload familiar syntax only when the type preserves familiar laws, explicit failure behavior, and a coherent value model.

Guided lab — Add checked compound assignment

Extend the quantity with `operator+=`. Compute a complete accepted result before replacing the current value, then return a reference to the current object for conventional chaining. Test normal accumulation, adding zero, lower-bound rejection, upper-bound rejection, and self-addition within a safe domain. Compare the target before and after every rejected operation.

Add stream tests for leading whitespace, a malformed first character, an out-of-range integer, and trailing nonnumeric text. Write down which cases are failures of token extraction and which require a separate whole-field parser. Enable stream exceptions in one test and verify the target still retains its old value when extraction reports a domain failure through an exception.

For an interface exercise, define two task orderings: priority then ID, and title then ID. Keep them as named comparators or projections rather than attempting to encode both in one `operator<`. Verify deterministic tie-breaking in each report.

Review and solution reasoning

Operators should preserve conventional meaning and the type's invariant. Constructor validation does not retroactively prevent arithmetic overflow. Parsing into a temporary provides preservation only if mutation waits until success. A stream operator participates in a larger input/output grammar; it should not pretend that token extraction and whole-record validation are the same operation.

Primary reference notes

Operator and stream rules: [overloaded operators](#), [three-way comparison](#), [stream extraction](#), and [stream insertion](#).

Chapter 8 — Reusing Behavior Through Inheritance

A sensor is a specialized device, but the team begins using inheritance simply to reuse a few utility functions. Soon derived classes depend on protected fields and construction order that were never part of a stable contract.

Inheritance creates a structural and semantic relationship, not just a shortcut for sharing code. This chapter examines a small base/derived pair and makes construction, accessibility, and destruction order visible before introducing runtime polymorphism.

Inheritance creates a relationship between types, not merely a shortcut for reusing a few lines. HarborWorks models a sensor with a small base component to expose construction and destruction order. Before introducing virtual dispatch, this chapter asks a simpler question: what promises does a derived object make when used through the base interface, and which parts exist at each stage of its lifetime?

The example's base is intentionally non-polymorphic and is never used for deletion through a base pointer. That restriction is part of the teaching contract. Chapter 9 introduces owning base pointers with virtual destruction. Keeping the two cases separate prevents a beginner from assuming that every base class automatically supports every kind of polymorphic use.

8.1 Defining Base and Derived Classes

Public inheritance expresses that a derived object can be used through an appropriate base interface. That substitution should remain meaningful: derived behavior must respect the promises clients rely on when they know only the base.

When the relationship is merely “uses a helper” or “contains a component,” composition is usually the clearer model. Chapter 10 develops that alternative.

8.1.1 Establishing Is-A Relationships

A **Sensor** publicly derives from **Device** in the example. Its base subobject stores a calibration scale and records lifetime events. The derived class adds sensor-specific behavior that uses the scale.

An is-a claim is a design judgment supported by behavioral compatibility, not just grammar. A derived class that refuses ordinary base operations or weakens their safety guarantees can violate clients' expectations even though the compiler accepts the hierarchy. Keep base interfaces small and meaningful rather than using inheritance as an unrestricted bucket of reusable implementation.

An is-a relationship should support substitution under the base interface's stated contract. A sensor can be a device when every operation exposed as a device remains meaningful for a sensor. Inheriting from a class solely to obtain storage or helper methods does not establish that relationship. Composition is often clearer when the intended statement is “this object uses a device configuration” rather than “this object is a device.”

Substitution includes failure and state rules. If a base operation promises to accept every nonnegative sample, a derived implementation that rejects half those samples without a documented reason narrows the contract. If the base promises a read-only query, a derived implementation that changes external state may surprise callers even though it has the same function signature. The compiler checks type compatibility, not every behavioral promise.

Public inheritance exposes a derived-to-base relationship to callers. Private inheritance instead restricts that relationship and is more often an implementation technique. This book uses public inheritance for explicit interfaces and composition for ordinary has-a relationships; it does not require a class hierarchy merely because several classes share an integer or string member.

A useful design test is to write a small function that knows only the base interface. Can it operate on every proposed derived type without special-case downcasts or name checks? Frequent questions such as “which derived type is this really?” can indicate that the base abstraction is missing an operation or that the hierarchy is modeling unrelated concepts.

8.1.2 Controlling Member Accessibility

Public, protected, and private access determine which code can name members through the relevant language rules. The scale is protected in this teaching example so the derived sensor can use it directly. That choice visibly couples the derived class to the base representation.

In a production design, a protected query or composed calibration object may preserve more freedom than a protected writable field. Derived code should not casually bypass an invariant established by the base. Private inheritance and protected inheritance have different interface implications; they are not needed merely to demonstrate basic specialization here.

Access control is about which code can name and use members. A derived class can access protected base members under the language’s rules, while ordinary callers cannot access them directly. That does not mean protected mutable data is well encapsulated. Every derived class that can write a protected field participates in maintaining the base’s invariant.

Suppose the base stores a positive calibration scale. Making the scale protected allows a derived class to assign zero or a negative value unless all derived code follows the rule. A protected operation that validates changes can preserve a narrower interface. Better still, a private immutable value can eliminate mutation when the application does not need it. Choose access based on required behavior, not on the idea that derived classes deserve unrestricted access to every detail.

A public getter can expose a representation without allowing mutation, but it can also make callers depend on that representation. Decide whether the useful public concept is a raw scale or an operation such as `calibrate`. An interface that exposes only the required behavior gives the implementation more freedom to change later.

The chapter’s simple scale member is visible to demonstrate inheritance mechanics. Its arithmetic is tested only for small representable inputs. Turning it into a general calibration API would require numeric range rules and a stronger invariant. A teaching hierarchy should not quietly become a production sensor library merely by acquiring a class name.

Reusing a Base Interface in a Derived Type

Purpose. Inheritance establishes an “is-a” relationship at the type level and composes the base subobject into every derived object.

```
#include <iostream>
#include <string>
struct Device{ std::string id; explicit Device(std::string x):id(std::move(x))
{} };
struct Sensor:Device{ double value; Sensor(std::string id,double
v):Device(std::move(id)),value(v){} };
int main(){ Sensor s{"T1",21.5}; std::cout<<s.id<<' '<<s.value<<'\n'; }
```

How it works.

- `Sensor` contains the `Device` base subobject.
- The derived constructor explicitly initializes the base before its own member.
- Public base state is accessible through the derived object.

Expected behavior/output. The program prints `T1 21.5`.

Experiment. Add a second derived type and identify which state truly belongs in the base.

8.2 Understanding Construction Through Inheritance Hierarchies

A derived object includes its base subobjects. Those bases must be constructed before the derived constructor body uses them, and they remain until after the derived destructor body finishes. Understanding that order prevents calling behavior that assumes a not-yet-ready or already-ended derived state.

Lifetime order is a language rule, not the order in which a developer happened to type log statements.

8.2.1 Base-Class Construction Order

The sensor constructor explicitly initializes its device base before initializing its own members. The log therefore records base construction before derived construction. A base without a suitable default constructor must receive appropriate constructor arguments from the derived constructor.

Do not call virtual behavior from a base constructor expecting dispatch into a fully initialized derived object; construction-time dispatch has special rules because the derived portion is not yet established. The current chapter deliberately avoids virtual dispatch so the structural order can be understood separately.

Base subobjects are initialized before the derived class’s own members and constructor body. Non-static data members are initialized in their declaration order, not the order in which they happen to appear in an initializer list. The order matters when one member’s initialization depends on another or when a later stage can throw.

For the sensor, the base constructor records its event first. The derived constructor records its event after the base has completed. At that point, base state is available for the derived initialization. A base constructor must not assume the final derived object's complete state already exists. Virtual calls during construction follow special rules tied to the currently constructing class; Chapter 9 explains why relying on derived overrides from base construction is a common design error.

Failure also follows the completed-subobject boundary. If the derived constructor throws, the already constructed base and members are destroyed. The destructor of the incomplete derived object does not run. A resource that needs cleanup should therefore live in a completed owning member or base subobject, not depend on a cleanup body that construction never reaches.

Use instrumentation that does not change the property being studied. A log vector that grows during destruction can allocate and potentially throw. The supplied example reserves sufficient capacity before creating the object, and the log entries are string views of static literals. The intended experiment records order without introducing dynamic string creation in cleanup.

8.2.2 Derived-Class Destruction Order

When the local sensor leaves scope, its derived destructor body runs first, followed by its member and base destruction in the required order. The log records derived destruction and then base destruction.

The base in this example is non-polymorphic and has no virtual destructor. Deleting a derived object through an owning base pointer would be invalid for this design, so the example never does that. Chapter 9 provides a virtual destructor for a polymorphic ownership interface. Do not copy the nonvirtual destructor into a base that clients will use for polymorphic deletion.

Destruction reverses the completed object's composition: the derived destructor body runs, then its members are destroyed in reverse declaration order, then base subobjects are destroyed in the appropriate reverse order. This lets outer behavior release its own responsibilities before the underlying base machinery disappears. Borrowed services used during destruction must still be alive at that point.

The logging example deliberately creates the log before the sensor and destroys the sensor before the log. If those lifetimes were reversed through retained references, recording destruction would use a dead observer. A lifetime trace must include instrumentation and dependencies, not only the object the chapter is nominally about.

Deleting a derived object through a base pointer requires an appropriate destruction design. For an ordinary polymorphic base intended for public deletion, a virtual destructor is the usual solution. A non-polymorphic base can instead prohibit that use, for example through a protected destructor when suitable. The example simply never performs base-pointer deletion; it should not be copied into a factory returning owning base pointers without revisiting the interface.

Avoid deriving from standard-library containers merely to add unrelated behavior. Their interfaces and destruction policies are not a promise that they are intended as your runtime-polymorphic base classes. A wrapper containing a vector can expose selected operations without making every container operation part of the new domain type's contract.

Tracing Base-Before-Derived Construction Order

Purpose. Construction order is determined by object structure, not by the textual order of statements in the derived constructor body.

```
#include <iostream>
struct Base{ Base(){std::cout<<"Base construct\n";} ~Base(){std::cout<<"Base
destroy\n";} };
struct Derived:Base{ Derived(){std::cout<<"Derived construct\n";} ~Derived()
{std::cout<<"Derived destroy\n";} };
int main(){ Derived d; }
```

How it works.

- Base construction completes before the derived constructor body runs.
- Destruction happens in the reverse order: derived first, then base.

Expected behavior/output. The four messages show base construct, derived construct, derived destroy, base destroy.

Experiment. Add a member object with trace messages and observe where member construction fits relative to the base and derived body.

Worked example — Trace base and derived lifetimes without hidden allocation in cleanup

Requirement and design

Construct a sensor using scale two, verify that a raw sample eight produces sixteen, and record the base/derived construction and destruction sequence. Keep the object local and never delete it through a base pointer.

The event log reserves four slots in advance and stores string views to string literals, avoiding allocation during these four destructor-log operations. The calibration arithmetic is restricted to the small representable fixture values.

Trace the complete lifetime, including the observer

The log is created first and reserves four entries. Constructing the sensor records base construction and then derived construction. A small calibration calculation verifies that base state is accessible through the derived behavior. Leaving the scope records derived destruction followed by base destruction. The log remains alive for the final assertions.

The sequence is the result being tested. The program does not depend on a particular memory offset for the base subobject or a compiler-specific object layout. It also does not perform dynamic dispatch or

delete through a base pointer. Those omissions are deliberate boundaries that keep this example about composition of lifetimes rather than several mechanisms at once.

When extending the example, count how many log entries will be needed before construction begins. A new member with its own lifecycle observation changes the required capacity. Instrumentation should not create a hidden allocation path inside a destructor while claiming to prove nonthrowing cleanup.

Complete implementation

The complete, buildable chapter project is `source/book2/ch08/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <string_view>
#include <vector>

class Device {
    std::vector<std::string_view> log_;
protected:
    int scale_;
public:
    Device(std::vector<std::string_view>& log, int scale) : log_(log), scale_(scale)
    {
        log_.push_back("base constructed");
    }
    ~Device() { log_.push_back("base destroyed"); }
    // No deletion through Device*: this is a non-polymorphic teaching base.
};

class Sensor : public Device {
    std::vector<std::string_view> log_;
public:
    Sensor(std::vector<std::string_view>& log, int scale) : Device(log, scale),
    log_(log) {
        log_.push_back("derived constructed");
    }
    ~Sensor() { log_.push_back("derived destroyed"); }
    int calibrated(int raw) const { return raw * scale_; }
};

int main() {
    std::vector<std::string_view> log;
    log.reserve(4); // Avoid allocating while recording these destructor events.
    {
        Sensor sensor{log, 2};
        CHECK(sensor.calibrated(8) == 16);
    }
    CHECK(log.size() == 4);
    CHECK(log[0] == "base constructed" && log[1] == "derived constructed");
    CHECK(log[2] == "derived destroyed" && log[3] == "base destroyed");
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch08 -B build/b2-ch08 \  
-DCMAKE_BUILD_TYPE=Debug  
cmake --build build/b2-ch08 --parallel 2  
ctest --test-dir build/b2-ch08 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch08`.

Evidence and interpretation

The expected sequence is base constructed, derived constructed, derived destroyed, base destroyed. The test checks all four events after the sensor has left scope. The log itself remains alive in the enclosing scope, satisfying the borrowed-reference lifetime requirement.

The source’s reserve call is relevant to cleanup safety: pushing into an already sufficient vector of string views avoids a new allocation for these events. General destructor logging through arbitrary strings or I/O could fail and needs a different policy. This teaching instrument is intentionally bounded.

The calibrated result demonstrates access to the protected scale, while the lifetime trace demonstrates the base subobject relationship. Neither test establishes a complete substitutability proof for a production hierarchy; that requires examining the base’s behavioral contracts.

Failure clinic — The initializer list that looked ordered

A class lists one member before another in its initializer list, but declares them in the opposite order. The programmer assumes the visual list controls initialization and reads a member that has not yet been initialized. A warning can reveal the mismatch, but the durable repair is to arrange declarations and dependencies intentionally, then write the initializer list in that same order for clarity.

Another defect stores a reference to a temporary observer in a base. The sensor’s own scope looks correct, but its cleanup log is already dead. The right question is not “does the sensor still exist?” but “does every borrowed dependency used during its lifetime still exist?” Composition and inheritance both create such dependencies.

Finally, a programmer adds a factory returning `unique_ptr<Device>` to this non-polymorphic example. Ownership is now explicit, but destruction through the base is not designed for that use. The smart pointer does not add a virtual destructor. Chapter 9 supplies the appropriate interface pattern rather than treating this as a minor syntactic conversion.

Practice and implementation lab

Add a second derived member with an observable constructor and destructor, and predict its position in the trace before running. Keep all observer storage alive and preallocated.

Refactor the scale into a private calibration component exposed through a query. Compare the derived class's dependence on representation before and after the change. Explain why an owning **Device*** pointing to a sensor would require a different destructor contract.

Review answers and solution notes

Bases are initialized before the derived portion is ready, and base destruction follows the derived destructor body. Member declaration order also matters regardless of initializer-list presentation.

A nonvirtual base destructor is not suitable for deleting a derived object through that base type. Protected representation increases coupling; composition or a narrow query can reduce it. Inheritance should describe a meaningful substitution relationship rather than merely save a few lines of duplicated code.

Engineering takeaway. Use inheritance for a defensible type relationship and understand subobject lifetime before relying on derived behavior.

Guided lab — Add a member with a visible lifetime

Add a small member object that records its construction and destruction without allocating during cleanup. Reserve all observation capacity before creating the sensor. Predict the full event order on paper, then compare the recorded sequence. Explain where the derived constructor body fits relative to member initialization and where the derived destructor body fits relative to member destruction.

Introduce a distinct controlled exception after the base and one member are complete. Verify that completed subobjects clean up and that the incomplete derived destructor body does not appear in the log. Do not catch a general test-failure exception as though it were the intentional failure. Keep the injection type separate from the harness's assertion type.

For a design exercise, rewrite the sensor to contain a **Calibration** value rather than inherit from a storage-only base. Compare the public interfaces. Which relationship better states the domain? Which version exposes fewer mutation paths? No universal answer is required; the acceptance criterion is a reasoned contract and a correct lifetime trace.

Review and solution reasoning

Inheritance supplies a type relationship and subobject structure, not behavioral correctness automatically. Declaration order controls member initialization. Completed subobjects clean up when later construction fails. A borrowed observer must outlive the events it records. Base-pointer ownership requires a destruction policy designed for that use, not merely a modern-looking smart pointer.

Primary reference notes

Inheritance and lifetime: [base classes](#), [base/member initialization](#), [destructors](#), and [construction-time behavior](#).

Chapter 9 — Designing Runtime Polymorphism

The report pipeline needs several filters selected at runtime. One accepts values above a threshold; another accepts even values. The caller should apply either filter without a chain of type tests or knowledge of its private representation.

Runtime polymorphism provides a stable interface with behavior selected by the actual object. Ownership, destruction, and interface contracts remain separate obligations that must be designed alongside dispatch.

A report can filter tasks in several ways: by minimum priority, by completion state, or by a policy loaded from configuration. Runtime polymorphism lets the application use a stable interface while choosing an implementation during execution. It is valuable when the set of active behaviors is not fully fixed at the call site, but it also creates lifetime and substitution obligations.

HarborWorks begins with two integer filters so the dispatch mechanism stays small enough to inspect. The example does not make every class virtual. It isolates one variation point, owns the implementations explicitly, and calls them through a base interface whose destructor supports that ownership path.

9.1 Using Virtual Functions

A virtual member function supports dispatch through a base-class interface to the appropriate overriding function for the dynamic object. This allows one caller to work with several implementations whose concrete types may not be known at compile time.

Use it where runtime substitution is a genuine requirement. Compile-time generic code and simple function objects can be better choices when the set of behavior is known statically.

9.1.1 Overriding Base-Class Behavior

The `Filter` interface declares `accepts(int) const`. `AtLeast` and `Even` implement the same contract with different predicates. Marking each derived implementation `override` asks the compiler to verify that it actually overrides a base virtual function.

A missing `const` qualifier or mismatched parameter can accidentally declare a different function instead of overriding the intended one. The `override` specifier turns that maintenance mistake into a diagnostic. `final` can prohibit further overriding or derivation where that restriction is part of the design, rather than being added merely as an unexplained optimization hint.

An `override` must match the base operation's relevant signature, including `const` qualification and reference qualifiers where present. The `override` specifier asks the compiler to reject a declaration that does not actually override a base virtual function. This prevents a subtle typo from silently introducing a different function while the old base behavior remains active through base references.

Overriding and overloading are different. A derived function with the same name but different parameters can hide base overloads rather than replace the intended virtual function. A `using` declaration can reintroduce hidden overloads when that is the design, but it does not convert an unrelated signature into an override. Start by inspecting the base declaration and the exact call expression.

Behavior matters after the signature matches. A filter's `accepts` query should apply the documented policy without taking ownership of the input or unexpectedly modifying the filter's externally meaningful state. A derived class can legally contain additional state, but callers should not need to know its representation to use the base interface correctly.

Marking a class or operation `final` can express that further overriding is not part of the design. It can also help an optimizer in some contexts, but do not add it as a performance claim without evidence. Its first role is an interface restriction: future derived implementations are intentionally prohibited at that point.

9.1.2 Understanding Dynamic Dispatch

Calling `accepts` through a `Filter*` or reference dispatches according to the actual derived object. Implementations commonly use virtual tables, but the language contract does not require one fixed object layout or forbid devirtualization when the optimizer knows the target.

Dispatch is not ownership. A borrowed base reference does not keep the object alive, and an owning base pointer needs a correct destruction interface. Do not measure one indirect call and assume it describes the complete cost of the design; allocation patterns, data locality, and work performed by the predicate can dominate.

Dynamic dispatch selects behavior according to the dynamic type of the object when a virtual function is called through an appropriate expression. The static type still determines which interface is available and participates in overload resolution. A `Filter&` exposes the filter interface even when the actual object is an `Even` filter. It does not automatically expose every member of the derived class.

Copying a derived object into a base object by value slices away the derived portion. The resulting base object is not a hidden polymorphic container that remembers the original derived type. Use references or owning pointers when the application needs polymorphic identity, or define a deliberate value-polymorphism mechanism if independent copying of heterogeneous behavior is required.

Typical implementations use indirect dispatch machinery, often described in terms of virtual tables. That is a useful implementation model, not a required portable memory layout to serialize or manipulate manually. The observable language contract is which override is selected and how lifetime behaves. Optimize the actual hot path later if indirect calls matter to the workload.

Virtual calls made during construction and destruction do not behave as calls on a fully initialized most-derived object at every stage. A base constructor should not rely on a derived override that needs derived members not yet initialized. Prefer explicit post-construction operations or composition when initialization would otherwise depend on such a call. This avoids making object validity depend on incomplete state.

Runtime Dispatch Through a Base Reference

Purpose. Virtual functions support substitution when callers know only the base interface but behavior depends on the dynamic type.

```
#include <iostream>
struct Shape{ virtual ~Shape()=default; virtual double area() const=0; };
struct Square:Shape{ double side; explicit Square(double s):side(s){} double
area() const override{return side*side;} };
void print_area(const Shape& s){ std::cout<<s.area()<<'\\n'; }
int main(){ Square s{4}; print_area(s); }
```

How it works.

- The base declares a virtual operation.
- `override` verifies that the derived signature actually overrides the base function.
- Calling through `Shape&` dispatches to `Square::area` at runtime.

Expected behavior/output. The program prints `16`.

Experiment. Add a `Circle` implementation and pass both shapes through the same `print\_area` function.

9.2 Designing Abstract Interfaces

An abstract interface expresses behavior without requiring a directly constructible base object. A pure virtual function marks a required operation whose implementation must be supplied for a concrete instantiable derived type.

Keep the contract precise enough that callers can substitute implementations without asking which class they received.

9.2.1 Declaring Pure Virtual Functions

`virtual bool accepts(int value) const = 0;` makes the acceptance operation pure virtual. The abstract base cannot be instantiated directly as an ordinary complete object, while derived classes become concrete when they implement the required interface.

The contract here is a read-only predicate over one integer. It does not perform I/O or mutate an externally visible acceptance history. A new filter requiring hidden global state would weaken local reasoning even if its signature matched. Semantic compatibility matters beyond type compatibility.

An abstract interface can state a small set of required operations without supplying a concrete object for the application to instantiate directly. A pure virtual `accepts(int) const` says that every concrete filter must provide the query. The base can still have a constructor, nonvirtual helpers, and data, but adding them expands the contract and its coupling.

Design the smallest useful interface. If a client only asks whether a sample is accepted, it should not also require every filter to expose a filename, network connection, mutable vector, and reset protocol. Broad interfaces make test implementations harder and force unrelated behaviors into every derived class. A narrow interface gives each implementation room to manage its own representation.

Document exceptional behavior as part of the interface. Can a query throw? Does it allocate? Can it retain input? Is it safe to call repeatedly? The compiler does not infer these domain promises from the word `virtual`. A later implementation that introduces a network request inside an apparently cheap predicate can dramatically change a report pipeline even while satisfying the function signature.

A nonvirtual public operation that validates inputs and delegates one protected virtual step can sometimes centralize common rules. That pattern is useful when all implementations must share an invariant. It is not required for the simple filters here, whose contract is intentionally small. Avoid adding a framework before the application has an actual family of policies to support.

9.2.2 Using Virtual Destructors Safely

The interface has `virtual ~Filter() = default;`, permitting correct destruction when an owning base pointer refers to a derived object. The vector stores `std::unique_ptr<Filter>`, so clearing it deletes through the base interface and must invoke the appropriate derived destruction.

Avoid object slicing: storing a derived object by value in a base-valued collection would keep only the base portion, and an abstract base cannot serve that purpose at all. Polymorphic objects are commonly accessed through references or pointers whose owners are explicit. Copying a polymorphic value, when needed, requires a deliberate cloning or value-wrapper design rather than an accidental base copy.

An owning pointer to a base must destroy the complete dynamic object correctly. A public virtual destructor makes destruction through the base interface dispatch appropriately to the derived object and its members. The base destructor can be defaulted when it has no custom cleanup; its virtual nature is the important interface property.

A pure virtual destructor still needs a definition because destruction of a derived object also destroys its base subobject. Declaring it pure can keep a class abstract, but it does not remove the base destruction step. The chapter uses an ordinary virtual defaulted destructor because abstraction is already supplied by the pure virtual query, making the intended mechanism clearer.

Destructors should not be used to report ordinary recoverable business errors. A filter may release its owned resources during destruction, but a request that must flush data or commit work should expose a checked operation before cleanup. The RAII distinction from Chapter 3 applies equally inside a polymorphic hierarchy.

An interface not intended for deletion through the base can use a different policy, such as protected nonvirtual destruction, when appropriate. The key is to make the intended use explicit. The supplied vector of `unique_ptr<Filter>` is explicitly designed for public polymorphic destruction, so a public virtual destructor is the right match for that example.

Abstract Storage Interface with a Concrete In-Memory Implementation

Purpose. An abstract interface should describe the stable capability consumers require while allowing implementation details to vary.

```
#include <iostream>
#include <string>
#include <unordered_map>
struct Store{ virtual ~Store()=default; virtual void put(std::string,int)=0;
virtual int get(const std::string&) const=0; };
struct MemoryStore:Store{ std::unordered_map<std::string,int> m; void
put(std::string k,int v) override{m[std::move(k)]=v;} int get(const std::string&
k) const override{return m.at(k);} };
int main(){ MemoryStore s; s.put("x",7); std::cout<<s.get("x")<<'\\n'; }
```

How it works.

- The interface contains behavior but no concrete storage representation.
- The implementation chooses an `unordered_map` privately.
- Callers depending on `Store` can be tested with alternate implementations.

Expected behavior/output. The program prints `7`.

Experiment. Create a second implementation backed by `std::map` without changing interface consumers.

Worked example — Apply interchangeable filters through exclusive base ownership

Requirement and design

Own a threshold filter and an even-number filter in one vector of unique pointers to the abstract base. Apply both to eleven and twelve, then clear the vector through the base ownership interface.

The threshold is ten. The two filters share only the interface contract, not a common implementation of their predicates.

Separate dispatch from ownership

The vector owns unique pointers, and each pointer owns one concrete filter. The call through `filters[0]` reaches the minimum-value policy; the call through `filters[1]` reaches the parity policy. The application does not downcast to identify which class it received. It depends only on the `accepts` contract.

Clearing the vector releases the owning pointers and destroys the complete filters through their virtual base destructors. The test checks that the vector is empty afterward. A deeper cleanup test can add a

resource-audit member to each concrete implementation and verify that all acquired resources are released. That would test lifetime separately from the query's Boolean answers.

Do not confuse the container's emptiness with proof that every conceivable external resource was released. The existing concrete filters own only small values. When a real implementation acquires a file or device, add evidence for that resource's release rather than assuming the old test covers a new responsibility.

Complete implementation

The complete, buildable chapter project is `source/book2/ch09/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <memory>
#include <vector>

class Filter {
public:
    virtual ~Filter() = default;
    virtual bool accepts(int value) const = 0;
};
class AtLeast final : public Filter {
    int minimum_;
public:
    explicit AtLeast(int minimum) : minimum_(minimum) {}
    bool accepts(int value) const override { return value >= minimum_; }
};
class Even final : public Filter {
public:
    bool accepts(int value) const override { return value % 2 == 0; }
};

int main() {
    std::vector<std::unique_ptr<Filter>> filters;
    filters.push_back(std::make_unique<AtLeast>(10));
    filters.push_back(std::make_unique<Even>());
    CHECK(filters[0]->accepts(11));
    CHECK(!filters[1]->accepts(11));
    CHECK(filters[0]->accepts(12) && filters[1]->accepts(12));
    filters.clear(); // Virtual destruction through owning base pointers.
    CHECK(filters.empty());
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch09 -B build/b2-ch09 \
  -DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build/b2-ch09 --parallel 2
ctest --test-dir build/b2-ch09 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch09`.

Evidence and interpretation

Eleven passes the threshold filter but fails the even filter. Twelve passes both. The caller invokes the same virtual operation through each base pointer without a type switch. Clearing the vector empties the owner collection and destroys each concrete filter through the virtual destructor.

The tests establish the selected predicates for representative boundary-adjacent inputs. They do not assume a particular virtual-table layout, allocation address, or number of indirect machine instructions. Those are implementation questions for later profiling.

Unique ownership keeps lifetime simple: the vector owns each filter, and the calls borrow access while those owners remain alive. No shared pointer is needed merely because multiple calls use the same object sequentially.

Failure clinic — The almost-matching override

A derived class writes `bool accepts(int)` while the base declares `bool accepts(int) const`. Without `override`, the code can introduce a different member and leave the class abstract or preserve an unexpected base implementation in a different hierarchy. With `override`, the compiler points directly at the mismatch. The correction is to match the intended interface, not to remove `override` to silence the diagnostic.

Another defect stores derived values in a `vector<Base>`. In a concrete base hierarchy this can slice each value, losing derived behavior. A vector of owning base pointers preserves the dynamic objects but changes the copy and allocation model. A variant can be an alternative when the set of implementations is closed and known at compile time. Chapter 19 develops that tradeoff rather than presenting inheritance as the only way to represent alternatives.

A third defect calls a virtual hook from a base constructor expecting the final derived policy. Move the operation to a phase where the object is complete, or redesign the dependency. Constructor order is not an implementation inconvenience to bypass; it defines which state can safely be used.

Practice and implementation lab

Add a filter accepting values in a closed interval and test both endpoints, just-outside values, and a negative interval. State what construction does when the lower endpoint exceeds the upper endpoint.

Create a composition that requires all filters to accept without adding a type switch. Then compare a runtime polymorphic interface with a templated function accepting a predicate when the behavior is known at compile time.

Review answers and solution notes

An interval filter should reject invalid construction or explicitly normalize the interval according to a documented policy. An all-filters operation can iterate through the base interface and short-circuit on the first rejection.

Override checking catches signature mismatches, but not a semantic violation of the predicate contract. A virtual destructor supports polymorphic deletion; a virtual operation alone does not automatically make a separately declared nonvirtual destructor safe. Runtime dispatch and lifetime ownership are related design concerns, not the same mechanism.

Engineering takeaway. Design polymorphic behavior and polymorphic destruction together, while keeping the caller independent of concrete types.

Guided lab — Add a range filter and test substitution

Implement an inclusive interval filter with validated lower and upper bounds. Its constructor should reject inverted bounds without creating a usable invalid filter. Test values below, at, inside, and above the interval. Run the same base-interface test function against the minimum, parity, and interval policies where their contracts permit.

Add a composite filter that owns several filters and accepts a value only when every component accepts it. Define the empty-composite policy explicitly. Short-circuiting evaluation may be appropriate, but do not claim that every component is called if the implementation stops at the first rejection. Use an observation fixture only when call ordering itself is part of the contract.

For cleanup testing, give one concrete filter a noncopyable resource-audit member and destroy it through `unique_ptr<Filter>`. Verify release through the registry rather than through console messages in a destructor. Keep all registry owners alive until after the polymorphic owners disappear.

Review and solution reasoning

Runtime polymorphism selects behavior through an interface; it does not define copying, ownership, or error policy automatically. `override` catches signature mistakes. Base-value copying can slice. Virtual destruction is required by the chosen owning-base-pointer use. A small interface with explicit behavior is easier to substitute and test than a large base class that exposes every implementation detail.

Primary reference notes

Polymorphism rules: [virtual functions](#), [abstract classes](#), [destructors](#), and [unique pointers](#).

Chapter 10 — Favoring Composition and Interface-Based Design

A reporter inherits from a file writer only because it needs to send formatted text somewhere. Tests now create files, production behavior depends on inherited configuration, and a future network destination would require another hierarchy branch. The relationship is not “a reporter is a file writer”; it is “a reporter uses an output destination.”

Composition makes that relationship explicit. This chapter combines an owned formatter with a borrowed sink interface and tests the reporter entirely in memory.

The reporter needs two collaborators: a formatter that turns a sample into text and a sink that receives the text. Neither relationship says the reporter is a formatter or is a sink. Composition expresses those has-a and uses-a relationships directly. It also lets the example give the formatter owned value semantics while borrowing the sink from a longer-lived caller.

The important design work is deciding who owns each component and which failures can cross the interface. A diagram of boxes is not enough. The reporting operation allocates formatted text, invokes the sink, and may fail at either stage. Its contract should not imply external publication guarantees that its collaborators do not provide.

10.1 Building Objects from Smaller Components

Composition builds behavior from member objects and delegated operations. Each component owns a focused responsibility: formatting converts a value into text, while a sink accepts the text. The reporter coordinates them without becoming a subclass of either implementation.

This often reduces coupling because components can change independently while their small contracts remain stable.

10.1.1 Modeling Has-A Relationships

A reporter has a formatter and uses a sink. The formatter is stored by value, so its prefix belongs to the reporter’s owned state. The sink is stored by reference, so the caller retains ownership and must keep it alive for the reporter’s use.

These two relationships are intentionally different. Converting both members to pointers would not automatically improve flexibility; it could conceal which one owns its target. The member declarations should help a reviewer draw the lifetime graph before reading the function bodies.

Composition puts smaller objects inside a larger object or connects them through explicit references and owners. A reporter can own a formatter as a value because it needs an independent formatting configuration. It can borrow a sink when the caller controls a destination that outlives the reporter. It could instead own a unique sink if the reporter itself should control that destination’s lifetime. Those are separate interface choices.

A has-a relationship does not automatically mean dynamic allocation. A member value has its lifetime tied to the enclosing object and can often avoid an extra allocation or indirection. Use a pointer when optionality, polymorphic identity, transfer, or a separate lifetime actually requires one. The smallest adequate ownership model is often easier to inspect than a graph of heap allocations created by habit.

Member declaration order becomes a design tool when components depend on one another. If a member borrows another member during its lifetime, arrange construction and destruction so the owner exists before the borrower is constructed and remains alive until after the borrower is destroyed. Better still, avoid persistent internal borrows when a call-time reference can express the relationship more simply.

Copying a composed object follows the semantics of each member. Copying the reporter copies its formatter value but preserves the reference member's observation of the same sink. That can be intentional, but it is not an independent copy of the destination. A class containing references may also have implicitly deleted assignment operations. Review these consequences before describing the reporter as a freely copyable value.

10.1.2 Delegating Responsibilities

The reporter's `sample` operation asks the formatter to create text and then asks the sink to write it. It does not duplicate numeric formatting or inspect a sink's concrete storage. Delegation keeps the coordinating function small while the actual responsibilities remain visible.

A failed formatting or sink operation can propagate through this simple interface. A production version should specify retry, partial-output, and error-reporting semantics according to the destination. Do not add blanket exception swallowing merely to make a void interface appear infallible. Small interfaces still need honest failure contracts.

Delegation means giving a collaborator a well-defined part of the work. The reporter asks the formatter for a string and then asks the sink to write it. The formatter does not choose the destination, and the sink does not decide how a sample should be represented. This division makes each component's behavior smaller and gives tests a clear place to observe results.

Do not split a function into objects merely to maximize the number of classes. A useful boundary corresponds to an independent policy, dependency, or invariant. A formatting rule that changes separately from a destination is a reasonable boundary. A wrapper with one method that merely forwards an integer to another wrapper may add indirection without providing a meaningful contract.

Delegation also determines failure boundaries. If formatting throws before the sink is called, no write should occur through this reporter invocation. If the sink fails after partially publishing text, the reporter cannot undo the destination's external state simply by catching the exception. It may report failure, retry under an explicit idempotency policy, or use a different destination protocol. The local object model does not automatically supply an external transaction.

For the memory sink in the example, successful writes append strings to a vector. That gives a controlled observable destination for tests. It is not intended to model every failure mode of a terminal, file, network socket, or database. Use a fake for the contract being exercised and a separate integration test for real external behavior.

Composition of Engine and Dashboard Components

Purpose. Composition builds a larger object from parts whose responsibilities remain separately testable and understandable.

```
#include <iostream>
struct Engine{ bool running=false; void start(){running=true;} };
struct Dashboard{ void show(bool running)
const{std::cout<<(running?"running":"stopped")<<'\\n';} };
struct Car{ Engine engine; Dashboard dash; void start()
{engine.start();dash.show(engine.running);} };
int main(){ Car c; c.start(); }
```

How it works.

- `Car` owns an `Engine` and a `Dashboard` as distinct subobjects.
- The higher-level operation coordinates their behaviors rather than inheriting from them.

Expected behavior/output. The program prints `running`.

Experiment. Add a `stop()` operation without changing either component's ownership relationship.

10.2 Reducing Coupling Through Interfaces

An interface should expose the operation the client needs, not the implementation's full configuration surface. A sink accepts text; it does not force every reporter to know about filenames, sockets, or test vectors.

Dependency injection means supplying such a dependency from outside rather than constructing a hidden concrete implementation internally.

10.2.1 Programming Against Abstractions

The abstract `Sink` declares a virtual `write` operation and a virtual destructor. `MemorySink` implements it by appending to a vector. The reporter depends on `Sink&`, so the same reporter can later use a file or network adapter without changing its formatting logic.

The in-memory sink is a fake with simple real behavior, not a guarantee that every external adapter behaves identically under failure. Tests can verify the reporter's contract quickly, while integration tests must verify actual file or network boundaries separately. Substitution helps isolation; it does not eliminate the need to test real dependencies.

Programming against an abstraction means that the caller depends on the required behavior, not on one concrete implementation's representation. The sink interface exposes writing a string. A reporter should not downcast the sink to a memory sink to inspect its vector during ordinary operation. The test can own a concrete memory sink and inspect its results from outside the reporter while the reporter itself remains decoupled.

Interface-based design works at compile time too. A template parameter can require a suitable callable or sink operation without a virtual base. Runtime polymorphism is useful when implementations are selected and stored through a stable runtime interface. Templates are useful when concrete types are known at instantiation and static composition fits the program. Neither mechanism is universally more modern than the other.

Keep error policy coherent across implementations. If one sink throws on failure and another silently drops text, substituting them changes the reporter's meaning. The interface should say whether a successful return means the text was accepted, buffered, or durably stored. Those are increasingly strong claims. A memory sink can support acceptance into its vector; it does not establish durability.

A dependency passed explicitly can be replaced in a test without editing hidden globals. That makes error paths reachable. It also makes lifetime obligations visible at construction. A global destination singleton may feel convenient, but it hides both the dependency and the order in which it becomes available or is destroyed.

10.2.2 Replacing Inheritance with Composition

Replacing inheritance with composition removes the false is-a relationship. The reporter no longer inherits unrelated destination methods or relies on protected destination state. Its constructor states exactly which components it requires.

Composition also makes ownership choices explicit. A borrowed sink is suitable when an enclosing scope manages several reporters and the destination. An exclusively owned sink may be appropriate when the reporter must determine its lifetime. A shared sink owner is appropriate only when several independent owners genuinely participate in lifetime. Select among those policies deliberately rather than treating one smart-pointer type as a universal interface.

Inheritance is often overused when the real goal is policy reuse. A reporter derived from a formatter exposes a type relationship that callers probably do not need. A reporter containing a formatter says exactly what is needed and can delegate formatting without inheriting unrelated operations. Changing the formatter's internal data does not automatically change the reporter's base-class representation contract.

Composition also permits independent replacement. The same reporter can use a different formatter value and a different sink implementation without creating a new subclass for every combination. A hierarchy with subclasses for "CSV-to-file," "CSV-to-memory," "text-to-file," and "text-to-memory" multiplies classes when two independent choices would be clearer.

Do not replace all inheritance mechanically. A well-defined runtime interface with genuine substitutable implementations can be an excellent design, as Chapter 9 showed. The aim is to avoid encoding every reuse relationship as public inheritance. Ask which relationship the application actually needs: value containment, temporary borrowing, exclusive ownership, shared lifetime, or runtime substitution.

When refactoring an existing hierarchy into composition, preserve behavior with tests at the public boundary. Keep expected output, failure reporting, and lifetime semantics stable unless the change intentionally revises the contract. A design improvement that silently drops errors or retains borrowed objects longer is not merely an internal cleanup.

Injecting a Logging Interface Instead of a Concrete Logger

Purpose. Depending on a narrow interface reduces coupling and enables testing or replacement without editing the service logic.

```
#include <iostream>
#include <string_view>
struct Logger{ virtual ~Logger()=default; virtual void
write(std::string_view)=0; };
struct ConsoleLogger:Logger{ void write(std::string_view s)
override{std::cout<<s<<'\\n';} };
struct Service{ Logger& log; void run(){log.write("service running");} };
int main(){ ConsoleLogger l; Service s{l}; s.run(); }
```

How it works.

- `Service` stores a reference to the capability it needs, not a concrete output mechanism.
- The concrete logger is supplied from outside, so the service is not responsible for constructing it.

Expected behavior/output. The program prints `service running`.

Experiment. Create a test logger that records messages in a vector and verify `Service::run` without console output.

Worked example — Test a composed reporter without external I/O

Requirement and design

Construct a reporter with the prefix `sample=` and an in-memory sink. Submit twelve and fourteen, then verify that the sink received exactly two messages in order.

The sink is declared before the reporter and remains alive until after the reporter is destroyed. The formatter is owned by the reporter, while the sink is borrowed.

Identify the owned value and the borrowed collaborator

The caller creates the memory sink first. The reporter stores its own formatter and a reference to that sink. Two sample calls format the numbers and append the resulting strings. The caller then inspects the sink's vector. The sink remains alive throughout the reporter's lifetime, so the reference is valid for every call.

The simplest alternative is a reporter that writes directly to `std::cout`. That can be useful in a tiny program, but it makes exact output tests and destination replacement harder. The explicit sink isolates the dependency without requiring the reporter to understand file paths, terminals, or network connections.

The test asserts the two complete strings and their order. It does not merely assert that “something was written.” Exact output is a useful contract for this small formatter. If later formatting becomes locale-dependent or configurable, update the requirements and test environment deliberately rather than treating every difference as a mysterious regression.

Complete implementation

The complete, buildable chapter project is `source/book2/ch10/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <string>
#include <vector>
#include <utility>

struct Sink {
    virtual ~Sink() = default;
    virtual void write(const std::string& text) = 0;
};
class MemorySink final : public Sink {
public:
    std::vector<std::string> messages;
    void write(const std::string& text) override { messages.push_back(text); }
};
class Formatter {
    std::string prefix_;
public:
    explicit Formatter(std::string prefix) : prefix_(std::move(prefix)) {}
    std::string format(int value) const { return prefix_ + std::to_string(value); }
};
class Reporter {
    Formatter formatter_; // Owned value.
    Sink& sink_;          // Borrowed; the caller must outlive this reporter.
public:
    Reporter(Formatter formatter, Sink& sink)
        : formatter_(std::move(formatter)), sink_(sink) {}
    void sample(int value) { sink_.write(formatter_.format(value)); }
};

int main() {
    MemorySink sink;
    Reporter reporter{Formatter{"sample="}, sink};
    reporter.sample(12);
    reporter.sample(14);
    CHECK(sink.messages == std::vector<std::string>({"sample=12", "sample=14"}));
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch10 -B build/b2-ch10 \  
-DCMAKE_BUILD_TYPE=Debug  
cmake --build build/b2-ch10 --parallel 2  
ctest --test-dir build/b2-ch10 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch10`.

Evidence and interpretation

The expected messages are `sample=12` and `sample=14`. The test compares the entire vector, so it checks ordering, count, and content rather than merely searching for one substring. No file, network connection, or global output stream is needed.

The formatter's prefix is moved into its owned member, and the reporter takes its formatter by value before moving it into place. The sink reference is never reseated and never deleted by the reporter. These choices give a compact lifetime model that can be verified from local declarations.

The memory sink appends strings, so allocation can still fail. This is not a hard real-time logging design. Later volumes examine bounded buffering and allocation policies; the present goal is clean responsibility separation and testable coordination.

Failure clinic — A borrowed sink escaped its owner

A helper creates a local memory sink, constructs a reporter that references it, and returns the reporter. The reporter's formatter survives by value, but its sink reference dangles as soon as the helper returns. Copying or moving the reporter does not extend the sink's lifetime. The repair is to move destination ownership into the returned object, return both components in a lifetime-safe arrangement, or require a caller-owned sink that lives long enough.

Another defect copies the reporter expecting independent output destinations. Its reference member still designates the same sink. Both copies write into the same vector. That may be a useful shared destination policy, but the documentation and tests should say so. A reference member is not a hidden deep copy of its referent.

A third defect catches every sink exception and reports success to the caller. This preserves control flow but destroys the error contract. Decide whether failure should propagate, return a typed result, or trigger a documented retry. Silencing the evidence is not a substitute for handling the operation.

Practice and implementation lab

Add a sink that counts messages without storing them and reuse the same reporter. Add a fake sink that throws on its second call, then document what the reporter promises after that failure.

Change the reporter to own a unique sink pointer in a separate version. Compare construction and destruction responsibilities with the borrowed-reference version. Explain why this is an ownership-policy change rather than merely a syntax refactor.

Review answers and solution notes

The counting sink tests coordination without retaining every message. A throwing sink demonstrates that external failure remains possible; the current reporter propagates it and does not promise rollback of a message already accepted by the destination.

Owning a unique sink makes the reporter responsible for its lifetime, while borrowing a sink requires an enclosing owner. Composition eliminates a false inheritance relationship and makes both dependencies visible. The right abstraction is the smallest stable contract the caller actually needs.

Engineering takeaway. Compose focused components, inject external behavior, and make each dependency's ownership and failure policy explicit.

Guided lab — Exercise a failing collaborator

Add a sink that accepts a configured number of writes and then throws a distinct simulated destination error before accepting another message. Verify that the reporter does not claim a successful write after rejection. Inspect the messages accepted before the failure and distinguish them from messages merely formatted in memory.

Add a formatter policy that rejects a chosen input before constructing output. Confirm that the sink receives no call for that input. This separates formatting failure from destination failure. Keep the fake's call counter and message storage owned by the test, and ensure they outlive the reporter.

Then redesign the reporter to own a unique sink. Compare construction, transfer, and copy behavior with the borrowing version. Neither version is automatically superior. The borrowing version supports a caller-managed destination; the owning version lets the reporter control destination lifetime. Write the intended use case for each.

Review and solution reasoning

Composition clarifies independent responsibilities, but each member still has a lifetime and copy policy. Explicit dependencies make failures testable. A fake sink demonstrates the reporter's response to a controlled collaborator, not the operating system's I/O behavior. Local formatting and external publication are separate boundaries, and a correct interface names the strength of the success it promises.

Primary reference notes

Composition mechanisms: [class members](#), [virtual interfaces](#), [references](#), and [unique ownership](#).

Chapter 11 — Creating Generic Functions with Templates

The project contains three copies of a “choose the larger value” function: one for integers, one for floating-point values, and one for strings. A template can share the algorithm, but a naive pointer instantiation compares addresses instead of text. Reuse requires a semantic contract, not just code that compiles for several spellings of a type.

This chapter creates generic functions, follows type deduction, and uses an overload to give C strings the intended content-based behavior.

HarborWorks has several functions that select, clamp, or combine values. Copying the same algorithm for `int`, `long`, and `double` creates several implementations to maintain. A function template can express one family of implementations while preserving concrete types at each instantiation. The important step is identifying the operations and semantic assumptions shared by that family.

A template is not a request for the compiler to make arbitrary types behave sensibly. If the implementation compares two values, the comparison must exist and mean what the algorithm expects. If it returns a value, the selected type must support the needed construction. Generic code is strongest when its assumptions are explicit enough that both the compiler and a human reviewer can examine them.

11.1 Defining Function Templates

A function template describes a family of functions parameterized by types or other template arguments. The compiler forms suitable specializations when the program uses them. The body still has requirements: every selected operation must be meaningful for the chosen type.

Write those requirements down even before expressing them through concepts. An unconstrained template is not an algorithm that works for every conceivable type.

11.1.1 Declaring Type Parameters

`template<class T> T larger(T left, T right)` uses one type parameter for both inputs and the result. Its implementation requires a usable ordering expression and a suitable value return. The parameters are values, so expensive types may incur copies according to the call and optimization context.

Returning by value avoids borrowing from a temporary argument. A reference-returning version might reduce copying but could dangle when called with temporaries or retained beyond the argument lifetime. Generic code should not trade a visible copy cost for a hidden lifetime defect. Choose the return relationship before optimizing it.

A type parameter stands for a type supplied explicitly or deduced from a call. In `template<class T> T larger(T left, T right)`, each instantiation has concrete parameter and return types.

The compiler checks the body under those concrete types when required. This is unlike a dynamically typed function that discovers every operation at runtime.

The implementation's syntax creates requirements. Copying parameters by value requires suitable initialization. Comparing them requires a valid `<` expression. Returning one requires the corresponding return construction. A string and an integer can both satisfy those operations while producing useful but different kinds of ordering. A file owner may be noncopyable and not meaningfully ordered, making it inappropriate for this interface even if it is otherwise a well-designed type.

Definition visibility matters. A caller instantiating a template usually needs access to its definition, so small function templates commonly live in headers. Merely putting a declaration in a header and an arbitrary definition in an unrelated source file does not provide all future instantiations automatically. Explicit instantiation can support a controlled set of types, but it is an intentional build and interface policy rather than an accidental workaround for missing symbols.

Write the template after identifying a useful common algorithm. Do not start by replacing every concrete type with `T`. Some differences belong to the domain: an integer count and a filesystem path may both be comparable, but combining them under an overly broad abstraction can erase the meaning a caller needs to understand.

11.1.2 Understanding Template Argument Deduction

For `larger(3, 9)`, deduction can select `T` as `int`. For two doubles it can select `double`. Mixed arguments such as an integer and a double do not automatically establish one agreed `T` under this simple signature; an explicit conversion or a different multi-parameter design is needed.

Deduction does not inspect the function's intended business meaning. It follows language rules for the parameter pattern and arguments. Read a deduction failure from the call outward: identify each candidate type, the conflicting positions, and whether a conversion should really be permitted. Broadening every parameter until a call compiles can weaken useful type distinctions.

Template argument deduction uses the function's parameter pattern and the call's argument types. It does not generally search for every conversion that would make two different argument types fit one `T`. A call like `larger(3, 4.5)` therefore needs a deliberate policy rather than assuming deduction will choose the most convenient common type.

One policy is to require matching types and ask the caller to convert explicitly. Another is to introduce two type parameters and choose a result through `std::common_type_t`, provided the domain and conversion rules justify that choice. A third is to return a selected reference rather than a value, which creates a different lifetime contract. None of these is universally correct; the public operation should state what it means to compare heterogeneous inputs.

By-value deduction also changes some types through the language's parameter adjustments. Arrays and function arguments can decay to pointer-like forms. That matters for string literals: comparing deduced C-string pointers is not the same operation as lexicographically comparing text. The supplied non-template overload constructs strings so the comparison uses textual values rather than accidental address ordering.

Reference parameters preserve different information and introduce different binding rules. A forwarding reference in a deduced context can preserve a caller's value category when used with `std::forward`, but forwarding is a separate design technique, not a reason to make every parameter T&. Start with a clear borrowing or owning contract. The later chapters build generic interfaces without requiring unexplained perfect-forwarding machinery everywhere.

One `maximum` Algorithm for Multiple Ordered Types

Purpose. Function templates express one algorithm over a family of types that satisfy the required operations.

```
#include <iostream>
#include <string>
template<class T> T maximum(T a,T b){ return b<a?a:b; }
int main(){ std::cout<<maximum(3,7)<<'
'<<maximum(std::string{"Ada"},std::string{"Grace"})<<'\\n'; }
```

How it works.

- The template parameter represents a type chosen at instantiation.
- The function body requires the selected type to support comparison with `<`.
- Separate specializations are generated as needed for `int` and `std::string`.

Expected behavior/output. The output is `7 Grace`.

Experiment. Call with two `double` values, then with mismatched `int` and `double` arguments and inspect deduction behavior.

11.2 Controlling Generic Function Behavior

A generic implementation can coexist with overloads for special cases whose semantics differ. Constraints can also restrict which types are eligible. These are interface-design tools, not merely ways to shorten compiler diagnostics.

Use an overload when the desired operation is genuinely different and a constraint when the generic operation has a meaningful domain.

11.2.1 Overloading Templates

The C-string overload accepts two non-null pointers to valid null-terminated strings, constructs owning strings, and delegates to the string-value comparison. It avoids accidentally choosing a pointer-based ordering for text. The null and termination requirements remain caller preconditions in this teaching overload.

Overload resolution considers candidates according to language rules; it does not always choose the implementation a programmer informally calls “more specific.” Keep overload sets small enough to

reason about and test representative argument categories. Distinguish ordinary overloading from explicit template specialization, which is introduced alongside class templates next.

An overload set can contain ordinary functions and templates. Overload resolution chooses among viable candidates using the language's ranking rules; it does not simply prefer the template because it looks more general. A non-template overload can handle a special domain case, as the C-string overload does, while the template supplies the regular typed-value path.

Overloads should remain understandable from a call site. If two nearly identical templates differ through obscure return-type substitutions, diagnostics can become difficult to interpret. C++20 constraints often express the intended distinction more directly. Chapter 13 shows how a named concept can make a type requirement visible rather than hiding it in a distant substitution error.

Avoid surprising overload semantics. If **larger** means ordinary value ordering for integers and strings, an overload that selects the shorter C string would violate the operation's apparent meaning. Special handling should adapt representation to the same contract or use a different name when the semantics genuinely differ.

Test each overload family explicitly. String literals, string objects, matching arithmetic types, and rejected mixed types exercise different paths. A single integer test cannot establish that text comparison uses content. Compile-time checks can verify accepted signatures, while runtime tests verify their meaning. Keep expected compile failures separate from the normal build so the delivered source remains buildable.

11.2.2 Constraining Acceptable Types

The clamp template uses a type trait and `static_assert` to limit this lesson's implementation to arithmetic types. It rejects inverted bounds at runtime. For floating-point inputs, its comparison logic assumes an ordered finite domain; NaN requires an additional policy because ordinary comparisons do not establish the expected interval relationships.

A static assertion inside a template body is not the same as a C++20 constraint participating in candidate selection. It can still yield a useful diagnostic after instantiation, but concepts express eligibility more directly. Chapter 13 develops that distinction and combines type-level requirements with value-level overflow checks.

The original clamp template uses an arithmetic type trait to reject unrelated types. That trait includes more types than some applications should accept, including Boolean and floating-point types. The body also requires sensible ordering of the bounds and values. A type-level restriction can reject a string, but it cannot prove that a runtime floating-point input is finite or that the lower bound is not greater than the upper bound.

Separate structural requirements from value requirements. The compiler can check that a type is an integer or supports a comparison expression. Runtime code checks actual bounds and input values. Tests then examine semantic properties such as the output lying within accepted bounds and an already in-range value remaining unchanged. Generic safety emerges from all three layers rather than from a single trait.

Be precise about rejected cases. The supplied example throws for inverted bounds. It is not advertised as a NaN-normalizing floating-point utility. A production clamp for a specific numeric domain should either

reject non-finite values explicitly, define their treatment, or restrict the accepted types and inputs so the ambiguity cannot arise.

The supplemental constrained-sum laboratory takes a narrower approach: integral values other than `bool`, checked representability, and a documented result type. Narrowness is useful when it captures the real application. “Works for every type that compiles” is a weak contract compared with “accepts these types and preserves these stated properties.”

Branching at Compile Time with ``if constexpr``

Purpose. Compile-time branching allows a generic function to adapt implementation details without requiring invalid branches to compile for every type.

```
#include <iostream>
#include <type_traits>
template<class T> void describe(const T&){ if constexpr(std::is_integral_v<T>)
std::cout<<"integral\n"; else std::cout<<"other\n"; }
int main(){ describe(3); describe(3.5); }
```

How it works.

- ``is_integral_v<T>`` is evaluated during template instantiation.
- ``if constexpr`` discards the nonselected branch for that type.

Expected behavior/output. The output is ``integral`` then ``other``.

Experiment. Add a string-specific branch using ``std::same_as<T,std::string>``.

Worked example — Generalize comparison without confusing text and addresses

Requirement and design

Use one template for integer and floating-point comparison, provide content ordering for C strings, and implement a clamp with explicit type and interval requirements. Test low, high, interior, and inverted-bound cases.

The fixed numeric fixtures are finite and representable. The C-string literals satisfy the overload’s non-null, null-terminated precondition.

Trace one instantiation and one overload

For the integer call, deduce `T` as `int`, substitute it into the parameters and return type, then evaluate the comparison. For the string-literal call, the ordinary C-string overload constructs string values and invokes the template on those values. The two paths have different representation work but the same high-level intent: return the larger value under the selected ordering.

The clamp checks cover below, inside, and above the interval, plus inverted bounds. Those four categories are more informative than several arbitrary in-range values. The accepted result should lie at the nearest boundary or remain unchanged. The inverted-bound case verifies a precondition check rather than a valid numeric result.

A template body can remain unexamined until an instantiation requires it. Keep small positive compile checks for the type families the library intends to support. Do not assume that a template declaration appearing in a header proves every advertised instantiation has been compiled.

Complete implementation

The complete, buildable chapter project is `source/book2/ch11/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <string>
#include <type_traits>

template<class T>
T larger(T left, T right) { return left < right ? right : left; }

// Give C strings content ordering rather than accidental pointer ordering.
std::string larger(const char* left, const char* right) {
    return larger(std::string{left}, std::string{right});
}

template<class T>
T clamp_value(T value, T low, T high) {
    static_assert(std::is_arithmetic_v<T>, "clamp_value requires arithmetic");
    if (high < low) throw std::invalid_argument("inverted bounds");
    return value < low ? low : (high < value ? high : value);
}

int main() {
    CHECK(larger(3, 9) == 9);
    CHECK(larger(2.5, 1.5) == 2.5);
    CHECK(larger("alpha", "beta") == "beta");
    CHECK(clamp_value(12, 0, 10) == 10);
    CHECK(clamp_value(-1, 0, 10) == 0);
    CHECK(clamp_value(5, 0, 10) == 5);
    CHECK(course::throws<std::invalid_argument>([] { (void)clamp_value(1, 10, 0); }));
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch11 -B build/b2-ch11 \
  -DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build/b2-ch11 --parallel 2
ctest --test-dir build/b2-ch11 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch11`.

Evidence and interpretation

The integer comparison returns nine, the double comparison returns 2.5, and the C-string call returns the owning string `beta`. Clamp maps twelve to ten, minus one to zero, and five to itself. Inverted bounds throw before a clamped value is returned.

The text overload is important because a syntactically available pointer comparison would not express lexical content ordering. Generic correctness has both syntactic and semantic layers: a compiler can verify that an expression exists without proving that it means what the application intends.

The result is returned by value, so callers can safely use a result formed from temporary string conversions. A borrowed result would require a different lifetime contract. This is a recurring theme: generic reuse must preserve ownership clarity as well as reduce duplication.

Failure clinic — Generic syntax, wrong semantics

The naïve text template compares two C-string pointer values. It compiles for many calls, but it does not express lexicographic ordering of the characters. The repair is to adapt the representation to a text value or view with the intended comparison semantics. Adding more template parameters does not solve the domain mismatch.

Another failure appears at link time because a template definition was placed in a source file without providing the needed explicit instantiation. A declaration allows the caller to form a call, but the complete program still needs the selected specialization’s definition. Either make the definition available where instantiation occurs or intentionally manage a fixed set of instantiations in the build.

A third failure is a test that equates “type accepted” with “all values safe.” A comparison-capable floating-point type can still contain NaN. An integral type can still overflow in an unchecked arithmetic body. Concepts and traits improve interfaces, but runtime domain boundaries remain real work.

Practice and implementation lab

Add tests for equal values and strings with a shared prefix. Attempt a mixed integer/double call in a separate compile exercise and explain the deduction conflict before choosing an explicit conversion policy.

Constrain clamp with a C++20 concept in a later revision. Add an explicit NaN policy for floating-point use or restrict the function to integers. Test null C-string input only after replacing the precondition with a checked interface; do not deliberately pass invalid pointers to the existing function.

Review answers and solution notes

Equal ordered values can return either equal value under this function’s value contract, while a reference-returning design might need a tie-identity policy. Mixed argument types require an intentional common-type or conversion decision.

A type trait does not prove runtime values satisfy an interval or arithmetic bound. C strings need content semantics rather than pointer ordering. Returning an owning string avoids a borrowed reference into a temporary conversion. Generalization is successful only when the algorithm’s requirements remain explicit.

Engineering takeaway. Generic code must preserve semantic meaning, type requirements, and lifetime—not merely accept more argument types.

Guided lab — Specify a generic selection contract

Design a `larger_value` function for a deliberately chosen set of value types. Decide how ties are handled, whether mixed numeric types are accepted, and whether the result is owned or borrowed. Implement the simplest interface supporting those choices and test integers, strings, equal values, and an explicitly rejected type.

Then design a checked clamp for integral types excluding `bool`. Test both bounds, inverted bounds, negative intervals for signed types, and the largest representable values without performing overflowing arithmetic in the test itself. Compare a trait-based restriction with the concept-based version developed in Chapter 13.

For the build exercise, place a template declaration and definition in separate files and predict the missing-instantiation failure in a disposable branch. Restore the complete header definition and verify the normal project. Record whether the problem was translation, linking, or runtime behavior; those categories need different fixes.

Review and solution reasoning

A template describes a family of concrete implementations. Deduction is governed by parameter patterns, not arbitrary conversion wishes. A valid expression can still have the wrong semantics, especially for pointer-based text. Type requirements, runtime validation, and behavioral tests cooperate; none of them proves the others automatically.

Primary reference notes

Template mechanics: [templates](#), [template argument deduction](#), [overload resolution](#), and [type traits](#).

Chapter 12 — Creating Reusable Generic Classes

A two-by-three integer matrix and a four-by-four floating-point matrix share the same indexing idea, but their shapes and element types differ. A class template can express those differences as parameters while keeping bounds checks and storage mapping in one implementation.

This chapter introduces type and non-type template parameters, then contrasts full and partial specialization with a small type-category example.

A reusable data structure needs a representation policy as well as a type parameter. HarborWorks uses a small fixed matrix to expose both a stored element type and dimensions known at compile time. The template can reject impossible shapes during compilation, while its checked accessor rejects an invalid coordinate during execution.

This division is useful because not all errors become knowable at the same stage. A zero dimension supplied as a template argument is a program-construction issue. A row number read from a user is runtime data. A robust class template makes each boundary explicit rather than expecting compile-time machinery to validate information it never sees.

12.1 Defining Class Templates

A class template defines a family of types. Each selected argument combination identifies a specialization with its own type identity. A matrix of one shape is not interchangeable with another shape merely because both have a member named `at`.

This can make compile-time structure visible in interfaces and prevent some mismatched-shape operations before runtime.

12.1.1 Parameterizing Data Types

The type parameter `T` determines the element type stored by the matrix. Its operations still impose requirements: the current representation value-initializes its elements and returns mutable references from checked access. Not every possible type supports every desired matrix operation.

Do not promise a fully generic numerical library from a storage template alone. Addition, multiplication, overflow policy, layout, and expression costs require further contracts. The chapter implements bounded storage access only, keeping the abstraction small enough that its guarantees can be inspected completely.

A class template defines a family of distinct types. `Matrix<int, 2, 3>` and `Matrix<double, 2, 3>` have different element types; `Matrix<int, 3, 2>` has a different shape from the first. These distinctions can prevent passing incompatible structures to operations that require a particular layout, but they can also increase the number of instantiations a build must manage.

Each instantiated class has its own member types and operations. A member function that requires addition of elements should state that requirement where it matters rather than making every matrix operation depend on addition. A storage-only matrix might be useful for a non-arithmetic record type even if a future numeric multiplication operation is not available for it.

Keep representation ownership simple. The fixed matrix contains a `std::array` whose size is the product of the dimensions. Its members manage their own lifetimes, so the class needs no custom destructor for storage. This is the Rule of Zero applied to a generic representation rather than to one concrete application record.

Returning a mutable element reference grants authority to replace that element. That is appropriate for the simple storage matrix, whose invariant concerns shape rather than cross-element numeric relationships. A matrix type that promises normalized rows or a fixed determinant would need a narrower mutation interface. Templates do not remove the same encapsulation questions encountered in Chapter 2.

12.1.2 Using Multiple Template Parameters

The non-type parameters `Rows` and `Columns` determine the fixed shape. Compile-time checks reject zero dimensions in this particular design and reject a product that would overflow `std::size_t`. A different design could intentionally support zero-sized matrices, but that must be a chosen contract.

The storage is `std::array<T, Rows * Columns>`, with row-major indexing `row * Columns + column`. Runtime access checks both coordinates before calculating the element position. The compile-time product check and runtime coordinate checks solve different problems: representable storage size and valid access within that shape.

Type parameters and non-type template parameters describe different aspects of an interface. `T` selects the element type. `Rows` and `Columns` select constant dimensions. The compiler can use those dimensions to calculate storage and reject invalid shapes, but a large shape can still produce an object too large for the intended storage context. A valid template argument is not a capacity-planning decision.

The product `Rows * Columns` must be representable before it is used as a size. The example checks the division-based condition before relying on the multiplication. This is the same general principle used in checked runtime arithmetic: do not perform an overflowing operation first and ask whether its result was valid afterward.

Multiple parameters should each have a clear role. Adding allocator, comparison, ownership, synchronization, and logging policies to a tiny matrix can make the type harder to use than the repeated code it replaces. Introduce a parameter when there is a real variation point and a coherent contract for each accepted policy.

Default template arguments can make common cases convenient while preserving explicit alternatives. They also become part of the public interface. A later change to a default can alter which specialization or representation a caller gets. Treat public defaults as versioned design decisions, not as invisible implementation details.

Fixed-Capacity Generic Buffer

Purpose. Class templates let a reusable data abstraction vary by type and compile-time policy while retaining full static typing.

```
#include <array>
#include <cstdint>
#include <iostream>
template<class T, std::size_t N> struct Buffer{ std::array<T, N> data{}; constexpr
std::size_t size() const{return N;} };
int main(){ Buffer<int, 4> b{{1, 2, 3, 4}}; std::cout<<b.size()<<'
'<<b.data[2]<<'\\n'; }
```

How it works.

- The element type and capacity are template parameters.
- Each distinct ``T, N`` pair defines a distinct concrete class type.
- ``std::array`` supplies fixed storage matching the compile-time capacity.

Expected behavior/output. The output is `4 3`.

Experiment. Instantiate ``Buffer<double, 2>`` and compare its type and size.

12.2 Customizing Templates

Specialization customizes a template for particular argument patterns. Full specialization selects one exact argument combination. Partial specialization selects a family of matching combinations and is available for class templates rather than ordinary function templates in the same form.

Use specialization when the abstraction's implementation genuinely differs, not to scatter unrelated behavior under one template name.

12.2.1 Applying Full Specialization

The primary `Category<T>` reports `value`. A full specialization for `bool` reports `boolean`. No type parameter remains in that specialization's selected argument; it addresses that exact type.

Specializations should preserve a coherent public interface so generic users can still ask the same question. If one specialization silently removes a key member or changes its meaning, callers may become harder to reason about than with separate named types. Header placement and one-definition requirements also remain relevant when specializations are shared across translation units.

A full specialization supplies a distinct implementation for one complete set of template arguments. The category example gives `bool` a specific label rather than using the primary template's general value label. This is a small, controlled demonstration of selection; it does not imply that Boolean values should be treated as ordinary numeric quantities in every generic algorithm.

A specialization is not merely a patch to one line of the primary template. It defines the specialized class's interface and members under the language rules. A careless specialization can therefore remove operations that callers expected from the primary template. Keep a common public contract when callers are intended to use all specializations uniformly.

Specialization placement matters. The relevant declaration must be visible before uses that would otherwise instantiate the primary template in ways affected by it. Do not scatter specializations into arbitrary source files and rely on link order to make them retroactively apply. Template selection is a compile-time matter, not a runtime registry.

Specializing your own templates is different from adding declarations to the standard namespace. The standard library permits only particular customization forms under specified rules. This book's examples specialize their own `Category` template and avoid presenting unrestricted specialization of standard-library internals as a normal extension technique.

12.2.2 Applying Partial Specialization

The partial specialization `Category<T*>` matches pointer types and reports `pointer`. The parameter `T` still varies, so it covers pointers to many pointee types. This is distinct from fully specializing one exact pointer type such as `int*`.

Patterns must be understood carefully: references, top-level `const` qualification, and nested types may not match a pattern the way a casual description suggests. Add compile-time checks for the actual categories the interface intends to support. For function customization, overloads or constrained templates are often clearer than trying to reproduce class partial-specialization techniques.

A partial specialization matches a pattern rather than one exact type. `Category<T*>` identifies pointer types while leaving the pointed-to type as a parameter. The primary template still handles ordinary values, and the full `bool` specialization handles its exact case. This lets the compiler choose among a family of related patterns.

Patterns can overlap. More elaborate partial specializations may become ambiguous when neither is more specialized under the language's ordering rules. A template library should keep its specialization structure understandable and test representative combinations, including `const`-qualified and nested forms where they are advertised.

Function templates do not support partial specialization in the same way class templates do. Function overloads and constraints are often the appropriate mechanisms for those cases. Confusing the two models can lead to surprising syntax and lookup behavior. Use class specialization when customizing a class family and overloads or constrained functions when selecting function behavior.

A trait class is a useful application because its interface can remain tiny and stable: a constant, a type alias, or a Boolean property. That makes specialization review manageable. A full application class with radically different member sets in every specialization is much harder to substitute safely. Prefer the smallest customization surface that represents the intended variation.

Partial Specialization for Pointer Formatting

Purpose. Template specialization allows a generic abstraction to provide different behavior for a well-defined subset of types.

```
#include <iostream>
template<class T> struct Kind{ static const char* name(){return "value";} };
template<class T> struct Kind<T*>{ static const char* name(){return
"pointer";} };
int main(){ std::cout<<Kind<int>::name()<<' '<<Kind<int*>::name()<<'\n'; }
```

How it works.

- The primary template handles general value types.
- The partial specialization matches any pointer type `T*`.
- Selection occurs at compile time from the template argument form.

Expected behavior/output. The output is ``value pointer``.

Experiment. Add a specialization for ``const T*`` and decide whether it should report a distinct category.

Worked example — Build a fixed-shape matrix and specialize a type category

Requirement and design

Create a two-by-three integer matrix, modify its last element, confirm zero initialization elsewhere, and reject an out-of-range row. Verify the category labels for an ordinary integer, a boolean, and an integer pointer at compile time.

The matrix is a fixed storage model with checked access, not a complete linear-algebra package.

Trace a coordinate into contiguous storage

For a two-row, three-column matrix, row one and column two map to offset $1 * 3 + 2$, which is five. The six-element array's valid offsets are zero through five. The accessor checks row and column bounds before performing the indexing calculation. A row equal to the row count is invalid even when an accidentally computed flat offset might fall within some other allocation.

The first check proves a written element can be read back. The zero-valued untouched element verifies value initialization for the integer array. The invalid-row test checks the accessor's rejection path. Compile-time assertions then verify which category specialization is selected for an integer, a Boolean, and a pointer.

Those tests cover different layers: shape at compile time, coordinate validity at runtime, initialization, and specialization selection. Keep them separate in the explanation so a learner can identify which mechanism is responsible for each outcome.

Complete implementation

The complete, buildable chapter project is `source/book2/ch12/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <array>
#include <cstdint>
#include <string_view>

template<class T, std::size_t Rows, std::size_t Columns>
class Matrix {
    static_assert(Rows > 0 && Columns > 0);
    static_assert(Columns == 0 || Rows <= static_cast<std::size_t>(-1) / Columns);
    std::array<T, Rows * Columns> data_{};
public:
    T& at(std::size_t row, std::size_t column) {
        if (row >= Rows || column >= Columns) throw std::out_of_range("matrix");
        return data_[row * Columns + column];
    }
};

template<class T> struct Category {
    static constexpr std::string_view name{"value"};
};

template<> struct Category<bool> {
    static constexpr std::string_view name{"boolean"};
};

template<class T> struct Category<T*> {
    static constexpr std::string_view name{"pointer"};
};

int main() {
    Matrix<int, 2, 3> matrix;
    matrix.at(1, 2) = 7;
    CHECK(matrix.at(1, 2) == 7 && matrix.at(0, 0) == 0);
    CHECK(course::throws<std::out_of_range>([&] { (void)matrix.at(2, 0); }));
    static_assert(Category<int>::name == "value");
    static_assert(Category<bool>::name == "boolean");
    static_assert(Category<int*>::name == "pointer");
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch12 -B build/b2-ch12 \
  -DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build/b2-ch12 --parallel 2
ctest --test-dir build/b2-ch12 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch12`.

Evidence and interpretation

Writing row one, column two updates the sixth element of the flat storage to seven. Row zero, column zero remains zero. Requesting row two fails because the valid row indices are zero and one, even though some flat arithmetic expression could still be formed.

The static assertions verify specialization selection without adding runtime branches. `Category<int>` uses the primary template, `Category<bool>` uses the full specialization, and `Category<int*>` uses the partial specialization.

The shape checks deliberately exclude a product too large for the indexing type. This does not guarantee that every representable matrix size can actually fit available memory or implementation object-size limits. Compile-time representability and resource feasibility remain separate constraints.

Failure clinic — One unchecked multiplication

A matrix template computes its storage size with an unchecked product of dimensions. On a target where that product wraps in the size type, the array can be smaller than the programmer intended. Later indexing then operates against the wrong representation. The repair is a representability constraint before multiplication, plus an application-level policy limiting impractically large shapes.

Another defect validates only the flat index. An invalid coordinate such as column equal to the column count can accidentally refer to the next row. The index is within the array but the requested coordinate is outside the matrix's contract. Validate the semantic coordinates, not only the final storage address.

A third defect adds a specialized class that lacks an accessor available on the primary template. Some instantiations compile and others fail far from the specialization. Maintain a consistent interface when the family is intended to be used uniformly, and add compile checks for every supported specialization category.

Practice and implementation lab

Add a `const at` overload and a read-only row/column count interface. Test the last valid coordinate and both independently invalid coordinate dimensions. Attempt a zero shape in a separate negative-compilation test.

Add compile-time checks for a pointer to const integer and a const pointer to integer, and explain why their pattern matching differs. Do not broaden the category template until the desired classification policy is written down.

Review answers and solution notes

A const accessor should return a const reference or value according to the interface’s lifetime policy. The row-major mapping is valid only after both coordinates satisfy the shape bounds. Compile-time dimensions make mismatched shapes distinct types.

A pointer to const integer still matches T^* with a const pointee type, while a top-level const pointer has a different outer type pattern. Full specialization selects an exact case; partial specialization selects a family. Neither removes the need for coherent semantics across the family.

Engineering takeaway. Parameterize genuine variation, validate shape at the right stage, and keep specializations faithful to one abstraction.

Guided lab — Extend a fixed matrix without breaking its contract

Add a const overload of `at`, public constant row and column queries, and a checked `fill` operation using the standard array’s facilities. Test every valid coordinate in a small matrix and verify that writing one coordinate changes no other coordinate. Test invalid rows and invalid columns separately.

Add a generic helper that computes a row total only for the numeric types and value domain it is designed to support. Do not make the matrix’s basic storage operation require summation. This keeps representation and numeric policy separate. Use a sufficiently wide or checked accumulator and include an empty-domain decision explicitly if you later permit zero dimensions.

For specialization practice, add a category for a const-qualified pointer pattern and predict which specialization should match several concrete types. Keep the exercise on your own templates. Record each accepted result in a compile-time assertion rather than relying on a comment that a particular pattern “should work.”

Review and solution reasoning

Class templates create concrete type families. Constant dimensions can enforce shape constraints, but runtime coordinates still need checks. Specializations select implementations under compile-time rules and should preserve the intended family interface. A reusable representation remains responsible for ownership, overflow boundaries, and understandable diagnostics just as a non-template class does.

Primary reference notes

Template structure: [class templates](#), [partial specialization](#), [explicit specialization](#), and [standard arrays](#).

Chapter 13 — Writing Constrained Generic Code

A generic addition function accepts both signed and unsigned integers. Its first implementation computes the sum and then checks whether it overflowed, which is already too late for signed overflow. Another version excludes floating-point values but accidentally accepts booleans as an intended arithmetic domain.

Concepts can express type eligibility, while runtime checks enforce value-level requirements. This chapter combines the two instead of expecting one mechanism to solve both problems.

A generic function can fail with a long diagnostic deep inside its body even when the real problem is simple: the caller supplied a type outside the function’s intended domain. C++20 concepts move many such requirements toward the interface. HarborWorks uses a checked-addition function to show a narrow type domain and runtime representability checks working together.

The concept excludes Boolean values even though `bool` is an integral type in the language’s classification. That is a domain choice: adding logical states as quantities is not the operation this function promises. The arithmetic body then handles signed and unsigned cases differently, because their representability rules are not interchangeable.

13.1 Understanding Compile-Time Type Requirements

Compile-time type inspection answers questions about a type: whether it is integral, signed, or the same as another type. Those answers can choose implementations or reject unsupported instantiations before runtime.

The result does not prove anything about a particular runtime value’s magnitude. Keep type properties and value properties separate in both code and tests.

13.1.1 Inspecting Types with Type Traits

Type traits such as `std::is_signed_v<T>` and `std::remove_cv_t<T>` provide composable compile-time information. The example removes `const`/`volatile` qualification when excluding `bool`, so its eligibility rule describes the underlying scalar category rather than an incidental qualifier.

Traits should answer a question that the algorithm actually needs. A broad label such as “arithmetic” includes types with substantially different overflow and ordering behavior. More precise requirements make the implementation easier to justify and diagnostics easier to interpret.

Type traits expose compile-time properties through library templates. A trait can identify integral types, signed types, constructibility, or whether an operation is declared nonthrowing under the relevant rules. These properties help select implementations and state constraints, but they are not a complete semantic description of a type.

Normalize the type only when normalization matches the question. `std::remove_cv_t<T>` removes top-level const and volatile qualification, while removing references requires a separate transformation or `remove_cvref_t`. A concept asking whether a deduced value type is Boolean should avoid accidentally accepting `const bool` because it compared only one spelling. Conversely, stripping qualifications indiscriminately can erase information an interface actually needs.

Traits can answer whether a copy construction expression is well-formed according to the type's interface. They do not prove that the copy creates independent domain values. A shared pointer is copyable precisely because sharing ownership is its intended operation. The distinction between syntactic capability and semantic meaning is the same one developed in Chapters 1 and 4.

Use traits where the algorithm has a concrete reason to branch or reject. A collection of dozens of unrelated static assertions can make a generic interface brittle without explaining its purpose. The checked-addition function uses a small set of directly relevant properties: integral, non-Boolean, and signed versus unsigned.

13.1.2 Performing Compile-Time Decisions

`if constexpr` selects a compile-time branch based on a constant expression in a template. The signed branch performs upper- and lower-bound checks, while the unsigned branch checks only against the maximum because unsigned values do not represent negative inputs.

The discarded dependent branch is not instantiated in the same way as a selected branch, allowing type-specific code. This does not mean arbitrary syntactically invalid text can be hidden anywhere behind `if constexpr`; parsing and nondependent validity rules still matter. Keep branches focused on the actual type distinction rather than using compile-time conditionals as a substitute for readable structure.

`if constexpr` selects a branch during template instantiation based on a compile-time condition. In the checked-addition function, the signed branch includes a lower-bound test, while the unsigned branch needs only the upper-bound test. A discarded dependent branch is not instantiated in the same way as the selected branch, allowing different type-specific operations within one template.

This is different from a runtime `if` that chooses a path while both branches still need to be valid under ordinary compilation requirements. It is also not a preprocessor directive: names and syntax still participate in C++ parsing, and nondependent errors cannot be hidden casually by placing them in a branch that happens not to execute.

Compile-time selection should preserve one public meaning. Both branches of checked addition either return the representable mathematical sum or report overflow. The implementation differs because the type domains differ. A branch that silently wraps unsigned input while the signed branch throws would establish a different contract and should be named and documented accordingly.

The arithmetic checks themselves avoid overflow. For positive signed `right`, compare `left` with `max - right`. For negative signed `right`, compare `left` with `min - right`. Those subtractions are selected under conditions that keep them representable. Only after the checks pass does the function perform addition. Explain this reasoning before trusting a sanitizer to discover every arithmetic defect.

Rejecting Unsupported Types with ``static_assert`` and Traits

Purpose. Generic code always has type requirements; making them explicit improves diagnostics and documents valid instantiations.

```
#include <type_traits>
template<class T> T twice(T v){ static_assert(std::is_arithmetic_v<T>,"twice
requires arithmetic"); return v+v; }
int main(){ return twice(3)==6 ? 0 : 1; }
```

How it works.

- The trait produces a compile-time boolean about ``T``.
- ``static_assert`` turns an undocumented assumption into an immediate compilation requirement.

Expected behavior/output. The program compiles and exits successfully for ``int``.

Experiment. Try ``twice(std::string{"x"})`` and inspect the intentional compile-time rejection.

13.2 Expressing Requirements with Concepts

A concept names a compile-time constraint. The `Integer` concept combines the standard integral concept with exclusion of `bool`. A `requires` clause or constrained template parameter then restricts which template candidates participate.

Constraints describe eligibility and some expression properties. They do not automatically prove algebraic laws, absence of races, or valid runtime bounds.

13.2.1 Defining Concepts

The declaration `template<class T> concept Integer = ...;` creates a reusable predicate over types. Static assertions verify that `int` and `unsigned` satisfy it while `bool` and `double` do not. These checks are executable documentation of the intended type domain.

A concept requiring an ordering expression can establish that the expression is available, but its semantic ordering laws may still rely on the type's implementation. Tests and review remain necessary. Prefer concepts named after meaningful requirements rather than long lists of incidental implementation expressions that merely happen to compile today.

A concept is a named compile-time predicate used in constraints. Some concepts are combinations of type traits, while others use `requires`-expressions to test whether particular expressions exist and have suitable types or exception properties. Naming the requirement helps both the interface and diagnostics communicate why an overload participates.

A `requires`-expression is not ordinary runtime validation. Its expressions are checked for well-formedness in the relevant context; they are not executed to test a particular value. A requirement that `a < b` is

convertible to Boolean says something about the expression's shape. It does not prove transitivity, irreflexivity, or any other ordering law needed by a sorting algorithm.

This distinction is particularly important for user-defined comparators. A callable returning `bool` can still implement `a <= b`, which is not a strict ordering because it is true when the arguments are equal. The compiler may accept the callable while the algorithm's semantic requirements are violated. A concept improves the visible type contract; property tests and reasoning examine the laws.

Choose concept names that describe stable capabilities rather than one temporary implementation detail. `Integer` in this chapter means an integral non-Boolean type for checked arithmetic. A broader public library might choose a more specific name such as `ArithmeticInteger` to avoid confusing that domain choice with the language's complete integral-type category.

13.2.2 Applying `requires` Clauses

`template<Integer T>` constrains the type parameter directly. An equivalent form can place `requires Integer<T>` after the template parameter list. The chapter includes both forms so the syntax can be connected to the same eligibility rule.

Inside the eligible function, value checks prevent overflow before addition. For signed values, a positive right operand is compared against `max - right`; a negative right operand is compared against `min - right`. The guards make those subtractions appropriate to their branches. For unsigned values, `left > max - right` detects an unrepresentable mathematical sum before the addition wraps. The function deliberately rejects wraparound rather than adopting it as its arithmetic contract.

A `requires`-clause restricts an overload or template according to a constraint. An abbreviated constrained parameter can provide another readable spelling when the relationship is simple. The important point is that unsuitable candidates are excluded through the interface rather than failing only after an unrelated expression deep in the body is instantiated.

Constraints can affect overload ordering, and logically similar-looking expressions are not always interchangeable for constraint subsumption. Reusing a named concept can make an intended relationship clearer than repeating slightly different trait expressions in several overloads. Keep introductory overload sets simple enough that readers can predict which candidate should participate.

A static assertion in the body and a constraint at the interface serve different purposes. A body assertion can provide a targeted explanation for an instantiated implementation, but it does not necessarily make the function disappear from overload selection in the same way as a constraint. Use a `requires`-clause when the type should not be a viable candidate; use an internal assertion for an implementation invariant that should hold after selection.

Runtime values still need checks. The concept accepts `int`, but it cannot know whether two particular runtime integers will overflow when added. The function's value checks remain essential. A successful call means both gates passed: the type belongs to the supported family, and the particular sum is representable.

Integral-Only Function Using a Standard Concept

Purpose. Concepts express semantic or syntactic requirements directly in the function declaration.

```
#include <concepts>
#include <iostream>
template<std::integral T> T gcd_like(T a,T b){ while(b!=0){T r=a%b;a=b;b=r;}
return a; }
int main(){ std::cout<<gcd_like(48,18)<<'\\n'; }
```

How it works.

- `std::integral` constrains template argument deduction to integral types.
- The `%` operation is valid because the concept guarantees the intended category.
- Invalid calls fail at the interface rather than deep in the body.

Expected behavior/output. The program prints `6`.

Experiment. Call with `double` values and compare the concept diagnostic to an unconstrained template failure.

Worked example — Constrain integer addition and reject overflow before evaluation

Requirement and design

Accept non-boolean integral types, add representable values, and throw on sums outside the selected type's representable range. Test ordinary signed and unsigned inputs together with maximum-plus-one and minimum-minus-one boundaries.

The function returns a value of the same selected type. It does not silently widen the public result to a different arithmetic domain.

Prove the checks do not overflow first

Take the signed upper-bound case. If `right` is positive, `max - right` is representable. Any `left` greater than that threshold would make the mathematical sum exceed `max`, so the function rejects before addition. For negative `right`, `min - right` is a representable threshold toward the positive direction, and a smaller `left` would underflow. The unsigned case uses `max - right`, which is representable because every unsigned `right` is at most `max`.

Now examine zero. Neither signed overflow branch needs to reject adding zero, and the returned value is unchanged. Mixed-sign additions can be representable even near a boundary, so a blanket “large absolute values are unsafe” rule would be unnecessarily restrictive. Test the actual threshold logic rather than approximating it with an arbitrary safety margin.

The compile-time assertions establish that integers are accepted while Boolean and floating-point types are rejected. The runtime cases establish ordinary sums and boundary failures. These are complementary forms of evidence, not interchangeable check counts.

Complete implementation

The complete, buildable chapter project is `source/book2/ch13/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <concepts>
#include <limits>
#include <type_traits>

template<class T>
concept Integer = std::integral<T> && !std::same_as<std::remove_cv_t<T>, bool>;

template<Integer T>
T checked_add(T left, T right) {
    const T high = std::numeric_limits<T>::max();
    if constexpr (std::is_signed_v<T>) {
        const T low = std::numeric_limits<T>::min();
        if ((right > 0 && left > high - right) ||
            (right < 0 && left < low - right)) {
            throw std::overflow_error("signed addition");
        }
    } else {
        if (left > high - right) throw std::overflow_error("unsigned addition");
    }
    return static_cast<T>(left + right);
}

template<class T> requires Integer<T>
constexpr bool accepted_type() { return true; }

int main() {
    static_assert(Integer<int> && Integer<unsigned>);
    static_assert(!Integer<bool> && !Integer<double>);
    static_assert(accepted_type<long>());
    CHECK(checked_add(2, 3) == 5);
    CHECK(checked_add(-2, -3) == -5);
    CHECK(checked_add(2U, 3U) == 5U);
    CHECK(course::throws<std::overflow_error>([] {
        (void)checked_add(std::numeric_limits<int>::max(), 1);
    }));
    CHECK(course::throws<std::overflow_error>([] {
        (void)checked_add(std::numeric_limits<int>::min(), -1);
    }));
    CHECK(course::throws<std::overflow_error>([] {
        (void)checked_add(std::numeric_limits<unsigned>::max(), 1U);
    }));
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch13 -B build/b2-ch13 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch13 --parallel 2
ctest --test-dir build/b2-ch13 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch13`.

Evidence and interpretation

The type checks reject `bool` and `double` at compile time. Runtime checks accept two plus three, minus two plus minus three, and two unsigned plus three unsigned. The three boundary tests require exceptions before a signed overflow or unintended unsigned wrap can occur.

The final cast accounts for the language’s integral promotions when smaller integer types participate in arithmetic. The prechecks establish that the mathematical result fits the intended return type. That combination is more precise than assuming every `T + T` expression is already evaluated in type `T`.

The algorithm’s proof is local: the right operand’s sign selects a safe bound expression, the comparison decides whether the sum fits, and only the accepted path evaluates addition. A sanitizer can help detect violations in tests, but it is not the reason the check is correct.

Failure clinic — A concept that promised too much

An engineer defines `Sortable` as “the type has a less-than expression” and assumes every accepted type can be passed to a sorting algorithm with correct results. A custom relation may still violate strict weak ordering. The concept checks syntax, while the algorithm requires semantic laws. Rename the constraint to reflect what it actually checks and document the stronger requirements where the algorithm is used.

Another defect detects signed overflow by computing the sum and then comparing it with an operand. The undefined operation has already occurred before the test. Rearrange the check so every intermediate expression is valid in the accepted type domain. The teaching function’s branch conditions are part of that proof.

A third defect admits `bool` because an integral trait returns true. The resulting operation may compile, but it does not match the function’s quantity semantics. A named concept can deliberately refine the language category. Type classifications are building blocks for a domain, not the domain itself.

Practice and implementation lab

Add tests for zero, opposite-sign cancellation, signed maximum plus zero, unsigned zero, and smaller integral types. Add compile-only checks showing that bool and floating-point instantiations are not eligible.

Design checked subtraction with the same discipline. Write the mathematical accepted range and derive preconditions before entering code. Do not implement it by blindly negating the right operand, because negating the minimum signed value introduces its own overflow problem.

Review answers and solution notes

Concepts select the type domain while runtime comparisons select valid values within that domain. Integral promotion can change the intermediate expression type, so the function still returns through a deliberate conversion after proving representability.

Negating a signed minimum is not a safe universal implementation of subtraction. Checking after a signed overflow is too late. A useful constraint should express the operation's real requirements, while tests cover boundaries that ordinary small examples never exercise.

Engineering takeaway. Constrain types at compile time and prove value safety before performing the operation that could violate it.

Guided lab — Build a checked range total

Use the constrained-sum laboratory in Appendix C. It accepts a sequence of supported integral values and combines them through checked addition. Define the accumulator type explicitly and return an owned result. Reject a sum before overflow rather than wrapping it or relying on undefined behavior detection afterward.

Test an empty range, one value, mixed signs, exact upper and lower boundaries, and a final element that would overflow after several valid additions. Compare against hand-computed small totals and a separately reasoned boundary table. Include compile-time checks for excluded Boolean and floating-point element types.

Then define a concept for a record exposing a readable integer priority. Decide whether the concept merely requires an expression or promises a domain range. The latter still needs runtime validation unless the type itself enforces the range. Write the distinction in the function's documentation.

Review and solution reasoning

Traits describe type properties, `if constexpr` selects implementations, and concepts constrain interfaces. None of them replaces runtime representability checks or semantic laws. A robust generic function states what the compiler can check, what the caller must guarantee, and what the function validates while executing. Clear boundaries make diagnostics shorter and correctness arguments stronger.

Primary reference notes

Generic constraints and arithmetic: [concepts](#), [constraints](#), [type traits](#), [numeric limits](#), and [integer conversions](#).

Chapter 14 — Writing Functional-Style Code with Lambdas

A threshold predicate is created when the cutoff is three, then the cutoff changes to five. Should the predicate remember three or observe five? A lambda capture list is a lifetime and state decision that answers exactly that question.

This chapter uses value capture, reference capture, generic parameters, and mutable closures. It also returns a stateful lambda safely by making the returned closure own its state.

A lambda is an object with state and behavior, even when its syntax fits on one line. HarborWorks uses small predicates to select tasks and transformations to prepare reports. The same compact syntax can either copy a configuration value into a stable snapshot or borrow a changing configuration object. The capture list determines that difference.

Read a lambda in two parts. Its parameters describe information supplied when it is called. Its captures describe information stored or borrowed when the closure is created. Confusing those moments is a common source of stale configuration, accidental sharing, and dangling callbacks.

14.1 Creating Lambda Expressions

A lambda expression creates a closure object with a callable operation. Parameters describe call-time inputs; captures describe state obtained from the surrounding scope. The distinction determines both behavior and lifetime.

Small lambdas are useful near algorithms because the operation stays close to the traversal that uses it. Large or widely reused behavior may deserve a named function or type.

14.1.1 Defining Parameters and Return Values

A lambda such as `[](int value) { return value * value; }` has no captures and accepts an integer parameter. Its return type is deduced from the return expression under the language's rules, or can be stated explicitly when needed.

The body still has ordinary arithmetic requirements. Squaring an arbitrary int can overflow, so the example uses the bounded values one, two, and three. A lambda does not make an operation safer simply because it is concise. Give named predicates or transformations clear preconditions when their domain extends beyond fixed teaching fixtures.

A captureless lambda is a convenient way to keep a small operation next to the algorithm that uses it. In a transformation over three known integers, squaring each integer is easy to understand locally. If the operation grows into several branches, performs I/O, or requires a long comment explaining its domain, a named function or function object can provide a clearer interface.

Return-type deduction follows the lambda's return expressions. Different branches need compatible return behavior under the language's deduction rules. An explicit trailing return type can state the intended result and make conversions visible. It is especially useful for a visitor whose alternatives must all produce one public result type.

Parameters can own values or borrow them just like ordinary function parameters. A predicate taking a large record by value can introduce a copy for every call. Taking `const Record&` can avoid that copy while requiring the record to stay alive during the call. A generic `const auto&` can express the same borrowing idea across several record types, subject to the body's actual requirements.

The algorithm also imposes requirements on the callable. A sorting relation must be a valid strict weak ordering. A transformation must produce a result suitable for the destination. A lambda that compiles is not automatically a valid comparator or a safe arithmetic operation. Keep the callable's semantic contract next to its implementation.

14.1.2 Capturing Local Variables

`[cutoff]` copies the cutoff into the closure, producing a snapshot for this scalar. `[&cutoff]` borrows the original variable, so later changes are observed while that variable remains alive. The example changes cutoff from three to five and tests the difference with input four.

A reference-capturing lambda returned from a function can dangle if it refers to an automatic local that has already ended. Capturing `this` also carries a lifetime dependency on the object; it is not an owning copy of all members. Prefer explicit captures where they clarify what escapes and who owns the needed state.

The cutoff experiment distinguishes creation time from call time. The value-capturing closure stores three when it is created. The reference-capturing closure continues to access the external cutoff. After the external value changes to five, input four is accepted by the fixed closure and rejected by the live closure. Both results are correct under their capture policies.

Copy capture follows the captured type's copy semantics. Capturing an integer copies a value. Capturing a shared pointer copies shared ownership of one pointee. Capturing a string view copies a view of characters owned elsewhere. The phrase "captured by value" therefore does not guarantee deep independence of every reachable object. Inspect the members of the closure just as you would inspect the members of a class.

Capturing `this` stores a relationship to the existing object; it does not keep that object alive automatically. Capturing `*this` copies the enclosing object under the applicable language rules, which can be useful for a snapshot but can also be expensive or semantically wrong for a noncopyable resource owner. An initializer capture can explicitly own a moved value when transfer is the intended policy.

Prefer explicit captures at a callback boundary. A broad reference capture can begin borrowing a newly added local variable after a later edit, silently expanding the lifetime obligation. When a closure escapes the creating scope, list every captured dependency and identify who keeps it alive through the last invocation.

Capturing a Threshold for Local Filtering Logic

Purpose. Lambdas are useful for local behavior that depends on nearby state such as a threshold.

```
#include <algorithm>
#include <iostream>
#include <vector>
int main(){ std::vector<int> v{2,7,4,9}; int threshold=5; auto
count=std::count_if(v.begin(),v.end(),[threshold](int x){return x>threshold;});
std::cout<<count<<'\n'; }
```

How it works.

- The lambda captures `threshold` by value.
- The parameter represents each candidate element supplied by the algorithm.
- The closure object provides behavior without requiring a separately named function.

Expected behavior/output. The program prints `2`.

Experiment. Capture by reference, change `threshold` before the algorithm call, and predict the new count.

14.2 Building Generic and Stateful Lambdas

Generic lambdas use type-deduced parameters to describe a family of call operations. Stateful lambdas store captured values as closure state. These features can be combined, but the resulting ownership and mutation rules should remain understandable.

Do not let compact syntax hide a callable object whose copies or retained references have surprising behavior.

14.2.1 Creating Generic Lambdas

The example's `[](auto left, auto right) { return left + right; }` supports suitable argument types through deduction. It inherits the semantics and requirements of their addition operator, including numeric conversion and overflow behavior.

Generic does not mean unconstrained in purpose. A predicate used by an algorithm may need purity, ordering, or consistency even when any callable with the right syntactic shape compiles. C++20 also supports more explicit template parameter and constraint syntax for lambdas when a generic callable needs a narrower interface.

A generic lambda has a templated call operation for its deduced parameters. The supplied addition lambda can be instantiated for integers or floating-point values because both support the expression. That does not mean it enforces a common numeric domain or overflow policy. Mixed types can introduce conversions that a domain-specific interface would prefer to reject.

C++20 permits explicit template parameters and constraints in a lambda when the callable needs a narrower family. For example, a small checked integer operation can reuse the chapter's `Integer` concept instead of accepting every type with a plus sign. Keep the constraint readable; a large inline requires-expression can be clearer as a named concept or named function template.

Generic lambdas are especially useful with algorithms and visitation because they keep a local policy close to its use. They can also conceal an overly broad fallback. A visitor with a generic `else` branch may continue compiling after a new variant alternative is added, even though the new case deserves separate behavior. Exhaustive handling should be a deliberate choice, not an accidental consequence of concise syntax.

Test representative instantiations. An integer call does not compile the same body for every future user-defined type. A type that supports addition might return a proxy or a different value type, changing return semantics. Generic code is a family of programs; define which members of that family are part of the supported interface.

14.2.2 Using Mutable Closures

A value-capturing lambda's call operator is ordinarily `const` with respect to its captured members. `mutable` permits modifying that closure state. `make_counter` returns a lambda that owns its integer counter, so it does not borrow the factory's local variable after return.

Copying the closure copies its captured integer, producing independent counter state. A captured shared pointer would instead copy shared ownership and preserve a shared pointee. The closure's copy behavior follows its member semantics just like any other object. The counter fixture uses only a few increments; an unbounded service counter would need an overflow policy.

A mutable lambda can modify its value-captured state through its call operator. The returned counter owns its integer, so it remains valid after the factory's parameter and local variables are gone. Copying the closure copies that integer, allowing two independent counters to continue from the same captured state.

Independence again depends on representation. A closure capturing `shared_ptr<State>` by value can be mutable or nonmutable while still accessing shared pointee state through the pointer. A closure capturing a reference aliases the same external counter. A closure capturing a unique pointer by move can become noncopyable, affecting which callback containers or algorithms can store it. The choice is not merely about whether the keyword `mutable` appears.

Stateful callables can be copied by algorithms or wrapper types. Do not assume that inspecting the original callable after an algorithm runs will reveal every update made inside a copy. When counting invocations is genuinely required, pass an explicit observation object with a documented lifetime or use an algorithm's returned callable where its contract provides one. Keep instrumentation separate from the algorithm's semantic predicate requirements.

An unbounded counter needs an overflow policy. The example calls it a few times with small values so every increment is representable. A long-running service counter might use a wider type, a saturating rule, a checked operation, or an explicit wrap policy depending on the requirement. Concise closure syntax does not make arithmetic unlimited.

Generic Accumulator Lambda with Mutable Captured State

Purpose. Generic parameters and captured state make lambdas small reusable function objects rather than only anonymous one-shot expressions.

```
#include <iostream>
int main(){ int total=0; auto add=[&total](auto value){ total +=
static_cast<int>(value); return total; }; std::cout<<add(2)<<'
'<<add(3.5)<<'\n'; }
```

How it works.

- `auto` makes the lambda call operator generic over argument type.
- Capturing `total` by reference preserves state across calls in the surrounding object.
- Each call updates the same accumulator.

Expected behavior/output. The output is `2 5`.

Experiment. Capture `total` by value with `mutable` and compare the closure's private state to the outer variable.

Worked example — Distinguish snapshot captures, live borrows, and owned counter state

Requirement and design

Create fixed and live threshold predicates, change the original cutoff, and compare their results. Return an owning mutable counter, copy it, and verify independent progression. Use a generic lambda for addition and a captureless lambda for in-place transformation.

Every reference capture remains within the lifetime of its referred-to variable. The returned counter captures by value.

Trace the two timelines

First trace configuration: create the fixed and live closures while cutoff equals three, change the external cutoff to five, and call both with four. Next trace owned state: create the counter at ten, call twice, copy the closure, and call both copies once. The expected counter results follow from copying the state after the second call, not from both closures secretly sharing one integer.

The transformation writes results back into the same vector over a range and operation for which that in-place form is intended. It uses small values to avoid arithmetic overflow. Do not generalize every algorithm into an overlap-safe operation; each algorithm defines its own input/output overlap requirements.

The example keeps borrowed state in scope for every invocation. The returned counter, by contrast, owns its state and needs no such external lifetime. These two valid patterns are intentionally shown side by side.

Complete implementation

The complete, buildable chapter project is `source/book2/ch14/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <vector>

auto make_counter(int initial) {
    return [value = initial]() mutable { return ++value; };
}

int main() {
    int cutoff{3};
    const auto fixed = [cutoff](int value) { return value >= cutoff; };
    const auto live = [&cutoff](int value) { return value >= cutoff; };
    cutoff = 5;
    CHECK(fixed(4) && !live(4));
    auto counter = make_counter(10);
    CHECK(counter() == 11);
    CHECK(counter() == 12);
    auto independent = counter;
    CHECK(independent() == 13 && counter() == 13);
    const auto add = [](auto left, auto right) { return left + right; };
    CHECK(add(2, 3) == 5 && add(2.0, 0.5) == 2.5);
    std::vector<int> data{1, 2, 3};
    std::transform(data.begin(), data.end(), data.begin(),
        [](int value) { return value * value; });
    CHECK(data == std::vector<int>({1, 4, 9}));
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch14 -B build/b2-ch14 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch14 --parallel 2
ctest --test-dir build/b2-ch14 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch14`.

Evidence and interpretation

After cutoff becomes five, the fixed predicate still accepts four because it captured three, while the live predicate rejects four because it observes five. The counter produces eleven and twelve; copying it then lets each independent closure produce thirteen.

The transformation replaces one, two, and three with one, four, and nine. In-place transform is suitable for this unary elementwise operation, but arbitrary overlapping input/output patterns need the specific algorithm's contract. Do not generalize from this one safe overlap to all algorithms.

The example illustrates that a lambda is an object, not magic executable text. Its captures have ownership, copying, and lifetime behavior. That mental model makes callback storage and asynchronous use much easier to review later.

Failure clinic — The callback outlived its capture

A function returns `[&local] { return local; }`. The closure survives, but the referenced automatic object does not. Calling the closure later uses an invalid borrow. The repair is to capture an owned value, transfer an owner, or require a longer-lived external object. Adding `const` to the captured reference does not extend its lifetime.

Another defect appears when a supposedly fixed configuration captures a shared pointer to mutable settings. The closure's pointer value is fixed, but the settings can still change through another owner. A genuine snapshot may require copying the settings into an immutable value rather than copying only the owner.

A third defect stores a move-only closure in a wrapper that requires copyability. The compiler's rejection is a clue about the ownership contract, not a reason to replace the unique pointer with shared ownership without analysis. Select a compatible storage mechanism or redesign the callback lifetime deliberately.

Practice and implementation lab

Return a threshold predicate that owns its cutoff and test it after the factory returns. Write a separate explanation of why a reference to the factory's local cutoff would dangle; do not execute that invalid version as a normal test.

Copy a counter before its first call and compare the two sequences. Then replace the owned scalar with deliberately shared state and document how the copy semantics change. Add a checked overflow policy for a counter intended to run without a fixed call limit.

Review answers and solution notes

A value capture owns its captured value, while a reference capture depends on an external lifetime. Mutable changes whether the closure can modify its own captured state; it does not turn a value capture into a reference capture.

Copied scalar counters progress independently. Shared-pointer captures can intentionally share a pointee, so closure copying is not automatically deep copying of every reachable object. Generic parameters inherit the argument types' operation semantics and still need a meaningful domain contract.

Engineering takeaway. Treat every lambda as a real object whose captures define ownership, copying, mutation, and escape safety.

Guided lab — Compare three configuration policies

Create predicates with a copied cutoff, a borrowed cutoff, and an owned immutable configuration object. Call them before and after changing the original configuration while every required owner remains alive. Record which results should change and why. Do not deliberately call a dangling closure in the normal suite; demonstrate safe lifetime boundaries instead.

Return an owning counter closure from a factory and copy it after several calls. Verify independent state afterward. Then use a shared state object and verify intentionally shared behavior. Give the two factories names that reveal the difference so callers do not have to inspect their capture lists.

For an algorithm exercise, write a transformation over bounded integers with an explicit result type and checked arithmetic. Test empty input and an input that would overflow. Decide whether the destination should remain unchanged after failure; achieving that stronger guarantee requires a candidate output rather than arbitrary in-place mutation.

Review and solution reasoning

A closure has ordinary object semantics. Capture time and call time are different events. Copy capture does not recursively copy every reachable resource, and reference capture does not extend lifetime. Mutable state, copyability, and arithmetic safety remain design questions even when the implementation is only one line long.

Primary reference notes

Lambda and algorithm rules: [lambda expressions](#), [object lifetime](#), and [transform algorithms](#).

Chapter 15 — Traversing Containers with Iterators

A loop erases unwanted vector elements and then increments the old iterator. It skips some values and sometimes accesses invalid state. The algorithm’s intention is simple, but its traversal state no longer matches the container after mutation.

Iterators connect algorithms to containers. Understanding their categories and invalidation rules is essential both for correctness and for interpreting complexity claims.

An iterator is a handle to a traversal position under a particular range’s rules. It is not an eternal address to an element. HarborWorks filters records, erases completed work, and reserves capacity while loading datasets. Each operation can change what previously obtained iterators, pointers, references, and indices mean.

This chapter distinguishes three forms of stability: the owner still exists, the element’s address remains valid, and a logical identity still refers to the same record. A container can satisfy one and violate another. Keeping those questions separate prevents many bugs that are otherwise described vaguely as “iterator problems.”

15.1 Understanding the Iterator Abstraction

An iterator represents a position in a traversal. A half-open range starts at a begin iterator and ends at a sentinel or iterator that is not itself an element to process. Empty ranges naturally have matching begin and end positions.

The available operations depend on the iterator’s capabilities, not just on whether its syntax resembles a pointer.

15.1.1 Working with Begin and End Iterators

`begin()` identifies the first element when one exists, and `end()` identifies the past-the-end position. A loop compares before dereferencing. Dereferencing end is invalid even when the underlying container has spare capacity or an implementation uses a nearby address.

Iterators belong to the ranges and validity conditions that produced them. Combining a begin iterator from one vector with an end iterator from another does not form a valid range merely because the types match. Preserve the ownership and range identity through function interfaces and tests.

A half-open range includes its beginning and excludes its end. This convention makes an empty range natural and lets adjacent ranges share a boundary without sharing an element. The end position is a stopping condition, not an extra element. Spare vector capacity does not make the past-the-end position dereferenceable.

The iterator pair must describe one valid range. A begin from one vector and an end from another do not become compatible merely because their types match. Likewise, an iterator retained before a mutating operation may no longer be valid to compare afterward. Some iterator errors occur during comparison or arithmetic, not only during dereference.

Sentinels can differ in type from iterators in modern range interfaces. A sentinel describes how traversal ends; it need not support every operation of the iterator. This lets ranges represent sources such as input streams or special termination conditions without pretending they are indexed arrays. Algorithms must use the capabilities their interface requires rather than assume every endpoint can be subtracted.

When writing a manual loop, establish the comparison before dereference and make progress explicit. For a nonmutating vector traversal, a range-based loop can reduce mechanical errors. For erasure, use the operation's returned position as the next state rather than applying a nonmutating-loop pattern to a mutating container.

15.1.2 Understanding Iterator Categories

Input iterators support single-pass reading under their contracts. Forward iterators support multipass traversal. Bidirectional iterators add backward movement, random-access iterators add constant-time indexed movement, and contiguous iterators additionally describe contiguous element storage in C++20's iterator model.

These distinctions explain why `std::distance` can be constant time for a vector yet linear for a forward list. A binary-search algorithm can use logarithmic comparisons on a forward range while still performing substantial iterator movement. Complexity must count the relevant operations, not assume every increment or midpoint access has the same cost.

Iterator categories express capabilities and complexity expectations. An input iterator can be single-pass; copying it does not necessarily create an independent reusable traversal of the same source. A forward iterator supports multipass use under its contract. Bidirectional and random-access iterators add movement operations, while contiguous iterators connect traversal to contiguous storage.

The category affects algorithm choice. `std::distance` over a vector can obtain a distance without visiting every element. Over a forward list it advances through the sequence. A binary-search algorithm on forward iterators can reduce the number of comparisons logarithmically while still performing linear-scale iterator movement. Counting comparisons alone can therefore misrepresent actual work.

Do not upgrade an iterator's category just because one operation is easy to implement in a teaching wrapper. A claimed category promises a family of behavior and complexity properties. Library algorithms may rely on those promises. An iterator that advertises random access but performs every jump through a linear scan defeats the advertised complexity model even if small tests return correct values.

For application code, the practical lesson is to ask what the operation needs. Sorting requires suitable random-access traversal for the ordinary sorting algorithms discussed here. A linked list supplies its own member sorting operation instead. Finding an element needs much less. Selecting a container and selecting an algorithm are connected decisions, not independent checklist items.

Same Summation Algorithm over Iterator Pairs

Purpose. Iterators decouple traversal algorithms from the concrete storage type being traversed.

```
#include <array>
#include <iostream>
#include <vector>
template<class It> int sum(It first, It last) { int s=0; for(; first!=last; ++first) s+=*first; return s; }
int main()
{ std::array<int, 3> a{1, 2, 3}; std::vector<int> v{4, 5}; std::cout<<sum(a.begin(), a.end())<<'\n';
  std::cout<<sum(v.begin(), v.end())<<'\n'; }
```

How it works.

- The algorithm knows only how to compare, increment, and dereference iterators.
- Array and vector iterators satisfy the same required operations.

Expected behavior/output. The output is `6 9`.

Experiment. Call the function with `v.begin()+1` to sum only a suffix.

15.2 Managing Iterator Validity

Container mutation can invalidate iterators, references, pointers, and endpoints according to operation-specific rules. “The container still exists” is not enough to justify an old element handle.

After a structural change, use the operation’s returned iterator or reacquire a handle through a stable logical identifier when appropriate.

15.2.1 Recognizing Container Reallocation

A vector reallocation relocates its elements into new storage and invalidates old element iterators and references. Reserving beyond the old capacity is a deliberate way to trigger that ownership change in the example. The program preserves an index, not an old iterator, and accesses the new storage through the vector afterward.

A saved index remains meaningful only if intervening operations preserve the intended element’s position. Insertion or erasure before that index changes positional meaning even without a dangling pointer. Distinguish address stability from identity stability. For long-lived application references, an explicit record identifier may be the correct abstraction.

A vector separates size from capacity. Increasing size within capacity may avoid relocating existing elements, while growing beyond capacity requires new storage under the usual vector operation model. Reallocation invalidates old element iterators, pointers, and references. An old address can still look plausible in a debugger even though using it is no longer permitted.

The end iterator needs its own reasoning. Appending an element changes the past-the-end position even when no reallocation occurs. A loop that cached the old end and then appends can therefore be wrong

despite a successful `reserve`. Capacity planning can prevent some relocations; it does not freeze every iterator-related property of the container.

A saved index avoids carrying an old storage address across reallocation. It does not preserve identity across insertion or erasure before that position. If record 17 was at index five and another record is inserted at index two, index five may now identify a different record. Long-lived application references often need explicit IDs and fresh lookup rather than raw indices.

The supplied example saves an index, reserves larger capacity, and reacquires access through the vector. No insertion or erasure changes the indexed element's position between those steps. That is why the index strategy is valid for this specific test. State the operations that preserve the strategy instead of declaring indices universally safe.

15.2.2 Avoiding Invalidated Iterators

`vector::erase` returns an iterator to the element following the erased position, or the new end when none follows. The loop assigns that result and does not increment again in the erase branch. In the keep branch, it increments normally.

Erasing can invalidate the erased position and later positions, including an old end iterator under the vector rules. Recompute through the returned state rather than carrying a stale cached endpoint into a mutating loop. Other containers have different guarantees, so read the specific operation contract instead of memorizing one global rule for all iterators.

The erase loop has two transitions. When an element is removed, `erase` returns the next valid position and the loop assigns it directly. When an element is kept, the loop increments the current iterator. Incrementing again after erasure would skip the element that shifted into the erased position. Retaining the old iterator would instead use invalid traversal state.

Other containers have different invalidation rules. Erasing one list node does not invalidate iterators to unrelated nodes. An unordered-container rehash invalidates iterators but does not by itself invalidate references or pointers to existing elements under the standard's specified guarantees. A vector erase affects positions at and after the erased location. Memorizing one rule for "containers" is not sufficient.

A stable iterator is not a stable business relationship. A node can remain alive while its value changes, and a pointer can remain valid while the application has logically retired the record. Conversely, a stable identifier can name a record that now lives at a different address. Match the handle to the requirement: traversal position, object lifetime, or domain identity.

Tests should never prove validity by dereferencing an iterator after an operation known to invalidate it. Reacquire a valid handle and compare logical values. Sanitizers and debug iterators can catch some invalid uses, but the absence of a diagnostic does not grant permission to use stale state.

Detecting Vector Reallocation Before Using an Iterator

Purpose. Iterator validity depends on container operations; vector growth is a classic invalidation boundary.

```
#include <iostream>
#include <vector>
int main(){ std::vector<int> v; v.reserve(2); v.push_back(1); auto
before=v.data(); v.push_back(2); auto stable=v.data()==before; v.push_back(3);
auto reallocated=v.data()!=before; std::cout<<std::boolalpha<<stable<<'
'<<reallocated<<'\n'; }
```

How it works.

- Reserving capacity allows the first two insertions without reallocation.
- Once capacity is exceeded, vector may allocate a new buffer and invalidate pointers/iterators to old storage.
- The code compares storage addresses without dereferencing any invalid iterator.

Expected behavior/output. Typical output is `true true` for the requested reserve and third insertion.

Experiment. Reserve 10 instead and verify that the third insertion keeps storage stable.

Worked example — Erase safely and preserve position across a deliberate reallocation

Requirement and design

Remove odd integers from a vector while traversing it, preserve the value at a chosen remaining index across reserve-induced reallocation, and compare traversal facilities for a vector and a forward list.

The example never dereferences an invalidated iterator. It uses the erase return value and reacquires access through the owner after reallocation.

Trace consecutive removals

The starting vector contains one through six. At the first position, one is erased and the returned iterator refers to two. Two is kept, so the iterator advances to three. Repeating this process produces two, four, and six. The loop never caches an endpoint across erasure and never increments the returned iterator in the erase branch.

A stronger manual trace includes two adjacent elements that both need removal. That exposes the common extra-increment defect immediately: after removing the first, the second shifts into its position and must be examined next. An alternating input alone can sometimes make a flawed traversal look plausible.

After filtering, the example saves an index and deliberately triggers reserve-driven reallocation. It then accesses through the current vector rather than through a stale iterator. Finally it contrasts distance over linked and indexed storage. Each step isolates one capability or validity rule.

Complete implementation

The complete, buildable chapter project is `source/book2/ch15/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <forward_list>
#include <iterator>
#include <vector>

int main() {
    std::vector<int> values{1, 2, 3, 4, 5, 6};
    for (auto it = values.begin(); it != values.end(); ) {
        if (*it % 2 != 0) it = values.erase(it);
        else ++it;
    }
    CHECK(values == std::vector<int>({2, 4, 6}));
    const std::size_t saved_index{1};
    const int before = values[saved_index];
    values.reserve(values.capacity() + 100);
    CHECK(values[saved_index] == before);
    const std::forward_list<int> singly{2, 4, 6};
    CHECK(std::distance(singly.begin(), singly.end()) == 3);
    CHECK(std::distance(values.begin(), values.end()) == 3);
    auto it = values.begin();
    std::advance(it, 2);
    CHECK(*it == 6);
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch15 -B build/b2-ch15 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch15 --parallel 2
ctest --test-dir build/b2-ch15 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch15`.

Evidence and interpretation

The erase loop leaves two, four, and six. Saving index one identifies four before and after reserve because reserve does not reorder the sequence. No iterator from the old allocation is used afterward.

Both containers report distance three, but the mechanisms differ: the vector supports random-access distance, while the forward list requires traversal. Advancing the vector iterator by two reaches six. Equal returned values do not imply equal complexity.

The loop's two branches encode its invariant: the current iterator always identifies the next unclassified element or the current end. Erasing replaces that iterator with the operation's valid successor. Keeping advances past the classified element. This reasoning scales better than trying to patch skipped values after a fragile loop has already run.

Failure clinic — Reserve did not save the cached end

A program reserves enough vector capacity, caches `end()`, and appends while iterating. The author concludes that all iterators remain safe because storage did not move. The conclusion overlooks the changed end position. Revisit the operation-specific invalidation rules, and redesign the loop to process an explicit original count or stage new elements separately when that is the intended behavior.

Another defect erases an element and then increments the invalid old iterator. Assign the returned iterator instead. A third defect saves an index across erasure before that index and later updates the wrong record. That is an identity error rather than necessarily a dangling-address error, so a memory sanitizer may not detect it.

The durable repair is to write the intended traversal and mutation policy before the loop. Are newly appended elements part of this pass? Are removals immediate or staged? Must a handle identify the same record after reordering? Different answers require different mechanisms.

Practice and implementation lab

Test the erase loop with an empty vector, all odd values, all even values, and alternating values ending in an odd value. Verify the exact remaining sequence for each case.

Save an index, insert before it, and explain why the same numeric index may now identify a different element without being out of bounds. Compare the complexity of repeated **advance** on a forward list with direct indexing on a vector. Separate correctness observations from performance measurements.

Review answers and solution notes

Assigning erase's returned iterator avoids both stale access and skipping the successor. A vector's reserved capacity does not make old iterators survive an actual reallocation. A saved index survives storage relocation only as a position, not as an immutable identity across insertions and erasures.

Iterator categories describe available operations and complexity expectations. Two calls to distance can return the same number while performing different amounts of work. Container-specific invalidation rules are part of every mutating algorithm's proof.

Engineering takeaway. Preserve a valid traversal state after every mutation, and distinguish element position, identity, and storage address.

Guided lab — Build an invalidation decision sheet

For vector `append`, `reserve`, `erase`, and `sort`, record what happens to iterators, references, endpoints, positions, and logical IDs. Keep the entries operation-specific. Sorting, for example, can preserve storage while changing which value occupies a position; that is different from reallocating the storage itself.

Implement removal tests for empty input, all elements removed, no elements removed, adjacent removals, and removal of the final element. Verify both the final values and the final size. Repeat the operation on a list using its appropriate `erase` behavior and compare the code without assuming identical invalidation rules.

Use the range-lifetime laboratory in Appendix C to examine a temporary span over a live array and an owning vector temporary whose range algorithm result is represented as `std::ranges::dangling`. The type-level result prevents some accidental uses, but it does not make every manually returned reference safe. Keep the underlying owner visible in the test.

Review and solution reasoning

Iterators describe valid traversal positions, not permanent identities. Reallocation, erasure, endpoint changes, and reordering are distinct events. A saved index survives only the operations that preserve its intended position. Correct mutation loops use returned state, and correctness arguments must remain valid even when a particular sanitizer reports nothing.

Primary reference notes

Iterator and mutation contracts: [iterator requirements](#), [vector modifiers](#), [vector capacity](#), and [iterator operations](#).

Chapter 16 — Selecting Standard Library Containers

A developer replaces every vector with a linked list because insertion is described as constant time. The application slows down: most work is indexed reading, and each insertion first searches for its position. A complexity label was applied without counting the whole operation.

Container choice depends on access patterns, mutation patterns, ordering, identity, and invalidation requirements. This chapter compares the major standard families through their contracts rather than a universal ranking.

Choosing a container begins with a workload, not with a favorite data structure. HarborWorks needs sequential input storage, unique-ID detection, deterministic reports, and occasional lookup by name. One container does not need to perform all of those jobs. A vector can own accepted records while a temporary set checks uniqueness and a separate sorted snapshot determines presentation order.

Write down the dominant operations, their input sizes, how positions are found, and which handles must remain valid. “Fast insertion” is incomplete without saying whether the insertion position is already known. “Fast lookup” is incomplete without saying what key is used, how it is hashed or compared, and whether output order matters.

16.1 Using Sequential Containers

Sequential containers arrange elements in an explicit sequence. They differ in storage layout, access capabilities, and the operations that preserve references or iterators. Those differences affect both algorithm choice and practical performance.

Begin with the workload: how are positions found, which operations dominate, and what handles must remain valid across changes?

16.1.1 Comparing vector, deque, and list

A vector provides contiguous element storage and efficient indexed access. A deque provides indexed access with a different, non-contiguous overall storage organization and efficient operations at both ends. A list provides linked bidirectional traversal and node-oriented insertion or removal at known positions.

The standard specifies behavior and complexity without requiring every implementation to use one exact internal layout. In particular, do not infer a portable deque block size or a list node’s byte size from one debugger session. Such observations can explain a measured implementation but are not the container’s universal contract.

A vector’s contiguous storage supports efficient traversal and indexed access. It often makes a good default for owned sequences because elements are close together and the representation is compact. Inserting in the middle can shift later elements, and growth can reallocate. Those are predictable tradeoffs to evaluate against the workload rather than reasons to reject vectors categorically.

A deque supports efficient growth at both ends and random access, but its overall storage is not one contiguous array. Code that assumes `&deque[0] + i` reaches every element is not using its documented model. A list provides node-based bidirectional traversal and operations such as splicing under the appropriate allocator conditions, but each node has storage and indirection costs.

The right comparison includes finding a position. A list can insert near a known iterator without shifting unrelated elements, but finding the tenth-thousandth element by repeated traversal is not constant time. A vector may outperform a list for many practical workloads despite a less favorable isolated insertion complexity because traversal and locality dominate. Measure a representative sequence of operations, not only the one operation that makes a preferred container look best.

The worked splice uses ordinary lists with compatible default allocators. Moving nodes between allocator-incompatible lists is not a harmless extension of that example. Allocator-aware operation requirements belong to the container contract even when introductory programs use defaults.

16.1.2 Evaluating Insertion and Access Costs

List insertion at an already known valid position can avoid shifting later elements, but locating that position may require linear traversal. Vector insertion in the middle moves later elements, while appending can be amortized constant time across growth. Count lookup, allocation, element movement, and subsequent traversal together.

Stable references can justify a node-based structure even when raw traversal is slower. Conversely, a dense read-heavy workload often benefits from contiguous storage. Use complexity to eliminate obviously unsuitable designs, then benchmark representative complete operations rather than an isolated instruction chosen to favor one container.

Separate access, insertion, erasure, allocation, and movement costs. A vector lookup by index is constant time, but finding a record by an arbitrary ID requires a search unless another index is maintained. A list insertion at a known position has a different cost from searching for that position. A deque's end operations differ from its middle operations.

Handle stability can be more important than the smallest asymptotic expression. A subsystem that retains references across insertions may need a representation or indirection strategy that preserves them. Another subsystem can avoid the requirement by retaining stable IDs and reacquiring records. Changing the interface can sometimes remove a costly representation constraint more effectively than changing the container.

Memory usage includes more than element payload. Node links, allocation metadata, spare vector capacity, hash buckets, and alignment all contribute. For a thousand tiny records, per-node overhead can be substantial relative to the data. For a few expensive objects requiring stable addresses, an extra level of ownership indirection may be reasonable. Use the actual element type and expected population in the analysis.

Do not claim a measured winner from an unoptimized toy program that prints inside the timed loop. Correctness tests, diagnostic instrumentation, and performance experiments have different purposes. This volume teaches the decision factors; Book III and the performance volume deepen algorithmic and hardware measurement.

Choosing Vector and Deque for Different Access Patterns

Purpose. Sequential containers share ordering semantics while differing in operations and performance tradeoffs.

```
#include <deque>
#include <iostream>
#include <vector>
int main(){ std::vector<int> history{1,2}; history.push_back(3); std::deque<int>
work{2,3}; work.push_front(1); std::cout<<history.front()<<'
'<<history.back()<<' '<<work.front()<<'\n'; }
```

How it works.

- Vector naturally models append-heavy contiguous history.
- Deque supports efficient insertion at both front and back.
- Both expose sequence order but with different storage and invalidation characteristics.

Expected behavior/output. The output is `1 3 1`.

Experiment. Replace the deque with vector and compare the operation needed to insert at the front.

16.2 Using Associative Containers

Associative containers organize elements by keys or uniqueness rather than by an arbitrary sequence position. Ordered containers use a comparison relation, while unordered containers use hashing together with key equality.

The key relationship is part of the container's invariant. Changing it inconsistently can make lookups fail even though the data appears present.

16.2.1 Comparing Ordered Maps and Sets

A set stores unique keys according to its comparison-equivalence relation. A map associates unique keys with mapped values and traverses them in comparison order. The comparator must meet the required ordering properties; using `<=` instead of a strict relation is not a harmless variation.

Ordered traversal is useful for range queries and deterministic reports. A map's `operator[]` can insert a missing key with an initialized mapped value, so it is not a read-only lookup operation. Use `find`, `contains`, or checked `at` according to the intended missing-key policy. Do not let observation accidentally mutate the index.

Ordered associative containers maintain key order according to a comparator. Their notion of key equivalence comes from that comparator, not necessarily from `operator==`. Two keys can be equivalent when neither compares before the other. If a case-insensitive comparator treats two spellings as equivalent, a set will not store them as distinct keys merely because their byte strings differ.

A comparator must remain a valid ordering relation for the stored keys. Mutating external comparator state so that existing keys would order differently can invalidate the container's assumptions. Likewise, a key cannot be casually modified in place through an ordinary map iterator. The container's structure depends on the ordering relationship used when elements were placed.

Use `find`, `contains`, or `at` according to the desired absence behavior. `Map operator[]` can insert a default-initialized mapped value for a missing key. That is useful when the operation intends to create or accumulate an entry, but it is a defect in a read-only existence check. The worked example verifies that looking up a missing key through `find` does not change size.

Deterministic key order can simplify reports and tests. It is not automatically the application's desired presentation order. A map ordered by ID does not produce priority order unless those relationships coincide. Keep storage identity and display sorting separate when the domain requires different orderings.

16.2.2 Comparing Hash-Based Containers

An unordered map uses a hash to organize candidate locations and equality to decide key matches. Equal keys must produce compatible hash values under the container's requirements. Average constant-time lookup depends on hashing and workload conditions; worst-case behavior can be linear.

Iteration order is not a sorted-order contract and may change after rehashing or between implementations. Rehashing also has iterator consequences distinct from element-reference rules. Use an ordered container or sort a separate result when output order is a requirement. Reserve and load-factor tuning can reduce avoidable rehashing but do not repair a poor or adversarial hash policy.

Unordered associative containers organize keys through a hash function and an equality predicate. Equivalent keys must produce compatible hash results under the container's requirements. A custom equality rule that ignores case paired with a case-sensitive hash can violate the intended model even though both functions compile separately.

Average lookup complexity is not a worst-case latency guarantee. Hash quality, load factor, adversarial keys, rehashing, and allocation affect behavior. A service with strict timing or untrusted key concerns needs a workload-specific analysis rather than the sentence "hash maps are constant time." The book's small fixtures use ordinary keys and do not claim denial-of-service resistance from the container choice.

Iteration order is not a stable output contract. Rehashing, insertion history, library implementation, or key distribution can change it. A report that requires reproducibility should collect keys or records and sort them by an explicit rule before publication. Tests should check membership and values for unordered storage, not one accidental traversal order.

Rehash affects iterators differently from references and pointers to existing elements. Erasure has its own rules. Reuse the operation-specific reasoning from Chapter 15 rather than assuming a hash table behaves like a vector. For application identity, a key-based lookup often communicates the intent more directly than retaining an iterator through unrelated mutations.

Key Lookup with `std::map` and Membership with `std::set`

Purpose. Associative containers organize data by key rather than by positional index.

```
#include <iostream>
#include <map>
#include <set>
#include <string>
int main(){ std::map<std::string,int> score{{"Ada",10},{"Grace",12}};
std::set<std::string> active{"Ada"}; std::cout<<score.at("Grace")<<'
'<<std::boolalpha<<active.contains("Ada")<<'\\n'; }
```

How it works.

- The map associates unique keys with values.
- The set stores keys without separate mapped data.
- `at` and `contains` express lookup intent directly.

Expected behavior/output. The output is `12 true`.

Experiment. Insert a duplicate key into the set and inspect whether its size changes.

Worked example — Exercise sequence and key-based container contracts

Requirement and design

Use a deque at both ends, splice one list into another while retaining an element iterator, compare ordered and hash-based key lookup, and verify uniqueness in a set. Confirm that a failed lookup does not insert a new key.

The list splice uses compatible default allocators. The example tests semantics, not elapsed-time superiority.

Match each container to one observable property

The deque demonstrates insertion at both ends. The lists demonstrate node transfer with a retained iterator to a moved element. The ordered map demonstrates key order, while the unordered map demonstrates equivalent lookup by key without a required iteration order. The set demonstrates uniqueness. These are separate mechanisms, not a benchmark ranking the containers.

The retained list iterator refers to an element moved into the destination list by splice. The element survives; ownership by the containing list changes. That is different from copying the element into a new node and erasing the old node. The example's iterator check is meaningful because the selected operation provides the relevant preservation guarantee.

For the missing-key lookup, record size before the call and compare afterward. This catches accidental insertion through `operator[]`, a behavior bug that produces no memory error. Container correctness often depends on whether an operation mutates state, not only on whether the returned value looks reasonable.

Complete implementation

The complete, buildable chapter project is `source/book2/ch16/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <deque>
#include <list>
#include <map>
#include <set>
#include <string>
#include <unordered_map>
#include <vector>

int main() {
    std::deque<int> jobs{2, 3};
    jobs.push_front(1); jobs.push_back(4);
    CHECK(jobs.front() == 1 && jobs.back() == 4);
    std::list<int> first{1, 2}, second{3, 4};
    auto stable = second.begin();
    first.splice(first.end(), second);
    CHECK(second.empty() && *stable == 3);
    CHECK(first.size() == 4);
    std::map<std::string, int> ordered{{"beta", 2}, {"alpha", 1}};
    std::unordered_map<std::string, int> hashed(ordered.begin(), ordered.end());
    CHECK(ordered.begin()->first == "alpha");
    CHECK(hashed.at("alpha") == ordered.at("alpha"));
    CHECK(!hashed.contains("missing"));
    const auto old_size = hashed.size();
    CHECK(hashed.find("missing") == hashed.end());
    CHECK(hashed.size() == old_size);
    const std::set<int> unique{3, 1, 3, 2};
    CHECK(unique.size() == 3);
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch16 -B build/b2-ch16 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch16 --parallel 2
ctest --test-dir build/b2-ch16 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch16`.

Evidence and interpretation

The deque's endpoints become one and four. Splicing transfers the second list's nodes into the first, leaves the second empty, and preserves the iterator to the transferred value three under this operation's contract. The first list then contains four elements.

The ordered map begins with `alpha` under its default lexical ordering. The unordered map retrieves the same mapped value but is not asked to produce a particular traversal order. Searching for a missing key leaves its size unchanged, making read-only lookup behavior explicit. The set removes duplicate-equivalent values and contains three keys.

These checks demonstrate why one container cannot be substituted for another solely because both expose `begin` and `end`. Ordering, reference stability, lookup semantics, and mutation effects are part of the application's behavior.

Failure clinic — A report inherited hash order

An application prints an unordered map directly and stores the output as a golden fixture. A later library or capacity change reorders the lines, and the test reports a regression even though the logical records are unchanged. Either output order is not part of the contract, in which case the test should compare unordered content, or deterministic output is required, in which case the implementation needs an explicit ordering stage.

Another defect uses `counts[name]` to check whether a name exists. Missing names are inserted with zero counts, changing the dataset during inspection. Use an absence-preserving query when the operation is read-only. A third defect compares list insertion with vector insertion while excluding the list's position search. That measurement answers a narrower question than the advertised workload.

A container decision should be revisited when requirements change. Adding long-lived references, deterministic ordering, bounded latency, or a second lookup key can change the tradeoff. Do not retain a container solely because it was chosen before those requirements existed.

Practice and implementation lab

Write a workload description for a task index that needs exact lookup, ordered reporting, and frequent updates. Compare one ordered map with an unordered map plus a sorted report snapshot. Include the cost of producing the report.

Add a test showing that `operator[]` inserts a missing mapped value, then replace it with a nonmutating query where observation is intended. Review the allocator preconditions for list splice before adapting it to custom allocators.

Review answers and solution notes

A missing-key query should not use an insertion operation unless creating the key is intended. Ordered maps provide key ordering directly, while unordered maps need additional work for sorted reports.

Constant-time list insertion assumes a position is already known and does not include an arbitrary search. Splice has allocator and iterator contracts that must be respected. Container selection is a workload decision balancing complete operation costs and handle-validity requirements, not a slogan about one method's complexity.

Engineering takeaway. Choose the container whose ordering, ownership, access costs, and invalidation rules fit the complete workload.

Guided lab — Separate ownership from indexes

Design a small catalog that owns records in a vector and builds an ID index for lookup. Decide whether the index stores positions, stable IDs, or copies. If it stores positions, rebuild or update it whenever mutations can change positions. Test that a failed duplicate-ID load leaves the previous catalog and its index consistent.

Compare an ordered and unordered index using the same accepted records. Verify lookup results and missing-key behavior. Generate report output through an explicitly sorted snapshot so both storage choices produce the same public order. This demonstrates that a storage optimization need not redefine presentation semantics.

For the workload worksheet, estimate counts for traversal, lookup, insertion, erasure, and retained-handle use. State which operations require searching for a position first. Record expected population and element size. Only then design a benchmark representative of those combined operations.

Review and solution reasoning

Container selection is a contract and workload decision. Ordered key equivalence follows the comparator; hash lookup depends on compatible hash and equality policies. Read-only queries should not insert accidentally. Stable output order should be explicit. A data structure's headline complexity is only one part of its practical cost and validity model.

Primary reference notes

Container specifications: [sequence containers](#), [associative requirements](#), [unordered requirements](#), and [list operations](#).

Chapter 17 — Processing Data with Standard Algorithms

A report sorts jobs by priority, but equal-priority jobs change relative order. Another routine calls binary search on unsorted data and treats an arbitrary result as a valid answer. Standard algorithms remove repetitive loops only when their preconditions and result semantics are understood.

This chapter chooses named algorithms for searching, stable ordering, transformation, and accumulation. The examples make ties, output growth, and accumulator type visible.

A standard algorithm is a named agreement between the caller and the library. The caller supplies a valid range, an appropriate operation, and any required ordering or destination storage. The library supplies the behavior described by that algorithm. Neither side can quietly omit its responsibilities. A beautifully named call with an invalid comparator is less reliable than a simple loop whose invariant is correct.

For HarborWorks, the important question is not whether a hand-written loop can be replaced. It is whether the replacement makes the report’s meaning easier to inspect. “Keep the original arrival order within each priority” is a stability requirement. “Produce exactly one converted value for every accepted sample” is an output-extent requirement. “Reject a total that cannot be represented” is a numeric requirement. State those requirements before choosing algorithms.

17.1 Searching and Sorting Collections

Searching and sorting algorithms answer different questions over ranges with different requirements. Linear search needs an equality or predicate relationship. Binary-search operations need the relevant partitioning or ordering precondition. Sorting requires a valid ordering relation and suitable iterator capabilities.

Start by writing the desired postcondition: first matching element, insertion boundary, fully ordered sequence, or stable relative order among equivalent keys.

17.1.1 Finding Elements with Standard Algorithms

`std::find` scans for an equal value and returns end when none exists. `std::lower_bound` finds the first position not less than a key under its comparison arrangement; that position may identify a larger value rather than an exact match.

The example searches for four in one, three, five, seven and obtains five as the lower-bound element. An exact-membership test must additionally compare the returned element with the requested key after checking end. The range is sorted, satisfying the stronger familiar precondition. Do not use the word “binary search” to imply that any arbitrary vector can be searched logarithmically without preparation.

A search result represents a position under a particular question. Equality search asks whether an element compares equal to the requested value. Predicate search asks whether an element satisfies a condition. A boundary search asks where a partition changes. These questions can yield the same iterator on one

example and different iterators on the next. Tests should make the distinction visible rather than relying on a conveniently chosen input.

Consider the ordered sequence 10, 20, 20, 40. The lower bound for 20 identifies the first 20, while the upper bound identifies 40. Together those positions delimit the equal values. The lower bound for 30 also identifies 40, but that does not mean 30 exists. The lower bound for 50 is end, which cannot be dereferenced. An empty sequence produces end immediately. Work through these cases on paper before writing the membership check.

For a sorted range, use the same meaningful comparison policy when sorting and searching. Sorting names without regard to case and then searching under ordinary byte ordering does not preserve the assumed arrangement. A comparator's consistency is part of the stored representation. The issue is not fixed by testing one spelling that happens to occupy the same position under both orderings.

The common description of binary search as logarithmic also needs context. Comparison count and iterator movement are different costs. A lower-bound operation over forward iterators can require linear iterator increments even while using logarithmically many comparisons. An ordered associative container's member lookup can exploit its own structure directly. Avoid replacing a container's suitable member search with a generic iterator algorithm merely because both have similar names.

Use a returned iterator only while the searched range remains valid. A successful search does not give the element an independent lifetime. If the owner changes, reallocates, or disappears, the search result may no longer identify a usable element. For an API that needs to retain identity across changes, return a stable identifier or an owning result under a documented contract rather than casually retaining an iterator.

17.1.2 Sorting According to Custom Criteria

`std::stable_sort` preserves the original relative order of elements equivalent under the comparator. Sorting jobs only by priority therefore retains their arrival sequence within each priority group. Ordinary `std::sort` does not promise that stability.

A comparator must express a strict weak ordering for these algorithms. Returning true for equal elements through `<=` violates that contract. Avoid comparators whose answer changes during sorting because of mutable external state. When deterministic tie-breaking is required rather than original-order stability, compare a tuple of explicit keys such as priority and identifier.

Strict weak ordering is a semantic requirement, not a compiler switch. The comparison must not say that an element precedes itself. It must behave consistently across repeated comparisons, and its ordering and equivalence relationships must satisfy the required laws. A comparator returning `left.priority <= right.priority` is not “more inclusive”; it gives the wrong answer when priorities are equal.

A practical review starts with three records. Ask whether comparing a record with itself returns false. Ask whether both directions can return true. Then test a chain of records and the equivalence groups induced by the comparison. These small tests do not prove every comparator correct, but they expose several common mistakes. A comparator with complex external state deserves a simpler redesign before it deserves a larger test suite.

Floating-point keys require deliberate handling of exceptional values. Ordinary less-than over a domain containing NaNs does not automatically provide a suitable ordering for every sorting use. Do not assume that a successful compilation or a familiar operator proves the semantic requirement. Restrict the input domain to finite values or define and test a policy for exceptional values before sorting such data.

Stability and determinism are related but distinct. Stable sorting preserves the prior order within comparator-equivalent groups. If input order itself changes between runs, the final order can change while remaining perfectly stable. A report that must be independent of arrival order needs an explicit secondary key, such as a unique ID. Sorting by `(priority, id)` expresses that contract directly; stable sorting by priority expresses a different contract.

Also decide whether sorting is permitted to mutate the accepted dataset. The report examples often sort an owning snapshot so the stored source remains in ingestion order. Sorting in place may be more economical, but it changes the owner's observable sequence. This is a design choice, not a harmless implementation detail when other operations depend on arrival order or positions.

Sorting Before Binary Search

Purpose. Standard search algorithms have preconditions; using the correct algorithm means establishing those conditions explicitly.

```
#include <algorithm>
#include <iostream>
#include <vector>
int main(){ std::vector<int> v{9,2,7,1}; std::sort(v.begin(),v.end());
std::cout<<std::boolalpha<<std::binary_search(v.begin(),v.end(),7)<<'\\n'; }
```

How it works.

- ``sort`` establishes the ordering precondition required by binary search.
- ``binary_search`` returns a boolean rather than an iterator.

Expected behavior/output. The program prints ``true``.

Experiment. Call ``binary_search`` before sorting and explain why the result is not meaningful under the algorithm contract.

17.2 Transforming and Reducing Collections

Transformation maps input elements into output elements, while reduction combines a sequence into an aggregate. These operations can express intent more clearly than a hand-written loop, but output extent and arithmetic behavior remain the caller's responsibilities.

An algorithm does not automatically resize every destination or choose a safe accumulator type for you.

17.2.1 Applying Element Transformations

`std::transform` applies the squaring operation and writes each result through an output iterator. `std::back_inserter` adapts a vector's `push_back`, so the initially empty destination grows as results are produced.

Passing `destination.begin()` for an empty vector would not create elements merely because the algorithm writes through it. Either size the destination first or use an insertion adapter. The sample values are bounded so squaring is representable. A general numeric transformation needs its own overflow, precision, and domain policy.

An output iterator grants a way to write; it does not necessarily grant room to write. `begin()` on an empty vector is not an append operation. Reserving capacity does not create elements either. A transformation into existing positions requires a destination with enough constructed elements. An insertion adapter grows a compatible container through its insertion interface. Choose one model and make its preconditions obvious.

A useful trace records input extent, output extent, and destination capacity separately. Suppose five input records produce five strings. A pre-sized vector of five strings supplies five writable objects. An empty vector with capacity five supplies storage but no string elements at those positions. A back-insertion adapter constructs elements as it appends them. Those three states can have similar memory allocations while permitting different operations.

A one-input transform can perform an in-place operation when the documented overlap rules are satisfied, such as replacing each integer with its own square. That does not authorize arbitrary overlapping input and output ranges. When unsure, choose a separate destination and inspect the algorithm's exact contract. An optimization that relies on a prohibited overlap is not an optimization of the same program.

Failure behavior also depends on the destination strategy. A transform that appends directly to a public vector can leave a prefix appended when the conversion throws. A transform into a private candidate can discard that candidate while leaving the accepted public value unchanged. RAII cleans up constructed strings in either case, but only the second arrangement supplies the stronger public-state guarantee. Cleanup and rollback remain different properties.

The mathematical operation needs its own proof. Widening the destination type does not automatically widen multiplication performed inside the lambda. If the lambda multiplies two narrow signed integers, overflow can occur before the result reaches a wide output element. Cast or model the inputs in an appropriate working domain, or perform a checked operation before committing the output. The supplied squaring example deliberately uses small representable values.

17.2.2 Accumulating Aggregate Results

`std::accumulate` begins with the supplied initial value, whose type influences the accumulation. The example uses `0LL` to accumulate squared integers into a `long long` result. Starting with integer zero and assigning the final result to a wider variable would not retroactively widen earlier intermediate accumulation.

`std::reduce` has different grouping and ordering allowances from a simple left-to-right accumulation, especially under execution policies. Floating-point addition is not mathematically associative in finite representation, so reordered reduction can change results. Select the algorithm according to both performance and numerical reproducibility requirements rather than replacing one name with another automatically.

The accumulator is part of the algorithm's state, not merely its final output. Starting accumulation with `0` selects an integer initial value even when the destination variable is wider. Starting with `0LL` chooses a different working type. The difference matters on every iteration. Draw the first two updates and label the type of the old accumulator, the input, the operation result, and the assigned accumulator.

A wider accumulator reduces risk but does not eliminate it for unbounded input. A million large values can exceed a wide type too. Decide whether the application rejects overflow, saturates, uses arbitrary precision, or operates under a proved input bound. The checked-total lab in Appendix C uses rejection and verifies the boundary before addition. It does not execute overflowing signed arithmetic and then inspect the damage.

Order can affect both numeric results and user-defined operations. `accumulate` follows a specified sequential progression over the input. `reduce` permits regrouping and rearrangement according to its contract, even without a parallel execution policy. An operation that encodes a sequence of stateful actions, or subtraction used as though it were addition, is not generally interchangeable between them. A parallel policy adds execution concerns; it is not the only reason results may differ.

Floating-point addition approximates real arithmetic and is not associative in finite precision. The totals from two valid evaluation orders may differ in low-order bits. This does not imply that either implementation is defective. It means the program must define its reproducibility and error-tolerance requirements. For a deterministic report, retain an agreed order and numeric policy; for a high-throughput reduction, measure and document the acceptable numerical error.

For empty input, the initial value is the result. That makes the initial value a semantic identity only when the chosen operation and domain support it. Zero is suitable for ordinary summation, while one is suitable for multiplication. A string join may require a different treatment of separators. Test empty input explicitly because it exposes assumptions hidden by every nonempty example.

Transforming Values and Reducing Them to a Total

Purpose. Transformation and reduction are separate data-flow stages that can often replace manual indexing loops.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
int main(){ std::vector<int> v{1,2,3}; std::vector<int> squares(v.size());
std::transform(v.begin(),v.end(),squares.begin(),[](int x){return x*x;});
std::cout<<std::accumulate(squares.begin(),squares.end(),0)<<'\n'; }
```

How it works.

- `transform` maps each input element to one output element.
- `accumulate` folds the transformed range into one result.

Expected behavior/output. The program prints `14`.

Experiment. Change the transformation to cube each value and predict the new total.

Worked example — Stably order jobs and transform a numeric report

Requirement and design

Sort four jobs by priority while preserving their original sequence within equal priorities. Find an insertion boundary in sorted data, verify a missing equality search, square the sequence, and sum the result with an explicit wide accumulator.

The expected squared sequence is one, nine, twenty-five, forty-nine, with total eighty-four.

Trace the report contract before selecting algorithms

The worked program performs four independent operations rather than claiming one algorithm solves the entire report. First, stable sorting arranges jobs under the priority-only comparison. Second, lower bound identifies a boundary in a separate sorted integer sequence. Third, transformation constructs squared values. Fourth, accumulation combines those values using an explicit working type.

For each operation, identify the owner of the input, whether the input changes, the result's lifetime, and the relevant precondition. The job vector changes order. The integer sequence remains sorted and unchanged during search. The squared-output vector owns newly constructed values. The final total is an independent scalar. This trace connects algorithms to ownership rather than treating the two subjects as separate chapters.

The reference program checks outcomes instead of counting internal comparisons or relying on a particular sorting implementation. A library is free to change its implementation while preserving its documented behavior. Tests that assert incidental internal details can reject a correct upgrade without revealing any customer-visible regression.

Complete implementation

The complete, buildable chapter project is `source/book2/ch17/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <iterator>
#include <numeric>
#include <vector>

struct Job { int priority; int sequence; };

int main() {
    std::vector<Job> jobs{{2, 0}, {1, 1}, {2, 2}, {1, 3}};
    std::stable_sort(jobs.begin(), jobs.end(), [](const Job& a, const Job& b) {
        return a.priority < b.priority;
    });
    CHECK(jobs[0].sequence == 1 && jobs[1].sequence == 3);
    CHECK(jobs[2].sequence == 0 && jobs[3].sequence == 2);
    const std::vector<int> data{1, 3, 5, 7};
    auto found = std::lower_bound(data.begin(), data.end(), 4);
    CHECK(found != data.end() && *found == 5);
    CHECK(std::find(data.begin(), data.end(), 2) == data.end());
    std::vector<int> squared;
    std::transform(data.begin(), data.end(), std::back_inserter(squared),
        [](int value) { return value * value; });
    CHECK(squared == std::vector<int>({1, 9, 25, 49}));
    CHECK(std::accumulate(squared.begin(), squared.end(), 0LL) == 84);
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch17 -B build/b2-ch17 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch17 --parallel 2
ctest --test-dir build/b2-ch17 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch17`.

Evidence and interpretation

After stable sorting, priority-one jobs retain sequence identifiers one and three, followed by priority-two jobs zero and two. The tests verify both groups, so an accidentally unstable tie policy is more likely to be exposed than by checking priorities alone.

Lower bound for four identifies five, illustrating an insertion boundary rather than membership. Find for two returns end. Transform grows the destination through back insertion, and accumulation returns 84 using the chosen result domain.

Each algorithm replaces a loop whose invariant is already meaningful: sorted order, preserved tie order, one output per input, or aggregate over the processed prefix. Knowing those invariants lets you select and verify the algorithm rather than treating the standard library as a bag of memorized names.

Failure clinic — Four plausible but incorrect replacements

Replacing stable sorting with ordinary sorting while leaving the requirement unchanged removes a promised tie property. The sample may still pass on one library and fail on another; accidental stability is not a contract. Replacing lower bound with a direct dereference removes the no-result branch. Replacing back insertion with an empty vector's begin iterator removes the destination extent. Replacing `OLL` with `O` can narrow accumulation before the final assignment.

These defects share a pattern: the replacement preserves the visual outline of the call while changing a hidden precondition or guarantee. Review algorithm changes with a small contract table, not only a line-by-line diff. Record range category, ordering, mutation, output extent, result lifetime, and failure behavior. The table makes semantic changes visible even when the source diff is one character.

Do not repair an ordering failure by repeatedly sorting until the expected output appears. Establish whether the comparator is valid and whether the selected algorithm promises the required tie behavior. Similarly, do not repair an output overrun by reserving more capacity without constructing elements or selecting an insertion interface. Memory allocation and object existence are different facts.

Practice and implementation lab

Add equal-priority jobs and verify stability. Replace the comparator with an explicit priority-and-sequence tie-break and explain how that changes the contract. Test lower bound before the first element, after the last, and among duplicates.

Write a transform into a pre-sized destination and compare it with back insertion. Use a dataset whose integer sum exceeds a small accumulator's range to motivate type selection, but keep the correct test implementation free of signed overflow.

Review answers and solution notes

Lower bound returns a boundary, so exact membership requires another equality decision. Stable sorting preserves input order only among comparator-equivalent elements; explicit secondary keys define a different order.

A destination iterator does not imply destination growth. The accumulation's initial value controls its working type, so widening only the final assignment is insufficient. Reductions with relaxed grouping can differ numerically from a fixed-order accumulation. Algorithm preconditions remain part of the caller's proof.

Engineering takeaway. Use algorithms to name the intended operation, then verify their range, ordering, output, and arithmetic contracts.

Guided lab — Make ties, boundaries, and failure visible

Create a report input with three priority groups and at least three records in one group. Keep a separate record of arrival order. Verify priority ordering and within-group arrival ordering independently. Then replace the requirement with priority-and-ID ordering and update the expected result deliberately. Explain why this is a specification change rather than a performance-only refactor.

Build a lower-bound table for empty input, a key below the first element, an exact key, a duplicate key, a missing interior key, and a key above the last element. For each, record the boundary and whether exact membership holds. Never dereference end merely to print a diagnostic. A diagnostic should preserve the same safety rules as the main operation.

Finally, transform records through a conversion that can reject one selected input. Test direct append into a working vector and candidate-then-swap publication. State exactly which destination changes after failure in each version. Use a distinct exception type for the planned conversion failure so the test does not accidentally catch its own assertion failure.

Review and solution reasoning

An algorithm name communicates useful intent only together with its requirements. Stable order does not imply arrival-independent order. A boundary does not imply membership. Reserved storage does not imply writable elements. A wide final variable does not repair narrow intermediate arithmetic. The strongest solution makes these distinctions part of the interface and tests, rather than leaving them as comments beside a fragile call.

Primary reference notes

Algorithm contracts: [sorting](#), [binary search](#), [nonmodifying sequence operations](#), [transform](#), and [numeric algorithms](#).

Chapter 18 — Building Data Pipelines with Ranges and Views

A report first copies every even reading into a temporary vector and then copies their squares into another vector. A range pipeline can describe the same selection and transformation without materializing both intermediate collections. But the pipeline borrows its source, so returning it after the source dies would be a lifetime defect.

Ranges improve composition; they do not suspend ownership rules. This chapter separates a lazy view from an owning result and shows why mutation of the source requires care.

A pipeline can look like a finished collection while being only a recipe for visiting another collection. That distinction is the central lesson of ranges and views. HarborWorks filters unfinished tasks and transforms them into report rows. The recipe may borrow the task vector, hold a predicate, and compute text only when traversed. Copying that recipe need not copy any tasks or precompute any rows.

The pipeline is useful because it exposes the sequence of operations. It is dangerous when its attractive notation hides ownership, invalidation, or repeated work. Before returning a view, ask what it refers to, what its callable captures, how long each dependency lives, and which mutations are allowed between construction and traversal. A view is not automatically a snapshot, a cache, or a safe asynchronous message.

18.1 Understanding Ranges

A range provides a begin/end traversal interface, potentially with different iterator and sentinel types. Range-based algorithms can accept that range directly, reducing repeated endpoint plumbing and enabling constrained interfaces.

The range is an access abstraction, not necessarily an owner. A vector owns elements; a span or many views borrow them.

18.1.1 Treating Sequences as Range Objects

Treating a sequence as a range lets an algorithm express what it traverses without hard-coding one container. It still needs capabilities such as input traversal or random access depending on the operation. C++20 concepts make many of those requirements visible in algorithm interfaces.

A range's lifetime can differ from the lifetime of its elements. Destroying a lightweight view object does not necessarily destroy the source, while keeping a view object alive does not necessarily keep borrowed source storage alive. Draw both objects and the borrowing edge whenever a range escapes a local scope.

A range supplies a way to obtain a beginning and an end under the range requirements. The two endpoints need not have the same type; an iterator and a sentinel can cooperate to describe termination. The abstraction includes standard containers, arrays, spans, and adapted sequences, but their ownership and traversal properties differ. "It is a range" is the start of an analysis, not a complete description.

A vector owns its elements. A span describes a contiguous sequence owned elsewhere. A filtering view may own a lightweight wrapper that refers to another range. A generating range can compute values rather than store a conventional array of them. Algorithms can operate through the common interface, but callers still need to know whether traversal is single-pass, multipass, random access, sized, contiguous, or borrowed.

A range being sized does not imply contiguous storage, and random access does not imply one contiguous allocation. Conversely, a range can be useful without knowing its size cheaply. Avoid adding a preliminary distance calculation to every pipeline merely to reserve output space. With some ranges, that calculation can consume work or require an extra traversal. Select the allocation strategy from the actual range category and workload.

The borrowed-range concept concerns whether iterators can remain usable after the range object itself is destroyed. It does not promise that the elements live forever. A temporary span over a live array can yield a usable iterator because the span does not own the array. The same span over storage that is about to disappear cannot manufacture a longer element lifetime. Distinguish the lifetime of the wrapper from the lifetime of the data.

Range algorithms use return types such as `std::ranges::dangling` to prevent certain unsafe results from temporary non-borrowed ranges. This is a useful compile-time boundary, not a complete lifetime checker. A view can still refer to an already-dead owner through an incorrectly constructed reference. The supplemental range-lifetime lab demonstrates safe compile-time distinctions without dereferencing a dangling object.

18.1.2 Applying Range-Based Algorithms

`std::ranges::sort(result, comparator)` operates on the owning result vector in this example. Range algorithms can also use projections to extract a key from each record without repeating that extraction inside a comparator.

A projection is not a promise of free computation or cached results. If extracting a key is expensive, its repeated evaluation may matter. Choose a representation or precomputed key strategy only after profiling a real workload. Range syntax makes an interface concise; it does not eliminate the algorithm's complexity or semantic ordering requirements.

Range-based algorithms accept a range directly and often support projections, reducing boilerplate around begin/end pairs and member extraction. A projection selects the value to compare without requiring the comparator to know the entire record. Sorting tasks by a tuple projection can make a deterministic priority-and-ID contract explicit. The projection must still return values whose comparison satisfies the algorithm's requirements.

A shorter call does not change invalidation rules or ownership. Sorting a vector through a ranges algorithm mutates the vector just as the corresponding iterator-based operation does. Finding an element returns a position whose continued use depends on the owner. Passing a temporary owning range can change the safe return type. Read the result type as carefully as the call expression.

Concept-based diagnostics can identify that a range lacks random-access capability or that a comparator cannot be invoked with the projected values. They cannot generally prove that the comparator establishes a strict weak ordering over every possible value. They also cannot prove that a borrowed source will

remain alive during later execution. Compile-time constraints remove some invalid programs; semantic review handles obligations the type system does not express.

The language baseline for this book is C++20. Later standards add convenient range facilities, but a compiler accepting C++20 does not promise those additions. The worked program materializes a view with an explicit loop rather than depending on a later conversion helper. That choice keeps the example's portability claim narrow and makes the construction of owned output visible.

When translating an iterator algorithm to a range algorithm, preserve the behavior first. Confirm the same selected elements, ordering, empty-input behavior, and ownership boundary. Only after those checks should you consider whether the new interface eliminates boilerplate or enables further composition. Syntax modernization is not a substitute for regression testing.

Range Algorithm Without Explicit Begin/End Pairs

Purpose. Ranges let algorithms work with range objects directly rather than always spelling separate iterator pairs.

```
#include <algorithm>
#include <iostream>
#include <ranges>
#include <vector>
int main(){ std::vector<int> v{4,1,3,2}; std::ranges::sort(v); for(int
x:v)std::cout<<x<<' '; }
```

How it works.

- The whole vector is passed as a range.
- The ranges algorithm obtains begin/end internally and preserves algorithm constraints in the type system.

Expected behavior/output. The values print `1 2 3 4`.

Experiment. Use `std::ranges::find(v,3)` and compare the returned iterator to `v.end()`.

18.2 Building Lazy Views

A view is intended to be a lightweight range abstraction that can adapt another range. Filtering and transformation commonly perform work as iteration requests elements rather than constructing all results eagerly.

Lazy evaluation changes when work and errors occur. Creating a pipeline is not the same as evaluating its full result.

18.2.1 Filtering Without Copying

The filter view examines source elements and yields those satisfying the even predicate. It does not create a new owning vector of selected integers. Iterating the view therefore depends on the source remaining alive and on operations that preserve the relevant iterator and predicate assumptions.

Some views cache traversal state. Mutating a source in ways that invalidate iterators or change predicate-relevant structure can make reuse subtle or invalid. The example rebuilds its filter after changing the source rather than presenting a view as an automatically maintained database query. For a stable snapshot, materialize an owning result at a deliberate boundary.

A filtering view selects elements during traversal according to a predicate. Constructing the view does not normally create a second container containing only the matches. Its iterators find qualifying positions in the underlying range. This saves an intermediate copy, but it also means later traversal depends on the underlying elements and the predicate's retained state.

A captured threshold illustrates the timing. Capturing an integer threshold by value stores the chosen value in the callable. Capturing it by reference makes later traversal depend on the current external integer and its lifetime. Both forms can be correct; they express different contracts. A pipeline sent to another component should not unexpectedly depend on a local threshold that has already gone out of scope.

Filtering may cache traversal state to meet its complexity requirements. In particular, a filter over a forward range can cache a beginning position. Changing source elements after a view has been used does not mean every future operation recomputes from the original first element. The teaching rule is to keep a view within a period of stable source state and reconstruct the view after a mutation that changes membership assumptions.

Even preserving storage addresses does not make every filtered mutation safe. The iterator and adaptor contracts restrict operations that would undermine their assumptions about selected elements. Do not use a filter over positive values and casually mutate the currently selected values so they no longer satisfy the predicate while assuming the same traversal remains valid. For an update, first identify stable targets or construct a separate result, then perform a reviewed mutation.

A const view variable does not automatically provide const access to all referenced elements, and the availability of const traversal varies by view type. Constness of the wrapper, ownership of the source, and mutability of the elements are separate questions. Use a const source or an interface that explicitly supplies read-only element access when read-only behavior is the requirement.

18.2.2 Transforming Through Composable Pipelines

The transform view computes a square for each selected value as that value is requested. Piping filter into transform describes selection followed by computation without an intermediate owned collection. Reiterating the pipeline may repeat computation and predicate evaluation according to the views' contracts.

The example materializes the pipeline into a vector with a loop because the teaching baseline is C++20 and it does not rely on later convenience facilities such as C++23 `ranges::to`. Keep language-version

claims precise. The resulting vector is independent of later source mutations, while the original views remain associated with their source and adapters.

A transform view computes its output as elements are accessed. Repeated traversal can repeat the computation. A transform that parses text, allocates strings, or queries external state therefore has observable work beyond the apparent simplicity of the pipeline expression. A view is not memoization. If repeated processing should use a stable result, materialize an owning collection at the appropriate boundary.

Returning references from a transform requires special care. Returning a reference to the current source element can be reasonable while the source remains valid. Returning a reference to a local temporary created inside the callable is not. Returning a string view into a temporary formatted string is equally unsafe despite the absence of an explicit reference symbol. Trace the owner of the returned bytes, not merely the type's spelling.

Adaptor order affects both meaning and work. Filtering records before formatting them can avoid formatting rejected records. Transforming first and then filtering formatted strings applies the predicate to a different domain and can allocate for every input. The operations commute only when their semantics justify that transformation. Do not reorder pipelines based solely on an intuition that filters should always come first.

Side effects make lazy evaluation harder to reason about. A predicate that increments a counter may be called more often than a casual reader expects as iterators advance or new traversals begin. A formatter that writes to a log on every dereference can produce duplicate entries. Prefer pure selection and transformation functions in ordinary views. When effects are required, use an explicit eager loop whose execution count and failure boundary are clear.

The final materialization point is an ownership decision. An owning vector of report rows can outlive the input, travel to another layer, and remain unchanged when the source is edited. It consumes storage and construction work, but buys a clear contract. The best design is not the one with the fewest allocations in isolation; it is the one that pays for ownership where independent lifetime is actually needed.

Filter-Then-Transform Pipeline with Views

Purpose. Views compose deferred range adaptors, allowing pipelines without materializing every intermediate result.

```
#include <iostream>
#include <ranges>
#include <vector>
int main(){ std::vector<int> v{1,2,3,4,5}; auto view=v|std::views::filter([](int
x){return x%2==1;})|std::views::transform([](int x){return x*x;}); for(int
x:view)std::cout<<x<<' '; }
```

How it works.

- The filter view selects odd values without constructing an intermediate container.
- The transform view squares values as they are iterated.
- Work occurs lazily when the final loop requests elements.

Expected behavior/output. The output is `1 9 25`.

Experiment. Change the source vector after constructing the view but before iterating and observe that the view reflects the underlying range.

Worked example — Build a lazy selection pipeline and an independent result

Requirement and design

Filter one through six to the even values, square those values lazily, and materialize the result. Verify that the source is unchanged, sort the owned result descending, then change the source and build a fresh filter over its current state.

The source stays alive throughout. The materialized result is the snapshot boundary.

Separate the recipe from the result

The worked sequence creates a view that selects even numbers and squares them. It then iterates to build an owning vector. The view is a recipe over the current source; the vector is a materialized result. Sorting the result changes only that owned result, not the original source or the mathematical definition of the recipe.

The second traversal after a source change deliberately uses a newly constructed filtering pipeline. This avoids teaching that a previously traversed view must forget all cached state when the source changes. The program demonstrates a controlled lifecycle: establish source state, construct a view, consume it within that state, end its use, change the source, and construct a fresh view.

The square operation uses small integers. The chapter is not asserting that view composition makes multiplication safe for arbitrary inputs. Numeric checking can be inserted into the transformation, but then a throwing conversion requires a decision about partially constructed output. A local candidate collection remains a straightforward way to avoid publishing a prefix as a complete report.

Complete implementation

The complete, buildable chapter project is `source/book2/ch18/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <ranges>
#include <vector>
#include <functional>

int main() {
    std::vector<int> source{1, 2, 3, 4, 5, 6};
    auto selected = source
        | std::views::filter([](int value) { return value % 2 == 0; })
        | std::views::transform([](int value) { return value * value; });
    std::vector<int> result;
    for (int value : selected) result.push_back(value);
    CHECK(result == std::vector<int>({4, 16, 36}));
    CHECK(source == std::vector<int>({1, 2, 3, 4, 5, 6}));
    std::ranges::sort(result, std::greater<>{});
    CHECK(result == std::vector<int>({36, 16, 4}));
    source[1] = 8;
    // Rebuild the view after structural/predicate-relevant source mutation.
    auto current = source | std::views::filter([](int v) { return v % 2 == 0; });
    CHECK(*current.begin() == 8);
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book2/ch18 -B build/b2-ch18 \
    -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch18 --parallel 2
ctest --test-dir build/b2-ch18 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch18`.

Evidence and interpretation

The pipeline yields four, sixteen, and thirty-six. The source remains one through six because the predicates and transformation only read it. Sorting the materialized vector produces thirty-six, sixteen, four without changing the source's order.

After replacing the second source element with eight, a newly built even filter begins at eight. The example does not assert that a previously cached view behaves like a continuously refreshed query after every possible mutation. It reestablishes the view from known current state.

This separation is useful in application architecture: borrow and compose during a bounded calculation, then own the result that must survive independently. Materialization costs memory and copying or moving, but those costs may buy a clear lifetime and snapshot contract that the caller actually needs.

Failure clinic — A returned pipeline that borrows a dead vector

A function creates a local vector, builds a filter over it, and returns the filter. The returned type may be perfectly well formed. The function still destroys its local vector on return, leaving the recipe without a valid source. Storing the returned view in a variable does not restore the destroyed elements. The appropriate repair is an owning result or a deliberately owning range arrangement whose dependencies are all valid.

Another failure captures a local configuration object by reference inside a view that outlives the function. Even if the underlying vector lives long enough, the predicate's configuration does not. Ownership analysis must include callable captures as well as the obvious element source. A diagram with only a view-to-vector arrow is incomplete when the predicate also points to local state.

The safe demonstration in Appendix C returns a materialized vector from a function with local input. It also compares a temporary span over a live array with a temporary owning vector using compile-time result types. These tests avoid intentionally executing undefined behavior. A sanitizer finding is valuable evidence, but deliberately invoking a lifetime defect is not necessary to teach why the contract is invalid.

Practice and implementation lab

Add an empty source and a source with no matching values. Count transformation calls using a carefully scoped observer and compare pipeline creation with actual iteration; do not depend on an exact call count beyond the specific instrumented traversal you control.

Return an owning vector from a helper that creates a local source. Explain why returning a borrowing pipeline into that local source would be invalid. Add a projection-based sort of records using an explicit key.

Review answers and solution notes

Creating a lazy pipeline does not necessarily compute all its values. Empty or nonmatching input yields an empty materialized result. A returned owning vector can outlive a local source; a borrowed view into destroyed local storage cannot.

Materialization creates a snapshot, while a view exposes a traversal relationship with its source. Source mutation and cached state require operation-specific reasoning. C++20 range facilities should not be silently mixed with later-standard conveniences in code advertised as C++20.

Engineering takeaway. Compose borrowed ranges within a known lifetime and materialize ownership where the result must become independent.

Guided lab — Choose an ownership boundary for report rows

Write a function that accepts a read-only collection of tasks and produces formatted rows for unfinished tasks. First return a view intended for immediate use by the caller, and document every borrowed dependency. Then write an owning-result version that can be retained after the input changes. Give the two functions names that communicate their different guarantees.

Test empty input, no matches, all matches, and multiple priority groups. For the owning version, change or destroy the original input after the function returns and verify that the rows remain unchanged. For the borrowed version, keep all dependencies alive and unchanged throughout consumption; do not manufacture an invalid test merely to hope that a sanitizer catches it.

Count formatting calls in a controlled experiment to show the difference between constructing a lazy recipe and materializing its result. Treat the counter as instrumentation, not as a promised universal call count for every adaptor implementation. Explain why repeating a traversal can repeat formatting and why a cached output vector behaves differently.

Review and solution reasoning

Ranges unify traversal interfaces; they do not unify ownership. Views often defer work and can retain borrows. Borrowed-range properties do not prolong the lifetime of external storage. Reconstructing views after relevant source changes is a clear teaching discipline. Materialization is justified when independent lifetime, stable values, or a controlled effect boundary is part of the requirement.

Primary reference notes

Range specifications: [ranges](#), [range adaptors](#), [filter views](#), and [range algorithms](#).

Chapter 19 — Modeling Optional and Variant Data

A parser returns zero on failure, but zero is also a valid reading. Another returns a boolean and fills several output parameters whose relationships callers must remember. Explicit result types can make absence, alternative outcomes, and grouped values visible in one object.

This chapter uses optional for presence, variant for a typed success-or-error alternative, and tuple decomposition for a small heterogeneous summary.

An absent reading, a malformed reading, and a valid reading of zero are three different states. A program that compresses them into one integer loses information before the reporting layer can make a sensible decision. Modern library vocabulary types help preserve those distinctions, but only when the type's alternatives match the domain rather than merely replacing a pointer or an error code mechanically.

HarborWorks uses optional values when the caller needs to know whether a result exists. It uses a variant when the caller needs to distinguish a valid result from a structured failure. It uses tuples for small combinations whose meaning is already clear at the point of use. These choices are complementary: the most general type is not automatically the clearest type for every interface.

19.1 Representing Missing Values with `std::optional`

`std::optional<T>` represents either an engaged value or no value. It is appropriate when absence itself is the important information and no richer error explanation is required by that interface.

It does not turn every failure into a missing value automatically; the function must decide which conditions map to absence and which may still throw or fail externally.

19.1.1 Returning Optional Results

The integer parser returns an optional int after requiring complete numeric conversion. Empty text, a trailing suffix, and conversion failure return no value. A successfully parsed zero returns an engaged optional containing zero.

This eliminates the sentinel ambiguity of returning zero or minus one for failure when those numbers belong to the valid domain. The optional owns its contained integer; it is not a pointer to a parser-local object. For larger results, ownership and move behavior follow the contained type.

An optional object represents either an engaged value or no value. It owns the contained value when engaged; it is not a non-owning pointer to someone else's object. An optional string can therefore safely return newly produced text from a function. An optional string view, however, owns only the view object, not the bytes it refers to. Adding optionality does not repair an underlying borrow lifetime.

The decision to return no value must have a documented meaning. A lookup can return no value for a missing ID. A parser can return no value for any rejected input when the caller does not need the reason.

A calculation can return no value for a mathematically undefined result under an agreed domain. These are different policies even though they share a representation. A caller should not have to guess whether absence means missing input, invalid syntax, unavailable infrastructure, or internal failure.

Sentinel values often fail when the domain evolves. Using minus one to indicate no count may work until the value represents a signed adjustment. Using zero to indicate no measurement already conflicts with a valid zero reading. Optionality keeps presence separate from payload and allows the entire payload domain to remain meaningful. It also makes absence checks visible during review rather than relying on an undocumented magic number.

Returning a value by copy can be a deliberate safety choice. A lookup returning an optional copy of a task allows the caller to retain a snapshot without depending on the collection's future mutations. A lookup returning a pointer or reference wrapper may avoid a copy but establishes a borrow. Neither is universally better. Choose the result according to whether the caller needs independent lifetime, direct mutation, or inexpensive immediate observation.

C++20 optional does not supply every convenience added in later language-library editions. This book uses explicit presence checks and ordinary construction so the examples remain within the stated baseline. The absence of a fashionable chaining syntax does not prevent a clear API. Preserve semantic distinctions first; select additional vocabulary only when the supported environment supplies it.

19.1.2 Checking Presence Safely

Check presence with `has_value()` or the contextual boolean conversion before dereferencing. `value()` provides checked access that can throw when disengaged, while `*optional` requires engagement. Choose one style consistently with the surrounding error policy.

Presence is not the same as truthiness of the contained value. An optional containing zero is present. A caller must not replace a presence test with an arithmetic test of the result and accidentally reject valid zeros. The example includes that boundary explicitly.

Checking presence is a control-flow decision. After a successful check, dereferencing the same unchanged optional accesses the contained value. Resetting or moving the optional between the check and use can change the relevant state. A presence check is not a permanent guarantee about all future operations, just as checking a pointer once does not protect it from later lifetime changes.

`value()` provides checked access that throws `std::bad_optional_access` when disengaged. Dereference access requires engagement. Choose the form that matches the surrounding contract rather than assuming all accessors have identical failure behavior. In ordinary expected-absence flow, an explicit branch is often clearer than using an exception to distinguish a routine missing record.

`value_or` returns a value using the contained object or a supplied fallback. Its fallback expression is evaluated as an ordinary function argument before the call, even when the optional is engaged. A costly database lookup or a side-effecting function is therefore not made lazy merely by appearing inside `value_or`. Use a branch when the fallback must run only for absence.

A fallback can also conceal a required decision. Replacing every absent configuration setting with zero may make a program continue while destroying the distinction between “explicitly disabled” and “not

configured.” Use defaults only where the domain defines them. Otherwise propagate the absence or convert it into a structured validation error at the boundary that owns the policy.

Test all three meaningful numeric states: engaged zero, engaged nonzero, and disengaged. Then test replacement and reset. For optional containers, distinguish an engaged empty container from absence when that distinction matters. A successfully loaded empty report and a report that could not be loaded are not the same outcome.

Parsing with an Explicit Optional Result

Purpose. Optional values model absence directly when a result may legitimately be missing.

```
#include <charconv>
#include <iostream>
#include <optional>
#include <string_view>
std::optional<int> parse(std::string_view s){int v{};auto
[p,e]=std::from_chars(s.data(),s.data()+s.size(),v);if(e!=std::errc{}||p!
=s.data()+s.size())return std::nullopt;return v;}
int main(){ auto v=parse("42"); std::cout<<(v?*v:-1)<<'\\n'; }
```

How it works.

- The return type makes “no parsed integer” part of the interface.
- ``nullopt`` represents failure without a magic integer sentinel.
- Callers must test presence before dereferencing.

Expected behavior/output. The program prints ``42``.

Experiment. Parse ``42x`` and verify the result is empty.

19.2 Representing Multiple Possible Types

`std::variant<A, B>` stores one active alternative from a fixed set of types. Distinct domain structs such as `Reading` and `Error` make alternatives meaningful even when their internal representations could otherwise overlap.

A variant is useful for a closed set of outcomes. It is not runtime inheritance and does not require every alternative to share a base class.

19.2.1 Using `std::variant`

The parse result holds either a `Reading` containing an integer or an `Error` containing explanatory text. Callers can inspect the active alternative with `holds_alternative`, use checked `get`, or use `get_if` for a pointer-based query.

Selecting the wrong alternative with `get` reports an error rather than reinterpreting arbitrary storage as the requested type. Some variant operations can produce a valueless-by-exception state under their

specified conditions, so generic infrastructure should not assume every conceivable variant operation is infallible. The simple construction and visitation paths here avoid pretending to cover every throwing-alternative case.

A variant represents one active alternative from a fixed set of types. The alternatives should describe meaningful domain states, such as `Reading` and `ParseError`, rather than unrelated types chosen simply because both can hold data. Named alternatives preserve meaning even when both happen to contain integers or strings internally.

A default-constructed variant attempts to construct its first alternative when that is possible. Consequently, alternative order can affect the default state. Do not assume that default construction means “nothing selected” unless the first alternative deliberately represents that state, often with a domain type or `std::monostate`. Prefer explicit construction when a meaningful initial alternative should be obvious.

Some type-changing operations can leave a variant valueless by exception. This is a distinct library state, not an extra domain alternative that the application intentionally selected. Designing alternatives with well-behaved move operations and controlling transitions can reduce the risk, but an API promising never to encounter that state needs an actual argument about its operations. The simple parser constructs a fresh result rather than repeatedly mutating a published variant between unrelated alternatives.

A variant stores its alternative object directly within its own representation, with implementation-dependent space for discrimination and alignment. It does not imply heap allocation for the alternative itself. An alternative such as `std::string` can still allocate its owned contents. Evaluate the complete representation rather than concluding that all variant-based designs are allocation-free.

Use variant for a closed set of alternatives known to the interface. Runtime polymorphism is useful for a different extension model, where new implementations can satisfy a stable base interface. Choosing between them depends on who adds new types, who adds new operations, and what ownership and coupling are acceptable. Neither mechanism is an automatic replacement for the other.

19.2.2 Visiting Variant Alternatives

`std::visit` applies a callable to the active alternative. The generic visitor in the example uses compile-time type inspection to format a reading or return the error message. Both branches return the same public result type, `std::string`.

Visitation makes handling the alternative set explicit, but an overly broad fallback branch can hide the need to update behavior when a new alternative is added. Consider exhaustive overload sets or compile-time checks for APIs where every new outcome must force a review. Match the visitor’s return and exception behavior to the caller’s requirements.

Visitation dispatches an operation according to the active alternative. The visitor must be valid for the combinations required by the call. With one variant, that means handling every possible alternative. With several variants, the combination space can grow quickly. This compile-time completeness is useful, but it does not guarantee that each handler implements the correct domain policy.

The worked generic visitor uses a compile-time type decision to format a reading or an error. The decision belongs to the type of the active alternative, not to a magic field value. Adding a new alternative

should trigger an intentional review of the visitor. A broad fallback that silently treats every unknown alternative as success defeats that advantage.

Keep the visitor's return contract consistent. A visitor returning text should return a suitable text value for every alternative, rather than a string in one branch and an unrelated integer in another. Separate formatting from side effects so a visitor can be tested with known inputs. Printing inside every branch may be convenient for a demonstration, but returning an owning string often makes a real reporting component easier to compose and verify.

Access through `get` with the wrong active alternative throws `std::bad_variant_access`. Pointer-style `get_if` permits an explicit no-match check without throwing for an inactive alternative. These are different interfaces to the same state. Select them according to whether the alternative is an established invariant or an expected branch of ordinary control flow.

A visit can still call code that throws, allocates, or mutates external state. The variant does not impose a rollback boundary around that work. If generating a report must not publish partial output, create the report in a local candidate and publish after successful visitation. The same candidate-and-commit discipline applies regardless of whether branching uses `if`, a virtual function, or a variant visitor.

Variant Result Carrying Either a Value or an Error Message

Purpose. A variant models a closed union of possible types without unsafe casts or ad-hoc tag fields.

```
#include <iostream>
#include <string>
#include <variant>
using Result=std::variant<int,std::string>;
Result divide(int a,int b){ if(b==0)return std::string{"divide by zero"}; return
a/b; }
int main(){ Result r=divide(8,0); std::visit([](const auto& x)
{std::cout<<x<<'\\n';},r); }
```

How it works.

- ``variant`` stores exactly one alternative at a time.
- The return type states both allowed result forms explicitly.
- ``visit`` dispatches based on the currently active alternative.

Expected behavior/output. The program prints ``divide by zero``.

Experiment. Call ``divide(8,2)`` and observe the integer alternative.

19.3 Grouping Heterogeneous Values

A tuple groups values of potentially different types by position. Structured bindings assign local names to those components. This is convenient for small internal results, but a named struct often communicates domain meaning better at a public boundary.

Choose positional grouping only when the positions remain obvious and stable.

19.3.1 Using Tuples

The example creates a tuple containing count, sum, and name. Its element types are deduced from the supplied values. Access by position is possible, but repeated `get<0>` and `get<1>` calls can obscure meaning in larger code.

A tuple does not validate that count and sum correspond to the same dataset. As with an aggregate, consistency belongs to the construction logic or a stronger domain type. For a reusable reporting API, a named `Summary` structure may be easier to evolve and document than a positional tuple.

A tuple combines heterogeneous values by position. It works well for a small local result whose structure is immediately understood, such as a total and a label produced together. It becomes less readable when callers must remember that position zero is an ID, position one is a priority, and position two is a count across distant modules. At that point a named structure often communicates the domain more effectively.

Tuple element types determine ownership. A tuple of strings owns strings. A tuple of string views borrows character data. A tuple of references aliases existing objects. Copying the tuple follows those element semantics; it does not turn references into independent values. An innocent-looking tuple return can therefore contain the same dangling-borrow defect as a direct reference return.

`std::tie` creates reference-based grouping for existing objects and can be useful for assignment or comparison. `std::make_tuple` ordinarily creates value elements with deduction rules that differ from tying references. These helpers are not interchangeable just because both produce tuple-shaped objects. Read the deduced element types when a lifetime or mutation surprise occurs.

Comparison of tuples proceeds lexicographically under the element comparisons. This makes a tuple projection convenient for deterministic compound keys, such as priority followed by ID. The resulting order is only as sound as the component comparisons and their domains. A tuple containing a problematic floating key does not automatically acquire a valid total ordering through composition.

Use a named result type at an API boundary when field meaning, invariants, or future evolution matter. A two-element tuple returned by a private helper can be clear. A public six-element tuple whose callers use numeric positions is harder to evolve without mistakes. Concision is valuable when it removes redundancy, not when it removes meaning.

19.3.2 Decomposing Values with Structured Bindings

`const auto& [count, sum, name] = summary;` decomposes the tuple through read-only bindings associated with the existing tuple. Using `auto` without a reference changes the hidden object involved and can introduce a copy; the binding declaration's type form therefore matters.

Do not return or retain references to decomposed components beyond the tuple owner's lifetime. Structured binding syntax shortens access but does not create independent ownership automatically. The example keeps the tuple and all bindings in one scope, making the lifetime relationship straightforward.

Structured bindings introduce names for components of a decomposable object. The declaration's reference and `const` qualifiers matter. A value binding can create a local object whose components are

then named. A reference binding aliases components of an existing object. A const reference binding offers read-only access through those names while depending on the original object's lifetime.

Consider iterating over a map. `auto [key, value]` creates a value-based binding for each element, while `auto& [key, value]` refers to the stored element. Updating the local value in the first form does not update the map. In the second form, mutation reaches the stored mapped value, subject to constness and key restrictions. The spelling difference is small; the ownership and mutation difference is substantial.

Bindings do not make the owner immortal. Returning a reference to a bound component of a local tuple can still dangle. Retaining a binding into a vector element across invalidating mutation is still unsafe. Draw the same owner-and-borrow arrows used for ordinary references. New syntax changes how names are introduced, not the fundamental lifetime rules.

Prefer descriptive binding names that expose the result's meaning. Names such as `count` and `label` are more useful than `first` and `second` when reading a calculation. But descriptive local names do not replace a well-designed public return type. A tuple that requires a paragraph to explain its positions may be a request for a named structure.

During debugging, inspect whether the binding refers to an original object or a local copy. A common false alarm is “the assignment succeeded but the container did not change,” when the program modified a copied binding. Another is “a supposedly local change altered the source,” when a reference binding was used. Both are resolved by tracing initialization and qualifiers rather than by changing the container implementation.

Tuple Return with Structured Binding

Purpose. Tuples are useful for small heterogeneous groups when defining a dedicated domain type would not add useful meaning.

```
#include <iostream>
#include <string>
#include <tuple>
auto user(){return std::tuple<int,std::string,bool>{7,"Ada",true};}
int main(){ auto [id,name,active]=user(); std::cout<<id<<' '<<name<<'
'<<std::boolalpha<<active<<'\n'; }
```

How it works.

- The tuple groups values with different types into one returned object.
- Structured binding gives names to each element at the call site.

Expected behavior/output. The output is `7 Ada true`.

Experiment. Reorder tuple elements in the return type and see how structured binding follows positional structure.

Worked example — Model absence, typed errors, and grouped report values

Requirement and design

Parse zero as a present value, reject incomplete numeric text, produce a typed reading or error, format the active alternative with a visitor, and inspect a heterogeneous summary through structured bindings.

The optional parser intentionally supplies only absence, while the variant wrapper supplies a human-readable error category.

Preserve information from parsing to presentation

The parser first demonstrates a compact optional result: an integer exists only when the complete input is accepted. It then demonstrates a richer variant result that retains an error message alongside successful reading values. These are two intentionally different interfaces to a similar parsing problem, not two layers that must always be stacked together.

Whole-input consumption matters. Parsing an initial number from `42tail` does not meet a grammar that requires one complete integer. The code checks both the error indicator and the end pointer returned by `from_chars`. It treats valid zero as a successful value rather than as a missing-result sentinel. Those checks preserve distinctions that the result type alone cannot enforce.

The tuple example groups a count and a label for decomposition. Its const-reference binding is used while the tuple remains alive. The example does not return references to that local tuple or retain them after destruction. Read these lifetime boundaries as part of the implementation, even when the short syntax makes them easy to overlook.

Complete implementation

The complete, buildable chapter project is `source/book2/ch19/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <charconv>
#include <optional>
#include <string>
#include <string_view>
#include <tuple>
#include <type_traits>
#include <variant>

std::optional<int> integer(std::string_view text) {
    if (text.empty()) return std::nullopt;
    int value{};
    const auto parsed = std::from_chars(text.data(), text.data() + text.size(),
    value);
    if (parsed.ec != std::errc{} || parsed.ptr != text.data() + text.size()) {
```

```

        return std::nullopt;
    }
    return value;
}
struct Reading { int value; };
struct Error { std::string message; };
using Result = std::variant<Reading, Error>;
Result parse(std::string_view text) {
    const auto value = integer(text);
    if (!value) return Error{"invalid integer"};
    return Reading{*value};
}

int main() {
    CHECK(integer("0").has_value());
    CHECK(!integer("12ms") && !integer(""));
    CHECK(std::get<Reading>(parse("42")).value == 42);
    CHECK(std::holds_alternative<Error>(parse("bad")));
    const std::string report = std::visit([](const auto& item) -> std::string {
        using T = std::decay_t<decltype(item)>;
        if constexpr (std::is_same_v<T, Reading>) return std::to_string(item.value);
        else return item.message;
    }, parse("42"));
    CHECK(report == "42");
    auto summary = std::tuple{3, 12, std::string{"north"}};
    const auto& [count, sum, name] = summary;
    CHECK(count == 3 && sum == 12 && name == "north");
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book2/ch19 -B build/b2-ch19 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch19 --parallel 2
ctest --test-dir build/b2-ch19 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch19`.

Evidence and interpretation

The presence test for zero succeeds, demonstrating why zero is not a suitable failure sentinel. `12ms` and empty text return no integer. Parsing `42` produces the reading alternative, while invalid text produces the error alternative. Visiting the valid result formats `42`.

The summary bindings observe count three, sum twelve, and name north from the existing tuple. They are read-only borrows, not separately owned copies under this declaration.

The layering is deliberate: the low-level integer parser answers a narrow presence question, and the higher-level parser chooses a domain result. A real parser with several error causes might return richer typed diagnostics rather than collapsing every failure into the same string.

Failure clinic — A default that erased an error

A reporting function replaces a rejected parse with zero and then emits a normal reading. The program no longer crashes, but the customer cannot distinguish malformed input from a genuine zero measurement. The defect is semantic information loss. The correction is not a different default number; it is preserving absence or a structured error until the policy-owning layer decides what to do.

Another defect constructs a variant with the wrong first alternative and assumes that default construction means empty. A third visitor catches all unknown types in a generic success branch, allowing a newly added error alternative to be formatted as ordinary data. Test the state model directly: valid zero, another valid value, malformed syntax, out-of-range value, and every declared alternative.

Finally, a tuple containing a string view into a local parse buffer is returned to the caller. Optional or variant wrapping around that tuple does not repair the buffer lifetime. Ownership must be corrected at the point the bytes cross the function boundary, usually by storing an owning string in the returned domain object.

Practice and implementation lab

Add distinct error alternatives for empty input and malformed numeric input, then make the visitor handle each explicitly. Test zero, negative values, representability limits, and trailing characters.

Replace the tuple with a named summary struct and compare readability at call sites. Write two structured-binding versions, one by value and one by reference, and use a safely observable mutable component to demonstrate which one affects the original owner.

Review answers and solution notes

Optional presence is independent of the contained integer's numeric value. A variant stores one typed alternative and visitation dispatches on that active type. Choosing a wrong alternative through checked access is not equivalent to reading a union member blindly.

Structured bindings follow the ownership and reference form of their declaration. Named structs often improve public domain interfaces, while tuples are convenient for small local groupings. Result types should remove ambiguity without hiding important error distinctions.

Engineering takeaway. Represent absence and alternative outcomes explicitly, and distinguish concise decomposition from independent ownership.

Guided lab — Design a result type that preserves the reason

Extend the parser with named error codes for empty input, malformed syntax, trailing characters, and out-of-range values. Decide whether leading spaces and a leading plus sign belong to the grammar. Test that decision explicitly rather than accepting whichever behavior a different parsing function happens to provide.

Return a variant of a valid reading and a parse-error structure containing a code and input position. Keep the input text out of the error object unless it is deliberately copied or its lifetime is guaranteed. Write a

visitor that formats every alternative into an owning string. Add a test for zero and verify that it follows the success path.

Compare the richer API with an optional-returning wrapper that intentionally discards the reason for a caller that needs only presence. Document the information loss in that wrapper. Then group a local count and label with a tuple, demonstrate value versus reference structured bindings, and explain which mutations affect the original objects.

Review and solution reasoning

Optional separates presence from payload. Variant preserves a closed set of distinct states. Tuple groups values whose positional meaning should remain clear. None of these types converts borrowed data into owned data automatically. Defaults, visitors, and bindings must preserve the intended state model, not merely satisfy the compiler.

Primary reference notes

Result-type contracts: [optional](#), [variant](#), [tuples](#), [structured bindings](#), and [character conversion](#).

Chapter 20 — Working with Files, Paths, Time, and Randomness

A report works in its author's current directory but fails when launched from a different location. Its binary output dumps a C++ struct, embedding padding and host byte order. Its timing measurement uses a wall clock that can be adjusted while the operation runs.

Files, paths, clocks, and random generators each have distinct contracts. This chapter combines their basic facilities while keeping serialization, completion, reproducibility, and measurement boundaries explicit.

The outside world does not behave like an in-memory unit test. A file can disappear between inspection and opening. Output can fail after some bytes have been written. A clock can serve a different purpose from the one suggested by its name. A random engine can be repeatable while a library distribution does not promise the same mapping across implementations. Modern C++ provides useful vocabulary for these boundaries, but it does not remove the need to define them.

HarborWorks stores input records, writes reports, inspects paths, times operations, and creates repeatable test workloads. Each activity needs a different contract. Text parsing requires a grammar; binary interchange requires an encoding; filesystem changes require error handling and scope; timing requires an appropriate clock; randomness requires a distinction between simulation and security. Keep those contracts separate even when they appear in one utility program.

20.1 Performing File Input and Output

File streams own file-related resources and expose formatted or unformatted I/O operations. Opening a file, writing all intended data, and successfully completing output are separate events that should be checked according to the application's requirements.

The fixture runner supplies a fresh temporary working directory, so the example's relative paths cannot overwrite the manuscript or another test's data.

20.1.1 Reading and Writing Text Files

An output file stream writes two newline-separated integers. Enabling exceptions for fail and bad state makes the example's output failures visible, and explicit close provides a checked completion boundary before success is reported. The destructor remains cleanup support for other exits.

Input extraction then reads both integers and verifies their values. A production record reader should also define trailing-data, encoding, and partial-record policies. Successfully reading the first two integers does not prove that an entire arbitrary file conforms to a larger grammar. The fixed fixture file is exactly the text just written.

A file stream owns an association with an external file and uses ordinary stream operations to transfer characters. Constructing the stream is not proof that opening succeeded. Inspect its state before

consuming input, and inspect the final state after operations whose success matters. By default, many stream failures are represented through state bits rather than exceptions. A program that never checks those bits can treat an incomplete result as success.

Extraction success and end-of-file are not interchangeable. A loop should normally continue while the requested extraction succeeds, then decide whether termination was expected. Testing `eof()` before reading asks about a condition that may not be known until a read attempts to pass the end. The classic extra-iteration bug arises because the program uses stale values after a failed extraction.

For structured text, define the grammar before selecting extraction operations. Whitespace-delimited extraction is appropriate for some formats. A delimiter-sensitive line protocol often needs whole-line parsing and explicit field rules. A successful prefix conversion is not a valid full record when trailing text is forbidden. A blank line may be ignored, rejected, or meaningful; the program should not decide accidentally through whichever helper is convenient.

Resource limits must be enforced at the right point. Checking a line's length after unrestricted `getline` limits accepted records but does not prevent allocation of a very large input line. The original Chapter 21 example uses that simpler pattern and documents its boundary. The hardened Appendix D reader limits retained line content while reading, before appending beyond the configured bound. Neither is a general defense against every possible hostile stream or process-level memory exhaustion.

Output needs explicit completion when failure must be reported. A destructor provides cleanup but cannot conveniently return a final write failure to the application. Check stream state after writing, flushing, and an explicit close where the file's completion is part of the operation. Even successful close does not, by itself, define crash-durable storage across every platform and filesystem. Durability is a stronger operating-system and storage contract than ordinary C++ stream completion.

20.1.2 Reading and Writing Binary Files

Binary mode controls platform text transformations; it does not make an in-memory object representation into a portable format. The example writes four explicit eight-bit octets on the documented platform baseline and reconstructs a 32-bit value by shifts in a chosen big-endian order.

Do not serialize a struct by dumping `sizeof(struct)` bytes when portability or confidentiality matters. Padding, endianness, type widths, and representation can differ, and padding can contain unintended data. A wire format should specify field widths, byte order, versioning, and validation independently of the compiler's object layout.

Writing the raw memory representation of a structure is not a portable file format. Padding, member layout, integer width, byte order, floating representation, compiler choices, and future version changes can all affect the bytes. A pointer stored in that structure is a process-local address, not a portable reference to another record. Trivially copyable does not mean suitable for persistent interchange without a defined encoding.

Define a binary format in terms of explicit fields and byte order. For example, a four-octet unsigned quantity in network-style big-endian order places its most significant octet first. Encode it using shifts and masks in an appropriate unsigned type, and decode by combining unsigned values. State the eight-bit-byte requirement when the format is defined in octets. Do not imply that every possible C++ implementation uses that representation.

Lengths in a binary format require the same care as values. Before allocating or reading a declared payload, check that the length is within an application bound and that size calculations do not overflow. A corrupt length field should not become a request for an enormous allocation. Also distinguish a valid zero-length payload from a truncated file that never supplied the length itself.

Versioning belongs in the format design, not only in a release note. A magic identifier and version field can help reject an unrelated or unsupported input early. They do not authenticate the file or prove that later fields are valid. Every field still needs structural and domain checks. Checksums can detect some corruption under an explicit scheme; they are not a substitute for authentication when an adversary can change both content and checksum.

The worked binary file is deliberately tiny: four explicitly chosen bytes with a known expected value. It teaches representation without pretending to be a production serialization framework. Extending it to records requires a grammar for field order, sizes, character encoding, optional fields, and compatibility. A bounded, documented format is easier to test than a dump of whatever an object happens to look like today.

Writing and Reading a Text File with Stream State Checks

Purpose. File I/O combines resource lifetime with stream-state validation.

```
#include <fstream>
#include <iostream>
#include <string>
int main(){ std::ofstream out("note.txt"); if(!out)return 1; out<<"hello\n";}
std::ifstream in("note.txt"); std::string line;
if(std::getline(in,line))std::cout<<line<<'\\n'; }
```

How it works.

- The output stream is checked after open.
- The extra scope guarantees the writer closes before the reader opens the same path.
- `getline` reports whether a line was successfully extracted.

Expected behavior/output. The program creates `note.txt` and prints `hello`.

Experiment. Change the path to a nonexistent directory and verify the output stream failure check catches it.

20.2 Manipulating Filesystem Paths

`std::filesystem::path` represents paths using the platform's path model and provides composition operations. It does not grant authorization, prevent symlink traversal, or make concurrent filesystem changes disappear.

Separate path construction from filesystem operations, and scope destructive cleanup to a directory whose ownership is established.

20.2.1 Inspecting Files and Directories

Filesystem queries can inspect existence, type, size, or directory entries. Many facilities offer throwing and error-code forms. Choose an error-handling policy deliberately and avoid treating a failed query as equivalent to an absent file.

A check followed by a later operation can race with another actor changing the filesystem. For example, checking that a path is safe and then opening it does not create an atomic security boundary. This chapter's isolated test directory avoids that shared-adversary scenario; secure production file handling requires platform-appropriate techniques and threat modeling.

A filesystem path is a representation of a name under platform rules. It is not an open file, a permission grant, or a guarantee that the same object will be found later. Path composition through `std::filesystem::path` is clearer than hand-inserting separators, but it does not resolve trust decisions about absolute paths, parent traversal, symbolic links, or allowed directories.

Inspecting a path and then opening it are separate operations. Another process may change the directory entry between them. A prior existence check is useful for user feedback, but it cannot generally establish a security property for the subsequent open. Security-sensitive access often needs platform-specific facilities and carefully designed directory-handle or permission boundaries beyond this portable introductory example.

`status` and `symlink_status` answer different questions about symbolic links. One follows the link when determining the target's status; the other reports the link itself. A broken link can therefore produce results that surprise code treating every entry as an ordinary file. Decide whether a traversal is meant to inspect links, follow targets, or reject links, and test that policy on an appropriate platform.

Directory traversal order is not a report-order contract. Collect entries and sort the relevant names when reproducible presentation is required. Likewise, a path that prints identically on two systems can still be subject to different case sensitivity, encoding, permission, or mount rules. Portable code should state the behavior it relies on and isolate platform assumptions instead of claiming that a path abstraction makes every filesystem identical.

Avoid logging private paths indiscriminately. A diagnostic can identify an operation and a safe relative name without exposing a user's full directory structure. The teaching fixture directory contains invented data only. A production tool should treat filenames and paths as potentially sensitive input, especially when forwarding logs to other systems.

20.2.2 Performing Portable Filesystem Operations

The `/` path operator combines directory and filename components without hard-coding one platform separator. The example creates a specifically named subdirectory inside its fresh test working directory and removes only that owned subtree when finished.

`remove_all` is destructive. Never adapt the cleanup by replacing the path with a user-supplied root or broad current-directory deletion. The test checks the expected removal count for its controlled three entries: the directory and two files. That number is a fixture fact, not a universal filesystem-cleanup expectation.

Filesystem functions often provide throwing and error-code interfaces. Choose a consistent policy at the layer boundary. A nonthrowing error-code overload can make expected failures explicit in control flow, but forgetting to inspect the error code is no safer than forgetting stream state. A throwing overload can simplify propagation, but the caller must still decide what state remains after an interrupted sequence of operations.

A sequence of filesystem changes is not automatically transactional. Creating a directory, writing a file, and renaming another file can fail midway. RAII can clean up selected temporary resources, but it cannot reverse every external effect without a designed recovery protocol. Document which paths may exist after failure and which component is responsible for retry or cleanup.

A temporary-file-and-rename strategy can improve publication, but its exact guarantees depend on platform and filesystem behavior, source and destination locations, replacement semantics, and durability requirements. Do not advertise a universally atomic, crash-safe update solely because `rename` appears in the code. For this volume, report rendering is separated from publication so the need for a stronger platform-specific publication layer remains visible.

Cleanup must be scoped to resources the operation actually owns. Calling `remove_all` on a user-supplied directory because a demonstration used it on a private test directory is a dangerous generalization. The chapter fixture runs in a fresh temporary working directory and removes only its deliberately created subtree. A production cleanup should verify ownership and retention policy rather than accepting arbitrary paths from a report field.

Idempotency means repeating an operation produces the intended state under defined conditions, not that every error can be ignored. “Directory already exists” can be acceptable when it is the intended directory with appropriate properties. It can be a problem when the name refers to a file or an unexpected link. Validate the resulting state instead of treating every unsuccessful creation as either fatal or harmless without context.

Building and Inspecting Paths with `std::filesystem`

Purpose. Filesystem paths are structured values with platform-aware operations, not merely arbitrary strings.

```
#include <filesystem>
#include <iostream>
int main(){ namespace fs=std::filesystem; fs::path root="data"; fs::path
file=root/"reports"/"run.txt"; std::cout<<file.generic_string()<<'
'<<file.extension().string()<<'\n'; }
```

How it works.

- Path concatenation uses `/` as a semantic path operator instead of manual separator strings.
- `generic_string` renders a portable slash-separated representation.
- `extension` extracts a path component without manual string parsing.

Expected behavior/output. The output includes `data/reports/run.txt` and `.txt`.

Experiment. Call `file.parent_path()` and inspect the resulting directory path.

20.3 Measuring Time and Generating Random Values

Time and randomness are stateful services whose choice affects interpretation. Use a monotonic clock for elapsed-duration measurement and a random engine/distribution pair for an explicitly defined sampling problem.

Neither a timestamp nor a pseudorandom seed automatically makes an experiment reproducible across every environment.

20.3.1 Using `std::chrono`

`std::chrono::steady_clock` is suited to elapsed time because its time points do not move backward under its clock contract. `system_clock` represents wall-clock time and can be appropriate for timestamps, but wall-clock adjustments complicate elapsed-duration assumptions.

A duration has a representation and period, so conversions should be explicit when presenting microseconds or milliseconds. The example only verifies nondecreasing steady time around its operations; it does not assert a speed threshold. Timing assertions in shared CI environments often test scheduler noise rather than application correctness.

A duration measures an amount of time; a time point identifies a point relative to a clock's epoch. Mixing these concepts through bare integers invites unit errors. `std::chrono` types keep units and clock relationships visible, and duration conversions make narrowing or truncation a deliberate operation. A variable named `timeout_ms` is helpful, but a duration type enforces more than a naming convention.

Use a monotonic clock such as `steady_clock` for elapsed intervals when monotonic progression is the requirement. Wall-clock time serves timestamps and calendar relationships but may be adjusted. `high_resolution_clock` is not a universal promise of monotonicity; its relationship to other clocks is implementation-dependent. Choose from the clock's properties, not from the most impressive name.

A measured duration includes everything between the two sample points: scheduling delays, page faults, allocation, cache state, instrumentation, and the operation itself. For a small function, those effects can dominate. Printing inside the timed interval changes the workload dramatically. Keep correctness checks outside the measurement where possible, repeat a controlled workload, and report enough context to interpret variation.

Converting a duration to whole milliseconds can yield zero for a real positive interval shorter than one millisecond. That is truncation, not proof that the operation took no time. Select a unit and representation suitable for the scale, and distinguish clock resolution from measurement accuracy. More decimal places do not automatically create more trustworthy evidence.

A timeout also needs a policy for expiry, cancellation, and cleanup. Sleeping for a duration is not proof that a thread resumes at an exact deadline. This volume does not claim hard-real-time behavior from standard timing utilities. The later systems and real-time volumes examine scheduling and deadline guarantees under explicit platform assumptions.

20.3.2 Using Standard Random Engines and Distributions

A random engine produces a deterministic sequence from its defined state, while a distribution maps engine output into a requested statistical domain. The example uses `mt19937` with a fixed seed and a uniform integer distribution from one through six.

Do not assume an exact distribution-generated sequence is portable across all standard-library implementations merely because the engine seed is fixed. Test the contract you need and record the engine, distribution, seed, library, and input generation method for experiments. This engine is not a cryptographic random source and should not be used to generate secrets.

A random engine produces a deterministic sequence from its state under its specified algorithm. A distribution maps engine output into a requested domain, such as integers from one through six. Separating the two lets the same engine support different distributions and lets tests control the engine state. It also means that engine reproducibility and distribution-result reproducibility are different claims.

Seeding a standard engine with a fixed seed is useful for repeatable tests and simulations. The exact distribution mapping is not generally a cross-library output guarantee. A test that requires the same dice sequence on every standard-library implementation may be asserting more than the distribution contract promises. The chapter checks that generated values are within the requested range and that repeated construction under the same local implementation behaves as expected, without publishing a universal sequence claim.

`std::random_device` is an interface intended to supply nondeterministic values where supported, but implementation behavior and availability vary. It is not a portable guarantee of a particular operating-system entropy service. Standard simulation engines such as Mersenne Twister are not suitable

substitutes for a reviewed cryptographic random-number facility when generating passwords, security tokens, or keys.

Modulo reduction can bias a bounded result when the source range is not evenly divisible by the target range. Prefer an appropriate standard distribution for ordinary statistical sampling instead of casually using `engine() % n`. For security-sensitive sampling, use the security library's documented facility and rejection or mapping rules rather than transplanting a simulation example.

For a failing randomized test, record the engine type, seed, input-generation rules, relevant library/compiler versions, and the generated failing case. Saving the concrete case is stronger than saving only a seed when implementation-dependent mapping is involved. Promote important failures into deterministic regression fixtures so future debugging does not depend on reproducing an entire random stream.

Timing Work and Generating a Reproducible Random Sequence

Purpose. The standard library separates random engines/distributions from clock selection and duration representation.

```
#include <chrono>
#include <iostream>
#include <random>
int main(){ std::mt19937 rng{1234}; std::uniform_int_distribution<int>d(1,6);
auto start=std::chrono::steady_clock::now(); int total=0; for(int i=0;i<1000;+
+i)total+=d(rng); auto end=std::chrono::steady_clock::now(); std::cout<<total<<
'<<std::chrono::duration_cast<std::chrono::microseconds>(end-
start).count()<<'\n'; }
```

How it works.

- A fixed seed makes the pseudo-random sequence reproducible for testing.
- `steady_clock` is appropriate for elapsed-time measurement because it is monotonic.
- The duration conversion expresses the measurement unit explicitly.

Expected behavior/output. The total is deterministic for the implementation's standard engine sequence while elapsed microseconds vary by machine.

Experiment. Replace the fixed seed with `std::random_device` and explain why exact output is no longer reproducible.

Worked example — Round-trip text and explicit binary data in an isolated workspace

Requirement and design

Create a local test directory, write and read two integers as text, write and read an explicitly ordered four-octet value, generate bounded pseudorandom integers, inspect a steady clock, and remove only the owned test files.

The supported baseline has eight-bit bytes and an available exact-width 32-bit unsigned type. The example is a serialization mechanism lesson, not a secure storage or cryptographic system.

Establish the experiment's boundaries

The fixture runner gives the chapter a private temporary working directory. The program creates a named subtree there, writes its own text and binary files, reads them back, and removes the resources it created. This isolation is part of the experiment. Running the same cleanup pattern against an arbitrary production path is outside the example's contract.

The text test verifies interpreted values. The binary test verifies an explicit byte sequence. The timing test checks a basic elapsed-time property rather than asserting a performance target. The random test checks the requested domain rather than an invented portable sequence. These are four different kinds of evidence, and the program keeps their assertions correspondingly narrow.

Before running, inspect the paths in the source and identify every operation that changes the filesystem. Then identify where final output errors are observed. This is the same observe-before-change discipline used in the Linux book, translated into application resource management.

Complete implementation

The complete, buildable chapter project is `source/book2/ch20/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <array>
#include <chrono>
#include <cstdint>
#include <filesystem>
#include <fstream>
#include <random>
#include <string>

int main() {
    namespace fs = std::filesystem;
    // The test runner supplies a fresh working directory for every test case.
    const fs::path directory{"chapter20-data"};
    CHECK(fs::create_directory(directory));
    const auto start = std::chrono::steady_clock::now();
    {
        std::ofstream out{directory / "readings.txt"};
        out.exceptions(std::ios::badbit | std::ios::failbit);
        out << "12\n24\n";
        out.close(); // Surface flush/close errors at the explicit boundary.
    }
    {
        std::ifstream in{directory / "readings.txt"};
        int a{}, b{};
        CHECK(static_cast<bool>(in >> a >> b));
        CHECK(a == 12 && b == 24);
    }
    const std::array<unsigned char, 4> bytes{0x12, 0x34, 0x56, 0x78};
    {
```

```

        std::ofstream out{directory / "value.bin", std::ios::binary};
        out.exceptions(std::ios::badbit | std::ios::failbit);
        out.write(reinterpret_cast<const char*>(bytes.data()), 4);
        out.close();
    }
    std::array<unsigned char, 4> decoded{};
    {
        std::ifstream in{directory / "value.bin", std::ios::binary};
        in.read(reinterpret_cast<char*>(decoded.data()), 4);
        CHECK(in.gcount() == 4 && decoded == bytes);
    }
    const std::uint32_t value = (std::uint32_t(decoded[0]) << 24)
        | (std::uint32_t(decoded[1]) << 16) | (std::uint32_t(decoded[2]) << 8)
        | std::uint32_t(decoded[3]);
    CHECK(value == 0x12345678U);
    std::mt19937 engine{42};
    std::uniform_int_distribution<int> distribution{1, 6};
    for (int i = 0; i < 100; ++i) {
        const int roll = distribution(engine);
        CHECK(roll >= 1 && roll <= 6);
    }
    CHECK(std::chrono::steady_clock::now() >= start);
    CHECK(fs::remove_all(directory) == 3);
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book2/ch20 -B build/b2-ch20 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch20 --parallel 2
ctest --test-dir build/b2-ch20 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch20`.

Evidence and interpretation

Text round-trip checks twelve and twenty-four. Binary round-trip checks all four bytes and reconstructs hexadecimal `0x12345678` independently of host byte order. The shifts operate on explicitly converted unsigned values, avoiding unintended signed-byte interpretation.

One hundred generated rolls must remain within one through six. Their exact sequence is intentionally not a fixture contract across standard libraries. The clock check requires nondecreasing steady time but asserts no duration target.

Finally, cleanup removes exactly the directory and its two files under the controlled fixture. Running the executable manually in a directory that already contains the same subdirectory will fail its ownership-establishment check rather than blindly reusing and deleting unknown contents. Use the fixture runner for the intended isolated execution.

Failure clinic — A successful computation with failed publication

The application calculates the correct report but fails while writing it. Returning success because the calculation completed misreports the externally visible operation. Check the output state and define an exit status for publication failure. At the same time, do not claim that an error status erased bytes already written to a terminal, pipe, or file. Detection and rollback are different properties.

A second failure assumes that a successful existence check proves a later open will succeed. A third reads a structure directly from disk and discovers that padding or byte order differs on another build. A fourth expects a fixed dice sequence from every library because the engine seed is fixed. Each defect arises from promoting a narrow observation into a stronger guarantee that the interface did not supply.

The hardened capstone tests a stream that fails after a prefix has been consumed and an output stream that accepts only a prefix. It verifies the application's accepted in-memory state separately from stream position and output bytes. That separation is the foundation for honest recovery behavior: the program can preserve one state boundary without pretending to control every external effect.

Practice and implementation lab

Add a truncated binary-file test and require rejection before reconstruction. Define a versioned record format with a magic marker, field widths, and explicit byte order. Keep that format separate from a C++ struct's memory representation.

Add a failure-path test for an unwritable destination using a controlled test environment, not a privileged system directory. Record random-engine and library details for a generated dataset, and explain which aspects of the result are intended to remain identical across platforms.

Review answers and solution notes

Binary mode does not define endianness or remove padding. A complete fixed-width record must verify the number of bytes read before decoding. Explicit close can surface output errors before success, while destructor cleanup alone cannot report every completion failure conveniently.

Steady time is appropriate for elapsed intervals, and a random engine is separate from its distribution. A fixed seed is not a cryptographic guarantee or a promise of identical distribution behavior across all libraries. Destructive filesystem cleanup must remain bounded to owned test state.

Engineering takeaway. Specify file formats, ownership boundaries, clock meaning, and randomness contracts instead of relying on convenient defaults.

Guided lab — Design a bounded file-processing contract

Write a small file-format specification with maximum record count, maximum line length, numeric grammar, empty-file behavior, and final-newline policy. Decide which limits are enforced before allocation and which only limit accepted state. Test a record exactly at the length boundary and one beyond it. Keep the input fixtures in a private temporary directory.

Add a four-byte binary field with an explicit byte order and a declared eight-bit-byte assumption. Reject truncated input before constructing a valid domain object. Do not serialize a native structure as the reference format. Test known bytes against a hand-calculated value so encoding and decoding are not merely validating each other's identical mistake.

For timing, measure a repeated in-memory operation with `steady_clock`, retain a result so the work remains meaningful, and report units explicitly. For randomness, save both a seed and a concrete generated case. Explain which results are intended to reproduce across implementations and which are only local observations.

Review and solution reasoning

A path is a name, not a held resource. Stream state must be checked where completion matters. Binary interchange needs an explicit encoding. Filesystem sequences need recovery boundaries. Elapsed time requires a suitable clock. Repeatable simulation is not cryptographic randomness. Modern types clarify these distinctions, but the application must still define the external contract.

Primary reference notes

Platform library facilities: [file streams](#), [filesystem](#), [chrono](#), and [random numbers](#).

Chapter 21 — Building a Modern C++ Application

Book I’s ledger kept a single in-memory session. The next application reads a complete task dataset, validates every record, and produces a deterministic priority report. The key new requirement is transactional loading: one bad record must not replace a previously accepted report with a half-loaded candidate.

This capstone combines value ownership, strict conversion, optional and variant results, views, projections, and separate compilation. Modern features are used where they clarify a boundary, not merely to display syntax.

The capstone is a small application with a complete behavioral contract, not a showcase that inserts every modern feature wherever it can fit. HarborWorks needs to accept task records, reject malformed updates, preserve the last accepted report after a failed replacement, and produce deterministic output. Ownership, parsing, containers, algorithms, and RAII all serve those requirements. Features that do not clarify the contract do not need to appear.

This chapter has two levels. The original chapter project, `source/book2/ch21`, is a compact multi-file reference that exposes the main architecture. The extended application in Appendix D, `source/labs/harbor_reports`, strengthens bounded input handling, structured error reporting, and failure testing. Keeping both versions makes the improvement visible. Do not attribute the extension’s pre-allocation line bound or richer tests to the simpler version.

The same discipline used throughout the Linux book applies here: establish the intended state, inspect the mechanism, make one controlled transition, verify the result, and state what recovery does not cover. For an application, the relevant state includes accepted objects, the input stream’s position, and bytes already published to an output stream. Preserving one of those states does not automatically preserve the others.

21.1 Designing the Ownership Model

The report owns its accepted task vector. Parsing temporarily owns a candidate vector and a duplicate-ID index. A successful load commits the candidate; a failed load leaves the previous accepted state unchanged.

This ownership model is the foundation of the application’s error guarantee. No global mutable collection or raw owning pointer is required.

21.1.1 Assigning Clear Resource Ownership

`Report` owns accepted `Task` values, each of which owns its title string. `Load` builds local candidate values and returns an optional `ParseError` describing an ordinary rejected input. The caller owns the input stream, and the report only borrows it during the call.

String views used while splitting one line borrow that line's buffer. Before the line is reused, the accepted task copies its title into an owning string. That is the precise point where temporary parsing views become durable application state. Retaining the views instead would make later lines overwrite or invalidate the stored titles.

Begin with a resource and state inventory. The command-line layer owns an input file stream only when a file argument is supplied. Otherwise it borrows standard input. The report owns accepted tasks. Each task owns its title string. The parser borrows the current line while splitting it into fields, then creates owning domain values before the line is reused. The renderer owns its temporary selected records and completed text. This inventory explains why most of the application needs no raw owning pointer.

The lifetime of field views is deliberately short. A `string_view` into the current line avoids allocating four temporary strings just to locate delimiters. It remains valid only while the line's storage remains suitable and unchanged. The parser copies the accepted title into a `std::string` before advancing to the next record. Returning the view instead would make the report depend on a reused parsing buffer and would violate the ownership boundary.

The report's accepted vector is private. Loading does not append directly into it. Instead, a candidate vector and a temporary uniqueness set accumulate validated tasks. Only after the entire input is accepted does a swap publish the candidate. Before that point, a malformed record, a duplicate, a stream failure, or an allocation exception destroys temporary resources without changing accepted task values. The vector's ordinary owning elements make cleanup automatic.

This is a strong guarantee for the report's accepted state, not for the world. The input stream has consumed bytes and may be in a failed state. Diagnostic output may already have been written by a caller. Allocation instrumentation or a test fault counter may have changed. A caller retrying from the beginning must supply a fresh stream or deliberately reposition a suitable stream. The report cannot promise to rewind a terminal or a network-like source.

Copying the report uses value semantics because its vector and strings own their data. A snapshot returned by value also owns independent tasks. That can be more expensive than returning a reference, but it is easy to reason about when a test or another component needs a stable before-state. The public API chooses clarity over exposing mutable storage. A future performance change should preserve the advertised value and lifetime contracts or explicitly introduce a different view API.

Shared ownership is not necessary merely because several functions read the report. Those functions can borrow a `const` reference during a bounded call. Shared ownership becomes relevant when independent lifetimes genuinely overlap, as in the snapshot publication lab. Avoid placing the whole report in a shared pointer simply to avoid deciding which scope owns it. The absence of shared ownership here is a positive design decision, not an omission from a modern C++ checklist.

21.1.2 Avoiding Raw Owing Pointers

The entry point optionally owns an input file stream and otherwise uses standard input. Its `std::istream*` is explicitly a borrowed selector between those existing streams, never a deletion owner. The selected stream remains alive throughout loading.

The report follows Rule-of-Zero composition for its accepted storage. Temporary sets, vectors, strings, and streams clean themselves up through their library ownership. A bad allocation can still propagate

rather than become a structured parse error, but the candidate-before-swap design prevents such a precommit failure from intentionally replacing the old accepted report.

The raw stream pointer in the command-line adapter is a non-owning selector. It points either to `std::cin` or to a local `istream` that remains alive for the complete load operation. No branch deletes the pointer. The owning object is visible in the surrounding scope. This is an appropriate limited use of a raw pointer: it expresses optional selection among existing objects without pretending to acquire their lifetime.

A reference could also express a selected input through a helper or conditional arrangement, but changing the syntax does not change the proof. The important facts are that the target exists throughout use, that no transfer of ownership occurs, and that cleanup belongs to the actual owner. Replacing the pointer with a smart pointer that deletes standard input would be a serious error introduced in the name of modernization.

Inside domain code, values and const references are sufficient. Task construction receives an owning string. Report loading receives a borrowed input stream. Rendering returns an owning string or writes through a borrowed output stream. None of those interfaces should retain a reference to a caller's temporary stream or local text unless a separate lifetime contract explicitly requires it.

Error objects should be ownership-conscious too. The extension stores an error code and a line number, then maps the code to static diagnostic text. It does not retain a view into the rejected line. This prevents an error message from becoming invalid when the parser reuses or destroys its buffer. A richer error that includes an excerpt would need to copy a bounded excerpt or retain a suitable owner, and should consider whether the input contains sensitive text.

Do not leak internal iterators as stable identities. A report consumer that needs to request a task later should use its validated ID, not a vector position retained across replacement. Whole-report replacement can invalidate every position and reference to the previous vector. An owning snapshot can preserve old values, while an ID can request a fresh lookup under new state. They answer different needs and should have distinct interfaces.

The application deliberately has no custom destructor, copy constructor, or move assignment for **Report**. Its member types provide the required ownership behavior. That is the Rule of Zero applied to a real boundary: custom validation and commit logic exist where the domain requires them, while ordinary resource lifecycle is delegated to library types. The rule does not prohibit meaningful methods; it prevents unnecessary manual lifecycle machinery.

Tree with Exclusive Child Ownership and Nonowning Parent Links

Purpose. An ownership model should distinguish lifetime-owning edges from observational relationships.

```
#include <iostream>
#include <memory>
#include <vector>
struct Node{ int value; Node* parent{}; std::vector<std::unique_ptr<Node>>
children; explicit Node(int v):value(v){} Node& add(int v){auto
n=std::make_unique<Node>(v);n-
>parent=this;children.push_back(std::move(n));return *children.back();} };
int main(){ auto root=std::make_unique<Node>(1); Node& child=root->add(2);
std::cout<<child.parent->value<<'\n'; }
```

How it works.

- Each child has exactly one owning `unique\_ptr` in its parent.
- The parent pointer is nonowning because the root-to-child ownership direction already determines lifetime.
- Destroying the root recursively destroys the whole tree.

Expected behavior/output. The program prints `1`.

Experiment. Add a grandchild and verify that no shared ownership is required.

21.2 Integrating Modern Language Features

The application integrates modern features around two clear phases: validation into an owned candidate, then read-only report generation. Each feature solves a specific problem in that flow.

The output is sorted by explicit keys so it does not depend on hash iteration order, file arrival order among different identifiers, or an unstated tie policy.

21.2.1 Combining Templates, Algorithms, and Ranges

The integer helper returns optional after complete `from_chars` consumption. The line parser returns a variant containing either a valid task or an error string. A set detects duplicate IDs, while a vector preserves the candidate task values. These types expose different meanings rather than forcing every outcome into a sentinel integer.

For output, a filter view selects open tasks from the accepted vector. Those tasks are copied into an owned report snapshot, then `std::ranges::sort` uses a tuple projection of priority and identifier. Priority one sorts before two and three; identifier provides deterministic tie-breaking. The owning snapshot can be reordered without mutating the accepted dataset.

The application's generic facilities serve concrete steps. `from_chars` checks numeric fields without locale-dependent whitespace skipping. A set detects duplicate IDs. A filter selects unfinished tasks. A

sorting projection establishes (`priority`, `id`) order. An optional error represents load success or rejection. A local variant in the extension distinguishes a parsed task from a parse error before publication. Each facility replaces a small, understandable mechanism rather than obscuring the whole control flow.

The grammar is intentionally explicit. A record has four fields separated by three vertical bars: ID, priority, completion flag, and title. Numeric fields require complete conversion. IDs must be positive and unique within the candidate. Priorities lie from one through three. Completion is zero or one. The original project accepts a bounded nonempty title under its documented rules; the extension additionally defines printable ASCII titles, excludes all-space titles, and limits retained line bytes during input. These differences are part of the versioned teaching contract.

Deterministic output requires more than selecting a container. The accepted vector remains in input order. Rendering selects unfinished tasks into a separate owning vector, then sorts by priority and unique ID. This produces the same public order for permutations of the same accepted records. An unordered uniqueness index could replace the set without changing the output contract because presentation does not depend on index traversal order.

A projection returning a tuple expresses compound ordering without repeating comparison logic in several branches. The integers involved are validated domain values, so the ordering is straightforward. A formatter then writes ordinary values and owned titles. The report does not rely on a lambda capturing local text by reference after the formatting call returns. Its pipeline stays within the lifetime of accepted state and publishes owned output.

Materializing selected tasks and report text has costs. It copies titles and allocates temporary storage. For this bounded instructional workload, those costs buy simple ownership and failure boundaries. A production optimization could sort pointers or indices into immutable accepted state during a bounded render call, or format directly into a reusable buffer. Such a change should be justified by measurements and should preserve the independent lifetime of any returned text.

Do not add a custom template abstraction solely because this is a modern C++ capstone. A small non-template parser with a clear domain can be more maintainable than a framework parameterized over every field policy. The generic checked-total and matrix labs already teach reusable constraints where type variation is meaningful. Application architecture should reuse those ideas selectively, not turn every local function into a library-design exercise.

21.2.2 Using RAII Throughout the Application

Local owners make each exit path easier to review. If parsing fails, the candidate and duplicate-ID set are destroyed while the accepted report remains intact. On success, swapping the candidate vector with the accepted vector is the commit point, after which the temporary destroys the old state.

The implementation checks input stream errors and the entry point checks output stream state after writing. It does not promise filesystem durability, bounded memory against hostile infinite lines, or a production-grade denial-of-service boundary. The per-line size limit is checked after `getline` has allocated the line, and the accepted row limit is ten thousand. These are teaching workload limits, not complete adversarial-ingestion protection.

RAII guarantees that owned strings, vectors, streams, and temporary objects are cleaned up as their lifetimes end. It does not automatically supply application rollback. The accepted-state guarantee comes from the candidate-and-swap algorithm. The cleanup guarantee comes from the owning members. Both are needed, and their responsibilities should be explained separately when reviewing an error path.

Consider a duplicate ID on the final input line. The candidate may contain thousands of fully constructed tasks. Returning an error destroys the candidate and uniqueness set, while the accepted vector remains unchanged because the swap was never reached. The input stream remains advanced to the duplicate. This is a useful, narrow guarantee that can be tested without pretending the entire process returned to an earlier moment.

Now consider output failure. The extension first renders to a local string, so a formatting exception does not publish a half-built textual report through `write`. Once insertion into the caller's output stream begins, however, the sink can accept a prefix and then fail. The application detects that failure but does not erase the prefix. Transactional external publication requires a separate sink-specific protocol. It cannot be obtained by adding another destructor to the in-memory report class.

Explicit stream-state checks and exception handling belong at the layer that can choose a response. The command-line layer converts validation and operational failures into documented exit statuses. The domain layer returns structured expected input errors and permits exceptional resource failures to propagate before commit. Catching every exception in the middle of parsing and pretending it was an ordinary bad record would lose the distinction between rejected data and an unavailable resource.

Destructor paths should not become a second reporting system. A file wrapper's destructor can attempt cleanup, but it cannot reliably communicate a final write failure through a return value. The chapter's explicit completion checks make those failures observable before normal success is returned. In more complex software, completion and fallback cleanup may be separate methods, exactly as the RAI resource lab demonstrates.

Finally, inspect object destruction order at the boundaries that borrow. The selected stream is used while its file owner is alive. The local report exists during rendering. The output stream belongs to the caller. Test stream buffers outlive the streams that borrow them. A complete ownership review includes these fixture lifetimes as well as the production-like code, because a dangling test double can invalidate the evidence used to judge an otherwise sound implementation.

Mini Pipeline Combining Optional, Ranges, and Strong Ownership

Purpose. Modern C++ features are most useful when combined to make ownership, absence, and algorithms explicit rather than merely using new syntax.

```
#include <iostream>
#include <optional>
#include <ranges>
#include <vector>
std::optional<int> first_even(const std::vector<int>& v){ auto evens=v|
std::views::filter([](int x){return x%2==0;}); auto it=evens.begin();
if(it==evens.end())return std::nullopt; return *it; }
int main(){ std::vector<int> v{1,3,8,10}; if(auto
x=first_even(v))std::cout<<*x<<'\n'; }
```

How it works.

- A range view expresses selection without a manual indexed loop.
- `optional` communicates that no even value may exist.`
- The vector remains an ordinary value-owning container with no manual memory management.

Expected behavior/output. The program prints `8``.

Experiment. Test a vector containing only odd values and handle the empty optional explicitly.

Worked example — Load a complete dataset transactionally and produce an ordered report

Requirement and design

Read records in the form `id|priority|done|title`. IDs are positive and unique; priorities are one through three; done is zero or one; titles contain one to two hundred characters and no pipe delimiter. Reject malformed numeric fields, duplicate identifiers, invalid ranges, and over-limit training inputs.

An empty file is a valid empty report. Blank lines are not silently skipped: they are invalid records under this grammar. On success, print the open-task count and total count, then open tasks ordered by priority and identifier. On a rejected load into an existing object, preserve the previous accepted state.

Acceptance contract for the compact application

The compact application accepts a complete input or reports a failure. Its output begins with the number of unfinished tasks and the total accepted count. Subsequent rows contain unfinished tasks ordered by priority and ID. A valid empty file produces an empty report rather than an error. A duplicate ID, malformed number, invalid priority, or empty title rejects the candidate. The in-process transaction test verifies that a failed replacement preserves the previous accepted report.

The command-line fixture tests and the transaction test serve different purposes. Separate process invocations verify input/output grammar and exit behavior. Reusing one **Report** object in an in-process test verifies state preservation across a sequence of calls. A command-line test starting from an empty report cannot by itself prove that an existing report survives a failed replacement. Both tests belong in the acceptance argument.

Follow one candidate to its commit point

Start with an accepted report containing ID 9. Load a candidate containing valid IDs 1 and 2 followed by a duplicate ID 1. The parser's line buffer owns current text. Temporary field views identify pieces of that text. Validated tasks copy titles into candidate ownership. The uniqueness set rejects the final ID before publication. The candidate's destruction releases its strings, and ID 9 remains the accepted value.

Run the same trace with a fully valid candidate. The final swap exchanges accepted and candidate storage. The old accepted vector now lives in the local candidate and is cleaned up when the function returns. No hand-written loop deletes old titles. The publication step is compact because the ownership model was designed before the update algorithm.

Complete implementation

The complete, buildable chapter project is `source/book2/ch21/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

report.hpp

```
#ifndef HARBOR_REPORT_HPP
#define HARBOR_REPORT_HPP
#include <cstdint>
#include <iosfwd>
#include <optional>
#include <string>
#include <vector>
namespace harbor {
struct Task { int id; int priority; bool done; std::string title; };
struct ParseError { std::size_t line; std::string message; };
class Report {
    std::vector<Task> tasks_;
public:
    std::optional<ParseError> load(std::istream& input);
    void write(std::ostream& output) const;
};
}
#endif
```

report.cpp

```
#include "report.hpp"
#include <algorithm>
#include <array>
#include <charconv>
#include <istream>
#include <ostream>
#include <ranges>
```

```

#include <set>
#include <string_view>
#include <tuple>
#include <variant>
#include <utility>
namespace harbor {
namespace {
std::optional<int> integer(std::string_view text) {
    if (text.empty()) return std::nullopt;
    int value{};
    const auto p = std::from_chars(text.data(), text.data() + text.size(), value);
    if (p.ec != std::errc{} || p.ptr != text.data() + text.size()) return
std::nullopt;
    return value;
}
std::variant<Task, std::string> parse_line(const std::string& line) {
    std::array<std::string_view, 4> fields{};
    const std::string_view view{line};
    std::size_t begin{};
    for (std::size_t i = 0; i < 3; ++i) {
        const auto end = view.find('|', begin);
        if (end == std::string_view::npos) return std::string{"expected four
fields"};
        fields[i] = view.substr(begin, end - begin);
        begin = end + 1;
    }
    fields[3] = view.substr(begin);
    auto id = integer(fields[0]);
    auto priority = integer(fields[1]);
    auto done = integer(fields[2]);
    if (!id || !priority || !done) return std::string{"invalid numeric field"};
    if (*id <= 0 || *priority < 1 || *priority > 3 || (*done != 0 && *done != 1)) {
        return std::string{"numeric field outside permitted range"};
    }
    if (fields[3].empty() || fields[3].size() > 200 ||
        fields[3].find('|') != std::string_view::npos) return std::string{"invalid
title"};
    return Task{*id, *priority, *done == 1, std::string{fields[3]}};
}
}
std::optional<ParseError> Report::load(std::istream& input) {
    std::vector<Task> candidate;
    std::set<int> seen;
    std::string line;
    std::size_t number{};
    while (std::getline(input, line)) {
        ++number;
        if (line.size() > 4096 || candidate.size() >= 10000) {
            return ParseError{number, "input exceeds teaching limit"};
        }
        auto parsed = parse_line(line);
        if (const auto* error = std::get_if<std::string>(&parsed)) {
            return ParseError{number, *error};
        }
        Task task = std::get<Task>(std::move(parsed));
        if (!seen.insert(task.id).second) return ParseError{number, "duplicate id"};
        candidate.push_back(std::move(task));
    }
    if (input.bad() || (!input.eof() && input.fail())) {
        return ParseError{number + 1, "input read failed"};
    }
    tasks_.swap(candidate); // Commit only after every record has passed validation.
    return std::nullopt;
}
void Report::write(std::ostream& output) const {
    std::vector<Task> open;

```

```

    for (const Task& task : tasks_ | std::views::filter([](const Task& t) {
        return !t.done;
    })) {
        open.push_back(task);
    }
    std::ranges::sort(open, {}, [](const Task& task) {
        return std::tuple{task.priority, task.id};
    });
    output << "open=" << open.size() << " total=" << tasks_.size() << '\n';
    for (const Task& task : open) {
        output << task.id << '|' << task.priority << '|' << task.title << '\n';
    }
}
}

```

main.cpp

```

#include "report.hpp"
#include <fstream>
#include <iostream>

int main(int argc, char** argv) {
    if (argc > 2) { std::cerr << "usage: report [input-file]\n"; return 1; }
    std::ifstream file;
    std::istream* input = &std::cin; // Borrow, never delete.
    if (argc == 2) {
        file.open(argv[1]);
        if (!file) { std::cerr << "error: cannot open input\n"; return 1; }
        input = &file;
    }
    harbor::Report report;
    if (const auto error = report.load(*input)) {
        std::cerr << "error line " << error->line << ": " << error->message << '\n';
        return 1;
    }
    report.write(std::cout);
    if (!std::cout) { std::cerr << "error: output failed\n"; return 1; }
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book2/ch21 -B build/b2-ch21 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b2-ch21 --parallel 2
ctest --test-dir build/b2-ch21 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b2_ch21`.

Evidence and interpretation

The application separates its public interface in `report.hpp`, implementation in `report.cpp`, and process-level input selection in `main.cpp`. The line parser uses views only while the current line is alive, then constructs an owning task. The loader does not expose a partially filled vector.

For the normal fixture, three tasks contain two open records. The priority-one task appears first, followed by the priority-two task; the completed record contributes to total count but not the open listing. A tie fixture verifies identifier ordering explicitly.

The additional transaction test exercises the guarantee across several calls on the same object. It loads an accepted record, attempts a candidate with duplicate IDs, and compares the output before and after rejection. It then accepts a replacement and finally an empty dataset. This is stronger evidence for rollback behavior than invoking a fresh process for every malformed input.

Cross-call transaction test

```
#include "course_test.hpp"
#include "../src/report.hpp"
#include <sstream>
#include <string>

int main() {
    harbor::Report report;
    std::istringstream good{"7|1|0|Keep this\n"};
    CHECK(!report.load(good));
    std::ostringstream before;
    report.write(before);
    std::istringstream invalid{"9|2|0|Candidate\n9|1|0|Duplicate\n"};
    const auto error = report.load(invalid);
    CHECK(error && error->line == 2);
    std::ostringstream after;
    report.write(after);
    CHECK(after.str() == before.str());
    std::istringstream replacement{"3|2|0|Replacement\n"};
    CHECK(!report.load(replacement));
    std::ostringstream changed;
    report.write(changed);
    CHECK(changed.str() == "open=1 total=1\n3|2|0|Replacement\n");
    std::istringstream empty;
    CHECK(!report.load(empty));
    std::ostringstream cleared;
    report.write(cleared);
    CHECK(cleared.str() == "open=0 total=0\n");
    course::report();
}
```

The remaining fixture suite checks malformed fields, priority bounds, empty titles, duplicates, missing files, and empty input. These are recorded cases, not exhaustive proof of every external failure. Output failure is checked at the process boundary, but this application does not retry partial output or provide an atomic external publication protocol.

The architecture's central invariant is simple: `tasks_` always contains the last completely accepted dataset. Local validation and a visible swap commit keep that invariant understandable even as several modern library features participate.

From the compact reference to the hardened extension

The original parser checks maximum line length after reading a line. That is a limit on accepted input, not a bound on the line buffer's peak allocation. Appendix D changes the reader so it stops retaining bytes before the configured line limit is exceeded. It consumes one excess byte to discover the violation and

does not drain or rewind the remaining input. The distinction is documented and tested at exact boundaries.

The extension adds structured error codes and consistent line numbers. It accepts LF and CRLF records under a declared policy, accepts a final record without a newline, rejects blank lines, and treats an empty file as a successful empty replacement. It uses a fixed ASCII title policy rather than implying support for every Unicode text-processing requirement. An internationalized application would need an explicit encoding and character-policy revision.

A bounded row count and line length help control retained input, but they do not define a precise total process-memory ceiling. Vector capacity, set nodes, strings, stream buffers, rendering copies, and library allocation overhead also consume memory. The extension is a bounded teaching design, not a hardened sandbox for arbitrary adversarial workloads. State the limit actually enforced rather than promoting it into a stronger claim.

Failure clinic — The test caught its own failed assertion

A fault-injection test throws `std::runtime_error` to simulate a resource failure, then catches `std::runtime_error` around both the operation and its checks. If the test harness also reports a failed check with that type, the catch can swallow a genuine test failure and incorrectly report success. Use distinct planned-failure types or keep assertions outside the protected operation. The resource and copy labs use separate injected failure types for this reason.

Another weak test compares output generated by the same sorting code with itself. It verifies consistency, not correctness. The extended report regression uses hand-written expected output and enumerates every permutation of four records. It tests that arrival-order changes do not change the public ordering. This finite experiment is meaningful evidence for those cases without pretending to prove the implementation for all possible datasets.

Separate three recovery questions

After a failed load, ask whether accepted tasks changed. Then ask where the input stream is positioned and which state bits are set. Finally, ask whether any diagnostics or output bytes were published. The answers can differ. A strong accepted-state guarantee can coexist with consumed input and emitted diagnostics. A runbook for a caller should say whether to retry with a fresh stream, preserve the old report, or ask the user to correct the source.

After a failed write, accepted tasks still exist, but the output sink may contain a prefix. Retrying to the same sink can duplicate that prefix. A file-publication adapter might use a separately validated temporary-file protocol; a message broker might use an application-level identifier and acknowledgement scheme. Those are different external contracts, intentionally outside this portable in-memory capstone.

Practice and implementation lab

Add fixtures for missing fields, an extra delimiter, invalid done values, negative IDs, leading numeric whitespace, duplicate IDs late in a file, and titles at both length boundaries. Extend the cross-call test so each rejection is followed by a comparison with the previously accepted output.

Add a new report mode that groups open tasks by priority without changing accepted storage. Define its output order and test an empty group. Keep parsing errors separate from report-format choices.

Write a production gap review covering maximum line ingestion before allocation, encoding and escaping, cancellation, output atomicity, persistence, and operational logging. Do not label the training row limit a complete security control. Identify which guarantees are local language/library properties and which require a platform or deployment design.

Review answers and solution notes

Transactional loading requires a candidate separate from accepted state and a commit that occurs only after all validation succeeds. A parser that appends directly into `tasks_` and returns an error later would violate the guarantee even if each individual task was valid.

Views into the line must not be retained after the line changes; owning title strings close that lifetime boundary. A raw stream pointer can be a legitimate borrow when its ownership is explicit. Sorting a separate snapshot avoids changing accepted data, and explicit priority/ID keys produce deterministic ordering.

The application is complete for its documented command-line report contract. Production hardening is a separate set of requirements, not a claim earned solely by using smart pointers, concepts, or ranges. Book III next examines the data structures and algorithms underlying these library choices.

Engineering takeaway. Build modern applications around explicit ownership, candidate-before-commit updates, and tests that verify state across failures.

Guided lab — Perform a release review rather than a feature demonstration

Build the compact project and run its fixtures first. Inspect every accepted-state mutation and identify the one publication point. Then build the supplemental application and run the `harbor.*` fixtures plus `lab.harbor_reports_regression`. Compare the two specifications before comparing outputs. The extension intentionally rejects some input that the compact teaching version does not reject under the same detailed policy.

Create an acceptance matrix with normal input, empty input, all-completed tasks, ties, missing fields, extra fields, whole-integer failures, invalid domains, duplicate IDs, boundary lengths, stream failures, and output failures. For every row, record expected exit or error code, accepted-state outcome, input-consumption boundary, and output-publication boundary. A test name should identify a behavior from that matrix, not merely a function name.

Add a new report option that groups unfinished tasks by priority. Keep the original accepted vector unchanged and produce priorities in an explicit order. Test that grouped counts agree with the selected records, that completed tasks do not contribute, and that empty input still produces a valid report. The supplied extension implements this option, so attempt the change in a copy before reading the reference implementation.

Next, inject an input failure after several valid records. Verify that previously accepted tasks remain unchanged, while the fixture records how many bytes were consumed. Inject an output failure after a

small prefix and verify both the error and the retained prefix. Do not assert an impossible rollback of a stream that the application does not own transactionally.

For the final review, write a one-page operational contract: supported language baseline, grammar, limits, invocation, exit meanings, ownership model, failure guarantees, test evidence, and known omissions. Include the fact that platform execution and stream behavior are verified only on the recorded environments. This document is part of the application, not paperwork added after the code is finished.

Review and solution reasoning

The capstone's strength comes from explicit boundaries. Values own accepted records; short-lived views parse borrowed text; a candidate preserves old state until commit; ranges and algorithms express selection and ordering; RAII supplies cleanup; structured errors preserve expected failure reasons; fixture and in-process tests verify different contracts. No individual feature makes the system reliable on its own. Their coordinated use makes the behavior explainable, testable, and revisable.

Primary reference notes

Capstone facilities: [character conversion](#), [optional](#), [variant](#), [ranges](#), [sorting](#), [vectors](#), and [file streams](#).

Appendix A — A Modern C++ Diagnostic Desk Reference

Start with the failed contract

A useful diagnosis begins with an observable discrepancy. “The application crashes” is a starting symptom. “The second render reads a title after its parsing buffer has been reused” identifies a lifetime hypothesis. “The destination changes after a rejected copy” identifies a state-guarantee discrepancy. “Two equal-priority records change order” identifies a presentation contract. State the discrepancy before changing implementation details.

Preserve the smallest reproducing input, the exact executable, compiler and library versions, build flags, arguments, working directory, exit status, and relevant output. Rebuilding the program before saving the failing binary can remove evidence. Reproducing with a new compiler can be useful, but it is a second experiment rather than a substitute for identifying what ran originally. The Linux book’s evidence-first habit applies directly to native application debugging.

Do not immediately replace a failing abstraction with a raw pointer or a hand-written loop. The defect may be a violated precondition, not a library limitation. Conversely, do not conceal a defect by adding a smart pointer around an object whose ownership is still ambiguous. Make one hypothesis-driven change and retain the case that demonstrated the original failure.

Gate 1 — Translation and constraints

At translation time, inspect the first relevant diagnostic rather than the longest cascade. Confirm the selected language mode and that all required headers are included directly. A header that happens to arrive through another header on one implementation is not a dependable interface. When a concept fails, identify the expression or requirement that was not satisfied before weakening the constraint.

Separate “the requested syntax is unavailable” from “the requested operation violates the template’s domain.” A C++20 build should not be repaired by assuming a C++23 library helper exists. A checked integer function rejecting `bool` should not be repaired by removing its concept. One failure concerns environment capability; the other concerns an intentional API boundary.

Template diagnostics often become easier when reduced to a named type and one operation. Replace a long pipeline temporarily with local variables whose types can be inspected. Check the callable parameter and return types. A reference qualifier or an unexpected `const` element can explain a failure more directly than adding an unconstrained forwarding layer.

Gate 2 — Linking and target composition

A declaration tells translation what an entity looks like. An out-of-line definition must also participate in the linked program when required. If the report's implementation file is absent from the target, repeatedly including its header will not resolve the missing definition. Inspect the CMake target and the actual build command, not only the editor's file tree.

Templates have additional visibility requirements because needed instantiations often require definitions to be available where used. Moving every template body to a source file without an explicit instantiation design can produce linking failures. Decide whether the template is header-defined or explicitly instantiated for a closed set of types. Both are designs; an accidental mixture is not.

Use separate build directories for different compilers and instrumentation modes. CMake caches toolchain decisions. A fresh directory makes it easier to establish what was configured than repeatedly editing a populated cache. Keep the source tree free of generated object files so a stale artifact cannot masquerade as a newly compiled result.

Gate 3 — Ownership and lifetime

For each suspect object, ask who constructed it, who destroys it, who can replace its storage, and which aliases escaped. Trace raw pointers, references, iterators, spans, string views, lambda captures, and references embedded inside tuples. The absence of the `*` character does not imply the absence of a borrow.

A pointer can be non-null and still be unusable. An iterator can have a plausible address and still be invalidated. A shared pointer can retain a control block while exposing an aliased subobject under a separate lifetime relationship. A const reference can prevent mutation through one name while another alias changes the owner. Diagnose the entire relationship, not a single reassuring property.

The first safe repair is often to shorten the borrow's scope or create an owning snapshot at a boundary. Retaining an ID and looking up the current object later can also be more appropriate than retaining a position. Do not use global storage merely to keep a local object alive; that changes lifetime, coupling, and concurrency assumptions throughout the program.

Gate 4 — Invariants and failure guarantees

Write the invariant as a statement that can be evaluated before and after the operation. A buffer's size must agree with its owned storage. A report's accepted IDs must be unique. A quantity must stay within its domain. A moved-from custom owner may promise an empty state. These statements identify what a test should inspect beyond the immediate return value.

A basic exception guarantee preserves validity and avoids leaks under the operation's contract. A strong guarantee preserves the specified observable state on failure. A nonthrowing operation promises that no exception escapes. These guarantees are not interchangeable, and each needs an explicit boundary. An operation can preserve a report while consuming input or changing a test fault counter.

Locate the commit point. If public state changes before the last potentially failing step, determine how rollback occurs or weaken the documented guarantee. Candidate construction followed by a suitable nonthrowing swap is a common solution for owning values, but allocator compatibility and member behavior still matter. Copy-and-swap is a technique, not a universal exemption from reviewing contracts.

Gate 5 — Algorithms and representation

Write the input domain, range category, comparator requirements, destination extent, mutation effects, and result semantics. A lower bound is not a membership result. Stable sorting does not make output independent of input order. Reserved vector capacity does not create destination elements. A wider receiving variable does not widen an earlier narrow multiplication.

For a container change, include how positions are found and which references are retained. A list's insertion cost near a known iterator does not include a search for that iterator. A hash table's average lookup behavior does not supply deterministic traversal order. A view's concise syntax does not supply independent element ownership or cached computation.

Use a hand-checkable reference case and a boundary case before adding a benchmark. Comparing two implementations that share the same faulty parser or comparator may confirm the same mistake twice. An independent oracle can be a manually computed result, a simpler algorithm over a bounded domain, or a reconstruction property whose assumptions are separately justified.

Gate 6 — External completion and evidence

Distinguish successful computation from successful publication. A completed report string can exist while writing to its destination fails. A successful file close is not a universal crash-durability guarantee. A prior path inspection is not a held file handle. A nonzero application exit can be the correct result of an expected rejection test.

When reporting verification, name the actual configurations and cases executed. GCC and Clang may share the same standard library on a host. Sanitizers detect classes of defects in exercised executions; they do not prove semantic correctness or every path. Keep performance measurements separate from instrumented correctness runs, and do not publish a speedup without the baseline and workload that produced it.

A reusable review record

Use one short record per investigated change: the requirement; the smallest failing case; the relevant owner and invariant; the proposed mechanism; the expected failure boundary; the observed result; and the remaining limitation. Preserve this record beside the regression fixture. It tells a future maintainer why the test exists and prevents a later “cleanup” from removing the condition that once exposed the defect.

Technical references: [object lifetime](#), [exception cleanup](#), [template constraints](#), [CMake](#), and [AddressSanitizer](#).

Appendix B — Executable Ownership and Failure Labs

These four experiments extend Chapters 3–6. Each is a complete C++20 program in `source/labs`. They use the same always-active check harness as the chapters. Run them through the root build; no device, network service, or privileged operation is required.

```
cmake -S source -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug --parallel 2
ctest --test-dir build/debug --output-on-failure \
  -R '^lab\.(resource_audit|copy_transaction|move_relocation|ownership_snapshot)$'
```

B.1 Audit a resource from acquisition to release

Requirement and model

A lease must release an acquired resource exactly once, including during exception unwinding. A failed acquisition must not be counted as a live lease. Moving a lease must transfer responsibility without duplicating it, and move assignment must first release the destination's previous resource. Repeated explicit close must be harmless under this class's contract.

The registry is a deterministic fake device with eight slots. It is not a wrapper around an operating-system handle API. Its counters make acquisition and release visible without relying on allocation addresses or external state. The registry owns the bookkeeping and must outlive every lease that borrows it. The lease owns responsibility for one slot, not the registry itself.

Complete source — `resource_audit.cpp`

```
#include "course_test.hpp"
#include <array>
#include <cstdint>
#include <stdexcept>
#include <type_traits>
#include <utility>

struct AcquisitionFailure final {};
struct PlannedFailure final {};

// A fake device registry, not an operating-system handle implementation.
// This owner must outlive every Lease that refers to it.
class Registry {
    std::array<bool, 8> occupied_{};
    unsigned acquired_{};
    unsigned released_{};
    unsigned invalid_releases_{};
public:
    int acquire(bool reject = false) {
        if (reject) throw AcquisitionFailure{};
        for (std::size_t i = 0; i < occupied_.size(); ++i) {
```

```

        if (!occupied_[i]) {
            occupied_[i] = true;
            ++acquired_;
            return static_cast<int>(i);
        }
    }
    throw AcquisitionFailure{};
}

bool release(int handle) noexcept {
    if (handle < 0 || static_cast<std::size_t>(handle) >= occupied_.size()
        || !occupied_[static_cast<std::size_t>(handle)]) {
        ++invalid_releases_;
        return false;
    }
    occupied_[static_cast<std::size_t>(handle)] = false;
    ++released_;
    return true;
}

unsigned live() const noexcept { return acquired_ - released_; }
unsigned acquired() const noexcept { return acquired_; }
unsigned released() const noexcept { return released_; }
unsigned invalid_releases() const noexcept { return invalid_releases_; }
};

class Lease {
    Registry* registry_{}; // Borrowed, with a documented enclosing lifetime.
    int handle_{-1};
public:
    explicit Lease(Registry& registry, bool reject = false)
        : registry_(&registry), handle_(registry.acquire(reject)) {}
    ~Lease() noexcept { (void)close(); }
    Lease(const Lease&) = delete;
    Lease& operator=(const Lease&) = delete;
    Lease(Lease&& other) noexcept
        : registry_(std::exchange(other.registry_, nullptr)),
          handle_(std::exchange(other.handle_, -1)) {}
    Lease& operator=(Lease&& other) noexcept {
        if (this != &other) {
            (void)close();
            registry_ = std::exchange(other.registry_, nullptr);
            handle_ = std::exchange(other.handle_, -1);
        }
        return *this;
    }
    bool close() noexcept {
        if (handle_ == -1) return true;
        const int previous = std::exchange(handle_, -1);
        return registry_->release(previous);
    }
    bool owns_resource() const noexcept { return handle_ != -1; }
};

int main() {
    static_assert(!std::is_copy_constructible_v<Lease>);
    static_assert(std::is_nothrow_move_constructible_v<Lease>);
    Registry registry;
    {
        Lease resource{registry};
        CHECK(registry.live() == 1 && resource.owns_resource());
    }
    CHECK(registry.live() == 0 && registry.released() == 1);
    try {
        Lease resource{registry};
        CHECK(registry.live() == 1);
        throw PlannedFailure{};
    } catch (const PlannedFailure&) {}
}

```

```

CHECK(registry.live() == 0 && registry.released() == 2);
CHECK(course::throws<AcquisitionFailure>([&] { Lease rejected{registry,
true}; }));
CHECK(registry.acquired() == 2 && registry.released() == 2);
{
    Lease first{registry};
    Lease second{std::move(first)};
    CHECK(!first.owns_resource() && second.owns_resource());
    CHECK(registry.live() == 1);
    Lease destination{registry};
    CHECK(registry.live() == 2);
    destination = std::move(second);
    CHECK(registry.live() == 1 && !second.owns_resource());
    Lease& alias = destination;
    destination = std::move(alias); // This class preserves self-move.
    CHECK(destination.owns_resource() && registry.live() == 1);
    CHECK(destination.close());
    const auto releases = registry.released();
    CHECK(destination.close());
    CHECK(registry.released() == releases && registry.live() == 0);
}
CHECK(registry.acquired() == registry.released());
CHECK(registry.invalid_releases() == 0);
course::report();
}

```

Evidence and interpretation

The initial scope tests ordinary destruction. The next scope throws **PlannedFailure** only after acquisition, proving that a completed lease is destroyed during unwinding. The acquisition-rejection test throws a different type before a resource is obtained. The counters therefore distinguish a failed constructor from a successfully constructed object that later leaves scope.

Move construction leaves the source without a resource. Move assignment releases the destination's previous slot before adopting the source's slot. The self-move test exercises this class's explicitly chosen preservation behavior; it is not a claim that every moved-from object in the standard library retains its value. The repeated-close check verifies that release count does not increase after the first close.

The destructor discards the close result because a destructor cannot return it to the caller. A real handle wrapper may need an explicit completion API when release or flush errors must be observed. The fake registry's release policy is intentionally simple so the experiment isolates ownership accounting. Do not generalize its handle reuse or failure behavior to a particular operating system without examining that API.

Practice boundary

Add a registry operation that reports a simulated completion failure while still consuming ownership. Decide whether retry is permitted. Then separate explicit completion from fallback cleanup and test both. Keep the registry alive throughout every lease scope. Do not “test” the borrowed lifetime by destroying the registry early and dereferencing it afterward; that would invoke a defect rather than verify a valid contract.

B.2 Preserve a destination when copying fails

Requirement and model

Assigning a sequence must either produce an independent copy of the source values or leave the destination values unchanged. Copying can fail after zero, one, or two elements have been copied. The test uses a copy budget, not actual memory exhaustion, to make those failure points reproducible.

Each cell stores an integer and a borrowed test policy. The source policy must outlive the source and its copies. The policy's counter is outside the promised value-state boundary: it changes when copying is attempted. The sequence's logical value is the collection of integers returned by `snapshot`, not the instrumentation state used to inject failures.

Complete source — `copy_transaction.cpp`

```
#include "course_test.hpp"
#include <initializer_list>
#include <utility>
#include <vector>

struct CopyFailure final {};
struct CopyBudget {
    int remaining{-1}; // Negative means copying is unrestricted.
    void consume() {
        if (remaining == 0) throw CopyFailure{};
        if (remaining > 0) --remaining;
    }
};

struct Cell {
    int value{};
    CopyBudget* budget{}; // Test policy must outlive every cell.
    Cell(int v, CopyBudget& b) : value(v), budget(&b) {}
    Cell(const Cell& other) : value(other.value), budget(other.budget) {
        budget->consume();
    }
    Cell(Cell&&) noexcept = default;
    Cell& operator=(const Cell&) = default;
    Cell& operator=(Cell&&) noexcept = default;
};

class Sequence {
    std::vector<Cell> cells_;
public:
    Sequence(CopyBudget& budget, std::initializer_list<int> values) {
        cells_.reserve(values.size());
        for (const int value : values) cells_.emplace_back(value, budget);
    }
    Sequence(const Sequence&) = default;
    Sequence& operator=(const Sequence& other) {
        Sequence candidate{other};
        cells_.swap(candidate.cells_); // Default allocator: nonthrowing commit.
        return *this;
    }
    std::vector<int> snapshot() const {
        std::vector<int> result;
        for (const Cell& cell : cells_) result.push_back(cell.value);
        return result;
    }
    void replace_first(int value) { cells_.at(0).value = value; }
```

```

};

int main() {
    CopyBudget source_budget, destination_budget;
    Sequence source{source_budget, {10, 20, 30}};
    const auto source_before = source.snapshot();
    for (int failure_point = 0; failure_point < 3; ++failure_point) {
        Sequence destination{destination_budget, {7, 8}};
        const auto before = destination.snapshot();
        source_budget.remaining = failure_point;
        CHECK(course::throws<CopyFailure>([&] { destination = source; }));
        CHECK(destination.snapshot() == before);
        CHECK(source.snapshot() == source_before);
        source_budget.remaining = -1;
        destination = source;
        CHECK(destination.snapshot() == source_before);
        destination.replace_first(99);
        CHECK(source.snapshot() == source_before);
        CHECK(destination.snapshot().front() == 99);
    }
    Sequence copy{source};
    copy.replace_first(-1);
    CHECK(source.snapshot() == source_before);
    const Sequence& alias = copy;
    const auto before_self = copy.snapshot();
    copy = alias;
    CHECK(copy.snapshot() == before_self);
    Sequence empty{source_budget, {}};
    copy = empty;
    CHECK(copy.snapshot().empty());
    Sequence empty_copy{empty};
    CHECK(empty_copy.snapshot().empty());
    course::report();
}

```

Evidence and interpretation

Assignment first constructs a private candidate from the source. If any cell copy throws, the partially constructed candidate is cleaned up and the destination has not been touched. With the default allocator used here, swapping the vector storage supplies the nonthrowing commit. The old destination then belongs to the local candidate and is destroyed on return.

The test repeats all three selected failure points, compares both source and destination snapshots, and then performs a successful assignment. Mutating the copied destination's first value must not change the source. Self-assignment is tested as a real copy-and-commit operation, and empty sources verify that replacing a nonempty sequence with an empty value is a valid success.

Do not catch the test harness's `std::runtime_error` as though it were an expected copy failure. The distinct `CopyFailure` type allows an assertion failure to escape and fail the test. This detail matters because a false-positive test can be more misleading than no test: it appears to provide evidence while hiding the discrepancy it was meant to detect.

Practice boundary

Add a second independently failing member to the destination type. Trace whether the combined assignment still has one nonthrowing publication point. A collection of individually safe members does

not automatically make a multi-step user-defined assignment transactional. State the complete object's invariant and test the failure between member updates before claiming a strong guarantee.

B.3 Observe relocation without overclaiming move behavior

Requirement and model

A vector must preserve its elements when it relocates storage under the applicable container and element contracts. The experiment compares a type with a nonthrowing move constructor and a type whose move constructor can throw. Copy and move counters are printed as observations. The assertions verify values and the explicitly defined behavior of the test type, not a universally mandated count of operations.

Complete source — `move_relocation.cpp`

```
#include "course_test.hpp"
#include <iostream>
#include <type_traits>
#include <utility>
#include <vector>

struct MoveFailure final {};
struct Metrics { unsigned copies{}; unsigned moves{}; bool reject_move{}; };

template<bool Nonthrowing>
class Item {
    int value_{};
    Metrics* metrics_{};
public:
    Item(int value, Metrics& metrics) : value_(value), metrics_(&metrics) {}
    Item(const Item& other) : value_(other.value_), metrics_(other.metrics_) {
        ++metrics_->copies;
    }
    Item(Item&& other) noexcept(Nonthrowing) : metrics_(other.metrics_) {
        if constexpr (!Nonthrowing) {
            if (metrics_->reject_move) throw MoveFailure{};
        }
        value_ = std::exchange(other.value_, 0);
        ++metrics_->moves;
    }
    Item& operator=(const Item&) = default;
    Item& operator=(Item&&) = default;
    int value() const noexcept { return value_; }
};

template<bool Nonthrowing>
void observe_relocation() {
    Metrics metrics;
    std::vector<Item<Nonthrowing>> values;
    values.reserve(3);
    values.emplace_back(11, metrics);
    values.emplace_back(22, metrics);
    values.emplace_back(33, metrics);
    const auto capacity = values.capacity();
    values.reserve(capacity + 8);
    CHECK(values.size() == 3);
}
```

```

    CHECK(values[0].value() == 11 && values[1].value() == 22
          && values[2].value() == 33);
    // Counts are observations, not portable fixture expectations.
    std::cout << "nothrow=" << Nonthrowing << " copies=" << metrics.copies
              << " moves=" << metrics.moves << '\n';
}
int main() {
    static_assert(std::is_nothrow_move_constructible_v<Item<true>>);
    static_assert(!std::is_nothrow_move_constructible_v<Item<false>>);
    observe_relocation<true>();
    observe_relocation<false>();
    Metrics metrics;
    Item<false> original{42, metrics};
    metrics.reject_move = true;
    CHECK(course::throws<MoveFailure>([&] { Item<false>
rejected{std::move(original)}; }));
    CHECK(original.value() == 42); // This class checks before changing the source.
    metrics.reject_move = false;
    Item<false> accepted{std::move(original)};
    CHECK(accepted.value() == 42 && original.value() == 0);
    course::report();
}

```

Evidence and interpretation

The vector begins with reserved storage for three elements and then reserves beyond its current capacity. This forces a capacity increase without relying on an implementation's growth factor. The program prints the observed copy and move counts for the current compiler/library combination. Different valid implementations may make different choices within their contracts, so exact counts are not fixture expectations.

The second part tests a direct move of the potentially throwing type. That type checks its rejection policy before changing the source value. Consequently, the failed direct move preserves the source under this class's design. A later successful move sets the source value to zero. These outcomes are properties of the code in front of you, not a general promise that every throwing move preserves its source or every successful move empties it.

The static assertions inspect the exception specification of construction. They do not measure runtime cost. Marking a function `noexcept` is a behavioral commitment; if an exception nevertheless escapes, termination rules apply. Do not add the keyword to influence vector behavior until the implementation genuinely meets the promise.

Practice boundary

Create a second type that has a different documented moved-from state. Verify that container consumers rely on its public contract rather than testing for zero by habit. Keep any performance measurement separate from the copy/move instrumentation because counters themselves change the executed work.

B.4 Retain immutable revisions with shared ownership

Requirement and model

Publishing a new snapshot must not destroy a previous snapshot while existing readers still own it. A weak observer must not keep an old revision alive. An aliasing shared pointer to a member must retain the enclosing object's ownership group while exposing the member's address.

The snapshot is constructed as a const object and offers only const accessors. The test is single-threaded. It demonstrates lifetime relationships and immutable values, not synchronization for concurrent access to the `current` variable or to arbitrary shared mutable data.

Complete source — `ownership_snapshot.cpp`

```
#include "course_test.hpp"
#include <memory>
#include <string>
#include <utility>
#include <vector>

class Snapshot {
    std::string name_;
    std::vector<int> values_;
public:
    Snapshot(std::string name, std::vector<int> values)
        : name_(std::move(name)), values_(std::move(values)) {}
    const std::string& name() const noexcept { return name_; }
    const std::vector<int>& values() const noexcept { return values_; }
};

int main() {
    auto current = std::make_shared<const Snapshot>("revision-one", std::vector{1, 2, 3});
    auto first_reader = current;
    auto second_reader = current;
    const std::weak_ptr<const Snapshot> old_revision = current;
    current = std::make_shared<const Snapshot>("revision-two", std::vector{8, 9});
    CHECK(current->name() == "revision-two");
    CHECK(first_reader->name() == "revision-one");
    CHECK(second_reader->values() == std::vector<int>({1, 2, 3}));
    first_reader.reset();
    CHECK(!old_revision.expired());
    {
        auto secured = old_revision.lock();
        CHECK(secured && secured->values().size() == 3);
    }
    second_reader.reset();
    CHECK(old_revision.expired());
    CHECK(!old_revision.lock());
    CHECK(current->values() == std::vector<int>({8, 9}));
    // Aliasing ownership: expose a member while retaining its enclosing owner.
    std::shared_ptr<const std::vector<int>> member(current, &current->values());
    const std::weak_ptr<const Snapshot> current_observer = current;
    current.reset();
    CHECK(!current_observer.expired());
    CHECK(member->size() == 2 && member->front() == 8);
    member.reset();
    CHECK(current_observer.expired());
}
```

```
    course::report();  
}
```

Evidence and interpretation

Two readers copy the owner of revision one. Replacing the current pointer publishes revision two for future readers but leaves revision one alive for the existing readers. Releasing the first reader is insufficient to expire the weak observer. Releasing the final strong reader allows expiry, and locking the expired observer produces no owner.

The aliasing constructor then exposes the vector member of revision two while sharing the snapshot's ownership group. Resetting the ordinary snapshot pointer does not destroy the enclosing object because the aliasing owner remains. The stored pointer and the owned object's address need not be identical; ownership identity and exposed address are distinct concepts.

This pattern requires the aliased member to remain valid for the lifetime of its enclosing owner. It is not permission to alias an element that may later be erased or reallocated. The snapshot's immutable layout makes the chosen member relationship straightforward. A mutable container would need a separate invalidation analysis.

Practice boundary

Add a reader API that returns an owning immutable snapshot and another that offers a short-lived `const borrow`. Document which caller can retain the result after publication changes. Do not use `use_count` as an authorization or thread-safety decision. The reference counts here explain lifetime in a controlled experiment, not a general lock-free publication protocol.

Technical references: [unique ownership](#) and [shared ownership](#), [exception cleanup](#), [copy and move construction](#), and the [C++ Core Guidelines](#).

Appendix C — Generic Arithmetic and Range-Lifetime Labs

These experiments connect constraints, numeric invariants, and lifetime reasoning. They complement Chapters 11–19 without requiring an additional framework. One test checks representability before addition; the other demonstrates safe range result types and deliberate materialization.

```
cctest --test-dir build/debug --output-on-failure \
-R '^lab\.(generic_pipeline|range_lifetime)$'
```

C.1 Constrain the domain and check the arithmetic

Requirement and simplest approach

Sum a read-only sequence of integer values. Empty input produces zero. Every intermediate addition must be representable in the selected type, or the operation rejects it without modifying caller-owned input. Boolean and floating-point arguments are outside this particular API’s domain. A generic signature must communicate those restrictions without pretending that a concept proves the arithmetic safe.

The unsafe shortcut adds first and checks the result afterward. For signed overflow, that can already have left the language’s defined behavior. The reference compares against safe thresholds before addition. For unsigned values, it checks whether the right operand exceeds the remaining room below the maximum. For signed values, it handles positive and negative right operands separately so the threshold expressions remain representable.

Complete source — `checked_numeric.hpp`

```
#ifndef HARBOR_CHECKED_NUMERIC_HPP
#define HARBOR_CHECKED_NUMERIC_HPP
#include <concepts>
#include <limits>
#include <span>
#include <stdexcept>
#include <type_traits>

namespace harbor::numeric {
template<class T>
concept Integer = std::integral<T> && !std::same_as<std::remove_cv_t<T>, bool>;

template<Integer T>
constexpr T checked_add(T left, T right) {
    constexpr T high = std::numeric_limits<T>::max();
    if constexpr (std::is_signed_v<T>) {
        constexpr T low = std::numeric_limits<T>::min();
        if ((right > 0 && left > high - right) ||
            (right < 0 && left < low - right)) {
            throw std::overflow_error("sum is not representable");
        }
    }
    else if (left > high - right) {

```

```

        throw std::overflow_error("sum is not representable");
    }
    return static_cast<T>(left + right);
}

template<Integer T>
T checked_total(std::span<const T> values) {
    T result{};
    for (T value : values) result = checked_add(result, value);
    return result;
}
}
#endif

```

Complete source — generic_pipeline.cpp

```

#include "course_test.hpp"
#include "checked_numeric.hpp"
#include <array>
#include <limits>
#include <span>
#include <vector>

int main() {
    using harbor::numeric::Integer;
    using harbor::numeric::checked_add;
    using harbor::numeric::checked_total;
    static_assert(Integer<int> && Integer<unsigned long>);
    static_assert(!Integer<bool> && !Integer<const bool> && !Integer<double>);
    static_assert(checked_add(2, 3) == 5);
    CHECK(checked_total<int>({}) == 0);
    const std::array mixed{10, -4, 7, -3};
    CHECK(checked_total<int>(mixed) == 10);
    constexpr int high = std::numeric_limits<int>::max();
    constexpr int low = std::numeric_limits<int>::min();
    CHECK(checked_add(high, 0) == high);
    CHECK(checked_add(low, 0) == low);
    CHECK(checked_add(high, -1) == high - 1);
    CHECK(checked_add(low, 1) == low + 1);
    CHECK(course::throws<std::overflow_error>([&] { (void)checked_add(high, 1); }));
    CHECK(course::throws<std::overflow_error>([&] { (void)checked_add(low, -1); }));
    const std::vector<int> overflow{high - 2, 1, 2};
    CHECK(course::throws<std::overflow_error>([&] {
        (void)checked_total<int>(overflow); }));
    CHECK(overflow == std::vector<int>({high - 2, 1, 2}));
    CHECK(course::throws<std::overflow_error>([&] {
        (void)checked_add(std::numeric_limits<unsigned>::max(), 1U);
    }));
    // Independent wider arithmetic oracle over a small exhaustive domain.
    for (int left = -40; left <= 40; ++left) {
        for (int right = -40; right <= 40; ++right) {
            const auto actual = checked_add(static_cast<signed char>(left),
                                           static_cast<signed char>(right));
            const int expected = left + right; // Always within [-80,80].
            CHECK(static_cast<int>(actual) == expected);
        }
    }
    course::report();
}

```

Evidence and interpretation

The concept expresses a compile-time category: integral types except Boolean. The runtime checks express value-dependent representability. These responsibilities are different. An integer type can satisfy the concept while two values of that type cannot be added safely. Conversely, a floating-point value that happens to be exactly integral remains outside the declared type domain.

The boundary cases include maximum plus zero, minimum plus zero, one step inward from each bound, and one step beyond each bound. A multi-element total fails late enough to demonstrate that earlier local progress does not mutate the caller's vector. The total is held in local state and returned only after all additions succeed.

The nested experiment covers every pair of signed-char inputs from minus forty through forty. Its independent integer oracle computes values only from minus eighty through eighty, which are representable for the selected test. This is an exhaustive test of that small chosen domain, not of every value of every integral type. The many internal checks remain part of one CTest case and should not be advertised as thousands of independent integration tests.

A different requirement might allow a wider accumulator than the input element type, or might use saturation instead of rejection. That is a different API and should be named and tested accordingly. Do not silently change this implementation to saturate merely to keep a calculation running; callers may rely on rejection to prevent publishing a misleading total.

Guided extension

Add a checked multiplication function for one explicitly chosen integer type. Derive safe preconditions for zero, positive, and negative operands before writing the operation. Pay particular attention to the minimum signed value multiplied or divided by minus one. Build a bounded independent oracle and boundary cases rather than testing the risky expression in order to discover whether it overflows.

C.2 Distinguish a range wrapper from its storage

Requirement and model

A function with local input must return an owning selected result. A temporary span over an external array may expose a valid iterator while the array remains alive. A temporary owning vector passed to a ranges search must not yield an ordinary retained iterator through that algorithm's safe return type. A view used before a source change should not be reused under an assumption that cached membership state automatically resets.

The experiment demonstrates these distinctions through valid executions and static assertions. It does not create a dangling view and dereference it merely to obtain a sanitizer screenshot. A type-level check or a correctly scoped ownership trace is often a better teaching experiment than deliberately invoking undefined behavior.

Complete source — range_lifetime.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <array>
#include <ranges>
#include <span>
#include <string>
#include <string_view>
#include <type_traits>
#include <vector>

std::vector<int> selected_snapshot() {
    std::vector<int> local{1, 2, 3, 4, 5, 6};
    auto view = local | std::views::filter([](int value) { return value % 2 == 0; })
                    | std::views::transform([](int value) { return value *
value; });
    std::vector<int> owned;
    for (int value : view) owned.push_back(value);
    return owned; // No borrow of local storage escapes.
}

int main() {
    static_assert(std::ranges::borrowed_range<std::span<int>>);
    static_assert(!std::ranges::borrowed_range<std::vector<int>>);
    using UnsafeResult = decltype(std::ranges::find(std::vector<int>{1, 2}, 2));
    static_assert(std::is_same_v<UnsafeResult, std::ranges::dangling>);
    std::array data{2, 4, 6};
    auto found = std::ranges::find(std::span<int>{data}, 4);
    CHECK(found != std::span<int>{data}.end() && *found == 4);
    const auto snapshot = selected_snapshot();
    CHECK(snapshot == std::vector<int>({4, 16, 36}));
    std::vector<int> source{1, 2, 3, 4};
    {
        auto selected = source | std::views::filter([](int value) { return value % 2
== 0; });
        std::vector<int> captured;
        for (int value : selected) captured.push_back(value);
        CHECK(captured == std::vector<int>({2, 4}));
    } // The old view is no longer used after source changes.
    source[0] = 8;
    auto current = source | std::views::filter([](int value) { return value % 2 ==
0; });
    CHECK(*current.begin() == 8);
    std::string line{"17|first title"};
    std::string saved;
    {
        const std::string_view field{line.data() + 3, line.size() - 3};
        saved = std::string{field};
    }
    line.assign(200, 'x');
    CHECK(saved == "first title");
    std::vector<int> empty;
    auto empty_view = empty | std::views::filter([](int v) { return v > 0; });
    CHECK(empty_view.begin() == empty_view.end());
    const std::vector<int> none{-3, -2, -1};
    auto no_matches = none | std::views::filter([](int v) { return v > 0; });
    CHECK(no_matches.begin() == no_matches.end());
    course::report();
}
```

Evidence and interpretation

`selected_snapshot` constructs a local input vector and a lazy pipeline, then materializes the selected squares into an owning vector. Returning that vector does not retain a borrow of the local source. Its values are checked after the function returns, when the source no longer exists. The result's independence comes from ownership, not from the fact that the pipeline used modern syntax.

The borrowed-range assertions distinguish the span wrapper from an owning vector. The temporary span's data belongs to a still-live local array in the caller. That valid array lifetime is essential; borrowed-range status would not rescue a span pointing into destroyed storage. The temporary-vector search instead has a **dangling** result type, making the unsafe retained-iterator use unavailable through that result.

The source-mutation exercise ends the old view's use before changing membership and constructs a fresh view afterward. The string-view exercise copies the desired field into an owning string before replacing the line buffer. Empty input and no-match input are checked separately so traversal does not assume that a begin iterator is dereferenceable.

Guided extension

Design two APIs for selecting task titles. One returns an owning vector of strings. The other returns a view intended for immediate use over a caller-owned immutable collection. Write their lifetime contracts side by side. Test the owning result after the input is replaced; consume the borrowed result only within its valid scope. Explain which interface is suitable for queuing work to a later stage and why.

Technical references: [concepts](#), [constraints](#), [ranges](#), [filter views](#), and [object lifetime](#).

Appendix D — A Bounded, Transactional Reporting Application

D.1 Define the extension's contract

The compact Chapter 21 application is the starting point. This extension retains its task-reporting purpose while making input limits, structured failures, and publication boundaries more explicit. Its implementation is separate so the reader can compare two designs without losing the simpler reference. The following contract applies to `harbor_reports`, not retroactively to every earlier example.

An input record contains `id|priority|done|title`. The ID is a positive representable `int`, unique within the candidate dataset. Priority is one through three. Completion is zero or one. Numeric fields require whole-field conversion: a leading plus sign, surrounding whitespace, or trailing characters is rejected by this grammar. Leading zeros are accepted. Negative numeric syntax may parse, but negative values fail the relevant domain checks.

Titles contain one through two hundred printable ASCII bytes, exclude vertical bars, and must include at least one non-space character. Leading and trailing ordinary spaces are retained. Tabs, NUL bytes, and non-ASCII text are rejected by this deliberately narrow teaching format. This is not a Unicode normalization or internationalization design. A broader format needs an explicit encoding and character-policy revision.

The default limits are ten thousand accepted rows and two hundred fifty-six retained bytes per line before LF. A trailing CR is removed for CRLF parsing, but it counts toward the line limit as delivered by the stream. The file adapter opens in binary mode. Standard input can be subject to platform text translation, so the limit concerns characters presented to the parser, not a universal physical-byte count before all platform processing.

An empty file is a valid empty replacement. A blank record is invalid. A final nonempty record without LF is accepted. On any returned input error, accepted tasks remain unchanged. Exceptional allocation failures also occur before publication. Input position and stream state are not restored. Rendering returns an owning string; writing can leave a prefix in an external sink on failure. The command-line adapter reports ordinary validation failures with exit two and detected operational failures with exit three. Operating-system signals and forced process termination are outside that exit-status contract.

D.2 Inspect the public boundary

`Task` is a simple transfer value; its fields are public so snapshots are easy to inspect. The invariant belongs to the private accepted collection inside `Report`, which only receives validated tasks through `load`. Constructing an arbitrary `Task` in a caller does not insert it into that collection. This is a deliberate boundary: the application does not claim that every possible standalone task object is valid at every moment.

The API returns copied snapshots and completed text. It does not expose mutable accepted storage or iterators that survive replacement. Error objects own only a code and line number, while diagnostic strings come from fixed literals. These choices keep ownership understandable at the cost of some copying, which is appropriate for the bounded instructional workload.

Complete source — report.hpp

```
#ifndef HARBOR_REPORTS_REPORT_HPP
#define HARBOR_REPORTS_REPORT_HPP
#include <cstdint>
#include <iosfwd>
#include <optional>
#include <string>
#include <string_view>
#include <vector>

namespace harbor {
struct Task {
    int id{};
    int priority{};
    bool done{};
    std::string title;
    bool operator==(const Task&) const = default;
};
enum class ErrorCode {
    input_io, line_limit, row_limit, field_count, integer_syntax,
    integer_range, id_domain, priority_domain, done_domain,
    title_domain, duplicate_id
};
struct LoadError {
    ErrorCode code;
    std::size_t line;
    bool operator==(const LoadError&) const = default;
};
std::string_view error_text(ErrorCode code) noexcept;
struct Limits {
    std::size_t max_rows{10'000};
    std::size_t max_line_bytes{256};
};

class Report {
public:
    // No returned error means the complete candidate replaced accepted state.
    // On a returned error or exception, accepted state is unchanged. Input
    // consumption is not rolled back. Invalid trusted limits throw.
    std::optional<LoadError> load(std::istream& input, Limits limits = {});
    std::vector<Task> snapshot() const { return accepted_; }
    std::string render(bool grouped = false) const;
    // Failure can leave a prefix in the external sink. It never changes tasks.
    void write(std::ostream& output, bool grouped = false) const;
private:
    std::vector<Task> accepted_;
};
} // namespace harbor
#endif
```

D.3 Follow parsing, validation, and commit

The bounded reader retains at most the configured number of characters in its line string. It reads one additional non-newline character to detect an overlong record but does not append it. The input is not drained after rejection. This gives a concrete retained-line bound without promising a precise process-memory ceiling: string capacity, stream buffers, candidate tasks, index nodes, and rendering copies still consume storage.

Each line is split into four short-lived views. Numeric conversion checks both conversion status and complete consumption. Title validation applies the declared ASCII policy. The successful alternative contains an owning **Task**; no view into the mutable line buffer escapes. A uniqueness set rejects duplicates before the task enters the accepted collection.

The private candidate accumulates validated values. The only accepted-state publication is the final vector swap after clean end-of-input. A valid empty candidate deliberately replaces old state with an empty value. A failed nonempty candidate is destroyed instead. This is why “the load returned no tasks” is not enough to classify success or failure; the optional error result and the accepted-state transition must be considered together.

Rendering takes a filtered owning copy of unfinished tasks and sorts by priority and ID. The optional grouped header is derived from those selected tasks, not from all accepted tasks. A classic locale makes numeric presentation stable under this renderer’s policy. The write method creates the whole string before touching the caller’s sink, checks that its size fits the stream write count, and uses an unformatted write to avoid inherited field-width formatting. A sink failure can still retain a prefix.

Complete source — report.cpp

```
#include "report.hpp"
#include <algorithm>
#include <array>
#include <charconv>
#include <istream>
#include <locale>
#include <limits>
#include <ostream>
#include <ranges>
#include <set>
#include <sstream>
#include <stdexcept>
#include <system_error>
#include <tuple>
#include <utility>
#include <variant>

namespace harbor {
namespace {
enum class ReadState { line, end, too_long, io_error };
ReadState read_line(std::istream& input, std::string& line,
                   std::size_t maximum) {
    line.clear();
    for (;;) {
        std::istream::int_type raw{};
        try {
            raw = input.get();
        } catch (const std::ios_base::failure&) {
```

```

        if (input.bad()) return ReadState::io_error;
        if (input.eof()) {
            return line.empty() ? ReadState::end : ReadState::line;
        }
        return ReadState::io_error;
    }
    if (std::istream::traits_type::eq_int_type(
        raw, std::istream::traits_type::eof())) {
        if (input.bad() || !input.eof()) return ReadState::io_error;
        return line.empty() ? ReadState::end : ReadState::line;
    }
    const char value = std::istream::traits_type::to_char_type(raw);
    if (value == '\n') return ReadState::line;
    // One excess byte is consumed, but it is never retained in line.
    if (line.size() == maximum) return ReadState::too_long;
    line.push_back(value);
}
}

std::optional<ErrorCode> parse_integer(std::string_view text, int& value) {
    if (text.empty()) return ErrorCode::integer_syntax;
    const auto [end, error] = std::from_chars(
        text.data(), text.data() + text.size(), value);
    if (error == std::errc::result_out_of_range) {
        return ErrorCode::integer_range;
    }
    if (error != std::errc{} || end != text.data() + text.size()) {
        return ErrorCode::integer_syntax;
    }
    return std::nullopt;
}

std::variant<Task, ErrorCode> parse_task(std::string_view line) {
    if (!line.empty() && line.back() == '\r') line.remove_suffix(1);
    std::array<std::string_view, 4> fields{};
    std::size_t start = 0;
    for (std::size_t index = 0; index < 3; ++index) {
        const auto separator = line.find('|', start);
        if (separator == std::string_view::npos) {
            return ErrorCode::field_count;
        }
        fields[index] = line.substr(start, separator - start);
        start = separator + 1;
    }
    fields[3] = line.substr(start);
    if (fields[3].find('|') != std::string_view::npos) {
        return ErrorCode::field_count;
    }
    int id{}, priority{}, done{};
    if (auto error = parse_integer(fields[0], id)) return *error;
    if (auto error = parse_integer(fields[1], priority)) return *error;
    if (auto error = parse_integer(fields[2], done)) return *error;
    if (id <= 0) return ErrorCode::id_domain;
    if (priority < 1 || priority > 3) return ErrorCode::priority_domain;
    if (done != 0 && done != 1) return ErrorCode::done_domain;
    const auto title = fields[3];
    const bool printable = std::ranges::all_of(title, [](char character) {
        const auto byte = static_cast<unsigned char>(character);
        return byte >= 0x20 && byte <= 0x7e;
    });
    const bool nonblank = std::ranges::any_of(title, [](char character) {
        return character != ' ';
    });
    if (title.empty() || title.size() > 200 || !printable || !nonblank) {
        return ErrorCode::title_domain;
    }
    return Task{id, priority, done == 1, std::string(title)};
}
}

```

```

} // namespace

std::string_view error_text(ErrorCode code) noexcept {
    switch (code) {
        case ErrorCode::input_io: return "input I/O failure";
        case ErrorCode::line_limit: return "line limit exceeded";
        case ErrorCode::row_limit: return "row limit exceeded";
        case ErrorCode::field_count: return "expected four fields";
        case ErrorCode::integer_syntax: return "invalid integer syntax";
        case ErrorCode::integer_range: return "integer out of range";
        case ErrorCode::id_domain: return "id must be positive";
        case ErrorCode::priority_domain: return "priority must be 1 through 3";
        case ErrorCode::done_domain: return "done must be 0 or 1";
        case ErrorCode::title_domain: return "invalid title";
        case ErrorCode::duplicate_id: return "duplicate id";
    }
    return "unknown input error";
}

std::optional<LoadError> Report::load(std::istream& input, Limits limits) {
    if (limits.max_rows == 0 || limits.max_rows > 1'000'000 ||
        limits.max_line_bytes == 0 || limits.max_line_bytes > 1'000'000) {
        throw std::invalid_argument("limits must be in [1, 1000000]");
    }
    std::vector<Task> candidate;
    candidate.reserve(std::min(limits.max_rows, std::size_t{256}));
    std::set<int> identifiers;
    std::string line;
    std::size_t number = 1;
    for (;;) {
        const auto state = read_line(input, line, limits.max_line_bytes);
        if (state == ReadState::end) break;
        if (state == ReadState::io_error) {
            return LoadError{ErrorCode::input_io, number};
        }
        if (state == ReadState::too_long) {
            return LoadError{ErrorCode::line_limit, number};
        }
        if (candidate.size() == limits.max_rows) {
            return LoadError{ErrorCode::row_limit, number};
        }
        auto parsed = parse_task(line);
        if (const auto* error = std::get_if<ErrorCode>(&parsed)) {
            return LoadError{*error, number};
        }
        auto task = std::get<Task>(std::move(parsed));
        if (!identifiers.insert(task.id).second) {
            return LoadError{ErrorCode::duplicate_id, number};
        }
        candidate.push_back(std::move(task));
        ++number;
    }
    accepted_.swap(candidate); // The only publication point for input state.
    return std::nullopt;
}

std::string Report::render(bool grouped) const {
    std::vector<Task> open;
    for (const auto& task : accepted_ | std::views::filter(
        [](const Task& value) { return !value.done; })) {
        open.push_back(task);
    }
    std::ranges::sort(open, {}, [](const Task& task) {
        return std::tuple{task.priority, task.id};
    });
    std::ostringstream output;

```

```

    output.imbue(std::locale::classic());
    output.exceptions(std::ios::badbit | std::ios::failbit);
    output << "open=" << open.size() << " total=" << accepted_.size() << '\n';
    if (grouped) {
        std::array<std::size_t, 3> counts{};
        for (const auto& task : open) {
            ++counts.at(static_cast<std::size_t>(task.priority - 1));
        }
        for (std::size_t index = 0; index < counts.size(); ++index) {
            output << "priority=" << index + 1
                << " count=" << counts[index] << '\n';
        }
    }
    for (const auto& task : open) {
        output << task.id << '|' << task.priority << '|' << task.title << '\n';
    }
    return output.str();
}

void Report::write(std::ostream& output, bool grouped) const {
    const std::string completed = render(grouped);
    if (completed.size() > static_cast<std::size_t>(
        std::numeric_limits<std::streamsize>::max())) {
        throw std::length_error("report exceeds stream write range");
    }
    output.write(completed.data(), static_cast<std::streamsize>(completed.size()));
    if (!output) throw std::ios_base::failure("report output failed");
}
} // namespace harbor

```

D.4 Keep command-line policy outside the domain

The adapter recognizes `--grouped`, `--help`, and at most one input file. With no file, it borrows standard input. With a file, a local stream owns the open association and remains alive through loading. The raw pointer selects one existing stream; it never owns or deletes either target.

The adapter reports structured validation errors with a line number and stable message. It checks final output and flushing before returning normal success. The broad final exception handler is an application boundary for unexpected standard exceptions, not a parser branch that disguises resource failure as malformed data. The core can therefore be tested independently with in-memory streams.

Complete source — main.cpp

```

#include "report.hpp"
#include <fstream>
#include <iostream>
#include <new>
#include <string_view>

int main(int argc, char* argv[]) {
    try {
        bool grouped = false;
        const char* filename = nullptr;
        for (int index = 1; index < argc; ++index) {
            const std::string_view argument(argv[index]);
            if (argument == "--grouped") {
                grouped = true;
            } else if (argument == "--help") {

```

```

        std::cout << "usage: harbor_reports [--grouped] [FILE]\n";
        return std::cout ? 0 : 3;
    } else if (argument.starts_with('-') || filename != nullptr) {
        std::cerr << "usage: harbor_reports [--grouped] [FILE]\n";
        return 2;
    } else {
        filename = argv[index];
    }
}
std::ifstream file;
std::istream* input = &std::cin; // Borrow; neither path owns cin.
if (filename != nullptr) {
    file.open(filename, std::ios::binary);
    if (!file) {
        std::cerr << "cannot open input\n";
        return 2;
    }
    input = &file;
}
harbor::Report report;
if (const auto error = report.load(*input)) {
    std::cerr << "line " << error->line << ": "
               << harbor::error_text(error->code) << '\n';
    return 2;
}
report.write(std::cout, grouped);
std::cout.flush();
return std::cout ? 0 : 3;
} catch (const std::bad_alloc&) {
    std::cerr << "resource allocation failed\n";
    return 3;
} catch (const std::exception& error) {
    std::cerr << "operation failed: " << error.what() << '\n';
    return 3;
}
}

```

Build and run a known case

Run these commands from the extracted package. The input contains an unfinished high-priority task, an unfinished lower-priority task, and one completed task. The output must exclude the completed task and use explicit priority/ID ordering.

```

cmake -S source -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug --parallel 2
printf '1|2|0|Later\n3|1|1|Done\n2|1|0|First\n' \
| ./build/debug/labs/harbor_reports --grouped

```

Expected output for this exact input:

```

open=2 total=3
priority=1 count=1
priority=2 count=1
priority=3 count=0
2|1|First
1|2|Later

```

The path shown is the usual single-configuration layout used by the recorded Linux build. Multi-configuration generators can place the executable beneath a configuration subdirectory. CTest resolves the target location automatically, making it the portable first choice for running the supplied fixtures.

D.5 Test the boundary, not just the happy path

The command-line fixture file contains thirty-six cases spanning valid input, empty input, ties, CRLF, final-line policy, title limits, syntax failures, numeric domains, duplicates, file selection, and option handling. The in-process regression below tests a different property: repeated operations on one existing report. It compares accepted state before and after failures instead of starting every experiment from an empty object.

BrokenInput throws at a selected byte position. The regression iterates through every failure position in a small two-record candidate, with both ordinary stream-state handling and `badbit` exceptions enabled. **LimitedOutput** accepts only eight bytes. The test confirms the error, retained prefix, and unchanged report separately. These fakes control external failure without requiring a full disk, a damaged file, or a network outage.

The permutation experiment enumerates all twenty-four orders of four records and compares the rendered output with one hand-written expected sequence. It verifies arrival-independent ordering for that finite set. It is not a general proof of the sorting implementation. The test also distinguishes an accepted empty replacement from an invalid blank record and confirms that owned titles do not depend on the original input string.

Complete source — tests.cpp

```
#include "report.hpp"
#include "course_test.hpp"
#include <algorithm>
#include <array>
#include <istream>
#include <limits>
#include <ostream>
#include <sstream>
#include <streambuf>
#include <string>
#include <utility>
#include <vector>

// These stream buffers fail deliberately; no disk or network is required.
class BrokenInput final : public std::streambuf {
public:
    BrokenInput(std::string text, std::size_t failure)
        : text_(std::move(text)), failure_(failure) {}
    std::size_t consumed() const noexcept { return position_; }
protected:
    int_type underflow() override {
        if (position_ == failure_) {
            throw std::ios_base::failure("injected read failure");
        }
        return position_ == text_.size() ? traits_type::eof()
            : traits_type::to_int_type(text_[position_]);
    }
    int_type uflow() override {
```

```

        const auto value = underflow();
        if (!traits_type::eq_int_type(value, traits_type::eof())) ++position_;
        return value;
    }
private:
    std::string text_;
    std::size_t failure_;
    std::size_t position_{};
};

class LimitedOutput final : public std::streambuf {
public:
    explicit LimitedOutput(std::size_t capacity) : capacity_(capacity) {}
    const std::string& bytes() const noexcept { return bytes_; }
protected:
    std::streamsize xsputn(const char* data, std::streamsize count) override {
        std::streamsize written = 0;
        while (written < count && bytes_.size() < capacity_) {
            bytes_.push_back(data[written++]);
        }
        return written;
    }
    int_type overflow(int_type character) override {
        if (traits_type::eq_int_type(character, traits_type::eof())) {
            return traits_type::not_eof(character);
        }
        if (bytes_.size() == capacity_) return traits_type::eof();
        bytes_.push_back(traits_type::to_char_type(character));
        return character;
    }
private:
    std::size_t capacity_;
    std::string bytes_;
};

int main() {
    using harbor::ErrorCode;
    harbor::Report report;
    std::istringstream initial("9|2|0|Existing\n");
    CHECK(!report.load(initial));
    const auto accepted = report.snapshot();
    const auto rendered = report.render();
    const std::vector<std::pair<std::string, ErrorCode>> invalid{
        {"", ErrorCode::field_count},
        {"1|2|0", ErrorCode::field_count},
        {"1|2|0|a|b", ErrorCode::field_count},
        {"+1|2|0|a", ErrorCode::integer_syntax},
        {"1x|2|0|a", ErrorCode::integer_syntax},
        {"0|2|0|a", ErrorCode::id_domain},
        {"-1|2|0|a", ErrorCode::id_domain},
        {"1|0|0|a", ErrorCode::priority_domain},
        {"1|4|0|a", ErrorCode::priority_domain},
        {"1|2|2|a", ErrorCode::done_domain},
        {"1|2|0|", ErrorCode::title_domain},
        {"1|2|0|   ", ErrorCode::title_domain},
        {"1|2|0|a\\tb", ErrorCode::title_domain},
        {"1|2|0|" + std::string(201, 'a'), ErrorCode::title_domain},
        {std::string(100, '9') + "|2|0|a", ErrorCode::integer_range},
        {"1|2|0|a\\n1|1|0|b", ErrorCode::duplicate_id}
    };
    for (const auto& [text, code] : invalid) {
        std::istringstream input(text + '\n');
        const auto error = report.load(input);
        CHECK(error && error->code == code);
        CHECK(report.snapshot() == accepted);
        CHECK(report.render() == rendered);
    }
}

```

```

// Rows and retained line bytes have independently tested limits.
{
    std::istringstream input("1|1|0|a\n2|1|0|b\n");
    CHECK(report.load(input, {1, 256}) ==
           harbor::LoadError{ErrorCode::row_limit, 2});
    CHECK(report.snapshot() == accepted);
}
{
    std::istringstream input("1|1|0|ab\n");
    CHECK(report.load(input, {1, 7}) ==
           harbor::LoadError{ErrorCode::line_limit, 1});
    CHECK(report.snapshot() == accepted);
    std::istringstream exact("1|1|0|a\n");
    CHECK(!report.load(exact, {1, 7}));
    CHECK(report.snapshot().at(0).title == "a");
}
{
    std::istringstream input("1|1|0|a\n");
    CHECK(course::throws<std::invalid_argument>([&] {
        report.load(input, {0, 256});
    }));
}
// Accepted strings do not borrow the parser's input storage.
{
    std::string source("4|3|0|Owned title");
    std::istringstream input(source);
    CHECK(!report.load(input));
    source.assign(100, 'x');
    CHECK(report.snapshot().at(0).title == "Owned title");
}
const auto before_io = report.snapshot();
const std::string candidate("1|1|0|New\n2|2|0|Next\n");
for (std::size_t failure = 0; failure < candidate.size(); ++failure) {
    for (const bool throw_on_bad : {false, true}) {
        BrokenInput buffer(candidate, failure);
        std::istream input(&buffer);
        if (throw_on_bad) input.exceptions(std::ios::badbit);
        const auto error = report.load(input);
        CHECK(error && error->code == ErrorCode::input_io);
        CHECK(report.snapshot() == before_io);
        CHECK(buffer.consumed() == failure);
    }
}
// Clean EOF remains success even with failbit/badbit exceptions enabled.
for (const std::string& text : {std::string{}, std::string{"1|1|0|a"},
                                std::string{"1|1|0|a\n"}}) {
    std::istringstream input(text);
    input.exceptions(std::ios::failbit | std::ios::badbit);
    CHECK(!report.load(input));
}
// Arrival order is not report order: verify every permutation of 4 rows.
std::array<std::string, 4> rows{
    "1|2|0|a\n", "2|1|0|b\n", "3|1|0|c\n", "4|3|1|d\n"};
const std::string expected("open=3 total=4\n2|1|b\n3|1|c\n1|2|a\n");
do {
    std::string text;
    for (const auto& row : rows) text += row;
    std::istringstream input(text);
    CHECK(!report.load(input));
    CHECK(report.render() == expected);
} while (std::next_permutation(rows.begin(), rows.end()));
CHECK(report.render(true) ==
       "open=3 total=4\npriority=1 count=2\npriority=2 count=1\n"
       "priority=3 count=0\n2|1|b\n3|1|c\n1|2|a\n");
const auto before_write = report.snapshot();
for (const bool throw_on_bad : {false, true}) {

```

```

    LimitedOutput buffer(8);
    std::ostream output(&buffer);
    if (throw_on_bad) output.exceptions(std::ios::badbit);
    CHECK(course::throws<std::ios_base::failure>([&] {
        report.write(output);
    }));
    CHECK(buffer.bytes() == expected.substr(0, 8));
    CHECK(report.snapshot() == before_write);
}
// A valid empty replacement is a successful state transition, not failure.
std::istringstream empty;
CHECK(!report.load(empty));
CHECK(report.snapshot().empty());
CHECK(report.render() == "open=0 total=0\n");
course::report();
}

```

D.6 Acceptance review and remaining work

The accepted-state guarantee is supported by a visible commit point and regression checks. The retained-line bound is supported by checking before append and exact-boundary tests. Input-consumption behavior is exposed by the failing stream buffer. Output non-atomicity is exposed by the prefix-retaining sink. These are separate pieces of evidence rather than a single vague claim that the program is “safe.”

The extension deliberately omits persistent configuration, authentication, Unicode text policy, crash-durable file publication, asynchronous processing, and concurrent access to a shared report. It also does not impose a total process-memory budget. Those omissions are not hidden behind the successful test count. Adding any of them requires a new contract and representative platform tests.

For a final exercise, replace the uniqueness set with a hash set while preserving all public results. Then optimize rendering by selecting indices into the immutable accepted vector during the bounded render call. Compare correctness first and performance second. The returned report text must remain owned, deterministic, and independent of later accepted-state replacement. A reduced copy count is valuable only when the original contract survives the change.

Technical references: [character conversion](#), [streams](#), [output operations](#), [sorting](#), [ranges](#), and [exception cleanup](#).

Appendix E — Test Harness, Fixtures, and Release Evidence

E.1 Keep assertions active in every build

The shared harness counts executed checks and throws when a condition fails. Unlike a disabled standard assertion in a build defining `NDEBUG`, these checks remain active in optimized configurations. The count is useful for identifying that an expected path executed, but it is not a percentage of correctness or a count of independent test scenarios.

The harness is intentionally small and single-threaded. It is not a replacement for a full test framework in a concurrent production repository. Its shared counter would need appropriate redesign before concurrent use. The examples in this volume use it within the stated single-threaded testing model.

Complete source — `course_test.hpp`

```
#ifndef CPP_COURSE_TEST_HPP
#define CPP_COURSE_TEST_HPP
#include <iostream>
#include <stdexcept>
#include <string>

// Test checks stay active even when NDEBUG is defined in optimized builds.
namespace course {
inline unsigned checks{};
inline void check(bool condition, const char* expression,
                  const char* file, int line) {
    ++checks;
    if (!condition) {
        throw std::runtime_error(std::string(file) + ':' + std::to_string(line)
                                + ": failed: " + expression);
    }
}
}
template<class Exception, class Function>
bool throws(Function&& function) {
    try { function(); }
    catch (const Exception&) { return true; }
    return false;
}
inline void report() { std::cout << "PASS checks=" << checks << '\n'; }
}
#define CHECK(...) ::course::check(static_cast<bool>((__VA_ARGS__)), \
                                   #__VA_ARGS__, __FILE__, __LINE__)
#endif
```

E.2 Run a contract in an isolated working directory

A fixture specifies arguments, standard input, expected exit status, optional exact output, and required diagnostic text. The runner gives each case a fresh temporary directory and creates only the declared local

files. That keeps the Chapter 20 filesystem experiment from interacting with another test's leftovers. The fixture definitions are trusted project inputs, not a sandbox format for arbitrary hostile instructions.

Exact output is appropriate when the application promises an exact textual contract. Substring checks are appropriate for a deliberately variable observation or an implementation-dependent diagnostic suffix. Do not loosen an exact expected result merely to make a regression pass. First decide whether the behavior changed intentionally, and update the specification together with the fixture.

Complete source — `run_case.py`

```
#!/usr/bin/env python3
"""Run one input/output contract in a fresh temporary working directory."""
from __future__ import annotations
import argparse, json, subprocess, tempfile
from pathlib import Path

def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument('--exe', type=Path, required=True)
    parser.add_argument('--cases', type=Path, required=True)
    parser.add_argument('--index', type=int, required=True)
    args = parser.parse_args()
    case = json.loads(args.cases.read_text(encoding='utf-8'))[args.index]
    exe = args.exe.resolve(strict=True)
    with tempfile.TemporaryDirectory(prefix='cpp-course-') as temp:
        for name, text in case.get('files', {}).items():
            target = Path(temp) / name
            target.parent.mkdir(parents=True, exist_ok=True)
            target.write_text(text, encoding='utf-8')
        result = subprocess.run([str(exe), *case.get('args', [])],
                                input=case.get('stdin', ''), text=True,
                                stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                                cwd=temp, timeout=30, check=False)

        problems = []
        if result.returncode != case.get('exit', 0):
            problems.append(f'exit {result.returncode}, expected {case.get("exit", 0)}')
        if 'stdout' in case and result.stdout != case['stdout']:
            problems.append(f'stdout {result.stdout!r}, expected {case["stdout"]!r}')
        for text in case.get('contains', []):
            if text not in result.stdout:
                problems.append(f'missing stdout text: {text!r}')
        if 'stderr_contains' in case and case['stderr_contains'] not in result.stderr:
            problems.append(f'missing stderr text: {case["stderr_contains"]!r}')
        print(f'CASE {case["name"]}: exit={result.returncode}')
        print(result.stdout, end='')
        if result.stderr:
            print('STDERR:\n' + result.stderr, end='')
        if problems:
            print('\nFAIL: ' + '; '.join(problems))
            return 1
        print('CONTRACT PASS')
        return 0

if __name__ == '__main__':
    raise SystemExit(main())
```

E.3 Interpret the verification matrix

Build and test evidence should name the source revision, compiler, standard library, flags, operating system, and test selection. A clean Clang build and a clean GCC build can share one libstdc++ implementation. That is useful front-end diversity, not proof of behavior on libc++, Microsoft's standard library, or every platform. The delivered `TEST_REPORT.md` records the actual boundary.

Sanitizer runs complement ordinary tests by detecting classes of invalid behavior during executed paths. They do not prove comparator laws, correct business requirements, acceptable performance, or absence of defects in unexecuted code. Keep sanitizer output and ordinary fixture output rather than summarizing every check as an undifferentiated green badge.

For a release, retain the source, fixtures, manuscript, publishing scripts, styles, dependency versions, and checksum manifest together. A future correction should regenerate listings from the same source used in tests. Book rebuilding and C++ testing are separate commands because they have different dependencies and failure modes. A successful EPUB build does not establish that any C++ example was compiled.

E.4 A practical acceptance worksheet

Record the intended audience and language baseline. Confirm every curriculum heading is present. Build from a clean directory with the documented command. Run all fixtures, then selected in-process failure tests. Inspect representative code listings in both EPUB and PDF. Confirm that navigation links resolve, code is not clipped, and references distinguish C++20 from living documentation.

Finally, identify unverified platforms and behaviors explicitly. A reading PDF is not automatically a printer-specific production interior; a structurally checked EPUB is not automatically certified by official EPUBCheck or every e-reader. The package documents the checks performed rather than converting their success into a broader unsupported publishing guarantee.

Appendix F — Working Vocabulary and Design Questions

F.1 Values, owners, and borrows

Value semantics. An object behaves as a meaningful value under its documented copying, comparison, and mutation rules. Independent value copies are a design goal for many domain types, but shared or borrowed members can change those semantics. Ask whether modifying one copy can affect another through an alias.

Owner. The object or subsystem responsible for a resource’s lifetime. Ownership includes responsibility for release, not merely permission to access. Ask which exact destructor or explicit completion path ends that responsibility and whether another object can accidentally release the same resource.

Borrow. Temporary access to an object owned elsewhere, commonly represented by a reference, pointer, iterator, span, or string view. Ask what must remain alive and unchanged during use, and whether the borrow escapes the scope in which those facts are established.

RAII. Associate resource responsibility with an object’s lifetime so cleanup occurs when that lifetime ends. Ask separately how acquisition failure, successful completion, and fallback cleanup are reported. RAII does not itself reverse external side effects or restore a previously modified domain value.

Rule of Zero. Use members with correct ownership behavior so the containing type does not need unnecessary custom copy, move, or destruction machinery. Ask whether memberwise operations also preserve relationships between members, such as a cached size and the storage it describes.

Moved-from state. The state left after a move operation has been applied. Its permitted uses come from the type’s contract. Ask whether the type promises emptiness, a valid unspecified value, or another explicit state; do not assume that every move sets a source to zero.

Aliasing shared ownership. A shared owner exposes one pointer while retaining the ownership group of another object, such as an enclosing snapshot. Ask whether the exposed subobject remains valid for the owner’s lifetime and whether later mutation can invalidate that relationship.

F.2 Interfaces and guarantees

Class invariant. A relationship that valid observable states of a class must satisfy. Ask which operations can change the relevant members and where validation occurs before those changes become public. A comment saying “valid range” is weaker than an interface that prevents invalid mutation.

Strong exception guarantee. Failure preserves the specified observable state. Ask where that state boundary ends: input positions, diagnostics, counters, and remote effects can lie outside it. A guarantee without a named boundary is easy to overstate.

Commit point. The operation at which a validated candidate replaces accepted state. Ask which earlier steps can fail, whether publication itself can fail, and which object cleans up the old state after success. A visible commit point simplifies both review and testing.

Runtime polymorphism. Dispatch behavior through a common interface to the active dynamic implementation. Ask whether base destruction is safe, overrides match the intended signatures, and derived behavior respects the interface’s semantic contract rather than only its function types.

Composition. Build behavior from smaller components with explicit ownership or borrowing relationships. Ask whether copying the composed object copies its dependencies or shares access to them. A reference member can make two apparent value copies write through the same external sink.

Concept. A named compile-time constraint on template arguments or expressions. Ask which semantic laws remain outside syntactic checking. A comparator can satisfy invocation requirements while violating ordering laws; an integer type can satisfy a concept while particular values overflow.

F.3 Traversal, algorithms, and result models

Iterator invalidation. A change that makes a previously obtained iterator unusable under the relevant container or range rules. Ask about cached end iterators as well as element iterators, and distinguish iterator stability from the stability of an element’s logical identity.

Projection. A function selecting the part of an element used by a range algorithm. Ask whether the projected value owns its data or borrows it and whether comparison over the projected domain satisfies the algorithm’s requirements.

Strict weak ordering. The semantic ordering relationship required by many sorting and ordered-container operations. Ask about self-comparison, contradictory directions, transitivity, and equivalence groups. A comparator using less-than-or-equal is not a valid substitute for strict precedence.

Stable sorting. Preserve prior relative order among comparator-equivalent elements. Ask whether prior order is itself stable across runs. If output must be independent of arrival order, define explicit tie-break keys instead of relying on stability alone.

View. A range representation intended for efficient manipulation under its specific ownership and complexity rules. Ask what it owns, what it borrows, what state it caches, and when computations occur. Do not assume that every view is non-owning or that every pipeline materializes data.

Materialization. Construct an owning collection or result from a traversal recipe. Ask whether the allocation and copying buy a needed independent lifetime, stable value, or effect boundary. Avoiding materialization is not inherently better when callers must retain the result.

Optional. An engaged value or absence of that value. Ask what absence means and whether a fallback would erase an important distinction. An optional string view still borrows bytes; the wrapper owns only the view object.

Variant. One active alternative from a closed set of types, with documented exceptional-state behavior. Ask whether every alternative has a meaningful handler and whether default construction chooses the state the domain actually intends.

F.4 External operations and evidence

Completion. The point at which an operation's required result has been checked, distinct from eventual destructor cleanup. Ask whether final output, flush, or close failures are observed before success is reported. Ordinary completion is not automatically crash durability.

Deterministic fixture. A saved input and expected contract whose result can be compared directly. Ask whether the expectation is hand-derived or independent of the implementation under test. A seed alone may not reproduce a library distribution's exact generated sequence across implementations.

Verification boundary. The scope supported by actual executed checks. Ask which compilers, standard libraries, platforms, configurations, and cases were tested. A precise limitation makes evidence more useful; it does not erase the value of the checks that genuinely passed.

Primary references and further reading

Language references use the C++20 working draft N4861 and its HTML rendering. This preserves the selected language baseline instead of silently linking a rule to a changing future draft. Compiler and build documentation can change independently; consult the installed version for unsupported or altered options. The links below support the chapter mechanisms. Original examples, proofs, scenarios, and exercises are not reproduced from these sources.

C++20 working draft N4861 — Abstract classes [class.abstract]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Algorithms library [algorithms]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

Clang AddressSanitizer documentation

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Associative-container requirements [associative.reqmts]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Binary-search operations [alg.binary.search]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Structured bindings [dcl.struct.bind]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Value categories [basic.lval]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Time library [time]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

CMake build-system manual

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Concepts library [concepts]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — constexpr declarations [dcl.constexpr]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Template constraints [temp.constr]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Construction and destruction behavior [class.ctor]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Standard conversions [conv]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Copy and move assignment [class.copy.assign]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Copy and move constructors [class.copy.ctor]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — C library file operations [c.files]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Template argument deduction [temp.deduct]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Destructors [class.dtor]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Construction, destruction, and exception cleanup [except.ctor]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Filesystem library [fs]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Filter view [range.filter]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Numeric parsing with from_chars [charconv.from.chars]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — File streams [file.streams]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Fundamental types [basic.fundamental]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

GCC: C dialect and language-standard options

Original project or institution documentation. [Open the primary reference.](#)

GCC: optimization options

Original project or institution documentation. [Open the primary reference.](#)

GCC: options controlling the kind of output

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Derived classes [class.derived]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — General principles [intro]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Input streams [istream]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Iterator operations [iterator.operations]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Iterator requirements [iterator.requirements]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Lambda expressions [expr.prim.lambda]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Object lifetime [basic.life]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Numeric limits [numeric.limits]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — List [list]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Initializing bases and members [class.base.init]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Member functions [class.mfct]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

Microsoft C++: selecting the language-standard version

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Nonmodifying sequence operations [alg.nonmodifying]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Generalized numeric operations [numeric.ops]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Overloaded operators [over.oper]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Optional objects [optional]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Output streams [ostream]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Overloading [over]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Partial specialization [temp.class.spec]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Random-number generation [rand]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Range adaptors [range.adaptors]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Ranges library [ranges]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — References [dcl.ref]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Sequence-container requirements [sequence.reqmts]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Shared-ownership pointers [util.smartptr.shared]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Sorting and related operations [alg.sorting]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Three-way comparison [expr.spaceship]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Explicit specialization [temp.expl.spec]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Array container [array]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Templates [temp]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Type traits [meta]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Transform algorithm [alg.transform]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Tuples [tuple]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

Clang UndefinedBehaviorSanitizer documentation

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Unique-ownership pointers [unique.ptr]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Unordered associative requirements [unord.req]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Utility components [utility]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Variant [variant]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Vector [vector]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Vector capacity [vector.capacity]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Vector modifiers [vector.modifiers]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Virtual functions [class.virtual]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Weak pointers [util.smartptr.weak]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

Additional primary references used in this expansion

The following living documents were consulted to cross-check mechanisms and tool behavior while expanding the manuscript. They can change after this edition. The fixed N4861 references above remain the language-baseline navigation for C++20; a feature in a newer draft is not automatically part of the companion's supported mode.

C++ Core Guidelines — resource, interface, and class design

Original guideline project by its named editors and contributors. Design guidance is distinguished from normative language requirements. [Open the C++ Core Guidelines](#).

Current C++ working draft — filter view behavior

Living working-draft text, used to cross-check adaptor state and lifetime discussions rather than to introduce later language features. [Open filter view wording](#). For the fixed teaching baseline, also see [N4861 filter views](#).

Current C++ working draft — library object states

Living working-draft text concerning library object requirements, including moved-from states. Type-specific guarantees still take precedence where specified. [Open library argument requirements](#).

CMake — requesting a target language standard

Original CMake property documentation. The companion explicitly sets `CXX_STANDARD`, `CXX_STANDARD_REQUIRED`, and `CXX_EXTENSIONS` rather than assuming a compiler's default mode. [Open CXX_STANDARD documentation](#).

GCC — diagnostic options

Original compiler documentation for warning options and their scope. A warning-clean build is evidence for enabled diagnostics, not a proof that all defects are absent. [Open warning-option documentation](#).